

Status Codes for Errors and Notifications

Table of Contents

Changes and deprecations 2
2025.05 2
2025.04 2
2025.01 2
Server notifications 3
GQL-status notification object 3
General codes for success 4
Neo4j-defined GQLSTATUS codes 5
Neo4j-status notification object 5
Server notification grouping and filtering 6
Notification internals 7
Server-driver version compatibility 8
List of notification codes 8
`PERFORMANCE` notifications 9
`HINT` notifications 19
`UNRECOGNIZED` notifications 22
`UNSUPPORTED` notifications 28
`DEPRECATION` notifications 29
`SECURITY` notifications 59
`TOPOLOGY` notifications 76
`SCHEMA` notifications 86
`GENERIC` notifications 93
Server errors 104
GQL-status error object 104
Classes of GQLSTATUS error codes 105
Neo4j-status error object 106
Types of server errors 106
Error internals 107
Query logging 107
Server-driver version compatibility 108
List of GQLSTATUS error codes 108
Connection exceptions 109
Data exceptions 112
Invalid transaction state 132
Invalid transaction termination 134
Transaction rollback 135
Syntax error or access rule violation 135
General processing exceptions 163

System configuration or operation exceptions 166
Procedure exceptions 177
Function exceptions 182
Dependent object errors 183
List of Neo4j error codes 326
Client errors 326
Transient errors 331
Database error 333

This manual covers all status codes for errors and notifications that a Neo4j server may return to indicate the result of a Cypher request.

Starting from 5.23 for notifications and 5.25 for errors, Neo4j supports the GQL standard.
GQL is the new ISO International Standard query language for graph databases. Cypher®, Neo4j's query language, supports most mandatory and a substantial portion of the optional GQL features (as defined by the ISO/IEC 39075:2024(en) - Information technology - Database languages - GQL Standard). For more information, see Cypher Manual → GQL conformance.

As part of this GQL compliance, Cypher also includes status codes that a GQL-compliant DBMS returns to indicate the outcome of a request. For more information on the GQL-status object framework for notifications and errors, see Server notifications and Server errors.

License: Creative Commons 4.0

Changes and deprecations

2025.05

Starting from 2025.05, the `failureReason` in the JSON template of the query log, is deprecated and switched off by default. Use the `errorInfo` entry instead.

For more information about `errorInfo`, refer to Operations manual → Logging → GQL error information.

2025.04

Using the error messages programmatically is deprecated and might change in a future release. Therefore, parsing the error messages or the status descriptions and incorporating them into scripts is not recommended. If you need to handle errors programmatically, please use the GQLSTATUS error codes, for example, `22NC5`.

2025.01

A new JSON template is available for the query log, which is the default set in *server_logs.xml*.
For more details, see the Operations manual → Monitoring → Logging.

Server notifications

After a successful query execution, the Neo4j server sends notifications to give additional information about the query execution or provide advice on how to improve the query's quality. The driver receives these notifications and sends them to the Neo4j tools (e.g. Browser, Bloom, Cypher Shell) or the user application, which display them to the user.

From version 5.23, Neo4j has a new GqlStatusObject API in addition to the existing Notification API (deprecated since 5.26). The GqlStatusObject API provides information about the status of a Cypher query or command execution in compliance with the ISO/IEC 39075:2024(en) - Information technology - Database languages - GQL Standard.

This page describes the GQL-status object and the Neo4j notification frameworks, their structure, the objects they provide for notification, and how to interpret them. It also explains the server notification grouping and filtering, the notification internals, and the server-driver compatibility for both the Notification and GqlStatusObject APIs.

GQL-status notification object

Version 5.23

In the GQL-status object framework, when a user executes a query to the server, it always produces a result known as the *execution outcome*. If no error occurs during execution, the outcome is recorded as a completion condition, indicating a successful result with a regular non-empty result, an omitted result, or no data. It is represented as a list of GQL-status objects, which are organized according to the following precedence rules, where the first object in the list is the primary GQL-status object:

1. `NO DATA` has precedence over `WARNING`.
2. `WARNING` has precedence over the `SUCCESSFUL COMPLETION` subclass.
3. `SUCCESSFUL COMPLETION` subclass has precedence over `INFORMATIONAL`.
4. `INFORMATIONAL` is the condition with the least precedence.

For more information about `SUCCESSFUL COMPLETION`, `NO DATA`, or `OMITTED RESULT`, see General codes for success.
The GQL-status object can contain multiple GQL-status objects, where each object represents a different condition of the query execution. The primary GQL-status object describes the condition with the greatest precedence and is always present. All other GQL-status objects in the list are additional GQL-status objects.

The GQL-status object also includes Neo4j-specific information, such as severity level and notification classification, which can be used for filtering. For more information about notification grouping and filtering, see Server notification grouping and filtering.

Each GQL-status object consists of the following fields:

Table 1. GQLSTATUS notification object

<table>
<tr><td>GQLSTATUS code</td><td>A 5-character string that is the concatenation of a 2-character class code followed by a 3-character subclass code, which identifies the condition of the notification.</td></tr>
<tr><td>StatusDescription</td><td>A human-readable description of the GQLSTATUS, which consists of a condition, a subcondition, and an optional additional text about the condition. The condition and subcondition are textual representations of the class and subclass codes, respectively.</td></tr>
<tr><td>DiagnosticRecord</td><td>Extra information about the status, given as key-value pairs, both on the server and driver side. To retrieve the full diagnostic record, you can use the <code>diagnosticRecord()</code> on the server side or the corresponding method on the driver sides. Additional helper methods are exposed for some useful fields.
<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td><code>OPERATION</code></td><td>The operation that the notification is related to. Always defaults to empty.</td></tr>
<tr><td><code>OPERATION_CODE</code></td><td>The operation code that the notification is related to. Always defaults to <code>0</code>.</td></tr>
<tr><td><code>CURRENT_SCHEMA</code></td><td>The current schema that the notification is related to. Always defaults to <code>/</code>.</td></tr>
<tr><td><code>_severity</code></td><td>The Neo4j severity level, which can be one of the following:
<code>WARNING</code>: There might be a problem with your query. Please, take a look.
<code>INFORMATION</code>: The query is correct, but this information can still be useful.</td></tr>
<tr><td><code>_classification</code></td><td>The Neo4j category of the notification.</td></tr>
<tr><td><code>_position</code></td><td>The position, given by row and column, where the notification is relevant in the query text.</td></tr>
</table></td></tr>
</table>

General codes for success

GQL has three general codes for success, indicated by different GQLSTATUS codes in categories S (successful completion) and N (no data), which are not covered by the Neo4j notification framework and are considered to be the default statuses for successful completion, omitted result, and no data, respectively. The Neo4j classification, severity, position, and status parameters are not meaningful for these GQL statuses, so they are not included in the diagnostic record and are set to default values either by the server or by the driver.

Table 2. GQLSTATUS general codes

GQLSTATUS	Condition	Subcondition	Description
00000	Successful completion		Successful completion with a regular non-empty result (n > 0 columns, m > 0 rows), for example, a `RETURN` clause with matches.
00001	Successful completion	Omitted result	Successful completion with no return columns (n = 0 columns, m = 0 rows), for example, `EXPLAIN` queries.
02000	No data		Successful completion with an empty result (n > 0 columns, m = 0 rows), for example, a `MATCH` clause with no matches.

GQLSTATUS general codes are filled in by the server unless the server is too old to be aware of GQL-status objects, in which case, it is polyfilled by the driver (see Server - driver compatibility).

Neo4j-defined GQLSTATUS codes

The Neo4j-defined GQLSTATUS codes are divided into classes and subclasses, where the class code is a 2-character string (one of `00`, `01`, or `03`) and the subclass code is a 3-character string. The class code indicates the general condition of the status (such as successful completion, warning, or information), and the subclass code provides more detailed information about the condition, such as classification and messages.

The following table lists the Neo4j-defined groups of GQLSTATUS codes and their meanings:

Table 3. GQLSTATUS groups of codes as defined by Neo4j

GQLSTATUS code	Description
01N0[y]	Deprecation warnings
01N3[y]	Hint warnings
01N4[y]	Unsupported warnings
01N5[y]	Unrecognized warnings
01N6[y]	Generic warnings
01N7[y]	Security warnings
01N8[y]	Topology warnings
03N6[y]	Generic information
03N8[y]	Topology information
03N9[y]	Performance information
00N5[y]	Unrecognized information under successful completion
00N6[y]	Generic information under successful completion
00N7[y]	Security information under successful completion
00N8[y]	Topology information under successful completion
00NA[y]	Schema information under successful completion

Neo4j-status notification object

Deprecated in 5.26

The Neo4j-status object for notifications contains diagnostic information representing the successful outcome of a Cypher query or command execution, including severity, the `ClientNotification` code,

category, title, description, and position in the query text where the notification is relevant. Depending on the application, some of the fields from the notification object might not be visible.

The notification object consists of the following fields:

Table 4. Neo4j notification object

Neo4j code	The Neo4j code in the form of `Neo.ClientNotification.[SubType].[Name]`.
Title	The title of the Neo4j code.
Description	The description of the specific notification.
Severity level	The severity can be one of the following: • `WARNING`: There might be a problem with your query. Please, take a look. • `INFORMATION`: The query is correct, but this information can still be useful.
Category	The category of the notification.
Position	The position, given by row and column, where the notification is relevant in the query text.

Server notification grouping and filtering

All server notifications are grouped by category (which is called classification in the GqlStatusObject framework) and severity level, which can be one of `WARNING`, `WARNING OR INFORMATION`, or `INFORMATION`.

The driver-side notification configuration used for filtering notifications by category and severity is the same for both Neo4j Notification and GQL-status object frameworks. The driver can filter notifications by category/classification and severity level, and the server will only send notifications that match the driver-side configuration.

The driver can also choose to ignore notifications. However, as per the GQLSTATUS framework, the server must always send the primary GQL-status object. Therefore, if notifications are off or the notification configuration filtering is set to filter out all notifications, the server will still send the primary GQL-status object with the status `SUCCESSFUL COMPLETION`, `OMITTED RESULT` or `NO DATA`.

The following notification groups exist in Neo4j, ordered by severity:

Table 5. Notification groups and severity levels

CATEGORY/CLASSIFICATION	SEVERITY	EXPLANATION	RECOMMENDED ACTION
`DEPRECATION`	`WARNING`	The query or command uses deprecated features that should be replaced.	Update to use the new functionality.
`HINT`	`WARNING`	The given hint cannot be satisfied.	Remove the hint or fix the query so the hint can be used.

CATEGORY/CLASSIFICATION	SEVERITY	EXPLANATION	RECOMMENDED ACTION
UNSUPPORTED	WARNING	The query or command is trying to use features not supported by the current system or using experimental features that should not be used in production.	Unsupported features cannot be trusted and should not be used in production.
UNRECOGNIZED	WARNING OR INFORMATION	The query or command mentions entities that are unknown to the system.	Make sure you have not misspelled the entity.
SECURITY	WARNING OR INFORMATION	The result of the query or command indicates a potential security issue.	Make sure that the behaviour is what you intended.
TOPOLOGY	INFORMATION	Information provided while executing database and server related commands.	
SCHEMA	INFORMATION	Information provided while managing indexes and constraints.	
GENERIC	WARNING OR INFORMATION	Notifications that are not part of a wider class.	Depends on the specific notification.
PERFORMANCE	INFORMATION	The query uses costly operations and might be slow. Consider if it is possible to rewrite the query in a different way.	

Notification internals

Deprecated in 5.26

The server and driver communicate with each other through the Bolt protocol. During the handshake process, they agree on using the newest possible Bolt protocol version that both the server and the driver support. For more information on the Bolt versions supported by different server versions, see the Bolt Protocol documentation.

On the server side, notifications are part of the Result Core API. A method called `getNotifications()`, which is deprecated since 5.26, returns a list of server-side notification objects. These notifications are then sent to the driver as success Bolt message metadata. On the driver side, notifications are part of the ResultSummary API, which has a method called `notifications()` that returns a list of driver-side Notification objects. The result of the `getCode()` or `code()` methods is known as the Neo4j status code. Driver-side notification configuration filters notifications by severity and/or category at both the driver and session levels. For more information, see Server notification grouping and filtering.

From version 5.23, Neo4j has a new GqlStatusObject API in addition to the existing Notification API. This can be used using the `.getGqlStatusObjects()` method in the Result Core API or by using the latest Neo4j drivers. The Notification API is deprecated since Neo4j 5.26.

Server-driver version compatibility

The GqlStatusObject API is available in Neo4j 5.22 and later versions on the server side and in the 5.23 driver and later versions on the driver side. The current Notification API is still present, and the GqlStatusObject API can be used in parallel with it.

To fully utilize the GqlStatusObject API, both your server and the driver must support it. Drivers that are older than 5.23 send only notifications from the Notification API, even if the server is 5.22 or later.

If a driver of version 5.23 or later talks to a server that is too old to be aware of GQL-status objects, the driver needs to poly-fill the GqlStatusObject API with information. The driver tries to deduce `SUCCESS`, `OMITTED RESULT`, or `NO DATA` from the returned number of records and columns. If that fails, the general GQLSTATUS code will be set to `02N42`. Then, the driver poly-fills the rest of the GQL-status object list with the notifications from the old notification API. These will get GQLSTATUS `01N42` for notifications with severity `WARNING` and `03N42` for notifications with severity `INFORMATION`. Finally, the list of poly-filled GQL-status objects is sorted according to the GQL precedence rules described in GQL-status notification object.

Table 6. GQLSTATUS compatibility codes

GQLSTATUS	Condition	Subcondition	Description
01N42	Warning	Unknown warning	Poly-filled notification with severity `WARNING`.
02N42	No data	Unknown subcondition	Poly-filled general status when `SUCCESS`, `OMITTED RESULT`, or `NO DATA` cannot be deduced.
03N42	Informational	Unknown notification	Poly-filled notification with severity `INFORMATION`.

List of notification codes

The following page provides an overview of all notifications in Neo4j, including the GQLSTATUS codes and Neo4j codes, along with some scenarios and their possible solutions.

- `PERFORMANCE` notifications
- `HINT` notifications
- `UNRECOGNIZED` notifications
- `UNSUPPORTED` notifications
- `DEPRECATION` notifications
- `SECURITY` notifications
- `TOPOLOGY` notifications
- `SCHEMA` notifications
- `GENERIC` notifications



Warning:

Please note that while GQLSTATUS codes remain stable (any changes to them will be breaking), changes to status descriptions associated with these codes are not breaking and may happen at any time. For this reason, parsing the status descriptions or incorporating them into scripts is not recommended.

`PERFORMANCE` notifications

Performance notifications are returned whenever the query uses costly operations and the performance may be improved by changing the query or adding an index.

Cartesian product

This notification is returned when there is a Cartesian product in the plan.

Table 7. Notification details

Neo4j code	`Neo.ClientNotification.Statement.CartesianProduct`
Title	This query builds a cartesian product between disconnected patterns.
Description	If a part of a query contains multiple disconnected patterns, this will build a cartesian product between all those parts. This may produce a large amount of data and slow down query processing. While occasionally intended, it may often be possible to reformulate the query that avoids the use of this cross product, perhaps by adding a relationship between the different parts or by using OPTIONAL MATCH (`%s`)
Category	`PERFORMANCE`
GQLSTATUS code	`03N90`
Status description	info: cartesian product. The disconnected patterns `{ [pat] }` build a cartesian product. A cartesian product may produce a large amount of data and slow down query processing.
Classification	`PERFORMANCE`
SeverityLevel	`INFORMATION`

Example 1. A query that contains many disconnected patterns

Query

```
MATCH (c:Child), (p:Parent) RETURN c, p
```

Returned GQLSTATUS code

03N90

Returned status description

info: cartesian product. The disconnected patterns `(c:Child), (p:Parent)` build a cartesian product. A cartesian product may produce a large amount of data and slow down query processing.

Suggestions for improvement

In case a Cartesian product is needed, nothing can be done to improve this query. In many cases, however, you might not need a combination of all children and parents, and that is when this query could be improved. If for example, you need the children and the children's parents, you can improve this query by rewriting it to the following:

```
MATCH (c:Child)-[:ChildOf]->(p:Parent) RETURN c, p
```

Query

```
MATCH (c:Child), (p:Parent) RETURN c, p
```

Description of the returned code

If a part of a query contains multiple disconnected patterns, this will build a cartesian product between all those parts. This may produce a large amount of data and slow down query processing. While occasionally intended, it may often be possible to reformulate the query that avoids the use of this cross product, perhaps by adding a relationship between the different parts or by using `OPTIONAL MATCH` (identifier is: (`p`))

Suggestions for improvement

In case a Cartesian product is needed, nothing can be done to improve this query. In many cases, however, you might not need a combination of all children and parents, and that is when this query could be improved. If for example, you need the children and the children's parents, you can improve this query by rewriting it to the following:

```
MATCH (c:Child)-[:ChildOf]->(p:Parent) RETURN c, p
```

Unbounded variable length pattern

This notification is returned when there is no upper bound specified on the variable length relationship.

Table 8. Notification details

Neo4j code	`Neo.ClientNotification.Statement.UnboundedVariableLengthPattern`
Title	The provided pattern is unbounded, consider adding an upper limit to the number of node hops.
Description	Using shortest path with an unbounded pattern will likely result in long execution times. It is recommended to use an upper limit to the number of node hops in your pattern.
Category	`PERFORMANCE`
GQLSTATUS code	`03N91`

Status description	info: unbounded variable length pattern. The provided pattern `{ [pat] }` is unbounded. Shortest path with an unbounded pattern may result in long execution times. Use an upper limit (e.g. `[*..5]`) on the number of node hops in your pattern.
Classification	PERFORMANCE
SeverityLevel	INFORMATION

Example 2. Shortest path with an unbounded pattern

Query

```
MATCH p=shortestPath((n)-[*]->(m))
WHERE n <> m
RETURN p
```

Returned GQLSTATUS code

03N91

Returned status description

info: unbounded variable length pattern. The provided pattern `(n)-[*]→(m)` is unbounded. Shortest path with an unbounded pattern may result in long execution times. Use an upper limit (e.g. `[*..5]`) on the number of node hops in your pattern.

Suggestions for improvement

If you have a big graph, this query might be very slow. Consider adding an upper limit.

```
MATCH p=shortestPath((n)-[*..8]->(m))
WHERE n <> m
RETURN p
```

Query

```
MATCH p=shortestPath((n)-[*]->(m)) RETURN p
```

Description of the returned code

Using shortest path with an unbounded pattern will likely result in long execution times. It is recommended to use an upper limit to the number of node hops in your pattern.

Suggestions for improvement

If you have a big graph, this query might be very slow. Consider adding an upper limit.

```
MATCH p=shortestPath((n)-[*..8]->(m)) RETURN p
```

Exhaustive shortest path

This notification is returned when a predicate, given on the shortest path, needs to inspect the whole path before deciding whether it is valid, the shortest path might fall back to the exhaustive search algorithm. For more information, see Cypher manual → Shortest path - additional predicate checks on the paths.

Table 9. Notification details

Neo4j code	`Neo.ClientNotification.Statement.ExhaustiveShortestPath`
Title	Exhaustive shortest path has been planned for your query that means that shortest path graph algorithm might not be used to find the shortest path. Hence an exhaustive enumeration of all paths might be used in order to find the requested shortest path.
Description	Using shortest path with an exhaustive search fallback might cause query slow down since shortest path graph algorithms might not work for this use case. It is recommended to introduce a `WITH` to separate the `MATCH` containing the shortest path from the existential predicates on that path.
Category	`PERFORMANCE`
GQLSTATUS code	`03N92`
Status description	info: exhaustive shortest path. The query runs with exhaustive shortest path due to the existential predicate(s) `{ [predList] }`. It may be possible to use `WITH` to separate the `MATCH` from the existential predicate(s).
Classification	`PERFORMANCE`
SeverityLevel	`INFORMATION`

Example 3. A query that runs with an exhaustive shortest path

Query

```
MATCH p = shortestPath(()-[*..42]-())
WHERE ANY(n in nodes(p) WHERE n:Label)
RETURN p
```

Returned GQLSTATUS code

03N92

Returned status description

info: exhaustive shortest path. The query runs with exhaustive shortest path due to the existential predicate(s) `ANY(n in nodes(p) WHERE n:Label)`. It may be possible to use `WITH` to separate the `MATCH` from the existential predicate(s).

Suggestions for improvement

Separate the predicate by introducing a `WITH` after the `MATCH` clause.

```
MATCH p = shortestPath(()-[*..42]-())
WITH p
WHERE ANY(n in nodes(p) WHERE n:Label)
RETURN p
```

Query

```
MATCH p = shortestPath(()-[*..42]-())
WHERE ANY(n in nodes(p) WHERE n:Label)
RETURN p
```

Description of the returned code

Using shortest path with an exhaustive search fallback might cause query slow down since shortest path graph algorithms might not work for this use case. It is recommended to introduce a `WITH` to separate the `MATCH` containing the shortest path from the existential predicates on that path.

Suggestions for improvement

Separate the predicate by introducing a `WITH` after the `MATCH` clause.

```
MATCH p = shortestPath(()-[*..42]-())
WITH p
WHERE ANY(n in nodes(p) WHERE n:Label)
RETURN p
```

No applicable index

This notification is returned when using `LOAD CSV` with a `MATCH` or a `MERGE` clause that matches a non-indexed label. This may not perform well on large data sets. Adding an index could improve the query speed.

Table 10. Notification details

Neo4j code	`Neo.ClientNotification.Statement.NoApplicableIndex`
Title	Adding a schema index may speed up this query.
Description	Using `LOAD CSV` followed by a `MATCH` or `MERGE` that matches a non-indexed label will most likely not perform well on large data sets. Please consider using a schema index.
Category	`PERFORMANCE`
GQLSTATUS code	`03N93`
Status description	info: no applicable index. `LOAD CSV` in combination with `MATCH` or `MERGE` on a label that does not have an index may result in long execution times. Consider adding an index for label `{ [label] }`.
Classification	`PERFORMANCE`
SeverityLevel	`INFORMATION`

Example 4. `LOAD CSV` with `MATCH` or `MERGE`

Query

```
LOAD CSV FROM 'file:///ignore/ignore.csv' AS line WITH * MATCH (n:Person{name:line[0]})
RETURN line, n
```

Returned GQLSTATUS code

03N93

Returned status description

info: no applicable index. `LOAD CSV` in combination with `MATCH` or `MERGE` on a label that does not have an index may result in long execution times. Consider adding an index for label `Person`.

Suggestions for improvement

Create an index on the label and property you match.

```
CREATE INDEX FOR (n:Person) ON (n.name)
```

Query

```
LOAD CSV FROM 'file:///ignore/ignore.csv' AS line WITH * MATCH (n:Person{name:line[0]})
RETURN line, n
```

Description of the returned code

Using `LOAD CSV` followed by a `MATCH` or `MERGE` that matches a non-indexed label will most likely not perform well on large data sets. Please consider using a schema index.

Suggestions for improvement

Create an index on the label and property you match.

```
CREATE INDEX FOR (n:Person) ON (n.name)
```

Eager operator

This notification is returned when the execution plan for a query contains the `Eager` operator.

Table 11. Notification details

Neo4j code	`Neo.ClientNotification.Statement.EagerOperator`
Title	The execution plan for this query contains the Eager operator, which forces all dependent data to be materialized in main memory before proceeding

Description	Using `LOAD CSV` with a large data set in a query where the execution plan contains the Eager operator could potentially consume a lot of memory and is likely to not perform well. See the Neo4j Manual entry on the Eager operator for more information and hints on how problems could be avoided.
Category	PERFORMANCE
GQLSTATUS code	03N94
Status description	info: eager operator. The query execution plan contains the `Eager` operator. `LOAD CSV` in combination with `Eager` can consume a lot of memory.
Classification	PERFORMANCE
SeverityLevel	INFORMATION

Example 5. `LOAD CSV` with an Eager operator

Query

```
LOAD CSV FROM 'file:///ignore/ignore.csv' AS line MATCH (n:Person{name:line[0]}) DELETE n
RETURN line
```

Returned GQLSTATUS code

03N94

Returned status description

info: eager operator. The query execution plan contains the `Eager` operator. `LOAD CSV` in combination with `Eager` can consume a lot of memory.

Suggestions for improvement

See the Cypher Manual → Eager operator for more information and hints on how to avoid problems. In this specific case, the query could be rewritten to the following:

```
LOAD CSV FROM 'file:///ignore/ignore.csv' AS line
CALL {
    WITH line
    MATCH (n:Person{name:line[0]}) DELETE n
}
RETURN line
```

`LOAD CSV` together with an Eager operator can take up a lot of memory.

Query

```
LOAD CSV FROM 'file:///ignore/ignore.csv' AS line MATCH (n:Person{name:line[0]}) DELETE n
RETURN line
```

Description of the returned code

Using `LOAD CSV` with a large data set in a query where the execution plan contains the Eager

operator could potentially consume a lot of memory and is likely to not perform well. See the Neo4j Manual entry on the Eager operator for more information and hints on how problems could be avoided.

Suggestions for improvement

See the Cypher Manual → Eager operator for more information and hints on how to avoid problems. In this specific case, the query could be rewritten to the following:

```
LOAD CSV FROM 'file:///ignore/ignore.csv' AS line
CALL {
    WITH line
    MATCH (n:Person{name:line[0]}) DELETE n
}
RETURN line
```

Dynamic property

Table 12. Notification details

Neo4j code	`Neo.ClientNotification.Statement.DynamicProperty`
Title	Queries using dynamic properties will use neither index seeks nor index scans for those properties
Description	Using a dynamic property makes it impossible to use an index lookup for this query (`%s`)
Category	`PERFORMANCE`
GQLSTATUS code	`03N95`
Status description	info: dynamic property. An index exists on label/type(s) `{ [labelList] }`. It is not possible to use indexes for dynamic properties. Consider using static properties.
Classification	`PERFORMANCE`
SeverityLevel	`INFORMATION`

Example 6. A dynamic node property key makes it impossible to use indexes

Query

```
MATCH (n:Person) WHERE n[$prop] IS NOT NULL RETURN n;
```

Returned GQLSTATUS code

03N95

Returned status description

info: dynamic property. An index exists on label/type(s) `Person`. It is not possible to use indexes for dynamic properties. Consider using static properties.

Suggestions for improvement

If there is an index for `(n:Person) ON (n.name)`, it will not be used for the above query because the query is using a dynamic property. Therefore, if there is an index, it is better to use the constant value. For example, if `[$prop]` is equal to `name`, the following query would be able to use the index:

```
MATCH (n:Person) WHERE n.name IS NOT NULL RETURN n;
```

Query

```
MATCH (n:Person) WHERE n[$prop] IS NOT NULL RETURN n;
```

Description of the returned code

Using a dynamic property makes it impossible to use an index lookup for this query (indexed label is: `Person`)

Suggestions for improvement

If there is an index for `(n:Person) ON (n.name)`, it will not be used for the above query because the query is using a dynamic property. Therefore, if there is an index, it is better to use the constant value. For example, if `[$prop]` is equal to `name`, the following query would be able to use the index:

```
MATCH (n:Person) WHERE n.name IS NOT NULL RETURN n;
```

Example 7. A dynamic relationship property key makes it impossible to use indexes

Query

```
MATCH ()-[r: KNOWS]->() WHERE r[$prop] IS NOT NULL RETURN r
```

Returned GQLSTATUS code

03N95

Returned status description

info: dynamic property. An index exists on label/type(s) `KNOWS`. It is not possible to use indexes for dynamic properties. Consider using static properties.

Suggestions for improvement

Similar to dynamic node properties, use a constant value if possible, especially when there is an index on the relationship property. For example, if `[$prop]` is equal to `since`, you can rewrite the query to:

```
MATCH ()-[r: KNOWS]->() WHERE r.since IS NOT NULL RETURN r
```

Query

```
MATCH ()-[r: KNOWS]->() WHERE r[$prop] IS NOT NULL RETURN r
```

Description of the returned code

Using a dynamic property makes it impossible to use an index lookup for this query (indexed type is: `KNOWS`)

Suggestions for improvement

Similar to dynamic node properties, use a constant value if possible, especially when there is an index on the relationship property. For example, if `[$prop]` is equal to `since`, you can rewrite the query to:

```
MATCH ()-[r: KNOWS]->() WHERE r.since IS NOT NULL RETURN r
```

Failed code generation

The `CodeGenerationFailed` notification is created when it is not possible to generate a code for a query, for example, when the query is too big. For more information about the specific query, see the stack trace in the *debug.log* file.

Table 13. Notification details

Neo4j code	`Neo.ClientNotification.Statement.CodeGenerationFailed`
Title	The database was unable to generate code for the query. A stacktrace can be found in the debug.log.
Description	The database was unable to generate code for the query. A stacktrace can be found in the debug.log. (method too big)
Category	`PERFORMANCE`
GQLSTATUS code	`03N96`
Status description	info: code generation failed Failed to generate code, falling back to interpreted `{ [cfgSetting] }` engine. A stacktrace can be found in the debug.log. Cause: `{ [cause] }`.
Classification	`PERFORMANCE`
SeverityLevel	`INFORMATION`

Virtual graph post-processing

This notification is returned when the query execution plan against a virtual graph contains a post-processing step that requires the local engine to fully materialize intermediate results from the remote source.

Table 14. Notification details

Neo4j code	`Neo.ClientNotification.Statement.VirtualGraphPostProcessing`
Title	The query plan against a virtual graph contains a potentially expensive post-processing step.
Description	The query execution plan contains a post-processing step that materialize intermediate results. This may transfer large amounts of data from the remote source and increase memory usage. Consider rewriting the query so that aggregation, `ORDER BY`, `DISTINCT`, and `LIMIT` can be pushed down to the remote source.
Category	`PERFORMANCE`
GQLSTATUS code	`03N97`
Status description	info: virtual graph post-processing. The query execution plan contains a post-processing step that materialize intermediate results. This may transfer large amounts of data from the remote source and increase memory usage. Consider rewriting the query so that aggregation, `ORDER BY`, `DISTINCT`, and `LIMIT` can be pushed down to the remote source.
Classification	`PERFORMANCE`
SeverityLevel	`INFORMATION`

`HINT` notifications

`HINT` notifications are returned by default when the Cypher planner or runtime cannot create a query plan to fulfill a specified hint, for example, `JOIN` or `INDEX`. This behavior of the Cypher planner or runtime can be changed by setting the configuration `dbms.cypher.hints_error` to `true`. In this case, the query will return an error.

Join hint unfulfillable

Table 15. Notification details

Neo4j code	`Neo.ClientNotification.Statement.JoinHintUnfulfillableWarning`
Title	The database was unable to plan a hinted join.
Description	The hinted join was not planned. This could happen because no generated plan contained the join key, please try using a different join key or restructure your query. (`%s`)
Category	`HINT`
GQLSTATUS code	`01N30`
Status description	warn: join hint unfulfillable. Unable to create a plan with `JOIN ON { [variableList] }`. Try to change the join key(s) or restructure your query.
Classification	`HINT`
SeverityLevel	`WARNING`

Example 8. Inability to fulfill the hint despite the given `JOIN` hint

Query

```
MATCH (a:A)
WITH a, 1 AS horizon
OPTIONAL MATCH (a)-[r]->(b:B)
USING JOIN ON a
OPTIONAL MATCH (a)--(c)
RETURN *
```

Returned GQLSTATUS code

01N30

Returned status description

warn: joint hint unfulfillable. Unable to create a plan with `JOIN ON a`. Try to change the join key(s) or restructure your query.

Suggestions for improvement

The `JOIN` hint cannot be applied because its specified variable is before the `OPTIONAL MATCH` and, therefore, is already bound. The only option for this query is to either remove the hint or modify the query to allow it to be used.

Query

```
MATCH (a:A)
WITH a, 1 AS horizon
OPTIONAL MATCH (a)-[r]->(b:B)
USING JOIN ON a
OPTIONAL MATCH (a)--(c)
RETURN *
```

Description of the returned code

The hinted join was not planned. This could happen because no generated plan contained the join key, please try using a different join key or restructure your query. (hinted join key identifier is: `a`)

Suggestions for improvement

The `JOIN` hint cannot be applied because its specified variable is before the `OPTIONAL MATCH` and, therefore, is already bound. The only option for this query is to either remove the hint or modify the query to allow it to be used.

Hinted index not found

Table 16. Notification details

Neo4j code	`Neo.ClientNotification.Schema.HintedIndexNotFound`
Title	The request (directly or indirectly) referred to an index that does not exist.

Description	The hinted index does not exist, please check the schema (`%s`)
Category	`HINT`
GQLSTATUS code	`01N31`
Status description	warn: hinted index not found. Unable to create a plan with `{ [idxDescr] }` because the index does not exist.
Classification	`HINT`
SeverityLevel	`WARNING`

Example 9. Inability to use the label index despite the given index hint

Query

```
MATCH (a: Label)
USING INDEX a:Label(id)
WHERE a.id = 1
RETURN a
```

Returned GQLSTATUS code

01N31

Returned status description

warn: hinted index not found. Unable to create a plan with `INDEX :Label(id)` because the index does not exist.

Suggestions for improvement

The hinted index does not exist, make sure the label and property are spelled correctly. If the spelling is correct, either create the index or remove the hint from the query.

Query

```
MATCH (a: Label)
USING INDEX a:Label(id)
WHERE a.id = 1
RETURN a
```

Description of the returned code

The hinted index does not exist, please check the schema (index is: INDEX FOR (`a`:`Label`) ON (`a.id`))

Suggestions for improvement

The hinted index does not exist, make sure the label and property are spelled correctly. If the spelling is correct, either create the index or remove the hint from the query.

Example 10. Inability to use the relationship index despite the given index hint

Query

```
MATCH ()-[r:Rel]-()
USING INDEX r:Rel(id)
WHERE r.id = 1
RETURN r
```

Returned GQLSTATUS code

01N31

Returned status description

warn: hinted index not found. Unable to create a plan with `INDEX :Rel(id)` because the index does not exist.

Suggestions for improvement

The hinted index does not exist, make sure the relationship type and property are spelled correctly. If the spelling is correct, either create the index or remove the hint from the query.

Query

```
MATCH ()-[r:Rel]-()
USING INDEX r:Rel(id)
WHERE r.id = 1
RETURN r
```

Description of the returned code

The hinted index does not exist, please check the schema (index is: INDEX FOR ()-[`r`:`Rel`]-() ON (`r`.`id`))

Suggestions for improvement

The hinted index does not exist, make sure the relationship type and property are spelled correctly. If the spelling is correct, either create the index or remove the hint from the query.

`UNRECOGNIZED` notifications

Unrecognized notifications are returned when the query or command mentions entities that are unknown to the system.

Home database not found

Table 17. Notification details

Neo4j code	`Neo.ClientNotification.Database.HomeDatabaseNotFound`
Title	The request referred to a home database that does not exist.

Description	The home database provided does not currently exist in the DBMS. This command will not take effect until this database is created. (`%s`)
Category	UNRECOGNIZED
GQLSTATUS code	00N50
Status description	note: successful completion - home database not found. The database `{ [db] }` does not exist. Verify that the spelling is correct or create the database for the command to take effect.
Classification	UNRECOGNIZED
SeverityLevel	INFORMATION

Example 11. Setting the `home` database to a database that does not exist

Query

```
CREATE USER john SET PASSWORD "secret" SET HOME DATABASE nej4
```

Returned GQLSTATUS code

00N50

Returned status description

note: successful completion - home database not found. The database `ne4j` does not exist. Verify that the spelling is correct or create the database for the command to take effect.

Suggestions for improvement

Verify that the home database name is not misspelled.

Query

```
CREATE USER john SET PASSWORD "secret" SET HOME DATABASE nej4
```

Description of the returned code

The home database provided does not currently exist in the DBMS. This command will not take effect until this database is created. (HOME DATABASE: `nej4`)

Suggestions for improvement

Verify that the home database name is not misspelled.

Unknown label

Table 18. Notification details

Neo4j code	Neo.ClientNotification.Statement.UnknownLabelWarning

Title	The provided label is not in the database.
Description	One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (`%s`)
Category	UNRECOGNIZED
GQLSTATUS code	01N50
Status description	warn: unknown label. The label `{ [label] }` does not exist in database `{ [db] }`. Verify that the spelling is correct.
Classification	UNRECOGNIZED
SeverityLevel	WARNING

Example 12. Matching on a node with a label that does not exist in the database

Query

```
MATCH (n:Perso) RETURN n
```

Returned GQLSTATUS code

01N50

Returned status description

warn: unknown label. The label `Perso` does not exist in database `neo4j`. Verify that the spelling is correct.

Suggestions for improvement

Verify that the label is not misspelled. If you plan to create nodes with that label in the future, no change is needed.

Query

```
MATCH (n:Perso) RETURN n
```

Description of the returned code

One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: `Perso`)

Suggestions for improvement

Verify that the label is not misspelled. If you plan to create nodes with that label in the future, then no change is needed.

Unknown relationship type

Table 19. Notification details

Neo4j code	`Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning`
Title	The provided relationship type is not in the database.
Description	One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (`%s`)
Category	`UNRECOGNIZED`
GQLSTATUS code	`01N51`
Status description	warn: unknown relationship type. The relationship type `{ [relType] }` does not exist in database `{ [db] }`. Verify that the spelling is correct.
Classification	`UNRECOGNIZED`
SeverityLevel	`WARNING`

Example 13. Matching a relationship with a type that does not exist

Query

```
MATCH (n)-[:NonExistingType]->() RETURN n
```

Returned GQLSTATUS code

01N51

Returned status description

warn: unknown relationship type. The relationship type `NonExistingType` does not exist in database `neo4j`. Verify that the spelling is correct.

Suggestions for improvement

Verify that the relationship type is not misspelled. If you plan to create relationships of this type in the future, no change is needed.

Query

```
MATCH (n)-[:NonExistingType]->() RETURN n
```

Description of the returned code

One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: `NonExistingType`)

Suggestions for improvement

Verify that the relationship type is not misspelled. If you plan to create relationships of this type in the future, no change is needed.

Unknown property key

Table 20. Notification details

Neo4j code	`Neo.ClientNotification.Statement.UnknownPropertyKeyWarning`
Title	The provided property key is not in the database
Description	One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (`%s`)
Category	`UNRECOGNIZED`
GQLSTATUS code	`01N52`
Status description	warn: unknown property key. The property `{ [propKey] }` does not exist in database `{ [db] }`. Verify that the spelling is correct.
Classification	`UNRECOGNIZED`
SeverityLevel	`WARNING`

Example 14. Matching a property key that does not exist

Query

```
MATCH (n:Person {nme:"Tom"})
RETURN n
```

Returned GQLSTATUS code

01N52

Returned status description

warn: unknown property key. The property `nme` does not exist in database `neo4j`. Verify that the spelling is correct.

Suggestions for improvement

Verify that the property key is not misspelled. If you plan to create that property key in the future, no change is needed.

Query

```
MATCH (n:Person {nme:"Tom"})
RETURN n
```

Description of the returned code

One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: `nme`)

Suggestions for improvement

Verify that the property key is not misspelled. If you plan to create that property key in the future, no change is needed.

Aggregation skipped null

Table 21. Notification details

Neo4j code	`Neo.ClientNotification.Statement.AggregationSkippedNull`
Title	The query contains an aggregation function that skips null values.
Description	The query contains an aggregation function that skips null values.
Category	`UNRECOGNIZED`
GQLSTATUS code	`01G11`
Status description	warn: null value eliminated in set function.
Classification	`UNRECOGNIZED`
SeverityLevel	`WARNING`

Example 15. Aggregation skipping a NULL value

Query

```
UNWIND [1, NULL, 2] AS i RETURN count(i) AS sum
```

Returned GQLSTATUS code

01G11

Returned status description

warn: null value eliminated in set function.

Query

```
UNWIND [1, NULL, 2] AS i RETURN count(i) AS sum
```

Description of the returned code

The query contains an aggregation function that skips null values.

`UNSUPPORTED` notifications

Unsupported notifications contain information about features not supported in the current context used to access them.

Unsupported value type

Table 22. Notification details

Neo4j code	`Neo.ClientNotification.Request.UnsupportedType`
Title	Client does not support this type.
Description	One or more values returned could not be handled by this version of the client and were replaced with placeholder map values. Please upgrade your client.
Category	`UNSUPPORTED`
GQLSTATUS code	`01N83`
Status description	warn: Client does not support this type. Client does not support type `{ [valueType] }`. Please upgrade your client.
Classification	`UNSUPPORTED`
SeverityLevel	`WARNING`

Example 16. Accessing a vector with an unsupported driver (e.g., 5.x driver)

Query

```
RETURN VECTOR([1, 2, 3], 3, INTEGER32)
```

Returned GQLSTATUS code

01N83

Returned status description

warn: Client does not support this type. Client does not support type `VECTOR`. Please upgrade your client.

Suggestions for improvement

Upgrade your client to a version that supports this type.

Query

```
RETURN VECTOR([1, 2, 3], 3, INTEGER32)
```

Description of the returned code

One or more values returned could not be handled by this version of the client and were replaced with placeholder map values. Please upgrade your client.

Suggestions for improvement

Upgrade your client to a version that supports this type.

`DEPRECATION` notifications

Deprecation notifications contain information about a feature or functionality that has been deprecated. It is important to change to the new functionality, otherwise, the query might break in a future version.

Feature deprecated

Table 23. Notification details

Neo4j code	`Neo.ClientNotification.Statement.FeatureDeprecationWarning`
Title	This feature is deprecated and will be removed in future versions.

Descriptions	• The procedure has a deprecated field. (`%s`) • The function has a deprecated field. (`%s`) • Creating an entity (`%s`) and referencing that entity in a property definition in the same CREATE is deprecated. • Merging an entity (`%s`) and referencing that entity in a property definition in the same MERGE is deprecated. • The Unicode character `%s` is deprecated for unescaped identifiers and will be considered as a whitespace character in the future. To continue using it, escape the identifier by adding backticks around the identifier `%s`. • The character with the Unicode representation `%s` is deprecated for unescaped identifiers and will not be supported in the future. To continue using it, escape the identifier by adding backticks around the identifier `%s`. • `%s` subquery without a variable scope clause is deprecated. Use CALL (`%s`) { ... } • All subqueries in a `UNION [ALL]` should have the same ordering for the return columns.  In versions 5.5 to 5.25, using differently ordered return items in a `UNION [ALL]` clause is deprecated. However, starting from version 5.26, the deprecation has been withdrawn following cost-benefit analysis and valuable user feedback. • Deprecated from 5.26 to 2025.05 Databases and aliases with unescaped `.` are deprecated unless the dots indicate that they belong to a composite database. Names containing `.` should be escaped. (`%s`)  Databases and aliases with unescaped `.` are deprecated in versions 5.26 to 2025.05. However, starting from version 2025.06, this deprecation has been withdrawn, and replaced by two new deprecations. The reason for this change is that the previous quoting rules have been found inconsistent and ambiguous, and will be improved starting from Cypher 25. For more information, see: ◦ Using separately backticked name parts in graph references ◦ Using separately backticked name parts in graph references in USE clause ◦ Cypher manual → Graph references. • Deprecated in 2025.06 Graph references with separately backticked name parts (`%s`) are deprecated. In future Cypher versions, use parameters or backtick the entire name (`%s`). • Deprecated in 2025.06 Graph references with separately backticked name parts (`%s`) are deprecated. In future Cypher versions, remove the backticks (`%s`). • Deprecated in 2025.11 The namespace of the invoked user-defined function is deprecated. (`%s`). • Deprecated in 2025.11 The namespace of the invoked user-defined procedure is deprecated. (`%s`). • Deprecated in 2025.11 The namespace of the invoked user-defined function is deprecated and the function is shadowing an internal function. (`%s`).
Category	`DEPRECATION`

GQLSTATUS code	01N00
Status description	warn: feature deprecated. { [item] }
Classification	DEPRECATION
SeverityLevel	WARNING

Example 17. Using separately backticked name parts in graph references

Query

```
CYPHER 5 CREATE ALIAS `foo`.`bar` FOR DATABASE ...
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. Graph references with separately backticked name parts (`foo`.`bar`) are deprecated. In future Cypher versions, use parameters or backtick the entire name (`foo.bar`).

Suggestions for improvement

In newer Cypher versions, either the whole graph reference name should be backticked or it should be sent in as a parameter.

```
CYPHER 25 CREATE ALIAS `foo.bar` FOR DATABASE ...
```

```
CYPHER 25 CREATE ALIAS $param FOR DATABASE ...
```

For more information on the semantics of graph references, see Cypher manual → Graph references.

Query

```
CYPHER 5 CREATE ALIAS `foo`.`bar` FOR DATABASE ...
```

Description of the returned code

Graph references with separately backticked name parts (`foo`.`bar`) are deprecated. In future Cypher versions, use parameters or backtick the entire name (`foo.bar`).

Suggestions for improvement

In newer Cypher versions, either the whole graph reference name should be backticked or it should be sent in as a parameter.

```
CYPHER 25 CREATE ALIAS `foo.bar` FOR DATABASE ...
```

```
CYPHER 25 CREATE ALIAS $param FOR DATABASE ...
```

For more information on the semantics of graph references, see Cypher manual → Graph references.

Example 18. Using separately backticked name parts in graph references in USE clause

Query

```
CYPHER 5 USE graph.byName("`a.b`.c") ...
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. Graph references with separately backticked name parts (`a.b`.`c`) are deprecated. In future Cypher versions, remove the backticks (a.b.c).

Suggestions for improvement

In newer Cypher versions, the backticks should be removed.

```
CYPHER 25 USE graph.byName("a.b.c") ...
```

For more information on the semantics of graph references, see Cypher manual → Graph references.

Query

```
CYPHER 5 USE graph.byName("`a.b`.c") ...
```

Description of the returned code

Graph references with separately backticked name parts (`a.b`.`c`) are deprecated. In future Cypher versions, remove the backticks (a.b.c).

Suggestions for improvement

In newer Cypher versions, the backticks should be removed.

```
CYPHER 25 USE graph.byName("a.b.c") ...
```

For more information on the semantics of graph references, see Cypher manual → Graph references.

Example 19. Create a database with an unescaped name containing a dot

Query

```
CREATE DATABASE foo.bar
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. Databases and aliases with unescaped `.` are deprecated unless to indicate that they belong to a composite database. Names containing `.` should be escaped. (Name: foo.bar)

Suggestions for improvement

If not intended for a composite database, escape the name with the character `` ` ``.

```
CREATE DATABASE `foo.bar`
```

Query

```
CREATE DATABASE foo.bar
```

Description of the returned code

Databases and aliases with unescaped `.` are deprecated unless to indicate that they belong to a composite database. Names containing `.` should be escaped. (Name: `foo.bar`)

Suggestions for improvement

If not intended for a composite database, escape the name with the character `` ` ``.

```
CREATE DATABASE `foo.bar`
```

Note:

Databases and aliases with unescaped `.` are deprecated in versions 5.26 to 2025.05. However, starting from version 2025.06, this deprecation has been withdrawn, and replaced by two new deprecations. The reason for this change is that the previous quoting rules have been found inconsistent and ambiguous, and will be improved starting from Cypher 25. For more information, see:

- Using separately backticked name parts in graph references
- Using separately backticked name parts in graph references in USE clause
- Cypher manual → Graph references.

Example 20. Using differently ordered return items in a `UNION` clause

Query

```
RETURN 'val' as one, 'val' as two
UNION
RETURN 'val' as two, 'val' as one
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. All subqueries in a UNION [ALL] should have the same ordering for the return columns. Using differently ordered return items in a UNION [ALL] clause is deprecated and will be removed in a future version.

Suggestions for improvement

Use the same order for the return columns in all subqueries combined by a `UNION` clause.

```
RETURN 'val' as one, 'val' as two
UNION
RETURN 'val' as one, 'val' as two
```

Query

```
RETURN 'val' as one, 'val' as two
UNION
RETURN 'val' as two, 'val' as one
```

Description of the returned code

All subqueries in a UNION [ALL] should have the same ordering for the return columns. Using differently ordered return items in a UNION [ALL] clause is deprecated and will be removed in a future version.

Suggestions for improvement

Use the same order for the return columns in all subqueries combined by a `UNION` clause.

```
RETURN 'val' as one, 'val' as two
UNION
RETURN 'val' as one, 'val' as two
```



Note:

In versions 5.5 to 5.25, using differently ordered return items in a `UNION [ALL]` clause is deprecated. However, starting from version 5.26, the deprecation has been withdrawn following cost-benefit analysis and valuable user feedback.

Example 21. Using the Unicode \u0085 in an unescaped identifier

Query

```
RETURN 1 as my\u0085identifier
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. The Unicode character `\u0085` is deprecated for unescaped identifiers and will be considered as a whitespace character in the future. To continue using it, escape the identifier by adding backticks around the identifier `my\u0085identifier`.

Query

```
RETURN 1 as my\u0085identifier
```

Description of the returned code

The Unicode character `\u0085` is deprecated for unescaped identifiers and will be considered as a whitespace character in the future. To continue using it, escape the identifier by adding backticks around the identifier `my\u0085identifier`.

Example 22. Using a CALL subquery scope without a variable scope clause

Query

```
WITH 42 AS nbr
CALL {
  WITH nbr
  RETURN nbr + 3 AS otherNbr
}
RETURN otherNbr
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. `CALL` subquery without a variable scope clause is deprecated. Use `CALL (nbr) {…}`

Suggestions for improvement

Replace the importing `WITH` with a variable scope clause.

```
WITH 42 AS nbr
CALL (nbr) {
  RETURN nbr + 3 AS otherNbr
```

```
}
RETURN otherNbr
```

Query

```
WITH 42 AS nbr
CALL {
  WITH nbr
  RETURN nbr + 3 AS otherNbr
}
RETURN otherNbr
```

Description of the returned code

`CALL` subquery without a variable scope clause is deprecated. Use `CALL (nbr) {…}`

Suggestions for improvement

Replace the importing `WITH` with a variable scope clause.

```
WITH 42 AS nbr
CALL (nbr) {
  RETURN nbr + 3 AS otherNbr
}
RETURN otherNbr
```

Example 23. Deprecated function namespace

Query

```
RETURN point.function() AS result
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. The namespace used by the user-defined function `point.function` is deprecated.

Suggestions for improvement

Change the namespace to a non-deprecated namespace e.g. `my.point.function` or `pointing.function`.

Query

```
RETURN point.function() AS result
```

Description of the returned code

The namespace of the invoked user-defined function is deprecated. (point.function)

Example 24. Deprecated procedure namespace

Query

```
CALL point.procedure() AS result
RETURN result
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. The namespace used by the user-defined procedure `point.procedure` is deprecated.

Suggestions for improvement

Change the namespace to a non-deprecated namespace e.g. `my.point.procedure` or `pointing.procedure`.

Query

```
CALL point.procedure() AS result
RETURN result
```

Description of the returned code

The namespace of the called user-defined procedure is deprecated. (point.procedure)

Example 25. Shadowing built-in function

Query

```
RETURN coll.flatten() AS result
```

Returned GQLSTATUS code

01N00

Returned status description

warn: feature deprecated. The namespace of the invoked user-defined function `coll.flatten` is deprecated and the function is shadowing an internal function.

Suggestions for improvement

Change the namespace to a non-deprecated namespace e.g. `my.coll.flatten` or `collection.flatten`.

Query

```
RETURN coll.flatten() AS result
```

Description of the returned code

The namespace of the invoked user-defined function is deprecated and the function is shadowing an internal function. (coll.flatten)

Feature deprecated with a replacement

Table 24. Notification details

Neo4j code	`Neo.ClientNotification.Statement.FeatureDeprecationWarning`
Title	This feature is deprecated and will be removed in future versions.
Descriptions	• The semantics of using colon in the separation of alternative relationship types will change in a future version. (`%s`) • The use of nodes or relationships for setting properties is deprecated and will be removed in a future version. Please use properties() instead. • The use of shortestPath and allShortestPaths with fixed length relationships is deprecated and will be removed in a future version. Please use a path with a length of 1 [r*1..1] instead or a Match with a limit. • The query used a deprecated function. (`%s`) • The query used a deprecated procedure. (`%s`) • The query used a deprecated runtime option. (`%s`) • Deprecated in 2025.09 `text-1.0` and `text-2.0` providers for text indexes are deprecated and will be removed in a future version. Please use `text-3.0` instead. • Deprecated in 2026.07 The subclause expression `%s` contains an ambiguous reference to variable `%s` and is deprecated. It is replaced by referencing the projection item expression by an alias or using the `GROUP BY` clause. • Deprecated in 2026.07 The subclause expression `%s` contains a reference to the complex projection item expression `%s` and is deprecated. It is replaced by referencing the projection item expression by an alias.
Category	`DEPRECATION`
GQLSTATUS code	`01N01`
Status description	warn: feature deprecated with replacement. `{ [feat] 1 }` is deprecated. It is replaced by `{ [feat] 2 }`.
Classification	`DEPRECATION`
SeverityLevel	`WARNING`

Example 26. Colon after the vertical bar `|:` in a relationship pattern

Query

```
MATCH (a)-[:A|:B|:C]-() RETURN *
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `:A|:B|:C` is deprecated. It is replaced by `:A|B|C`.

Suggestions for improvement

Remove the colon inside the relationship type expression.

```
MATCH (a)-[:A|B|C]-() RETURN *
```

Query

```
MATCH (a)-[:A|:B|:C]-() RETURN *
```

Description of the returned code

The semantics of using colon in the separation of alternative relationship types will change in a future version. (Please use ':A|B|C' instead)

Suggestions for improvement

Remove the colon inside the relationship type expression.

```
MATCH (a)-[:A|B|C]-() RETURN *
```

Example 27. Setting properties using a node

Query

```
CYPHER 5
MATCH (a)-[]-(b)
SET a = b
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `SET a = b` is deprecated. It is replaced by `SET a = properties(b)`.

Suggestions for improvement

Use the `properties()` function to get all properties from `b`.

```
CYPHER 5
MATCH (a)-[]-(b)
SET a = properties(b)
```

Query

```
MATCH (a)-[]-(b)
SET a = b
```

Description of the returned code

The use of nodes or relationships for setting properties is deprecated and will be removed in a future version. Please use `properties()` instead.

Suggestions for improvement

Use the `properties()` function to get all properties from `b`.

```
MATCH (a)-[]-(b)
SET a = properties(b)
```

Example 28. Setting properties using a relationship

Query

```
CYPHER 5
MATCH (a)-[r]-(b)
SET a += r
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `SET a += r` is deprecated. It is replaced by `SET a += properties(r)`.

Suggestions for improvement

Use the `properties()` function to get all properties from `r`.

```
CYPHER 5
MATCH (a)-[r]-(b)
SET a += properties(r)
```

Query

```
MATCH (a)-[r]-(b)
SET a += r
```

Description of the returned code

The use of nodes or relationships for setting properties is deprecated and will be removed in a future version. Please use `properties()` instead.

Suggestions for improvement

Use the `properties()` function to get all properties from `r`.

```
MATCH (a)-[r]-(b)
SET a += properties(r)
```

Example 29. Shortest path with a fixed relationship length

Query

```
MATCH (a:Start), shortestPath((a)-[r]->()) RETURN a
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `shortestPath((a)-[r]→())` is deprecated. It is replaced by `shortestPath((n)-[r*1..1]→(m))`.

Suggestions for improvement

If the relationship length is fixed, there is no reason to search for the shortest path. Instead, you can rewrite it to the following:

```
MATCH (a: Start)-[r]->(b: End) RETURN b LIMIT 1
```

Query

```
MATCH (a:Start), shortestPath((a)-[r]->()) RETURN a
```

Description of the returned code

The use of `shortestPath` and `allShortestPaths` with fixed length relationships is deprecated and will be removed in a future version. Please use a path with a length of `1 [r*1..1]` instead or a `Match` with a `limit`.

Suggestions for improvement

If the relationship length is fixed, there is no reason to search for the shortest path. Instead, you can rewrite it to the following:

```
MATCH (a: Start)-[r]->(b: End) RETURN b LIMIT 1
```

Example 30. Using a deprecated runtime option

Query

```
CYPHER runtime = interpreted MATCH (n) RETURN n
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `runtime=interpreted` is deprecated. It is replaced by `runtime=slotted`.

Suggestions for improvement

Runtime `interpreted` is deprecated and another runtime is used instead. Alternatively, you can remove the runtime option to use the default runtime.

```
MATCH (n) RETURN n
```

Query

```
CYPHER runtime = interpreted MATCH (n) RETURN n
```

Description of the returned code

The query used a deprecated runtime option. (`'runtime=interpreted'` is deprecated, please use `'runtime=slotted'` instead)

Suggestions for improvement

Runtime `interpreted` is deprecated and another runtime is used instead. Alternatively, you can remove the runtime option to use the default runtime.

```
MATCH (n) RETURN n
```

Example 31. Using the `text-1.0` index provider when creating a text index

Query

```
CREATE TEXT INDEX FOR (n:Label) ON (n.prop) OPTIONS {indexProvider : 'text-1.0'}
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `text-1.0` is deprecated. It is replaced by `text-2.0`.

Suggestions for improvement

Update the option `indexProvider` with the value `text-2.0`.

```
CREATE TEXT INDEX FOR (n:Label) ON (n.prop) OPTIONS {indexProvider : 'text-2.0'}
```

Query

```
CREATE TEXT INDEX FOR (n:Label) ON (n.prop) OPTIONS {indexProvider : 'text-1.0'}
```

Description of the returned code

The `text-1.0` provider for text indexes is deprecated and will be removed in a future version. Please use `text-2.0` instead.

Suggestions for improvement

Update the option `indexProvider` with the value `text-2.0`.

```
CREATE TEXT INDEX FOR (n:Label) ON (n.prop) OPTIONS {indexProvider : 'text-2.0'}
```

Example 32. Using a renamed or a deprecated procedure

```
CALL cdc.query()
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `cdc.query` is deprecated. It is replaced by `db.cdc.query`.

Suggestions for improvement

Use the procedure `db.cdc.query` instead.

```
CALL db.cdc.query()
```

Query

```
CALL cdc.query()
```

Description of the returned code

The query used a deprecated procedure: `cdc.query`.

Suggestions for improvement

Use the procedure `db.cdc.query` instead.

```
CALL db.cdc.query()
```

Example 33. Using id() function

Query

```
MATCH (a)
RETURN id(a)
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. id is deprecated. It is replaced by elementId or consider using an application-generated id.

Suggestions for improvement

Use the function `elementId()` instead.

```
MATCH (a)
RETURN elementId(a)
```

Query

```
MATCH (a)
RETURN id(a)
```

Description of the returned code

The query used a deprecated function: `id`.

Suggestions for improvement

Use the function `elementId()` instead.

```
MATCH (a)
RETURN elementId(a)
```

Example 34. Using an unescaped variable named 'where' in a node pattern

Query

```
MATCH (where {p: 5})
RETURN where
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `(where {p: 5})` is deprecated. It is replaced by `(`where` {p: 5})`.

Suggestions for improvement

To continue using variables with this name, use backticks to escape the variable name:

```
MATCH (`where` {p: 5})
RETURN `where`.p
```

Query

```
MATCH (where {p: 5})
RETURN where
```

Description of the returned code

'(where {p: 5})' is deprecated. It is replaced by '(`where` {p: 5})'.

Suggestions for improvement

To continue using variables with this name, use backticks to escape the variable name:

```
MATCH (`where` {p: 5})
RETURN `where`.p
```

Example 35. Using an unescaped variable named 'where' in a relationship pattern

Query

```
MATCH ()-[where {p: 5}]->()
RETURN where
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `-[where {p: 5}]-` is deprecated. It is replaced by `-[`where` {p: 5}]-`.

Suggestions for improvement

To continue using variables with this name, use backticks to escape the variable name:

```
MATCH ()-[`where` {p: 5}]->()
RETURN `where`.p
```

Query

```
MATCH ()-[where {p: 5}]->()
RETURN where
```

Description of the returned code

'-[where {p: 5}]-' is deprecated. It is replaced by '-[`where` {p: 5}]-'.

Suggestions for improvement

To continue using variables with this name, use backticks to escape the variable name:

```
MATCH ()-[`where` {p: 5}]->()
RETURN `where`.p
```

Example 36. Using an unparenthesized label expression predicate as the right-hand side operand of `+`

Query

```
MATCH (n)-[r]->(m)
WITH m, n.truthCodes AS listOfBooleans
RETURN listOfBooleans + m:A
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `… + m:A` is deprecated. It is replaced by `… + (m:A)`.

Suggestions for improvement

Parenthesize the label expression predicate on the right-hand side of `+`:

```
MATCH (n)-[r]->(m)
WITH m, n.truthCodes AS listOfBooleans
RETURN listOfBooleans + (m:A)
```

Query

```
MATCH (n)-[r]->(m)
WITH m, n.truthCodes AS listOfBooleans
RETURN listOfBooleans + m:A
```

Description of the returned code

'... + m:A' is deprecated. It is replaced by '... + (m:A)'.

Suggestions for improvement

Parenthesize the label expression predicate on the right-hand side of `+`:

```
MATCH (n)-[r]->(m)
WITH m, n.truthCodes AS listOfBooleans
RETURN listOfBooleans + (m:A)
```

Example 37. Using an unparenthesized label expression predicate as the right-hand side operand of `+`

Query

```
MATCH (n)-[r]->(m)
WITH r, n.truthCodes AS listOfBooleans
RETURN listOfBooleans + r:C|D
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `… + r:C|D` is deprecated. It is replaced by `… + (r:C|D)`.

Suggestions for improvement

Parenthesize the label expression predicate on the right-hand side of `+`:

```
MATCH (n)-[r]->(m)
WITH r, n.truthCodes AS listOfBooleans
```

```
RETURN listOfBooleans + (r:C|D)
```

Query

```
MATCH (n)-[r]->(m)
WITH r, n.truthCodes AS listOfBooleans
RETURN listOfBooleans + r:C|D
```

Description of the returned code

'... + r:C|D' is deprecated. It is replaced by '... + (r:C|D)'.

Suggestions for improvement

Parenthesize the label expression predicate on the right-hand side of `+`:

```
MATCH (n)-[r]->(m)
WITH r, n.truthCodes AS listOfBooleans
RETURN listOfBooleans + (r:C|D)
```

Example 38. Using an unescaped variable named `is` as a `WHEN` operand in a simple `CASE` expression

Query

```
MATCH (n)
WITH n, n.internationalStandard AS is
RETURN CASE n
  WHEN is :: INTEGER THEN "ISO/IEC" + is
  ELSE is
END AS standardsName
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `WHEN is :: INTEGER` is deprecated. It is replaced by `WHEN \`is\` :: INTEGER`.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH (n)
WITH n, n.internationalStandard AS `is`
RETURN CASE n
  WHEN `is` :: INTEGER THEN "ISO/IEC" + `is`
  ELSE `is`
END AS standardsName
```

Query

```
MATCH (n)
WITH n, n.internationalStandard AS is
RETURN CASE n
  WHEN is :: INTEGER THEN "ISO/IEC" + is
  ELSE is
END AS standardsName
```

Description of the returned code

'WHEN is

INTEGER' is deprecated. It is replaced by 'WHEN `is` :: INTEGER'.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH (n)
WITH n, n.internationalStandard AS `is`
RETURN CASE n
  WHEN `is` :: INTEGER THEN "ISO/IEC" + `is`
  ELSE `is`
END AS standardsName
```

Example 39. Using an unescaped variable named `contains` in addition operations within a `WHEN` operand in a simple `CASE` expression

Query

```
MATCH p = (:A)-[:HAS]->(:B)
WITH p, size(relationships(p)) AS contains
RETURN CASE size(nodes(p))
  WHEN contains + 1 THEN "okay"
  ELSE "not okay"
END AS check
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `WHEN contains + 1 INTEGER` is deprecated. It is replaced by `WHEN `contains` + 1 INTEGER`.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH p = (:A)-[:HAS]->(:B)
WITH p, size(relationships(p)) AS `contains`
RETURN CASE size(nodes(p))
  WHEN `contains` + 1 THEN "okay"
  ELSE "not okay"
```

```
END AS check
```

Query

```
MATCH p = (:A)-[:HAS]->(:B)
WITH p, size(relationships(p)) AS contains
RETURN CASE size(nodes(p))
  WHEN contains + 1 THEN "okay"
  ELSE "not okay"
END AS check
```

Description of the returned code

'WHEN contains + 1 INTEGER' is deprecated. It is replaced by 'WHEN `contains` + 1 INTEGER'.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH p = (:A)-[:HAS]->(:B)
WITH p, size(relationships(p)) AS `contains`
RETURN CASE size(nodes(p))
  WHEN `contains` + 1 THEN "okay"
  ELSE "not okay"
END AS check
```

Example 40. Using an unescaped variable named `contains` in subtraction operations within a `WHEN` operand in a simple `CASE` expression

Query

```
MATCH p = (:A)-[:HAS]->(:B)
WITH p, size(nodes(p)) AS contains
RETURN CASE size(relationships(p))
  WHEN contains - 1 THEN "okay"
  ELSE "not okay"
END AS check
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `WHEN contains - 1 INTEGER` is deprecated. It is replaced by `WHEN `contains` - 1 INTEGER`.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH p = (:A)-[:HAS]->(:B)
```

```
WITH p, size(nodes(p)) AS `contains`
RETURN CASE size(relationships(p))
  WHEN `contains` - 1 THEN "okay"
  ELSE "not okay"
END AS check
```

Query

```
MATCH p = (:A)-[:HAS]->(:B)
WITH p, size(nodes(p)) AS contains
RETURN CASE size(relationships(p))
  WHEN contains - 1 THEN "okay"
  ELSE "not okay"
END AS check
```

Description of the returned code

'WHEN contains - 1 INTEGER' is deprecated. It is replaced by 'WHEN `contains` - 1 INTEGER'.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH p = (:A)-[:HAS]->(:B)
WITH p, size(nodes(p)) AS `contains`
RETURN CASE size(relationships(p))
  WHEN `contains` - 1 THEN "okay"
  ELSE "not okay"
END AS check
```

Example 41. Using the `[]` operator on an unescaped variable named `in` within a `WHEN` operand in a simple `CASE` expression

Query

```
MATCH (c:Client)-[:MAKES]->(t:Transaction)
WITH t, c.ibanNumbers AS in
RETURN CASE t.ibanNumber
  WHEN in[0] THEN "used main account"
  ELSE "used different account"
END AS check
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `WHEN in[0] INTEGER` is deprecated. It is replaced by `WHEN `in`[0] INTEGER`.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to

escape the variable name:

```
MATCH (c:Client)-[:MAKES]->(t:Transaction)
WITH t, c.ibanNumbers AS `in`
RETURN CASE t.ibanNumber
  WHEN `in`[0] THEN "used main account"
  ELSE "used different account"
END AS check
```

Query

```
MATCH (c:Client)-[:MAKES]->(t:Transaction)
WITH t, c.ibanNumbers AS in
RETURN CASE t.ibanNumber
  WHEN in[0] THEN "used main account"
  ELSE "used different account"
END AS check
```

Description of the returned code

'WHEN in[0] INTEGER' is deprecated. It is replaced by 'WHEN `in`[0] INTEGER'.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH (c:Client)-[:MAKES]->(t:Transaction)
WITH t, c.ibanNumbers AS `in`
RETURN CASE t.ibanNumber
  WHEN `in`[0] THEN "used main account"
  ELSE "used different account"
END AS check
```

Example 42. Using the `[]` operator on an unescaped variable named `in` within a `WHEN` operand in a simple `CASE` expression

Query

```
MATCH (in:Client)-[:MAKES]->(t:Transaction)
RETURN CASE t.ibanNumber
  WHEN in["mainAccount"] THEN "used main account"
  ELSE "used different account"
END AS check
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. `WHEN in["mainAccount"] INTEGER` is deprecated. It is replaced by `WHEN `in`["mainAccount"] INTEGER`.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH (in:Client)-[:MAKES]->(t:Transaction)
RETURN CASE t.ibanNumber
  WHEN `in`["mainAccount"] THEN "used main account"
  ELSE "used different account"
END AS check
```

Query

```
MATCH (in:Client)-[:MAKES]->(t:Transaction)
RETURN CASE t.ibanNumber
  WHEN in["mainAccount"] THEN "used main account"
  ELSE "used different account"
END AS check
```

Description of the returned code

'WHEN in["mainAccount"] INTEGER' is deprecated. It is replaced by 'WHEN `in`["mainAccount"] INTEGER'.

Suggestions for improvement

To continue using variables with this name in simple `CASE` expressions, use backticks to escape the variable name:

```
MATCH (in:Client)-[:MAKES]->(t:Transaction)
RETURN CASE t.ibanNumber
  WHEN `in`["mainAccount"] THEN "used main account"
  ELSE "used different account"
END AS check
```

Example 43. Ambiguous reference to a variable in a subclause expression

Query

```
MATCH (a), (b)
RETURN a.prop AS b, b AS a
  ORDER BY a.prop
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. The subclause expression `a.prop` contains an ambiguous reference to variable `a` and is deprecated. It is replaced by referencing the projection item expression by an alias or using the `GROUP BY` clause.

Suggestions for improvement

Reference the projection item by its alias rather than repeating the expression.

```
MATCH (a), (b)
RETURN a.prop AS b, b AS a
  ORDER BY b
```

Query

```
MATCH (a), (b)
RETURN a.prop AS b, b AS a
  ORDER BY a.prop
```

Description of the returned code

The subclause expression `a.prop` contains an ambiguous reference to variable `a` and is deprecated. It is replaced by referencing the projection item expression by an alias or using the `GROUP BY` clause.

Suggestions for improvement

Reference the projection item by its alias rather than repeating the expression.

```
MATCH (a), (b)
RETURN a.prop AS b, b AS a
  ORDER BY b
```

Example 44. Reference to a complex projection item expression in a subclause expression

Query

```
MATCH (a)
RETURN a.prop * 2 AS x, count(*) AS count
  ORDER BY a.prop * 2 + 1
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. The subclause expression `a.prop * 2 + 1` contains a reference to the complex projection item expression `a.prop * 2` and is deprecated. It is replaced by referencing the projection item expression by an alias.

Suggestions for improvement

Reference the projection item by its alias rather than repeating the expression.

```
MATCH (a)
RETURN a.prop * 2 AS x, count(*) AS count
  ORDER BY x + 1
```

Query

```
MATCH (a)
RETURN a.prop * 2 AS x, count(*) AS count
  ORDER BY a.prop * 2 + 1
```

Description of the returned code

The subclause expression `a.prop * 2 + 1` contains a reference to the complex projection item expression `a.prop * 2` and is deprecated. It is replaced by referencing the projection item expression by an alias.

Suggestions for improvement

Reference the projection item by its alias rather than repeating the expression.

```
MATCH (a)
RETURN a.prop * 2 AS x, count(*) AS count
  ORDER BY x + 1
```

Example 45. Ambiguous and complex reference in a subclause expression

Query

```
MATCH (a), (b)
RETURN a.prop * 2 AS x, b AS a, count(*) AS count
  ORDER BY a.prop * 2 + 1
```

Returned GQLSTATUS code

01N01

Returned status description

warn: feature deprecated with replacement. The subclause expression `a.prop * 2 + 1` contains an ambiguous reference to variable `a` and a reference to the complex projection item expression `a.prop * 2` and is deprecated. It is replaced by referencing the projection item expression by an alias or using the `GROUP BY` clause.

Suggestions for improvement

Reference the projection item by its alias rather than repeating the expression.

```
MATCH (a), (b)
RETURN a.prop * 2 AS x, b AS a, count(*) AS count
  ORDER BY x + 1
```

Query

```
MATCH (a), (b)
RETURN a.prop * 2 AS x, b AS a, count(*) AS count
  ORDER BY a.prop * 2 + 1
```

Description of the returned code

The subclause expression `a.prop * 2 + 1` contains an ambiguous reference to variable `a` and a reference to the complex projection item expression `a.prop * 2` and is deprecated. It is replaced by referencing the projection item expression by an alias or using the `GROUP BY` clause.

Suggestions for improvement

Reference the projection item by its alias rather than repeating the expression.

```
MATCH (a), (b)
RETURN a.prop * 2 AS x, b AS a, count(*) AS count
  ORDER BY x + 1
```

Deprecated features without a future replacement

Table 25. Notification details

Neo4j code	`Neo.ClientNotification.Statement.FeatureDeprecationWarning`
Title	This feature is deprecated and will be removed in future versions.
Descriptions	• The Cypher query option `connectComponentsPlanner` is deprecated and will be removed without a replacement. The product's default behavior of using a cost-based IDP search algorithm when combining sub-plans will be kept. For more information, see Cypher Manual → Cypher planner. • The query used a deprecated function%s • The query used a deprecated procedure%s
Category	`DEPRECATION`
GQLSTATUS code	`01N02`
Status description	warn: feature deprecated without replacement. `{ [feat] }` is deprecated and will be removed without a replacement.
Classification	`DEPRECATION`
SeverityLevel	`WARNING`

Example 46. Using Cypher query option `connectComponentsPlanner`

Query

```
CYPHER connectComponentsPlanner=greedy MATCH (a), (b) RETURN *
```

Returned GQLSTATUS code

01N02

Returned status description

warn: feature deprecated without replacement. `connectComponentsPlanner` is deprecated and will be removed without a replacement.

Query

```
CYPHER connectComponentsPlanner=greedy MATCH (a), (b) RETURN *
```

Description of the returned code

The Cypher query option `connectComponentsPlanner` is deprecated and will be removed without a replacement. The product's default behavior of using a cost-based IDP search algorithm when combining sub-plans will be kept. For more information, see Cypher manual → Cypher planner.

Example 47. Using a deprecated procedure

Query

```
CALL unsupported.dbms.shutdown()
```

Returned GQLSTATUS code

01N02

Returned status description

warn: feature deprecated without replacement. `unsupported.dbms.shutdown` is deprecated and will be removed without a replacement.

Suggestions for improvement

Remove the use of the deprecated procedure. If there is a suggested replacement, update to use the replacement instead.

Query

```
CALL unsupported.dbms.shutdown()
```

Description of the returned code

The query used a deprecated procedure: `'unsupported.dbms.shutdown'`.

Suggestions for improvement

Remove the use of the deprecated procedure. If there is a suggested replacement, update to use the replacement instead.

Procedure field deprecated

Table 26. Notification details

Neo4j code	`Neo.ClientNotification.Statement.FeatureDeprecationWarning`
Title	This feature is deprecated and will be removed in future versions.
Description	The query used a deprecated field from a procedure. (`%s`)
Category	`DEPRECATION`
GQLSTATUS code	`01N03`
Status description	warn: procedure field deprecated. The field `{ [procField] }` of procedure `{ [proc] }` is deprecated.
Classification	`DEPRECATION`
SeverityLevel	`WARNING`

Feature deprecated with replacement - API Request

Table 27. Notification details

Neo4j code	`Neo.ClientNotification.Request.FeatureDeprecationWarning`
Title	This feature is deprecated and will be removed in future versions.
Description	`%s` is deprecated. It is replaced by `%s`.
Category	`DEPRECATION`
GQLSTATUS code	`01N01`
Status description	warn: feature deprecated with replacement. `{ [feat] 1 }` is deprecated. It is replaced by `{ [feat] 2 }`.
Classification	`DEPRECATION`
SeverityLevel	`WARNING`

Feature deprecated with replacement - DeprecatedFormat

Table 28. Notification details

Neo4j code	`Neo.ClientNotification.Request.DeprecatedFormat`
Title	The client made a request for a format which has been deprecated.
Description	The requested format has been deprecated. (`%s`)
Category	`DEPRECATION`
GQLSTATUS code	`01N01`

Status description	warn: feature deprecated with replacement. `{ [feat] 1 }` is deprecated. It is replaced by `{ [feat] 2 }`.
Classification	DEPRECATION
SeverityLevel	WARNING

`SECURITY` notifications

Security notifications indicate that the result of the query or command might have a potential security issue. Verify that this is the intended behavior of your query or command.

Role or privilege not assigned

Table 29. Notification details

Neo4j code	Neo.ClientNotification.Security.CommandHasNoEffect
Title	`<command>` has no effect.*
Descriptions	• The user does not have the role. See Status Codes documentation for more information. • The auth rule does not have the role. See Status Codes documentation for more information. • The role does not have the privilege. See Status Codes documentation for more information.
Category	SECURITY
GQLSTATUS code	00N71
Status description	note: successful completion - role or privilege not assigned. The command `{ [cmd] }` has no effect. The role or privilege is not assigned.
Classification	SECURITY
SeverityLevel	INFORMATION

* `<command>` *and* `cmd` *could be either the full command given by the user or a subset of the given command.*

Example 48. Revoking a role from a user who does not have that role

Command

```
REVOKE ROLE admin, reader FROM jane
```

Returned GQLSTATUS code

00N71

Returned status description

note: successful completion - role or privilege not assigned. `REVOKE ROLE reader FROM jane` has no effect. The role or privilege is not assigned.

Suggestions for improvement

Verify that this is the intended role and user.

Command

```
REVOKE ROLE admin, reader FROM jane
```

Title of the returned code

`REVOKE ROLE reader FROM jane` has no effect.

Description of the returned code

The user does not have the role. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended role and user.

Example 49. Revoking a role from an auth rule that does not have that role

Command

```
REVOKE ROLE admin, reader FROM AUTH RULE authrule
```

Returned GQLSTATUS code

00N71

Returned status description

note: successful completion - role or privilege not assigned. `REVOKE ROLE reader FROM AUTH RULE authrule` has no effect. The role or privilege is not assigned.

Suggestions for improvement

Verify that this is the intended role and auth rule.

Command

```
REVOKE ROLE admin, reader FROM AUTH RULE authrule
```

Title of the returned code

`REVOKE ROLE reader FROM AUTH RULE authrule` has no effect.

Description of the returned code

The auth rule does not have the role. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended role and auth rule.

Example 50. Revoking a privilege from a role that does not have that privilege

Command

```
REVOKE WRITE ON GRAPH * FROM reader
```

Returned GQLSTATUS code

00N71

Returned status description

note: successful completion - role or privilege not assigned. `REVOKE DENY WRITE ON GRAPH * FROM reader` has no effect. The role or privilege is not assigned.

Suggestions for improvement

Verify that this is the intended privilege and role.

Command

```
REVOKE WRITE ON GRAPH * FROM reader
```

Title of the returned code

`REVOKE DENY WRITE ON GRAPH * FROM reader` has no effect.

Description of the returned code

The role does not have the privilege. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended privilege and role.

Role or privilege already assigned

Table 30. Notification details

Neo4j code	`Neo.ClientNotification.Security.CommandHasNoEffect`
Title	`<command>` has no effect.*
Descriptions	• The user already has the role. See Status Codes documentation for more information. • The auth rule already has the role. See Status Codes documentation for more information. • The role already has the privilege. See Status Codes documentation for more information.

Category	SECURITY
GQLSTATUS code	00N70
Status description	note: successful completion - role or privilege already assigned. The command `{ [cmd] }` has no effect. The role or privilege is already assigned.
Classification	SECURITY
SeverityLevel	INFORMATION

* `<command>` *and* `cmd` *could be either the full command given by the user or a subset of the given command.*

Example 51. Granting a role to a user who already has that role

Command

```
GRANT ROLE admin TO john
```

Returned GQLSTATUS code

00N70

Returned status description

note: successful completion - role or privilege already assigned. `GRANT ROLE admin TO john` has no effect. The role or privilege is already assigned.

Suggestions for improvement

Verify that this is the intended role and user.

Command

```
GRANT ROLE admin TO john
```

Title of the returned code

`GRANT ROLE admin TO john` has no effect.

Description of the returned code

The user already has the role. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended role and user.

Example 52. Granting a role to an auth rule that already has that role

Command

```
GRANT ROLE admin TO AUTH RULE authrule
```

Returned GQLSTATUS code

00N70

Returned status description

note: successful completion - role or privilege already assigned. `GRANT ROLE admin TO AUTH RULE authrule` has no effect. The role or privilege is already assigned.

Suggestions for improvement

Verify that this is the intended role and auth rule.

Command

```
GRANT ROLE admin TO AUTH RULE authrule
```

Title of the returned code

`GRANT ROLE admin TO AUTH RULE authrule` has no effect.

Description of the returned code

The auth rule already has the role. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended role and auth rule.

Example 53. Granting or denying a privilege to a role that already has that privilege

Command

```
GRANT TRAVERSE ON GRAPH * TO reader
```

Returned GQLSTATUS code

00N70

Returned status description

note: successful completion - role or privilege already assigned. `GRANT TRAVERSE ON GRAPH * TO reader` has no effect. The role or privilege is already assigned.

Suggestions for improvement

Verify that this is the intended privilege and role.

Command

```
GRANT TRAVERSE ON GRAPH * TO reader
```

Title of the returned code

`GRANT TRAVERSE ON GRAPH * NODE * TO reader` has no effect.

Description of the returned code

The role already has the privilege. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended privilege and role.

Impossible revoke command

Table 31. Notification details

Neo4j code	`Neo.ClientNotification.Security.ImpossibleRevokeCommand`
Title	`<command>` has no effect.*
Description	Role does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information.
Category	`SECURITY`
GQLSTATUS code	`01N70`
Status description	warn: inoperational revoke command. The command `{ [cmd] }` has no effect. Make sure nothing is misspelled. This notification will become an error in a future major version. Cause: `{ [msg] }`
Classification	`SECURITY`
SeverityLevel	`WARNING`

* `<command>` *and* `cmd` *could be either the full command given by the user or a subset of the given command.*

Example 54. Revoking a non-existing role from a user

Command

```
REVOKE ROLE manager, reader FROM jane
```

Returned GQLSTATUS code

01N70

Returned status description

warn: impossible revoke command. `REVOKE ROLE manager FROM jane` has no effect. Role does

not exist. Make sure nothing is misspelled. This notification will become an error in a future major version.

Suggestions for improvement

Verify that this is the intended role and that it is spelled correctly.

Command

```
REVOKE ROLE manager, reader FROM jane
```

Title of the returned code

`REVOKE ROLE manager FROM jane` has no effect.

Description of the returned code

Role does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended role and that it is spelled correctly.

Example 55. Revoking a role from a non-existing user

Command

```
REVOKE ROLE reader FROM alice
```

Returned GQLSTATUS code

01N70

Returned status description

warn: impossible revoke command. `REVOKE ROLE reader FROM alice` has no effect. User does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information. o

Suggestions for improvement

Verify that this is the intended user and that it is spelled correctly.

Command

```
REVOKE ROLE reader FROM alice
```

Title of the returned code

`REVOKE ROLE reader FROM alice` has no effect.

Description of the returned code

User does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version.

Suggestions for improvement

Verify that this is the intended user and that it is spelled correctly.

Example 56. Revoking a privilege from a non-existing role

Command

```
REVOKE GRANT WRITE ON GRAPH * FROM manager
```

Returned GQLSTATUS code

01N70

Returned status description

warn: impossible revoke command. `REVOKE GRANT WRITE ON GRAPH * FROM manager` has no effect. Role does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version.

Suggestions for improvement

Verify that this is the intended role and that it is spelled correctly.

Command

```
REVOKE GRANT WRITE ON GRAPH * FROM manager
```

Title of the returned code

`REVOKE GRANT WRITE ON GRAPH * FROM manager` has no effect.

Description of the returned code

Role does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended role and that it is spelled correctly.

Example 57. Revoking a privilege on a non-existing graph from a role

Command

```
REVOKE GRANT WRITE ON GRAPH neo3j FROM editor
```

Returned GQLSTATUS code

01N70

Returned status description

warn: impossible revoke command. `REVOKE GRANT WRITE ON GRAPH neo3j FROM editor` has no effect. Database `neo3j` does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version.

Suggestions for improvement

Verify that this is the intended graph and that it is spelled correctly.

Command

```
REVOKE GRANT WRITE ON GRAPH neo3j FROM editor
```

Title of the returned code

`REVOKE GRANT WRITE ON GRAPH neo3j FROM editor` has no effect.

Description of the returned code

Database `neo3j` does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended graph and that it is spelled correctly.

Example 58. Revoking a privilege on a non-existing database from a role

Command

```
REVOKE GRANT ACCESS ON DATABASE neo3j FROM editor
```

Returned GQLSTATUS code

01N70

Returned status description

warn: impossible revoke command. `REVOKE GRANT ACCESS ON DATABASE neo3j FROM editor` has no effect. Database `neo3j` does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version.

Suggestions for improvement

Verify that this is the intended database and that it is spelled correctly.

Command

```
REVOKE GRANT ACCESS ON DATABASE neo3j FROM editor
```

Title of the returned code

`REVOKE GRANT ACCESS ON DATABASE neo3j FROM editor` has no effect.

Description of the returned code

Database `neo3j` does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information.

Suggestions for improvement

Verify that this is the intended database and that it is spelled correctly.

Example 59. Revoking a privilege from a role with wildcard graph parameter

Parameter

```
{
    "graph": "*"
}
```

Command

```
REVOKE GRANT CREATE ON GRAPH $graph FROM PUBLIC
```

Returned GQLSTATUS code

01N70

Returned status description

warn: impossible revoke command. `REVOKE GRANT CREATE ON GRAPH $graph FROM PUBLIC` has no effect. Database `*` does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version.

Suggestions for improvement

Use `GRAPH *` without the parameter to revoke the privilege on all graphs.

Parameter

```
{
    "graph": "*"
}
```

Command

```
REVOKE GRANT CREATE ON GRAPH $graph FROM PUBLIC
```

Title of the returned code

`REVOKE GRANT CREATE ON GRAPH $graph FROM PUBLIC` has no effect.

Description of the returned code

Parameterized database and graph names do not support wildcards. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information.

Suggestions for improvement

Use `GRAPH *` without the parameter to revoke the privilege on all graphs.

Example 60. Revoking a privilege from a role with a wildcard database parameter

Parameter

```
{
    "database": "*"
}
```

Command

```
REVOKE GRANT ACCESS ON DATABASE $database FROM PUBLIC
```

Returned GQLSTATUS code

01N70

Returned status description

warn: impossible revoke command. `REVOKE GRANT ACCESS ON DATABASE $database FROM PUBLIC` has no effect. Database `*` does not exist. Make sure nothing is misspelled. This notification will become an error in a future major version.

Suggestions for improvement

Use `DATABASE *` without the parameter to revoke the privilege on all databases.

Parameter

```
{
    "database": "*"
}
```

Command

```
REVOKE GRANT ACCESS ON DATABASE $database FROM PUBLIC
```

Title of the returned code

`REVOKE GRANT ACCESS ON DATABASE $database FROM PUBLIC` has no effect.

Description of the returned code

Parameterized database and graph names do not support wildcards. Make sure nothing is misspelled. This notification will become an error in a future major version. See Status Codes documentation for more information.

Suggestions for improvement

Use `DATABASE *` without the parameter to revoke the privilege on all databases.

The auth provider is not defined

Table 32. Notification details

Neo4j code	`Neo.ClientNotification.Security.AuthProviderNotDefined`
Title	The auth provider is not defined.
Description	The auth provider `{ $provider }` is not defined in the configuration. Verify that the spelling is correct or define `{ $provider }` in the configuration.
Category	`SECURITY`
GQLSTATUS code	`00N72`
Status description	note: successful completion - undefined auth provider. The auth provider `{ [auth] }` is not defined in the configuration. Verify that the spelling is correct or define { $auth } in the configuration.
Classification	`SECURITY`
SeverityLevel	`INFORMATION`

Example 61. Create a user with an auth provider that is not defined in the configuration

Command

```
CREATE USER foo SET AUTH 'unknownProvider' { SET ID 'idString' }
```

Returned GQLSTATUS code

00N72

Returned status description

note: successful completion - undefined auth provider. The auth provider `unknownProvider` is not defined in the configuration. Verify that the spelling is correct or define `unknownProvider` in the configuration.

Suggestions for improvement

Make sure that the given provider is correct, or replace it if not. If it is correct, make sure to add it as a known auth provider in one or both of `dbms.security.authentication_providers` and `dbms.security.authorization_providers`.

Command

```
CREATE USER foo SET AUTH 'unknownProvider' { SET ID 'idString' }
```

Description of the returned code

The auth provider `unknownProvider` is not defined in the configuration. Verify that the spelling is correct or define `unknownProvider` in the configuration.

Suggestions for improvement

Make sure that the given provider is correct, or replace it if not. If it is correct, make sure to add it as a known auth provider in one or both of `dbms.security.authentication_providers` and `dbms.security.authorization_providers`.

Example 62. Alter a user to add an auth provider that is not defined in the configuration

Command

```
ALTER USER foo SET AUTH 'unknownProvider' { SET ID 'idString' }
```

Returned GQLSTATUS code

00N72

Returned status description

note: successful completion - undefined auth provider. The auth provider `unknownProvider` is not defined in the configuration. Verify that the spelling is correct or define `unknownProvider` in the configuration.

Suggestions for improvement

Make sure that the given provider is correct, or replace it if not. If it is correct, make sure to add it as a known auth provider in one or both of `dbms.security.authentication_providers` and `dbms.security.authorization_providers`.

Command

```
ALTER USER foo SET AUTH 'unknownProvider' { SET ID 'idString' }
```

Description of the returned code

The auth provider `unknownProvider` is not defined in the configuration. Verify that the spelling is correct or define `unknownProvider` in the configuration.

Suggestions for improvement

Make sure that the given provider is correct, or replace it if not. If it is correct, make sure to add it as a known auth provider in one or both of `dbms.security.authentication_providers` and `dbms.security.authorization_providers`.

External auth for user is not enabled

Table 33. Notification details

Neo4j code	`Neo.ClientNotification.Security.ExternalAuthNotEnabled`
Title	External auth for user is not enabled.
Description	Use setting `dbms.security.require_local_user` to enable external auth.
Category	`SECURITY`
GQLSTATUS code	`01N71`
Status description	warn: external auth disabled. Use the setting `dbms.security.require_local_user` to enable external auth.
Classification	`SECURITY`
SeverityLevel	`WARNING`

Example 63. Create a user with an external auth provider when linked users are not enabled

Command

```
CREATE USER foo SET AUTH 'exampleProvider' { SET ID 'idString' }
```

Returned GQLSTATUS code

01N71

Returned status description

warn: external auth disabled. Use the setting 'dbms.security.require_local_user' to enable external auth.

Suggestions for improvement

Enable linked users through the `dbms.security.require_local_user` setting. Until enabled, the new external auth will be ignored, and current external auth behaviors will continue to apply.

Command

```
CREATE USER foo SET AUTH 'exampleProvider' { SET ID 'idString' }
```

Suggestions for improvement

Enable linked users through the `dbms.security.require_local_user` setting. Until enabled, the new external auth will be ignored, and current external auth behaviors will continue to apply.

Example 64. Alter a user to add an external auth provider when linked users are not enabled

Command

```
ALTER USER foo SET AUTH 'exampleProvider' { SET ID 'idString' }
```

Returned GQLSTATUS code

01N71

Returned status description

warn: external auth disabled. Use the setting 'dbms.security.require_local_user' to enable external auth.

Suggestions for improvement

Enable linked users through the `dbms.security.require_local_user` setting. Until enabled, the new external auth will be ignored, and current external auth behaviors will continue to apply.

Command

```
ALTER USER foo SET AUTH 'exampleProvider' { SET ID 'idString' }
```

Suggestions for improvement

Enable linked users through the `dbms.security.require_local_user` setting. Until enabled, the new external auth will be ignored, and current external auth behaviors will continue to apply.

Query uses an insecure protocol

Table 34. Notification details

Neo4j code	`Neo.ClientNotification.Statement.InsecureProtocol`
Title	Query uses an insecure protocol
Description	The query uses an insecure protocol. Please consider using 'https' instead.
Category	`SECURITY`
GQLSTATUS code	`01N72`
Status description	warn: insecure URL protocol. Query uses an insecure protocol. Consider using 'https' instead.
Classification	`SECURITY`
SeverityLevel	`WARNING`

OIDC credential forwarding is not enabled

Table 35. Notification details

Neo4j code	`Neo.ClientNotification.Security.OidcCredentialForwardingNotEnabled`
Title	OIDC credential forwarding is not enabled.
Description	Use setting 'dbms.security.allow_oidc_credential_forwarding_enabled' to enable OIDC credential forwarding.
Category	`SECURITY`
GQLSTATUS code	`01N74`
Status description	warn: OIDC credential forwarding disabled. Use the setting `dbms.security.allow_oidc_credential_forwarding_enabled` to enable OIDC credential forwarding.
Classification	`SECURITY`
SeverityLevel	`WARNING`

Example 65. Create a remote database alias with remote credentials set to OIDC credential forwarding when OIDC credential forwarding is not enabled.

Command

```
CREATE ALIAS `remote-neo4j` FOR DATABASE `neo4j` AT "neo4j+s://location:7687" OIDC
CREDENTIAL FORWARDING
```

Returned GQLSTATUS code

01N74

Returned status description

warn: OIDC credential forwarding disabled. Use the setting 'dbms.security.allow_oidc_credential_forwarding_enabled' to enable OIDC credential forwarding.

Suggestions for improvement

The remote database alias has been successfully created, but OIDC credential forwarding is not enabled. To use the alias to connect to the remote Neo4j DBMS, allow OIDC credential forwarding by setting `dbms.security.allow_oidc_credential_forwarding_enabled` to `true`.

Command

```
CREATE ALIAS `remote-neo4j` FOR DATABASE `neo4j` AT "neo4j+s://location:7687" OIDC
CREDENTIAL FORWARDING
```

Suggestions for improvement

The remote database alias has been successfully created, but OIDC credential forwarding is

not enabled. To use the alias to connect to the remote Neo4j DBMS, allow OIDC credential forwarding by setting `dbms.security.allow_oidc_credential_forwarding_enabled` to `true`.

Sharded privilege performance

Table 36. Notification details

Neo4j code	`Neo.ClientNotification.Security.ShardedPrivilegePerformance`
Title	Privilege with a severe performance impact on sharded databases.
Description	The specified privilege severely reduces the performance of queries run on sharded databases. Consider excluding sharded databases for now.
Category	`SECURITY`
GQLSTATUS code	`01N73`
Status description	warn: sharded privilege performance. The specified privilege severely reduces the performance of queries run on sharded databases. Consider excluding sharded databases for now.
Classification	`SECURITY`
SeverityLevel	`WARNING`

Example 66. Grant a property access rule privilege on all databases

Command

```
GRANT TRAVERSE ON GRAPH * TO ROLE exampleRole
```

Returned GQLSTATUS code

01N73

Returned status description

warn: sharded privilege performance. The specified privilege severely reduces the performance of queries run on sharded databases. Consider excluding sharded databases for now.

Suggestions for improvement

Grant the privilege explicitly to databases instead of to all.

Command

```
GRANT TRAVERSE ON GRAPH * TO ROLE exampleRole
```

Suggestions for improvement

Grant the privilege explicitly to databases instead of to all.

`TOPOLOGY` notifications

Topology notifications provide additional information related to managing databases and servers.

Server already enabled

Table 37. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.ServerAlreadyEnabled`
Title	`<command>` has no effect.
Description	Server `%s` is already enabled. Verify that this is the intended server.
Category	`TOPOLOGY`
GQLSTATUS code	`00N80`
Status description	note: successful completion - server already enabled. The command `ENABLE SERVER` has no effect. Server `{ [server] }` is already enabled. Verify that this is the intended server.
Classification	`TOPOLOGY`
SeverityLevel	`INFORMATION`

Example 67. Enabling an already enabled server

Command

```
ENABLE SERVER "123e4567-e89b-12d3-a456-426614174000"
```

Returned GQLSTATUS code

00N80

Returned status description

note: successful completion - server already enabled. `ENABLE SERVER` has no effect. Server `123e4567-e89b-12d3-a456-426614174000` is already enabled. Verify that this is the intended server.

Command

```
ENABLE SERVER "123e4567-e89b-12d3-a456-426614174000"
```

Description of the returned code

Server `123e4567-e89b-12d3-a456-426614174000` is already enabled. Verify that this is the intended server.

Server already cordoned

Table 38. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.ServerAlreadyCordoned`
Title	`<command>` has no effect.
Description	Server `%s` is already cordoned. Verify that this is the intended server.
Category	`TOPOLOGY`
GQLSTATUS code	`00N81`
Status description	note: successful completion - server already cordoned. The command `CORDON SERVER` has no effect. Server `{ [server] }` is already cordoned. Verify that this is the intended server.
Classification	`TOPOLOGY`
SeverityLevel	`INFORMATION`

Example 68. Cordoning an already cordoned server

Command

```
CORDON SERVER "123e4567-e89b-12d3-a456-426614174000"
```

Returned GQLSTATUS code

00N81

Returned status description

note: successful completion - server already cordoned. `CORDON SERVER` has no effect. Server `123e4567-e89b-12d3-a456-426614174000` is already cordoned. Verify that this is the intended server.

Command

```
CORDON SERVER "123e4567-e89b-12d3-a456-426614174000"
```

Description of the returned code

Server `123e4567-e89b-12d3-a456-426614174000` is already cordoned. Verify that this is the intended server.

No databases reallocated

Table 39. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.NoDatabasesReallocated`

Title	`<command>` has no effect.
Description	No databases were reallocated. No better allocation is currently possible.
Category	`TOPOLOGY`
GQLSTATUS code	`00N82`
Status description	note: successful completion - no databases reallocated. The command `REALLOCATE DATABASES` has no effect. No databases were reallocated. No better allocation is currently possible.
Classification	`TOPOLOGY`
SeverityLevel	`INFORMATION`

Example 69. Reallocating databases resulted in no allocation changes

Command

```
REALLOCATE DATABASES
```

Returned GQLSTATUS code

00N82

Returned status description

note: successful completion - no databases reallocated. `REALLOCATE DATABASES` has no effect. No databases were reallocated. No better allocation is currently possible.

Example scenarios

Scenario 1: The cluster is already balanced. For example, when there are three servers, each hosting databases `foo` and `bar`, meaning all databases are allocated to all servers.

Scenario 2: The cluster appears unbalanced, but server constraints prevent you from moving to a better, more balanced, allocation. For example, assuming server 1 hosts databases `foo` and `bar`, server 2 hosts only `foo`, and server 3 hosts no databases. Then, a better allocation would move `foo` from server 1 to server 3, but if server 3 has the constraint `deniedDatabases:['foo']}`, then the cluster is already balanced subject to this constraint.

Command

```
REALLOCATE DATABASES
```

Description of the returned code

No databases were reallocated. No better allocation is currently possible.

Example scenarios

Scenario 1: The cluster is already balanced. For example, when there are three servers, each hosting databases `foo` and `bar`, meaning all databases are allocated to all servers.

Scenario 2: The cluster appears unbalanced, but server constraints prevent you from moving to a better, more balanced, allocation. For example, assuming server 1 hosts databases `foo` and `bar`, server 2 hosts only `foo`, and server 3 hosts no databases. Then, a better allocation would move `foo` from server 1 to server 3, but if server 3 has the constraint `deniedDatabases:['foo']}`, then the cluster is already balanced subject to this constraint.

Cordoned servers existed during allocation

This notification is returned when a Cypher administration command triggers an allocation decision and some of the servers are cordoned. For example, `CREATE DATABASE`, `ALTER DATABASE`, `DEALLOCATE DATABASES FROM SERVER[S]`, and `ALTER DATABASE` return this notification. However, `REALLOCATE DATABASES` requires that there are no cordoned servers and, therefore, does not return it.

Table 40. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.CordonedServersExistedDuringAllocation`
Title	Cordoned servers existed when making an allocation decision.
Description	Server(s) `%s` are cordoned. This can impact allocation decisions.
Category	`TOPOLOGY`
GQLSTATUS code	`00N83`
Status description	note: successful completion - cordoned servers existed during allocation. Cordoned servers existed when making an allocation decision. Server(s) `{ [serverList] }` are cordoned. This can impact allocation decisions.
Classification	`TOPOLOGY`
SeverityLevel	`INFORMATION`

Example 70. Cordoned servers existed during an allocation decision

The example assumes that you have a cluster with three servers, of which server `123e4567-e89b-12d3-a456-426614174000` is cordoned using the `dbms.cluster.cordonServer` procedure. Then the below command will return this notification.

Command

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES
```

Returned GQLSTATUS code

00N83

Returned status description

note: successful completion - cordoned servers existed during allocation. Cordoned servers existed when making an allocation decision. Server(s) `123e4567-e89b-12d3-a456-`

`426614174000` are cordoned. This can impact allocation decisions.

The example assumes that you have a cluster with three servers, of which server `123e4567-e89b-12d3-a456-426614174000` is cordoned using the `dbms.cluster.cordonServer` procedure. Then the below command will return this notification.

Command

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES
```

Description of the returned code

Server(s) `123e4567-e89b-12d3-a456-426614174000` are cordoned. This can impact allocation decisions.

Requested topology matched current topology

Table 41. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.RequestedTopologyMatchedCurrentTopology`
Title	`<command>` has no effect.
Description	The requested topology matched the current topology. No allocations were changed.
Category	`TOPOLOGY`
GQLSTATUS code	`00N84`
Status description	note: successful completion - requested topology matched current topology. The command `ALTER DATABASE` has no effect. The requested topology matched the current topology. No allocations were changed.
Classification	`TOPOLOGY`
SeverityLevel	`INFORMATION`

Example 71. Requested topology matched current topology

The example assumes that you have a cluster with three servers and a database `foo` with a topology of two primaries and one secondary.

Command

```
ALTER DATABASE foo SET TOPOLOGY 2 PRIMARIES 1 SECONDARY
```

Returned GQLSTATUS code

00N84

Returned status description

note: successful completion - requested topology matched current topology. The command `ALTER DATABASE` has no effect. The requested topology matched the current topology. No allocations were changed.

The example assumes that you have a cluster with three servers and a database `foo` with a topology of two primaries and one secondary.

Command

```
ALTER DATABASE foo SET TOPOLOGY 2 PRIMARIES 1 SECONDARY
```

Description of the returned code

The requested topology matched the current topology. No allocations were changed.

Server has caught up during `WAIT` command

Introduced in 2025.01 | Cypher 25

Table 42. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.ServerCaughtUp`
Title	Server has caught up.
Description	Server `(%s)` at address `(%s)` has caught up.
Category	`TOPOLOGY`
GQLSTATUS code	`03N85`
Status description	info: server has caught up. Server `{ [server] }` at address `{ [serverAddress] }` has caught up.
Classification	`TOPOLOGY`
SeverityLevel	`INFORMATION`

Example 72. Successful completion of a `WAIT` command with two servers

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Returned GQLSTATUS code

03N85

Returned status description

info: server has caught up. Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Returned GQLSTATUS code

03N85

Returned status description

info: server has caught up. Server `ServerId{0e020000}` at address `localhost:20026` has caught up.

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Description of the returned code

Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Description of the returned code

Server `ServerId{0e020000}` at address `localhost:20026` has caught up.

Server failed during `WAIT` command

Introduced in 2025.01 | Cypher 25

Table 43. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.ServerFailed`
Title	Server failed.
Description	Server `(%s)` at address `(%s)` failed: (%s)
Category	`TOPOLOGY`
GQLSTATUS code	`01N80`
Status description	warn: server failed. Server `{ [server] }` at address `{ [serverAddress] }` failed: `{ [msg] }`
Classification	`TOPOLOGY`
SeverityLevel	`WARNING`

Example 73. One out of two servers failed during `WAIT` command

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Returned GQLSTATUS code

03N85

Returned status description

info: server has caught up. Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Returned GQLSTATUS code

01N80

Returned status description

warn: server failed. Server `ServerId{0e020000}` at address `localhost:20026` failed: Caught up but has failure for DatabaseId{0db00002[foo]} Failure: java.nio.file.FileAlreadyExistsException: <...>

Suggestions for improvement

Investigate the failing server using the provided message.

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Description of the returned code

Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Description of the returned code

Server `ServerId{0e020000}` at address `localhost:20026` failed: Caught up but has failure for DatabaseId{0db00002[foo]} Failure: java.nio.file.FileAlreadyExistsException: <...>

Suggestions for improvement

Investigate the failing server using the provided message.

Server is still catching up during `WAIT` command

Introduced in 2025.01 | Cypher 25

Table 44. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.ServerCatchingUp`
Title	Server is still catching up.
Description	Server `(%s)` at address `(%s)` is still catching up.
Category	`TOPOLOGY`
GQLSTATUS code	`01N81`
Status description	warn: server is catching up. Server `{ [server] }` at address `{ [serverAddress] }` is still catching up.
Classification	`TOPOLOGY`
SeverityLevel	`WARNING`

Example 74. One out of two servers is still catching up during `WAIT` command

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Returned GQLSTATUS code

03N85

Returned status description

info: server has caught up. Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Returned GQLSTATUS code

01N81

Returned status description

warn: server is catching up. Server `ServerId{0e020000}` at address `localhost:20026` is still catching up.

Suggestions for improvement

This behaviour indicates that one of the servers is lagging behind. Investigate the server and network for performance issues or increase the wait timeout.

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Description of the returned code

Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Description of the returned code

Server `ServerId{0e020000}` at address `localhost:20026` is still catching up.

Suggestions for improvement

This behaviour indicates that one of the servers is lagging behind. Investigate the server and network for performance issues or increase the wait timeout.

Server is not available during `WAIT` command

Introduced in 2025.01 | Cypher 25

Table 45. Notification details

Neo4j code	`Neo.ClientNotification.Cluster.ServerNotAvailable`
Title	Server is not available.
Description	Server `(%s)` is not available.
Category	`TOPOLOGY`
GQLSTATUS code	`01N82`
Status description	warn: server is not available. Server `{ [server] }` is not available.
Classification	`TOPOLOGY`
SeverityLevel	`WARNING`

Example 75. One out of two servers is not available during `WAIT` command

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Returned GQLSTATUS code

03N85

Returned status description

info: server has caught up. Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Returned GQLSTATUS code

01N82

Returned status description

warn: server is not available. Server `ServerId{0e020000}` at address `localhost:20026` is not available.

Suggestions for improvement

Investigate the server to determine why it is not available.

Query

```
CREATE DATABASE foo TOPOLOGY 2 PRIMARIES WAIT
```

One notification is returned for each server in the cluster:

Description of the returned code

Server `ServerId{0e010000}` at address `localhost:20025` has caught up.

Description of the returned code

Server `ServerId{0e020000}` at address `localhost:20026` is not available.

Suggestions for improvement

Investigate the server to determine why it is not available.

`SCHEMA` notifications

Schema notifications provide additional information related to indexes and constraints.

Index or constraint already exists

Table 46. Notification details

Neo4j code	`Neo.ClientNotification.Schema.IndexOrConstraintAlreadyExists`
Title	`<command>` has no effect.
Description	`<conflicting>` already exists.
Description	`%s` already exists.
Category	`SCHEMA`
GQLSTATUS code	`00NA0`
Status description	note: successful completion - index or constraint already exists. The command `{ [cmd] }` has no effect. The index or contstraint specified by `{ [idxOrConstrPat] }` already exists.

SeverityLevel	INFORMATION

* `<command>` and `cmd` could be either the full command given by the user or a subset of the given command.

Example 76. Creating an index when an equivalent index already exists

Given a range index on `(:Label {property})` named `existingRangeIndex`.

Command

```
CREATE INDEX labelProperyRangeIndex IF NOT EXISTS FOR (n:Label) ON (n.property)
```

Returned GQLSTATUS code

00NA0

Returned status description

note: successful completion - index or constraint already exists. `CREATE RANGE INDEX labelProperyRangeIndex IF NOT EXISTS FOR (e:Label) ON (e.property)` has no effect. `RANGE INDEX existingRangeIndex FOR (e:Label) ON (e.property)` already exists.

Given a range index on `(:Label {property})` named `existingRangeIndex`.

Command

```
CREATE INDEX labelProperyRangeIndex IF NOT EXISTS FOR (n:Label) ON (n.property)
```

Title of the returned code

`CREATE RANGE INDEX labelProperyRangeIndex IF NOT EXISTS FOR (e:Label) ON (e.property)` has no effect.

Full description of the returned code

`RANGE INDEX existingRangeIndex FOR (e:Label) ON (e.property)` already exists.

Example 77. Creating an index when another unrelated index using that name already exists

Given a range index on `(:Label {property})` named `myIndex`.

Command

```
CREATE TEXT INDEX myIndex IF NOT EXISTS FOR ()-[r:REL_TYPE]-() ON (r.property)
```

Returned GQLSTATUS code

00NA0

Returned status description

note: successful completion - index or constraint already exists. `CREATE TEXT INDEX myIndex IF NOT EXISTS FOR ()-[e:REL_TYPE]-() ON (e.property)` has no effect. `RANGE INDEX myIndex FOR (e:Label) ON (e.property)` already exists.

Suggestions for improvement

Choose a different name for the new index and try again.

```
CREATE TEXT INDEX myIndex2 IF NOT EXISTS FOR ()-[r:REL_TYPE]-() ON (r.property)
```

Given a range index on `(:Label {property})` named `myIndex`.

Command

```
CREATE TEXT INDEX myIndex IF NOT EXISTS FOR ()-[r:REL_TYPE]-() ON (r.property)
```

Title of the returned code

`CREATE TEXT INDEX myIndex IF NOT EXISTS FOR ()-[e:REL_TYPE]-() ON (e.property)` has no effect.

Full description of the returned code

`RANGE INDEX myIndex FOR (e:Label) ON (e.property)` already exists.

Suggestions for improvement

Choose a different name for the new index and try again.

```
CREATE TEXT INDEX myIndex2 IF NOT EXISTS FOR ()-[r:REL_TYPE]-() ON (r.property)
```

Example 78. Creating a constraint when an identical constraint already exists

Given a node key constraint on `(:Label {property})` named `nodeKeyLabelPropertyConstraint`.

Command

```
CREATE CONSTRAINT nodeKeyLabelPropertyConstraint IF NOT EXISTS FOR (n:Label) REQUIRE
(n.property) IS NODE KEY
```

Returned GQLSTATUS code

00NA0

Returned status description

note: successful completion - index or constraint already exists. `CREATE CONSTRAINT nodeKeyLabelPropertyConstraint IF NOT EXISTS FOR (e:Label) REQUIRE (e.property) IS NODE KEY` has no effect. `CONSTRAINT nodeKeyLabelPropertyConstraint FOR (e:Label)`

`REQUIRE (e.property) IS NODE KEY` already exists.

Given a node key constraint on `(:Label {property})` named `nodeKeyLabelPropertyConstraint`.

Command

```
CREATE CONSTRAINT nodeKeyLabelPropertyConstraint IF NOT EXISTS FOR (n:Label) REQUIRE
(n.property) IS NODE KEY
```

Title of the returned code

`CREATE CONSTRAINT nodeKeyLabelPropertyConstraint IF NOT EXISTS FOR (e:Label) REQUIRE (e.property) IS NODE KEY` has no effect.

Full description of the returned code

`CONSTRAINT nodeKeyLabelPropertyConstraint FOR (e:Label) REQUIRE (e.property) IS NODE KEY` already exists.

Example 79. Creating a constraint when another unrelated constraint using that name already exists

Given a node key constraint on `(:Label {property})` named `myConstraint`.

Command

```
CREATE CONSTRAINT myConstraint IF NOT EXISTS FOR (n:Label2) REQUIRE (n.property2) IS NOT
NULL
```

Returned GQLSTATUS code

00NA0

Returned status description

note: successful completion - index or constraint already exists. `CREATE CONSTRAINT myConstraint IF NOT EXISTS FOR (e:Label2) REQUIRE (e.property2) IS NOT NULL` has no effect. `CONSTRAINT myConstraint FOR (e:Label) REQUIRE (e.property) IS NODE KEY` already exists.

Suggestions for improvement

Choose a different name for the new constraint and try again.

```
CREATE CONSTRAINT myConstraint2 IF NOT EXISTS FOR (n:Label2) REQUIRE (n.property2) IS NOT
NULL
```

Given a node key constraint on `(:Label {property})` named `myConstraint`.

Command

```
CREATE CONSTRAINT myConstraint IF NOT EXISTS FOR (n:Label2) REQUIRE (n.property2) IS NOT
NULL
```

Title of the returned code

`CREATE CONSTRAINT myConstraint IF NOT EXISTS FOR (e:Label2) REQUIRE (e.property2) IS NOT NULL` has no effect.

Full description of the returned code

`CONSTRAINT myConstraint FOR (e:Label) REQUIRE (e.property) IS NODE KEY` already exists.

Suggestions for improvement

Choose a different name for the new constraint and try again.

```
CREATE CONSTRAINT myConstraint2 IF NOT EXISTS FOR (n:Label2) REQUIRE (n.property2) IS NOT
NULL
```

Index or constraint does not exist

Table 47. Notification details

Neo4j code	`Neo.ClientNotification.Schema.IndexOrConstraintDoesNotExist`
Title	`<command>` has no effect.
Description	`%s` does not exist.
Category	`SCHEMA`
GQLSTATUS code	`00NA1`
Status description	note: successful completion - index or constraint does not exist. The command `{ [cmd] }` has no effect. The specified index or constraint `{ [idxOrConstr] }` does not exist.
SeverityLevel	`INFORMATION`

Example 80. Attempting to drop a non-existing index

Command

```
DROP INDEX nonExistingIndex IF EXISTS
```

Returned GQLSTATUS code

00NA1

Returned status description

note: successful completion - index or constraint does not exist. `DROP INDEX`

`nonExistingIndex IF EXISTS` has no effect. `nonExistingIndex` does not exist.

Suggestions for improvement

Verify that this is the intended index and that it is spelled correctly.

Command

```
DROP INDEX nonExistingIndex IF EXISTS
```

Title of the returned code

`DROP INDEX nonExistingIndex IF EXISTS` has no effect.

Full description of the returned code

`nonExistingIndex` does not exist.

Suggestions for improvement

Verify that this is the intended index and that it is spelled correctly.

Example 81. Attempting to drop a non-existing constraint

Command

```
DROP CONSTRAINT nonExistingConstraint IF EXISTS
```

Returned GQLSTATUS code

00NA1

Returned status description

note: successful completion - index or constraint does not exist. `DROP CONSTRAINT nonExistingConstraint IF EXISTS` has no effect. `nonExistingConstraint` does not exist.

Suggestions for improvement

Verify that this is the intended constraint and that it is spelled correctly.

Command

```
DROP CONSTRAINT nonExistingConstraint IF EXISTS
```

Title of the returned code

`DROP CONSTRAINT nonExistingConstraint IF EXISTS` has no effect.

Full description of the returned code

`nonExistingConstraint` does not exist.

Suggestions for improvement

Verify that this is the intended constraint and that it is spelled correctly.

Vector index dimensions not specified

Table 48. Notification details

Neo4j code	`Neo.ClientNotification.Schema.VectorIndexDimensionsNotSpecified`
Title	Vector index dimensions not specified.
Description	When creating a vector index, `vector.dimensions` should be specified. Omitting it is allowed, but specifying dimensions ensures that only vectors of that size are indexed and makes dimension mismatches fail clearly at query time. For example, set `OPTIONS { indexConfig: { vector.dimensions : 1536 } }` when creating the index.
Category	`SCHEMA`
GQLSTATUS code	`00NA2`
Status description	note: successful completion - vector index dimensions not specified. When creating a vector index, `vector.dimensions` should be specified. Omitting it is allowed, but specifying dimensions ensures that only vectors of that size are indexed and makes dimension mismatches fail clearly at query time. For example, set `OPTIONS { indexConfig: { vector.dimensions : 1536 } }` when creating the index.
SeverityLevel	`INFORMATION`

Example 82. Creating a vector index without configured dimensions

Command

```
CREATE VECTOR INDEX movieEmbeddingIndex
FOR (m:Movie) ON (m.embedding)
```

Returned GQLSTATUS code

00NA2

Returned status description

note: successful completion - vector index dimensions not specified. When creating a vector index, `vector.dimensions` should be specified. Omitting it is allowed, but specifying dimensions ensures that only vectors of that size are indexed and makes dimension mismatches fail clearly at query time. For example, set `OPTIONS { indexConfig: { vector.dimensions : 1536 } }` when creating the index.

Suggestions for improvement

If the indexed vectors are expected to have the same size, recreate the index with an explicit `vector.dimensions` value.

```
CREATE VECTOR INDEX movieEmbeddingIndex
```

```
FOR (m:Movie) ON (m.embedding)
OPTIONS {
  indexConfig: {`vector.dimensions`: 1536}
}
```

Command

```
CREATE VECTOR INDEX movieEmbeddingIndex
FOR (m:Movie) ON (m.embedding)
```

Title of the returned code

Vector index dimensions not specified.

Full description of the returned code

When creating a vector index, `vector.dimensions` should be specified. Omitting it is allowed, but specifying dimensions ensures that only vectors of that size are indexed and makes dimension mismatches fail clearly at query time. For example, set `OPTIONS { indexConfig: { vector.dimensions : 1536 } }` when creating the index.

Suggestions for improvement

If the indexed vectors are expected to have the same size, recreate the index with an explicit `vector.dimensions` value.

```
CREATE VECTOR INDEX movieEmbeddingIndex
FOR (m:Movie) ON (m.embedding)
OPTIONS {
  indexConfig: {`vector.dimensions`: 1536}
}
```

`GENERIC` notifications

`GENERIC` notification codes do not belong to any wider type and do not have any connection to each other.

Subquery variable shadowing

Table 49. Notification details

Neo4j code	`Neo.ClientNotification.Statement.SubqueryVariableShadowing`
Title	Variable in subquery is shadowing a variable with the same name from the outer scope.
Description	Variable in subquery is shadowing a variable with the same name from the outer scope. If you want to use that variable instead, it must be imported into the subquery using a variable scope clause. (`%s`)
Category	`GENERIC`
GQLSTATUS code	`03N60`

Status description	info: subquery variable shadowing. The variable `{ [variable] }` in the subquery uses the same name as a variable from the outer query. Use `{ [clause] } ({ [variable] })` to import the one from the outer scope unless you want it to be a new variable.
Classification	GENERIC
SeverityLevel	INFORMATION

Example 83. Shadowing of a variable from the outer scope

Query

```
MATCH (n)
CALL {
  MATCH (n)--(m)
  RETURN m
}
RETURN *
```

Returned GQLSTATUS code

03N60

Returned status description

info: subquery variable shadowing. The variable `n` in the subquery uses the same name as a variable from the outer query. Use `CALL (n)` to import the one from the outer scope unless you want it to be a new variable.

Suggestions for improvement

If the intended behavior of the query is for the variable in the subquery to be a new variable, then nothing needs to be done. If the intended behavior is to use the variable from the outer query, it needs to be imported to the subquery using a variable scope clause.

```
MATCH (n)
CALL (n) {
  MATCH (n)--(m)
  RETURN m
}
RETURN *
```

Query

```
MATCH (n)
CALL {
  MATCH (n)--(m)
  RETURN m
}
RETURN *
```

Description of the returned code

Variable in subquery is shadowing a variable with the same name from the outer scope. If you want to use that variable instead, it must be imported into the subquery using a variable scope clause. (the shadowing variable is: `n`)

Suggestions for improvement

If the intended behavior of the query is for the variable in the subquery to be a new variable, then nothing needs to be done. If the intended behavior is to use the variable from the outer query, it needs to be imported to the subquery using a variable scope clause.

```
MATCH (n)
CALL (n) {
  MATCH (n)--(m)
  RETURN m
}
RETURN *
```

Redundant optional procedure

Table 50. Notification details

Neo4j code	`Neo.ClientNotification.Statement.RedundantOptionalProcedure`
Title	The use of `OPTIONAL` is redundant when the procedure calls a void procedure.
Description	The use of `OPTIONAL` is redundant as `CALL %s` is a void procedure.
Category	`GENERIC`
GQLSTATUS code	`03N61`
Status description	info: redundant optional procedure. The use of `OPTIONAL` is redundant as `CALL { [proc] }` is a void procedure.
Classification	`GENERIC`
SeverityLevel	`INFORMATION`

Example 84. Redundant use of `OPTIONAL` in a procedure call

Query

```
OPTIONAL CALL db.createLabel("A")
```

Returned GQLSTATUS code

03N61

Returned status description

info: redundant optional procedure. The use of `OPTIONAL` is redundant as `CALL db.createLabel` is a void procedure.

Suggestions for improvement

If the intended behavior of the query is to use a void procedure, the `OPTIONAL` keyword can be removed without impacting the query.

```
CALL db.createLabel("A")
```

Query

```
OPTIONAL CALL db.createLabel("A")
```

Description of the returned code

The use of `OPTIONAL` is redundant as `CALL db.createLabel` is a void procedure.

Suggestions for improvement

If the intended behavior of the query is to use a void procedure, the `OPTIONAL` keyword can be removed without impacting the query.

```
CALL db.createLabel("A")
```

Redundant optional subquery

Table 51. Notification details

Neo4j code	`Neo.ClientNotification.Statement.RedundantOptionalSubquery`
Title	The use of `OPTIONAL` is redundant when `CALL` is a unit subquery.
Description	The use of `OPTIONAL` is redundant as `CALL` is a unit subquery.
Category	`GENERIC`
GQLSTATUS code	`03N62`
Status description	info: redundant optional subquery. The use of `OPTIONAL` is redundant as `CALL` is a unit subquery.
Classification	`GENERIC`
SeverityLevel	`INFORMATION`

Example 85. Redundant use of `OPTIONAL` in a `CALL` subquery

Query

```
UNWIND [1, 2, 3] AS x
OPTIONAL CALL (x) {
   CREATE({i:x})
}
```

Returned GQLSTATUS code

03N62

Description of the returned code

info: redundant optional subquery. The use of `OPTIONAL` is redundant as `CALL` is a unit subquery.

Suggestions for improvement

If the intended behavior of the query is for the subquery not to return any values, the `OPTIONAL` keyword can be removed without impacting the query.

```
UNWIND [1, 2, 3] AS x
CALL (x) {
   CREATE({i:x})
}
```

Query

```
UNWIND [1, 2, 3] AS x
OPTIONAL CALL (x) {
   CREATE({i:x})
}
```

Description of the returned code

Optional is redundant in the case of a unit subquery. The use of `OPTIONAL` on unit subqueries have no effect and can be removed.

Suggestions for improvement

If the intended behavior of the query is for the subquery not to return any values, the `OPTIONAL` keyword can be removed without impacting the query.

```
UNWIND [1, 2, 3] AS x
CALL (x) {
   CREATE({i:x})
}
```

Identifier shadowing variable

Table 52. Notification details

Neo4j code	`Neo.ClientNotification.Statement.IdentifierShadowingVariable`
Title	An identifier is shadowing a variable in scope.
Description	The identifier `%s` in the `%s` clause has the same name as a variable in scope. Regardless of what the variable evaluates to, it is the literal `%s` that will be used.
Category	`GENERIC`
GQLSTATUS code	`03N63`

Status description	info: identifier shadowing variable. The identifier { [ident] } in the { [clause] } clause has the same name as a variable in scope. Regardless of what the variable evaluates to, it is the literal { [ident] } that will be used.
Classification	GENERIC
SeverityLevel	INFORMATION

Example 86. An identifier used for the index name in a `SEARCH` clause shadowing a variable in scope

Query

```
CYPHER 25
WITH "myPlotString" AS moviePlots
MATCH (m:Movie {title:'Matrix, The'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    LIMIT 5
  ) SCORE AS score
RETURN movie.title AS title, score, moviePlots
```

Returned GQLSTATUS code

03N63

Description of the returned code

info: identifier shadowing variable. The identifier `moviePlots` in the `VECTOR INDEX` clause has the same name as a variable in scope. Regardless of what the variable evaluates to, it is the literal `moviePlots` that will be used.

Suggestions for improvement

Consider to rename the variable to something else to avoid confusion.

```
CYPHER 25
WITH "myPlotString" AS plotString
MATCH (m:Movie {title:'Matrix, The'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    LIMIT 5
  ) SCORE AS score
RETURN movie.title AS title, score, plotString
```

Query

```
CYPHER 25
WITH "myPlotString" AS moviePlots
MATCH (m:Movie {title:'Matrix, The'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    LIMIT 5
  ) SCORE AS score
```

```
RETURN movie.title AS title, score, moviePlots
```

Description of the returned code

The identifier `moviePlots` in the `VECTOR INDEX` clause has the same name as a variable in scope. Regardless of what the variable evaluates to, it is the literal `moviePlots` that will be used.

Suggestions for improvement

Consider to rename the variable to something else to avoid confusion.

```
CYPHER 25
WITH "myPlotString" AS plotString
MATCH (m:Movie {title:'Matrix, The'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    LIMIT 5
  ) SCORE AS score
RETURN movie.title AS title, score, plotString
```

Parameter missing

Table 53. Notification details

Neo4j code	`Neo.ClientNotification.Statement.ParameterNotProvided`
Title	The statement refers to a parameter that was not provided in the request.
Description	Did not supply query with enough parameters. The produced query plan will not be cached and is not executable without EXPLAIN. (`%s`)
Category	`GENERIC`
GQLSTATUS code	`01N60`
Status description	warn: parameter missing. The query plan cannot be cached and is not executable without `EXPLAIN` due to the undefined parameter(s) `{ [paramList] }`. Provide the parameter(s).
Classification	`GENERIC`
SeverityLevel	`WARNING`

Example 87. Using an `EXPLAIN` query with parameters without providing them

Query

```
EXPLAIN WITH $param as param
RETURN param
```

Returned GQLSTATUS code

01N60

Returned status description

warn: parameter missing. The query plan cannot be cached and is not executable without `EXPLAIN` due to the undefined parameter(s) `{ $param }`. Provide the parameter(s).

Suggestions for improvement

Provide the parameter to be able to cache the plan.

Query

```
EXPLAIN WITH $param as param
RETURN param
```

Description of the returned code

Did not supply query with enough parameters. The produced query plan will not be cached and is not executable without `EXPLAIN`. (Missing parameters: `param`)

Suggestions for improvement

Provide the parameter to be able to cache the plan.

Procedure or function execution warning

Table 54. Notification details

Neo4j code	`Neo.ClientNotification.Procedure.ProcedureWarning`
Title	The query used a procedure that generated a warning.
Description	The query used a procedure that generated a warning. (`%s`)
Category	`GENERIC`
GQLSTATUS code	`01N62`
Status description	warn: procedure execution warning. The procedure `{ [proc] }` generated the warning `{ [msg] }`.
Classification	`GENERIC`
SeverityLevel	`WARNING`

Unsatisfiable relationship type expression

When matching on a relationship type expression that can never be satisfied, for example asking for zero, more than one or contradictory types.

Table 55. Notification details

Neo4j code	`Neo.ClientNotification.Statement.UnsatisfiableRelationshipTypeExpression`
Title	The query contains a relationship type expression that cannot be satisfied.

Description	Relationship type expression cannot possibly be satisfied. (`%s`)
Category	`GENERIC`
GQLSTATUS code	`01N61`
Status description	warn: unsatisfiable relationship type expression. The expression `{ [labelExpr] }` cannot be satisfied because relationships must have exactly one type.
Classification	`GENERIC`
SeverityLevel	`WARNING`

Example 88. Matching on a relationship type expression that can never be satisfied

Query

```
MATCH ()-[r:R1&R2]->() RETURN r
```

Returned GQLSTATUS code

01N61

Returned status description

warn: unsatisfiable relationship type expression. The expression `R1&R2` cannot be satisfied because relationships must have exactly one type.

Query

```
MATCH ()-[r:R1&R2]->() RETURN r
```

Description of the returned code

Relationship type expression cannot possibly be satisfied. (`R1&R2` can never be fulfilled by any relationship. Relationships must have exactly one type.)

Repeated relationship reference

Table 56. Notification details

Neo4j code	`Neo.ClientNotification.Statement.RepeatedRelationshipReference`
Title	The query returns no results because a relationship variable is bound more than once.
Description	• A relationship is referenced more than once in the query, which leads to no results because relationships must not occur more than once in each result. (`%s`) • A variable-length relationship variable is bound more than once, which leads to no results because relationships must not occur more than once in each result. (`%s`)
Category	`GENERIC`

GQLSTATUS code	`01N63`
Status description	warn: repeated relationship reference. `{ [variable] }` is repeated in `{ [pat] }`, which leads to no results.
Classification	`GENERIC`
SeverityLevel	`WARNING`

Example 89. Binding a relationship variable more than once

Query

```
MATCH (:A)-[r]->(), ()-[r]->(:B) RETURN r
```

Returned GQLSTATUS code

01N63

Returned status description

warn: repeated relationship reference. `r` is repeated in `(:A)-[r]→(), ()-[r]→(:B)`, which leads to no results.

Suggestions for improvement

Use one pattern to match all relationships that start with a node with the label `A` and end with a node with the label `B`:

```
MATCH (:A)-[r]->(:B) RETURN r
```

Query

```
MATCH (:A)-[r]->(), ()-[r]->(:B) RETURN r
```

Description of the returned code

A relationship is referenced more than once in the query, which leads to no results because relationships must not occur more than once in each result. (Relationship `r` was repeated)

Suggestions for improvement

Use one pattern to match all relationships that start with a node with the label `A` and end with a node with the label `B`:

```
MATCH (:A)-[r]->(:B) RETURN r
```

Example 90. Binding a variable-length relationship variable more than once (when run on version 5.6 or newer)

Query

```
MATCH ()-[r*]->()<-[r*]-() RETURN count(*) AS count
```

Returned GQLSTATUS code

01N63

Returned status description

warn: repeated relationship reference. `r` is repeated in `()-[r*]→()←[r*]-()`, which leads to no results.

Query

```
MATCH ()-[r*]->()<-[r*]-() RETURN count(*) AS count
```

Description of the returned code

A variable-length relationship variable is bound more than once, which leads to no results because relationships must not occur more than once in each result. (Relationship r was repeated)

Server errors

Neo4j returns a server error to indicate that the outcome of a Cypher query or command execution is unsuccessful. The driver receives these errors and sends them to the Neo4j tools (e.g. Browser, Bloom, Cypher Shell) or the user application, which display them to the user.

Starting from version 5.26, the Neo4j error codes have an additional GQL-status object along with the Neo4j exception that provides information about the status of a Cypher query or command execution in compliance with the ISO/IEC 39075:2024(en) - Information technology - Database languages - GQL Standard. This additional GQL-status object is also displayed in the query log from Neo4j 5.25 onwards. For more information, see Operations Manual → GQL error information.

This page describes the GQL-status and the Neo4j-status object frameworks for errors, their structure, the objects they provide for errors, and how to interpret them. It also explains the error internals and the server-driver compatibility for both the GQLSTATUS and errors.

GQL-status error object

Version 5.25

In the GQL-status object framework, when a user executes a query to the server, it always produces a result known as the *execution outcome*. If an error occurs during execution, the outcome is recorded as an exception condition, indicating a failed outcome with no result. The execution outcome for errors is represented as a list of GQL-status objects with a failed outcome, which are organized according to the following precedence rules:

1. Every exception that results in a transaction rollback takes precedence over other exceptions.
2. Every exception takes precedence over any completion condition. For completion conditions, see the GQL-status notification object.

The GQL-status object also includes Neo4j-specific information, such as severity level and error classification.

Each GQL-status object consists of the following fields:

Table 57. GQLSTATUS error object

GQLSTATUS code	A 5-character string that is the concatenation of a 2-character class code followed by a 3-character subclass code, which identifies the condition of the error.
StatusDescription	A human-readable description of the GQLSTATUS, which consists of a condition, a subcondition, and an optional additional text about the condition. The format is `error: condition - subcondition. AdditionalInfo`. The condition and subcondition are textual representations of the class and subclass codes, respectively. The subcondition for the subclass 000 is empty.

<table>
<tr><td>DiagnosticRecord</td><td>Extra information about the status, given as key-value pairs, both on the server and driver side. To retrieve the full diagnostic record, you can use the <code>diagnosticRecord()</code> on the server side or the corresponding method on the driver sides. Additional helper methods are exposed for some useful fields.

<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td><code>OPERATION</code></td><td>The operation that the error is related to. Always defaults to empty.</td></tr>
<tr><td><code>OPERATION_CODE</code></td><td>The operation code that the error is related to. Always defaults to <code>0</code>.</td></tr>
<tr><td><code>CURRENT_SCHEMA</code></td><td>The current schema that the error is related to. Always defaults to <code>/</code>.</td></tr>
<tr><td><code>_classification</code></td><td>The Neo4j error classification, which can be <code>CLIENT_ERROR</code>, <code>TRANSIENT_ERROR</code>, or <code>DATABASE_ERROR</code>.</td></tr>
<tr><td><code>_position</code></td><td>(Optional) The position, given by offset, line and column, where the error is relevant in the query text.</td></tr>
</table></td></tr>
</table>

A GQL-status error object can also contain an optional GQL-status object that represents the cause of the error and is used to provide additional, more specific diagnostic information.

Classes of GQLSTATUS error codes

The GQLSTATUS codes are divided into classes and subclasses. The class code is a 2-character string that indicates the general condition of the status, such as connection exception, data exception, etc. The subclass code is a 3-character string that provides more detailed information about the condition.

The following table lists the GQLSTATUS classes and their meanings:

Table 58. GQLSTATUS code classes

Class	Condition
08	connection exception
22	data exception
25	invalid transaction state
2D	invalid transaction termination
40	transaction rollback
42	syntax error or access rule violation
50	general processing exception
51	system configuration or operation exception
52	procedure exception
53	function exception

G1	dependent object error

Neo4j-status error object

The Neo4j-status object for errors contains diagnostic information representing the **unsuccessful** outcome of a Cypher query or command execution, including severity, status code, category, description, message, and position in the query text where the error is relevant. Depending on the application, some of the fields from the error object might not be visible.

The error object consists of the following fields:

Table 59. Neo4j error object

Neo4j code	The Neo4j code in the form of `Neo.[Type].[SubType].[Name]`.
Description	The description of the specific error.
Message	The error message.
Severity level	`ERROR`
Category	The category of the error. Available categories are `CLIENT_ERROR`, `TRANSIENT_ERROR`, and `DATABASE_ERROR`.
Position	(Optional) The position, given by offset, line and column, where the error is relevant in the query text.

For more information, see the List of Neo4j error codes.

Types of server errors

The fact that an error is returned by the server does not always mean that it is a fatal error. Status codes can also indicate transient problems that may go away if you retry the request. The server error group determines the effect on the transaction.

Table 60. Server error types

Type	Description	Effect on the transaction
`ClientError`	These errors are caused by the client (user input or user application) and are usually related to the request itself. Changing the request might yield a successful outcome. Neo4j codes have the prefix `Neo.ClientError`, while the GQLSTATUS codes have `ErrorClassification` of type `CLIENT_ERROR`.	Rollback

Type	Description	Effect on the transaction
`TransientError`	These errors are detected by the server and are usually related to some kind of database unavailability, such as limits reached, out-of-memory, timeouts, etc. The error can be temporary, therefore retrying later might yield a successful outcome. Neo4j codes have the prefix `Neo.TransientError`, while the GQLSTATUS codes have `ErrorClassification` of type `TRANSIENT_ERROR`.	Rollback
`DatabaseError`	These errors are caused by the database and are usually related to the database state and mean that the database failed to service the request. Neo4j codes have the prefix `Neo.DatabaseError`, while the GQLSTATUS codes have `ErrorClassification` of type `DATABASE_ERROR`.	Rollback

Error internals

Neo4j supports server errors in the form of Java exceptions. Most of these implement the `HasStatus` interface, which means they have a status code in addition to the exception message.

On the server side, an exception contains normal Java constructors and methods like `getMessage()`, `getCause()`, etc., and additionally the `status()` method from the `HasStatus` API, which returns the status code.

The exceptions also get new compulsary fields for `gqlStatus`, `statusDescription`, `diagnosticRecord`, and an optional field for `cause`. The cause field in turn has its own GQLSTATUS, status description, diagnostic record, and message.
The `getMessage()` method is kept as Java exceptions inherently have this method. And a new classification field is added to cover the division of client errors, transient errors, and database errors, which today is part of the Neo4j code. All of these fields construct the GQLSTATUS object, which is sent to the driver as part of the Failure Bolt message. Exactly how this looks, depends on the combination of driver and server versions. See Server-driver version compatibility for more information.

On the driver side, the Neo4jException is extended with the corresponding methods as on the server side. The driver receives the Failure Bolt message and extracts the status code and the error message. Then, it constructs an exception with the status code, error message, and other relevant information, and sends it to the client.

Query logging

Since the query log is server-side and DBMS wide, multiple clients connected to the same DBMS write to the same query log. As the clients can have separate driver versions, they may have different error framework formats.

In Neo4j 5.25, the default JSON template for the query log is updated to include an `errorInfo` entry. This entry contains `GQLSTATUS`, `statusDescription`, `classification`, `position` (if applicable), and `cause` (if

applicable) with the same entries.

Server-driver version compatibility

The server and driver communicate with each other through the Bolt protocol. During the handshake process, they agree on using the newest possible Bolt protocol version that both the server and the driver support. For more information on the Bolt versions supported by different server versions, see the Bolt Protocol documentation.

The new error framework with the additional GQL-status object for errors is available in the JSON format query log for Neo4j server Neo4j 5.25 and later versions. It is supported over Bolt since Bolt 5.7, which corresponds to version 5.26 or later on both server and driver side.

To fully utilize the new error framework, both your server and the driver must support it. Drivers that are older than 5.26 will not send any GQL-status object for exceptions, even if server is 5.26 or later.

If a driver of version 5.26 or later talks to a server that is older than 5.26, the driver needs to poly-fill the exceptions with a GQL-status object. In this case, the default GQLSTATUS code 50N42 is returned for all exceptions.

Table 61. Error compatibility matrix

	Driver 5.25 or older	Driver 5.26 or newer
Server 5.24 or older	Bolt: existing error information Query log: existing error information	Bolt: additional GQL-status object with default values Query log: existing error information
Server 5.25	Bolt: existing error information Query log: additional errorInfo entry	Bolt: additional GQL-status object with default values Query log: additional errorInfo entry
Server 5.26 or newer	Bolt: existing error information Query log: additional errorInfo entry	Bolt: additional GQL-status object Query log: additional errorInfo entry

List of GQLSTATUS error codes

The following page provides an overview of all GQLSTATUS server error codes in Neo4j. All errors in Neo4j have severity level `ERROR`.

Warning:

Please note that while GQLSTATUS codes remain stable (any changes to them will be breaking), changes to status descriptions associated with these codes are not breaking and may happen at any time. For this reason, parsing the status descriptions or incorporating them into scripts is not recommended.

Connection exceptions

Connection exceptions occur when the client (e.g. Browser/Bloom/Cypher Shell) is unable to connect to the server for various reasons such as network issues, server-side routing being disabled, or the database being unavailable, etc.

08000

Status description

error: connection exception

08007

Status description

error: connection exception - transaction resolution unknown

08N00

Status description

error: connection exception - unable to connect to database. Unable to connect to database `{ [db] }`. Unable to get bolt address of the leader. Check the status of the database. Retrying your request at a later time may succeed.

08N01

Status description

error: connection exception - unable to write to database. Unable to write to database `{ [db] }` on this server. Server-side routing is disabled. Either connect to the database leader directly or enable server-side routing by setting `{ [cfgSetting] }=true`.

08N02

Status description

error: connection exception - unable to connect to database. Unable to connect to database `{ [db] }`. Server-side routing is disabled. Either connect to `{ $db }` directly, or enable server-side routing by setting `{ [cfgSetting] }=true`.

08N03

Status description

error: connection exception - failed to write to graph. Failed to write to graph `{ [graph] }`. Check the defined access mode in both driver and database.

08N04

Status description

error: unable to route use clause. Routing with `{ [clause] }` is not supported in embedded sessions. Connect to the database directly or try running the query using a Neo4j driver or the HTTP API.

08N05

Status description

error: connection exception - unable to route administration command. Routing administration commands is not supported in embedded sessions. Connect to the system database directly or try running the query using a Neo4j driver or the HTTP API.

08N06

Status description

error: connection exception - protocol error. General network protocol error.

08N07

Status description

error: connection exception - not the leader. This member is not the leader.

08N08

Status description

error: connection exception - database is read only. This database is read only on this server.

08N09

Status description

error: connection exception - database unavailable. The database `{ [db] }` is currently unavailable. Check the database status. Retry your request at a later time.

08N10

Status description

error: connection exception - invalid server state. Message `{ [msg] }` cannot be handled by session in the `{ [boltServerState] }` state.

08N11

Status description

error: connection exception - request error. The request is invalid and could not be processed by the server. See cause for further details.

08N12

Status description

error: connection exception - failed to parse bookmark. Failed to parse the supplied bookmark. Verify it is correct or check the debug log for more information.

08N13

Status description

error: connection exception - database not up to requested bookmark. The database `{ [db] }` is not up to the requested bookmark `{ [transactionId] 1 }`. The latest transaction ID is `{ $transactionId2 }`.

08N14

Status description

error: connection exception - alias chains are not permitted. Unable to provide a routing table for the database identifed by the alias `{ [alias] 1 }` because the request comes from another alias `{ $alias2 }` and alias chains are not permitted.

08N15

Status description

error: connection exception - no such routing policy. Policy definition of the routing policy `{ [routingPolicy] }` could not be found. Verify that the spelling is correct.

08N16

Status description

error: connection exception - general driver client error. Remote execution failed with message `{ [msg] }`.

08N17

Status description

error: connection exception - general driver transient error. Remote execution failed with message `{ [msg] }`.

08N18

Status description

error: connection exception - general driver database error. Remote execution failed with message `{ [msg] }`.

08N19

Status description

error: connection exception - shard execution transient error. Communication with shard `{ [graph] }` failed with message `{ [msg] }`.

08N20

Status description

error: connection exception - shard execution database error. Communication with shard `{ [graph] }` failed with message `{ [msg] }`.

08N21

Status description

error: connection exception - shard execution client error. Communication with shard `{ [graph] }` failed with message `{ [msg] }`.

Data exceptions

Data exceptions occur when a client request contains the wrong format, types, or other unsupported input. Some examples are data and constraint creation, which conflicts with existing constraints, properties of non-storable type, and spatial and temporal values with invalid components.

22000

Status description

error: data exception

22003

Status description

error: data exception - numeric value out of range. The numeric value `{ [value] }` is outside the required range.

22004

Status description

error: data exception - null value not allowed

22007

Status description

error: data exception - invalid date, time, or datetime format

22012

Status description

error: data exception - division by zero

22015

Status description

error: data exception - interval field overflow

22G03

Status description

error: data exception - invalid value type

22G05

Status description

error: data exception - invalid date, time, or datetime function field name

22G06

Status description

error: data exception - invalid date, time, or datetime function field value

22G08

Status description

error: data exception - invalid duration function field value

22G0I

Status description

error: data exception - invalid duration field. `{ [field] }` is not a valid duration field.

22N00

Status description

error: data exception - unsupported value. The provided value is unsupported and cannot be processed.

22N01

Status description

error: data exception - invalid type. Expected the value `{ [value] }` to be of type `{ [valueTypeList] }`, but was of type `{ [valueType] }`.

22N02

Status description

error: data exception - specified negative numeric value. Expected `{ [option] }` to be a positive number but found `{ [value] }` instead.

22N03

Status description

error: data exception - specified numeric value out of range. Expected `{ [component] }` to be of type `{ [valueType] }` and in the range `{ [lower] }` to `{ [upper] }` but found `{ [value] }`.

22N04

Status description

error: data exception - invalid input value. Invalid input `{ [input] }` for `{ [context] }`. Expected `{ [inputList] }`.

22N05

Status description

error: data exception - input failed validation. Invalid input `{ [input] }` for `{ [context] }`.

22N06

Status description

error: data exception - required input missing. Invalid input. `{ [inputList] }` needs to be specified.

22N07

Status description

error: data exception - invalid pre-parser option key. Invalid pre-parser option(s): `{ [optionList] }`.

22N08

Status description

error: data exception - invalid pre-parser combination. Invalid pre-parser option, cannot combine `{ [option] 1 }` with `{ $option2 }`.

22N09

Status description

error: data exception - conflicting pre-parser combination. Invalid pre-parser option, cannot specify multiple conflicting values for `{ [option] }`.

22N10

Status description

error: data exception - invalid pre-parser option value. Invalid pre-parser option, specified `{ [input] }` is not valid for option `{ [option] }`. Valid options are: `{ [optionList] }`.

22N11

Status description

error: data exception - invalid argument. Invalid argument: cannot process `{ [input] }`.

22N12

Status description

error: data exception - invalid date, time, or datetime format. Invalid argument: cannot process `{ [input] }`.

22N13

Status description

error: data exception - invalid time zone. Specified time zones must include a date component.

22N14

Status description

error: data exception - invalid temporal value combination. Cannot select both `{ [temporal] 1 }` and `{ $temporal2 }`.

22N15

Status description

error: data exception - invalid temporal component. Cannot read the specified `{ [component] }` component from `{ [temporal] }`.

22N16

Status description

error: data exception - invalid import value. Importing entity values to a graph with a USE clause is not supported. Attempted to import `{ [expr] }` to `{ [graph] }`.

22N18

Status description

error: data exception - incomplete spatial value. A `{ [crs] }` POINT must contain `{ [mapKeyList] }`.

22N19

Status description

error: data exception - invalid spatial value. A `POINT` must contain either `x` and `y`, or `latitude` and `longitude`.

22N20

Status description

error: data exception - invalid spatial value dimensions. Cannot create POINT with `{ [dim] 1 }D` coordinate reference system (CRS) and `{ [value] }` coordinates. Use the equivalent `{ $dim2 }D` coordinate reference system instead.

22N21

Status description

error: data exception - unsupported coordinate reference system. Unsupported coordinate reference system (CRS): `{ [crs] }`.

22N22

Status description

error: data exception - invalid spatial value combination. Cannot specify both coordinate reference system (CRS) and spatial reference identifier (SRID).

22N23

Status description

error: data exception - invalid latitude value. Cannot create WGS84 POINT with invalid coordinate: `{ [coordinates] }`. The valid range for the latitude coordinate is [-90, 90].

22N24

Status description

error: data exception - invalid coordinate arguments. Cannot construct a `{ [valueType] }` from `{ [coordinates] }`.

22N25

Status description

error: data exception - invalid temporal arguments. Cannot construct a `{ [valueType] }` from `{ [temporal] }`.

22N26

Status description

error: data exception - unsupported rounding mode. Unknown rounding mode. Valid values are: `CEILING`, `FLOOR`, `UP`, `DOWN`, `HALF_EVEN`, `HALF_UP`, `HALF_DOWN`, `UNNECESSARY`.

22N27

Status description

error: data exception - invalid entity type. Invalid input `{ [input] }` for `{ [context] }`. Expected to be `{ [valueTypeList] }` `{ [hint] }`.

22N28

Status description

error: data exception - overflow error. The result of the operation `{ [operation] }` has caused an overflow.

22N29

Status description

error: data exception - unknown coordinate reference system. Unknown coordinate reference system (CRS).

22N30

Status description

error: data exception - missing temporal unit. At least one temporal unit must be specified.

22N31

Status description

error: data exception - invalid properties in merge pattern. The `{ [entityType] }` property `{ [propKey] }` in `{ [pat] }` is invalid. `MERGE` cannot be used with a graph element property value that is `{ [value] }`.

22N32

Status description

error: data exception - non-deterministic sort expression. `ORDER BY` expressions must be deterministic.

22N33

Status description

error: data exception - invalid shortest path expression. Shortest path expressions must contain start and end nodes. Cannot find: `{ [variable] }`.

22N34

Status description

error: data exception - invalid use of aggregate function. Cannot use `{ [funType] }` function inside an aggregate function.

22N35

Status description

error: data exception - invalid date format. Cannot parse `{ [input] }` as a DATE. Calendar dates need to be specified using the format `YYYY-MM`, while ordinal dates need to be specified using the format `YYYY-DDD`.

22N36

Status description

error: data exception - invalid temporal format. Cannot parse `{ [input] }` as a `{ [valueType] }`.

22N37

Status description

error: data exception - invalid coercion. Cannot coerce `{ [value] }` to `{ [valueType] }`.

22N38

Status description

error: data exception - invalid function argument. Invalid argument to the function `{ [fun] }`.

22N39

Status description

error: data exception - unsupported property value type. Value `{ [value] }` cannot be stored in properties.

22N40

Status description

error: data exception - non-assignable temporal component. Cannot assign `{ [component] }` of a `{ [valueType] }`.

22N41

Status description

error: data exception - merge node uniqueness constraint violation. The `MERGE` clause did not find a matching node `{ [variable] }` and cannot create a new node due to conflicts with existing uniqueness constraints.

22N42

Status description

error: data exception - merge relationship uniqueness constraint violation. The `MERGE` clause did not find a matching relationship `{ [variable] }` and cannot create a new relationship due to conflicts with existing uniqueness constraints.

22N43

Status description

error: data exception - unable to load external resource. Could not load external resource from `{ [url] }`.

22N44

Status description

error: data exception - parallel runtime disabled. Parallel runtime has been disabled, enable it or upgrade to a bigger Aura instance.

22N46

Status description

error: data exception - unsupported use of parallel runtime. Parallel runtime does not support updating queries or a change in the state of transactions. Use another runtime.

22N47

Status description

error: data exception - invalid parallel runtime configuration. No workers are configured for the parallel runtime. Set `server.cypher.parallel.worker_limit` to a larger value.

22N48

Status description

error: data exception - unable to use specified runtime. Cannot use the specified runtime `{ [runtime] }` due to `{ [cause] }` not being supported. Use another runtime.

22N49

Status description

error: data exception - CSV buffer size overflow. Cannot read a CSV field larger than the set buffer size. Ensure the field does not have an unterminated quote, or increase the buffer size via `dbms.import.csv.buffer_size`.

22N51

Status description

error: data exception - graph reference not found. A graph reference with the name `{ [db] }` was not found. Verify that the spelling is correct.

22N52

Status description

error: data exception - invalid combination of `PROFILE` and `EXPLAIN`. `PROFILE` and `EXPLAIN` cannot be combined.

22N53

Status description

error: data exception - invalid use of PROFILE. Cannot `PROFILE` query before results are materialized.

22N54

Status description

error: data exception - invalid map. Multiple conflicting entries specified for `{ [mapKey] }`.

22N55

Status description

error: data exception - required key missing from map. Map requires key `{ [mapKey] }` but was missing from field `{ [field] }`.

22N56

Status description

error: data exception - protocol message length limit overflow. Protocol message length limit exceeded (limit: `{ [boltMsgLenLimit] }`).

22N57

Status description

error: data exception - invalid protocol type. Protocol type is invalid. Invalid number of struct components (received `{ [count] 1 }` but expected `{ [count] 2 }`).

22N58

Status description

error: data exception - invalid spatial component. Cannot read the specified `{ [component] }` component from `{ [value] }`.

22N59

Status description

error: data exception - token does not exist. The `{ [tokenType] }` token with id `{ [tokenId] }` does not exist.

22N60

Status description

error: data exception - illegal element. Encountered illegal `{ [item] }` element. Reason: `{ [msg] }`

22N62

Status description

error: data exception - relationship type does not exist. The relationship type `{ [relType] }` does not exist.

22N63

Status description

error: data exception - property key does not exist. The property key `{ [propKey] }` does not exist.

22N64

Status description

error: data exception - constraint does not exist. The constraint `{ [constrDescrOrName] }` does not exist.

22N65

Status description

error: data exception - equivalent constraint already exists. An equivalent constraint already exists: `{ [constrDescrOrName] }`.

22N66

Status description

error: data exception - conflicting constraint already exists. A conflicting constraint already exists: `{ [constrDescrOrName] }`.

22N67

Status description

error: data exception - duplicated constraint name. A constraint with the same name already exists: `{ [constr] }`.

22N68

Status description

error: data exception - dependent constraint managed individually. Dependent constraints cannot be managed individually and must be managed together with its graph type.

22N69

Status description

error: data exception - index does not exist. The index `{ [idxDescrOrName] }` does not exist.

22N70

Status description

error: data exception - equivalent index already exists. An equivalent index already exists: `{ [idxDescrOrName] }`.

22N71

Status description

error: data exception - duplicated index name. An index with the same name already exists: `{ [idx] }`.

22N73

Status description

error: data exception - constraint conflicts with existing index. Constraint conflicts with already existing index `{ [idxDescrOrName] }`.

22N74

Status description

error: data exception - index conflicts with existing constraint. An index that belongs to the constraint `{ [constr] }` contains a conflicting index.

22N75

Status description

error: data exception - constraint contains duplicated tokens. The constraint specified by `{ [constrDescrOrName] }` includes a label, relationship type, or property key with name `{ [token] }` more than once.

22N76

Status description

error: data exception - index contains duplicated tokens. The index specified by `{ [idxDescrOrName] }` includes a label, relationship type, or property key with name `{ [token] }` more than once.

22N77

Status description

error: data exception - property presence verification failed. `{ [entityType] }` (`{ [entityId] }`) with `{ [tokenType] }` `{ [token] }` must have the following properties: `{ [propKeyList] }`.

22N78

Status description

error: data exception - property type verification failed. `{ [entityType] }` (`{ [entityId] }`) with `{ [tokenType] }` `{ [token] }` must have the property `{ [propKey] }` with value type `{ [valueType] }`.

22N79

Status description

error: data exception - property uniqueness constraint violated. Property uniqueness constraint violated: `{ [reasonList] }`.

22N80

Status description

error: data exception - index entry conflict. Index entry conflict: `{ [value] }`.

22N81

Status description

error: data exception - expression type unsupported here. Invalid input: `{ [exprType] }` is not supported in `{ [context] }`.

22N82

Status description

error: data exception - input contains invalid characters. Input `{ [input] }` contains invalid characters for `{ [context] }`. Special characters may require that the input is quoted using backticks.

22N83

Status description

error: data exception - input consists of too many components. Expected name to contain at most `{ [upper] }` components separated by `'.'`.

22N84

Status description

error: data exception - string too long. Expected the string to be no more than `{ [upper] }` characters long.

22N85

Status description

error: data exception - string too short. Expected the string to be at least `{ [lower] }` characters long.

22N86

Status description

error: data exception - numeric range 0 disallowed. Expected a nonzero number.

22N88

Status description

error: data exception - not a valid CIDR IP. `{ [input] }` is not a valid CIDR IP.

22N89

Status description

error: data exception - new password cannot be the same as the old password. Expected the new password to be different from the old password.

22N90

Status description

error: data exception - property type unsupported in constraint. `{ [valueType] }` is not supported in property type constraints.

22N91

Status description

error: data exception - cannot convert alias local to remote or remote to local. Failed to alter the specified database alias `{ [alias] }`. Altering remote alias to a local alias or vice versa is not supported. Drop and recreate the alias instead.

22N92

Status description

error: data exception - missing RETURN. This query requires a RETURN clause.

22N93

Status description

error: data exception - missing `YIELD`. A required `YIELD` clause is missing.

22N94

Status description

error: data exception - invalid `YIELD *`. `YIELD *` is not supported in this context. Explicitly specify which columns to yield.

22N95

Status description

error: data exception - parsing JSON exception. Invalid JSON input. Please check the format.

22N96

Status description

error: data exception - mapping JSON exception. Unable to map the JSON input. Please verify the structure.

22N97

Status description

error: data exception - unexpected struct tag. Unexpected struct tag: `{ [value] }`.

22N98

Status description

error: data exception - wrong first field during deserialization. Unable to deserialize request. Expected first field to be `{ [field] }`, but was `{ [value] }`.

22N99

Status description

error: data exception - wrong token during deserialization. Unable to deserialize request. Expected `{ [token] }`, found `{ [value] }`.

22NA0

Status description

error: data exception - invalid property-based access control rule. Failed to administer property rule.

22NA1

Status description

error: data exception - invalid property-based access control rule involving non-commutative expressions. The property `{ [propKey] }` must appear on the left hand side of the `{ [operation] }` operator.

22NA2

Status description

error: data exception - invalid property-based access control rule involving multiple properties. The expression: `{ [expr] }` is not supported. Property rules can only contain one property.

22NA3

Status description

error: data exception - invalid property-based access control rule involving NaN. 'NaN' is not supported for property-based access control.

22NA4

Status description

error: data exception - invalid property-based access control rule involving comparison with `NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`.

22NA5

Status description

error: data exception - invalid property-based access control rule involving `IS NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`. Use `IS NULL` instead.

22NA6

Status description

error: data exception - invalid property-based access control rule involving `IS NOT NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`. Use `IS NOT NULL` instead.

22NA7

Status description

error: data exception - invalid property-based access control rule involving nontrivial predicates. The expression: `{ [expr] }` is not supported. Only single, literal-based predicate expressions are allowed for property-based access control.

22NA8

Status description

error: data exception - parsing JSON failure. Underlying error: `{ [cause] }`.

22NA9

Status description

error: data exception - unexpected map entry. Invalid input. Unexpected key `{ [mapKey] }`, expected keys are `{ [mapKeyList] }`.

22NAA

Status description

error: data exception - invalid list for property-based access control rule. The expression `{ [expr] }` is not supported. Lists containing `{ [exprType] }` values are not supported for property-based access control.

22NAB

Status description

error: data exception - mixed type list for property-based access control rule. The expression `{ [expr] }` is not supported. All elements in a list must be literals of the same type for property-based access

control.

22NAC

Status description

error: data exception - characters after quote in CSV field. Characters after an ending quote in a CSV field are not supported. See `{ [input] }` at position `{ [pos] }`. This is read as `{ [variable] }`.

22NAD

Status description

error: data exception - missing end quote in CSV field. Missing end quote at position `{ [pos] }` in `{ [input] }`.

22NAE

Status description

error: data exception - multi-line field in illegal CSV context. Multi-line fields are illegal in this context. Verify that there is not a missing end quote in `{ [input] }` at position `{ [pos] }`.

22NB0

Status description

error: data exception - invalid property-based access control rule involving `WHERE` and `IS NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`. Use `WHERE` syntax in combination with `IS NULL` instead.

22NB1

Status description

error: data exception - type mismatch. Type mismatch: expected to be `{ [valueTypeList] }` but was `{ [input] }`.

22NB2

Status description

error: data exception - incompatible graph type dependence. Graph type `{ [graphTypeDependence] 1 }` constraint `{ [constrDescrOrName] 1 }` is incompatible with graph type `{ $graphTypeDependence2 }` constraint `{ $constrDescrOrName2 }` because they have different graph type dependence.

22NB3

Status description

error: data exception - token presence verification failed. `{ [entityType] }` (`{ [entityId] }`) with `{ [tokenType] 1 }` `{ [token] 1 }` must have the `{ $tokenType2 }` `{ $token2 }`.

22NB4

Status description

error: data exception - endpoint label presence verification failed. Relationship (`{ [entityId] 1 }`) with type `{ [relType] }` requires its `{ [endpointType] }` node (`{ $entityId2 }`) to have the label `{ [label] }`.

22NB5

Status description

error: data exception - unsupported time zone identifier. Unknown time zone identifier `{ [input] }`.

22NB6

Status description

error: data exception - input empty. Invalid input. `{ [item] }` is not allowed to be an empty string.

22NB7

Status description

error: data exception - element id unsupported on composite database. It is not supported to create element ids on composite databases. Create the element id for `{ [entityType] }` `{ <<$entityId>> }` on the constituent instead.

22NB8

Status description

error: data exception - invalid Neo4j type. `{ [input] }` is not a recognized Neo4j type.

22NB9

Status description

error: data exception - invalid inner list type. Lists cannot have `{ [typeDescription] }` as an inner type in this context.

22NBA

Status description

error: data exception - omitting mandatory field for property type constraints for vectors. Property type constraints for vectors need to define both coordinate type and dimension.

22NBC

Status description

error: data exception - index belongs to constraint. Index belongs to constraint `{ [constrDescrOrName] }`.

22NBD

Status description

error: data exception - unsupported struct tag. Unsupported struct tag: `{ [value] }`.

22NBE

Status description

error: data exception - invalid vector dimensions. Invalid vector dimensions. The number of vector dimensions must be between `{ [count] 1 }` and `{ $count2 }`, but is `{ $count3 }`.

22NBF

Status description

error: data exception - property value too big. Property value of type `{ [typeDescription] }` is too big (more than `{ [bytes] }` bytes): `{ [value] }`

22NBG

Status description

error: data exception - invalid vector coordinates. Invalid vector coordinates. The vector coordinates must be finite, but are `{ [value] }`.

22NBH

Status description

error: data exception - property value size: `{ [bytes] }` of element id: `{ [entityId] }` is too large for the index: `{ [idx] }` you are trying to add it to. See the Operations Manual for index limitations.

22NC1

Status description

error: data exception - graph type element contains duplicated tokens. The graph type element includes a property key with name `{ [propKey] }` more than once.

22NC2

Status description

error: data exception - node element type has no effect. The node element type `{ [label] }` must contain one or more implied labels, or at least one property type.

22NC3

Status description

error: data exception - relationship element type has no effect. The relationship element type `{ [relType] }` must define a source, destination, or at least one property type.

22NC4

Status description

error: data exception - a label cannot be both identifying and implied. The label(s) `{ [labelList] }` are defined as both identifying and implied.

22NC5

Status description

error: data exception - graph type element not found. The `{ [entityType] }` element type referenced by `{ [graphTypeReference] }` does not exist.

22NC6

Status description

error: data exception - independent constraint and node element type have the same label. The independent constraint `{ [constrDescrOrName] }` is using the same label `{ [label] }` as a node element type.

22NC7

Status description

error: data exception - independent constraint and relationship element type have the same relationship type. The independent constraint `{ [constrDescrOrName] }` is using the same relationship type `{ [relType] }` as a relationship element type.

22NC8

Status description

error: data exception - graph type contains duplicated tokens. The graph type includes a label, a relationship type, or an alias with the name `{ [token] }` more than once.

22NC9

Status description

error: data exception - invalid element type constraints in graph type. A `{ [entityType] 1 }` element type property `{ [context] }` constraint cannot be specified inline of a `{ [entityType] 2 }` element type.

22NCA

Status description

error: data exception - node element type already exists. A node element type identified by the label `{ [label] }` already exists in the graph type.

22NCB

Status description

error: data exception - relationship element type already exists. A relationship element type identified by the relationship type `{ [relType] }` already exists in the graph type.

22NCC

Status description

error: data exception - node element type specified incorrectly. The node element type `{ [graphTypeElement] 1 }` identified by the label `{ [label] }` is different to the one specified in the query `{ [graphTypeElement] 2 }`.

22NCD

Status description

error: data exception - relationship element type specified incorrectly. The relationship element type `{ [graphTypeElement] 1 }` identified by the relationship type `{ [relType] }` is different to the one specified in the query `{ [graphTypeElement] 2 }`.

22NCE

Status description

error: data exception - node element type in use. The node element type identified by the label `{ [label] }` is referenced in the graph type element `{ [graphTypeReference] }` and cannot be dropped.

22NCF

Status description

error: data exception - graph type constraint not supported. Graph type constraint definitions are not supported in the `{ [graphTypeOperation] }` operation.

22NCG

Status description

error: data exception - wrong index type. Expected the index `{ [idx] }` to be a `{ [idxType] 1 }` index but was a `{ [idxType] 2 }` index.

22ND1

Status description

error: data exception - operation not allowed for roles that are granted to an `AUTH RULE`. Invalid input: `'{ [query] }'` is not allowed for roles that are granted to an `AUTH RULE`.

22ND2

Status description

error: data exception - operation not allowed for roles with `DENY` privileges. Invalid input: `'{ [query] }'`

is not allowed for roles with `DENY` privileges.

22ND3

Status description

error: data exception - wrong property for vector search with filters. The property `{ [propKey] }` is not an additional property for vector search with filters on the vector index `{ [idx] }` .

Invalid transaction state

Invalid transaction state errors occur when the transaction is in an invalid state, such as when the transaction is terminated or closed, or when there is a conflict between the transaction state and applied updates.

25G02

Status description

error: invalid transaction state - catalog and data statement mixing not supported

25N01

Status description

error: invalid transaction state - invalid combination of statement types. Failed to execute the query `{ [query] }` due to conflicting statement types (read query, write query, schema modification, or administration command). To execute queries in the same transaction, they must be either of the same type, or be a combination of schema modifications and read commands.

25N02

Status description

error: invalid transaction state - unable to complete transaction. Unable to complete transaction. See debug log for details.

25N03

Status description

error: invalid transaction state - concurrent access violation. Transaction is being used concurrently by another request.

25N04

Status description

error: invalid transaction state - specified transaction does not exist. Transaction `{ [transactionId] }` does not exist.

25N05

Status description

error: invalid transaction state - transaction closed. Transaction has been closed.

25N06

Status description

error: invalid transaction state - transaction start failed. Failed to start transaction. See debug log for details.

25N08

Status description

error: invalid transaction state - invalid transaction lease. The lease for the transaction is no longer valid.

25N09

Status description

error: invalid transaction state - internal transaction failure. The transaction failed due to an internal error.

25N11

Status description

error: invalid transaction state - conflicting transaction state. There was a conflict detected between the transaction state and applied updates. Please retry the transaction.

25N12

Status description

error: invalid transaction state - index was dropped. Index `{ [idx] }` was dropped in this transaction and cannot be used.

25N13

Status description

error: invalid transaction state - cannot access entity after removal. A `{ [entityType] }` was accessed after being deleted in this transaction. Verify the transaction statements.

25N14

Status description

error: invalid transaction state - transaction termination client error. The transaction has been terminated. Retry your operation in a new transaction, and you should see a successful result. Reason: `{ [msg] }`

25N15

Status description

error: invalid transaction state - transaction termination database error. The transaction has been terminated. Retry your operation in a new transaction, and you should see a successful result. Reason: `{ [msg] }`

25N16

Status description

error: invalid transaction state - transaction termination transient error. The transaction has been terminated. Retry your operation in a new transaction, and you should see a successful result. Reason: `{ [msg] }`

25N17

Status description

error: invalid transaction state - implicit transaction required. The attempted operation requires an implicit transaction.

Invalid transaction termination

Invalid transaction termination errors occur when the transaction termination fails, such as when the transaction or constituent transaction fails to commit, or when the transaction termination fails to apply or append the transaction.

2DN01

Status description

error: invalid transaction termination - commit failed. Failed to commit transaction.

2DN02

Status description

error: invalid transaction termination - constituent commit failed. Failed to commit constituent transaction. See debug log for details.

2DN03

Status description

error: invalid transaction termination - transaction termination failed. Failed to terminate transaction. See debug log for details.

2DN04

Status description

error: invalid transaction termination - constituent transaction termination failed. Failed to terminate constituent transaction. See debug log for details.

2DN05

Status description

error: invalid transaction termination - failed to apply transaction. There was an error on applying the transaction. See logs for more information.

2DN06

Status description

error: invalid transaction termination - failed to append transaction. There was an error on appending the transaction. See logs for more information.

2DN07

Status description

error: invalid transaction termination - inner transactions still open. Unable to commit transaction because it still have non-closed inner transactions.

Transaction rollback

Transaction rollback errors occur when there is a failure in a transaction or a constituent transaction rollback.

40000

Status description

error: transaction rollback

40003

Status description

error: transaction rollback - statement completion unknown

40N01

Status description

error: transaction rollback - rollback failed. Failed to rollback transaction. See debug log for details.

40N02

Status description

error: transaction rollback - constituent rollback failed. Failed to rollback constituent transaction. See debug log for details.

Syntax error or access rule violation

Syntax error or access rule violation errors occur when a Cypher query contains invalid syntax or when a client request violates the access rules, such as when a query tries to access a database without enough privileges, etc.

42000

Status description

error: syntax error or access rule violation

42001

Status description

error: syntax error or access rule violation - invalid syntax

42006

Status description

error: syntax error or access rule violation - number of edge labels below supported minimum

42007

Status description

error: syntax error or access rule violation - number of edge labels exceeds supported maximum

42008

Status description

error: syntax error or access rule violation - number of edge properties exceeds supported maximum

42009

Status description

error: syntax error or access rule violation - number of node labels below supported minimum

42010

Status description

error: syntax error or access rule violation - number of node labels exceeds supported maximum

42011

Status description

error: syntax error or access rule violation - number of node properties exceeds supported maximum

42012

Status description

error: syntax error or access rule violation - number of node type key labels below supported minimum

42013

Status description

error: syntax error or access rule violation - number of node type key labels exceeds supported maximum

42014

Status description

error: syntax error or access rule violation - number of edge type key labels below supported minimum

42015

Status description

error: syntax error or access rule violation - number of edge type key labels exceeds supported maximum

42I00

Status description

error: syntax error or access rule violation - invalid case expression. `CASE` expressions must have the same number of `WHEN` and `THEN` operands.

42I01

Status description

error: syntax error or access rule violation - invalid `FOREACH`. Invalid use of `{ [clause] }` inside `FOREACH`.

42I02

Status description

error: syntax error or access rule violation - invalid comment. Failed to parse comment. A comment starting with `/*` must also have a closing `*/`.

42I04

Status description

error: syntax error or access rule violation - invalid expression. `{ [expr] }` cannot be used in a `{ [clause] }` clause.

42I05

Status description

error: syntax error or access rule violation - invalid FIELDTERMINATOR. The FIELDTERMINATOR specified for LOAD CSV can only be one character wide.

42I06

Status description

error: syntax error or access rule violation - invalid input. Invalid input `{ [input] }`, expected: `{ [valueList] }`.

42I07

Status description

error: syntax error or access rule violation - invalid integer literal. The given `{ [valueType] }` literal `{ [input] }` is invalid.

42I08

Status description

error: syntax error or access rule violation - invalid lower bound. The lower bound of the variable length relationship used in the `{ [fun] }` function must be 0 or 1.

42I10

Status description

error: syntax error or access rule violation - invalid label expression. Mixing label expression symbols (`|`, `&`, `!`, and `%`) with colon (`:`) between labels is not allowed. This expression could be expressed as `{ [syntax] }`.

42I11

Status description

error: syntax error or access rule violation - invalid name. A `{ [tokenType] }` name cannot be empty or contain any null-bytes: `{ [input] }`.

42I12

Status description

error: syntax error or access rule violation - invalid nesting of quantified path patterns. Quantified path patterns cannot be nested.

42I13

Status description

error: syntax error or access rule violation - invalid number of procedure or function arguments. The procedure or function call does not provide the required number of arguments; expected `{ [count] 1 }`

but got `{ $count2 }`. The procedure or function `{ [procFun] }` has the signature: `{ [sig] }`.

42I14

Status description

error: syntax error or access rule violation - invalid number of relationship types. Exactly one relationship type must be specified for `{ [variable] }`.

42I15

Status description

error: syntax error or access rule violation - invalid number of statements. Expected exactly one statement per query but got: `{ [count] }`.

42I16

Status description

error: syntax error or access rule violation - invalid point. Map with keys `{ [mapKeyList] }` is not a valid POINT. Use either Cartesian or geographic coordinates.

42I17

Status description

error: syntax error or access rule violation - invalid quantifier. A quantifier must not have a lower bound greater than the upper bound.

42I18

Status description

error: syntax error or access rule violation - reference to non-grouping sub-expression. The expression contains a non-grouping sub-expression `{ [variableList] }`. In an aggregating context, only grouping sub-expressions and constants are allowed.

42I19

Status description

error: syntax error or access rule violation - invalid string literal. Failed to parse string literal. The query must contain an even number of non-escaped quotes.

42I20

Status description

error: syntax error or access rule violation - invalid symbol in expression. Label expressions and relationship type expressions cannot contain `{ [input] }`. To express a label disjunction use `{ [labelExpr] }` instead.

42I21

Status description

error: syntax error or access rule violation - invalid reference to variable out of scope. Not allowed to reference `{ [variableList] }` from within a parenthesized/quantified path pattern like `{ [pat] }` in the same graph pattern.

42I23

Status description

error: syntax error or access rule violation - invalid quantified path pattern in shortest path. The `{ [fun] }` function cannot contain a quantified path pattern.

42I24

Status description

error: syntax error or access rule violation - invalid use of aggregate function. Aggregate expression `{ [expr] }` is not allowed in this context.

42I25

Status description

error: syntax error or access rule violation - invalid use of `CALL IN TRANSACTIONS`. `CALL { … } IN TRANSACTIONS` is not supported after a write clause.

42I26

Status description

error: syntax error or access rule violation - invalid `DELETE`. `DELETE …` does not support removing labels from a node. Use `REMOVE …` instead.

42I27

Status description

error: syntax error or access rule violation - invalid use of `DISTINCT` with non-aggregate function. `DISTINCT` cannot be used with the `{ [fun] }` function.

42I28

Status description

error: syntax error or access rule violation - invalid use of importing `WITH`. Importing `WITH` can consist only of direct references to outside variables. `{ [input] }` is not allowed.

42I29

Status description

error: syntax error or access rule violation - invalid use of `IS`. The `IS` keyword cannot be used together with multiple labels in `{ [input] }`. Rewrite the expression as `{ [replacement] }`.

42I30

Status description

error: syntax error or access rule violation - invalid use of label expressions. Label expressions cannot be used in `{ [expr] }`.

42I31

Status description

error: syntax error or access rule violation - invalid use of `MATCH`. `MATCH …` cannot directly follow an `OPTIONAL MATCH …`. Use a `WITH` clause between them.

42I32

Status description

error: syntax error or access rule violation - invalid use of node and relationship pattern predicate. Node and relationship pattern predicates cannot be used in `{ [context] }`. They can only be used in a `MATCH` clause or inside a pattern comprehension.

42I33

Status description

error: syntax error or access rule violation - invalid use of `NOT NULL`. Closed Dynamic Union types cannot be appended with `NOT NULL`, specify `NOT NULL` on inner types instead.

42I34

Status description

error: syntax error or access rule violation - invalid use of pattern expression. A pattern expression can only be used to test the existence of a pattern. Use a pattern comprehension instead.

42I35

Status description

error: syntax error or access rule violation - invalid use of relationship type expression. Relationship type expressions can only be used in `MATCH …`.

42I36

Status description

error: syntax error or access rule violation - invalid use of `REPORT STATUS`. `REPORT STATUS` can only be used when specifying `ON ERROR CONTINUE` or `ON ERROR BREAK`.

42I37

Status description

error: syntax error or access rule violation - invalid use of `RETURN *`. `RETURN *` is not allowed when there are no variables in scope.

42I38

Status description

error: syntax error or access rule violation - invalid use of `RETURN`. `RETURN …` can only be used at the end of a query or subquery.

42I39

Status description

error: syntax error or access rule violation - invalid use of shortest path function. Mixing the `{ [fun] }` function with path selectors, explicit match modes, or explicit path modes is not allowed.

42I40

Status description

error: syntax error or access rule violation - invalid use of `UNION` and `UNION ALL`. `UNION` and `UNION ALL` cannot be combined.

42I41

Status description

error: syntax error or access rule violation - invalid use of variable length relationship. Variable length relationships cannot be used in `{ [value] }`.

42I42

Status description

error: syntax error or access rule violation - invalid use of `YIELD`. Cannot use `YIELD` on a call to a void procedure.

42I43

Status description

error: syntax error or access rule violation - invalid use of `YIELD *`. `YIELD *` can only be used with a standalone procedure call.

42I45

Status description

error: syntax error or access rule violation - invalid use of multiple path patterns. Multiple path patterns

cannot be used in the same clause in combination with a selective path selector. `{ [action] }`

42I46

Status description

error: syntax error or access rule violation - invalid use of a node pattern pair. Node pattern pairs are only supported for quantified path patterns.

42I47

Status description

error: syntax error or access rule violation - parser error. Parser Error: `{ [msg] }`.

42I48

Status description

error: syntax error or access rule violation - invalid use of a subquery in `MERGE`. Subqueries are not allowed in a `MERGE` clause.

42I49

Status description

error: syntax error or access rule violation - invalid inequality operator. Unknown inequality operator `!=`. The operator for inequality in Cypher is `<>`.

42I50

Status description

error: syntax error or access rule violation - token name too long. Invalid input `{ [input] }...`. A `{ [tokenType] }` name cannot be longer than `{ [value] }`.

42I51

Status description

info: invalid call signature. The procedure or function `{ [procFun] }` must have the signature: `{ [sig] }`.

42I52

Status description

error: syntax error or access rule violation - no longer valid syntax. `{ [msg] }`

42I53

Status description

error: syntax error or access rule violation - unsupported coordinate type. Unknown coordinate type: `{`

`[input] }`.

42I54

Status description

error: syntax error or access rule violation - invalid use of `INSERT`. `{ [cause] }` is not allowed in `INSERT`. Use `CREATE` or `{ [replacement] }`.

42I55

Status description

error: syntax error or access rule violation - invalid use of dynamic label or type. Dynamic `{ [tokenType] }` using `$any()` are not allowed in `{ [clause] }`.

42I56

Status description

error: syntax error or access rule violation - invalid relationship direction. Only directed relationships are supported in `{ [clause] }`.

42I57

Status description

error: syntax error or access rule violation - invalid query ending. `{ [exprType] }` cannot contain a query ending with `{ [clause] }`.

42I58

Status description

error: syntax error or access rule violation - invalid entity reference. Entity, `{ [expr] }`, cannot be created and referenced in the same clause.

42I59

Status description

error: syntax error or access rule violation - dynamic entity type not allowed. Dynamic label and types are only allowed in `{ [clauseList] }` clauses.

42I60

Status description

error: syntax error or access rule violation - invalid glob escaping. Each part of the glob (a block of text up until a dot) must either be fully escaped or not escaped at all.

42I61

Status description

error: syntax error or access rule violation - missing LOOKUP INDEX function name. Missing function name for the LOOKUP INDEX.

42I62

Status description

error: syntax error or access rule violation - unsupported distance metric. Unknown distance metric: `{ [input] }`.

42I63

Status description

error: syntax error or access rule violation - wrong subclause order. `ORDER BY`, `{ [clause] }` and `LIMIT` can only be used in this order in `RETURN`.

42I64

Status description

error: syntax error or access rule violation - unsupported operation. `{ [msg] }` currently not supported after `NEXT` `{ [context] }`.

42I65

Status description

error: syntax error or access rule violation - invalid pattern character. An invalid character is used in the pattern. Verify that all characters are supported by `{ [valueType] }`.

42I66

Status description

error: syntax error or access rule violation - pattern parsing failed. Pattern parsing failed. Make sure that an even number of escapes are used in the pattern.

42I67

Status description

error: syntax error or access rule violation - unsupported language feature. The query is parsable in `CYPHER { [version] 2 }`, but it is run in `CYPHER { [version] 1 }`. Consider changing the database default Cypher version using `ALTER DATABASE SET DEFAULT LANGUAGE` or prefix the query with `CYPHER { [feat] 2 }`.

42I68

Status description

error: syntax error or access rule violation - mismatched pattern. Pattern, `{ [input] 1 }`, does not match

input, `{ [input] 2 }`. Verify that the pattern is valid for constructing `{ [valueType] }`.

42I69

Status description

error: syntax error or access rule violation - invalid search variable reference. `{ [variable] }` must reference a variable from the same `MATCH` statement.

42I70

Status description

error: syntax error or access rule violation - search clause with multiple bound variables. In order to have a search clause, a `MATCH` statement can only have one bound variable.

42I71

Status description

error: syntax error or access rule violation - search clause with invalid predicates. In order to have a search clause, a `MATCH` statement can only have predicates on the bound node or relationship.

42I72

Status description

error: syntax error or access rule violation - search clause with too complex pattern. In order to have a search clause, a `MATCH` statement can only have a single node or relationship pattern and no selectors.

42I73

Status description

error: syntax error or access rule violation - invalid predicate for vector search with filters. The vector search filter predicate `{ [expr] }` must consist of one or more property predicates joined by `AND`, and the combined property predicates for each property must specify either an exact value (e.g. `x.prop = 1`), a half-bounded range (e.g. `x.prop >= 1`), or a bounded range (e.g. `x.prop > 1 AND x.prop < 100`). Note that this is not an exhaustive list of valid predicates, see documentation for all rules.

42I74

Status description

error: syntax error or access rule violation - wrong variable for vector search with filters. The variable `{ [variable] 1 }` in a vector search filter property predicate must be the same as the search clause binding variable `{ [variable] 2 }`.

42I75

Status description

error: syntax error or access rule violation - self-referencing in search. The expression `{ [expr] }` in the

search clause may not depend on the search clause binding variable `{ [variable] }`.

42I76

Status description

error: syntax error or access rule violation - index or constraint value too long. The provided index or constraint `{ [item] }` `{ [input] }` (`{ [bytes] 1 }` bytes) exceeded limit of `{ [bytes] 2 }` bytes.

42I78

Status description

error: syntax error or access rule violation - unsupported procedure or function in language version. The procedure or function is available in `CYPHER { [version] }`. Consider changing the database default Cypher version using `ALTER DATABASE SET DEFAULT LANGUAGE` or prefix the query with `CYPHER { [version] }`.

42I79

Status description

error: syntax error or access rule violation - invalid reference in subclause expression. Aggregation in subclause expression is not allowed to reference variables declared in the same clause `{ [argList] }`.

42I80

Status description

error: syntax error or access rule violation - invalid grouping element. The grouping element `{ [expr] }` is not a valid grouping key. `{ [cause] }`

42N00

Status description

error: syntax error or access rule violation - graph reference not found. A graph reference with the name `{ [db] }` was not found. Verify that the spelling is correct.

42N01

Status description

error: syntax error or access rule violation - no such constituent graph exists in composite database. The constituent graph `{ [graph] }` was not found in the in composite database `{ [db] }`. Verify that the spelling is correct.

42N02

Status description

error: syntax error or access rule violation - writing in read access mode. Writing in read access mode not allowed.

42N03

Status description

error: syntax error or access rule violation - writing to multiple graphs. Writing to multiple graphs in the same transaction is not allowed. Use `CALL IN TRANSACTION` or create separate transactions in your application.

42N04

Status description

error: syntax error or access rule violation - unsupported access of composite database. Failed to access database identified by `{ [db] 1 }` while connected to session database `{ $db2 }`. Connect to `{ $db3 }` directly.

42N05

Status description

error: syntax error or access rule violation - unsupported access of standard database. Failed to access database identified by `{ [db] 1 }` while connected to composite session database `{ $db2 }`. Connect to `{ $db3 }` directly or create an alias in the composite database.

42N06

Status description

error: syntax error or access rule violation - unsupported action on composite database. `{ [action] }` is not supported on composite databases.

42N07

Status description

error: syntax error or access rule violation - variable shadowing. The variable `{ [variable] }` is shadowing a variable with the same name from the outer scope and needs to be renamed.

42N08

Status description

error: syntax error or access rule violation - no such procedure. The procedure `{ [procFun] }` was not found. Verify that the spelling is correct.

42N09

Status description

error: syntax error or access rule violation - no such user. A user with the name `{ [user] }` was not found. Verify that the spelling is correct.

42N0A

Status description

error: syntax error or access rule violation - invalid shard target. `{ [action] }` is not allowed with a shard target. Connect to `{ [db] 1 }` in order to read or write data from `{ db2 }`.

42N0B

Status description

error: syntax error or access rule violation - cannot replace sharded database. The database identified by `{ [db] 1 }` is sharded. Drop the database `{ [db] 2 }` before recreating.

42N0D

Status description

error: syntax error or access rule violation - cannot call function from this context. The function `{ [fun] }` cannot be called from the current context. It can only be used `{ [context] }`.

42N0E

Status description

error: syntax error or access rule violation - cannot call function without metadata. The function `{ [fun] }` cannot be called without metadata.

42N0F

Status description

error: syntax error or access rule violation - cannot call function without metadata for realm. The function `{ [fun] }` cannot be called without metadata for realm: `{ [auth] }`.

42N10

Status description

error: syntax error or access rule violation - no such role. A role with the name `{ [role] }` was not found. Verify that the spelling is correct.

42N11

Status description

error: syntax error or access rule violation - graph reference already exists. A graph reference with the name `{ [dbList] }` already exists.

42N12

Status description

error: syntax error or access rule violation - user already exists. A user with the name `{ [user] }` already exists.

42N13

Status description

error: syntax error or access rule violation - role already exists. A role with the name `{ [role] }` already exists.

42N14

Status description

error: syntax error or access rule violation - invalid use of command. `{ [clause] }` cannot be used together with `{ [cmd] }`.

42N15

Status description

error: syntax error or access rule violation - invalid use of reserved keyword. `{ [syntax] }` is a reserved keyword and cannot be used in this place.

42N16

Status description

error: syntax error or access rule violation - unsupported index or constraint. Only single property `{ [idxType] }` are supported.

42N17

Status description

error: syntax error or access rule violation - unsupported request. `{ [input] }` is not allowed on the `system` database.

42N18

Status description

error: syntax error or access rule violation - read-only database. The database is in read-only mode.

42N19

Status description

error: syntax error or access rule violation - duplicate clause. Duplicate `{ [syntax] }` clause.

42N20

Status description

error: syntax error or access rule violation - empty list range operator. The list range operator `[ ]` cannot be empty.

42N21

Status description

error: syntax error or access rule violation - unaliased return item. Expression in `{ [clause] }` must be aliased (use AS).

42N22

Status description

error: syntax error or access rule violation - single return column required. A `COLLECT` subquery must end with a single return column.

42N23

Status description

error: syntax error or access rule violation - missing reference to aggregation function. The aggregating function must be included in the `{ [clause] }` clause for use in `ORDER BY …`.

42N24

Status description

error: syntax error or access rule violation - missing `WITH`. A `WITH` clause is required between `{ [input] 1 }` and `{ $input2 }`.

42N25

Status description

error: syntax error or access rule violation - missing `YIELD`. Procedure call inside a query does not support naming results implicitly. Use `YIELD` instead. Available output columns are: `{ [variableList] }`.

42N26

Status description

error: syntax error or access rule violation - multiple join hints on same variable. Multiple join hints for the same variable `{ [variable] }` are not supported.

42N28

Status description

error: syntax error or access rule violation - patterns or variables not statically inferrable. Only statically inferrable patterns and variables are allowed in `{ [input] }`.

42N29

Status description

error: syntax error or access rule violation - unbound variables in pattern expression. Pattern expressions are not allowed to introduce new variables: `{ [variable] }`.

42N31

Status description

error: syntax error or access rule violation - specified number out of range. Expected `{ [component] }` to be `{ [valueType] }` in the range `{ [lower] }` to `{ [upper] }` but found `{ [value] }`.

42N32

Status description

error: syntax error or access rule violation - invalid use of parameter map. Parameter maps cannot be used in `{ [keyword] }` patterns. Use a literal map instead.

42N34

Status description

error: syntax error or access rule violation - path bound in quantified path pattern. Path cannot be bound in a quantified path pattern.

42N35

Status description

error: syntax error or access rule violation - unsupported path selector or explicit path mode in path pattern. The path selector or explicit path mode `{ [selectorOrPathMode] }` is not supported within quantified or parenthesized path patterns.

42N36

Status description

error: syntax error or access rule violation - procedure call without parentheses. Procedure call is missing parentheses.

42N37

Status description

error: syntax error or access rule violation - invalid use of relationship pattern predicates in variable length relationships. Relationship pattern predicates cannot be use in variable length relationships.

42N38

Status description

error: syntax error or access rule violation - duplicate return item name. Return items must have unique names.

42N39

Status description

error: syntax error or access rule violation - incompatible return column names. All `{ [context] }` must have the same return column names. Use `AS` to ensure columns have the same name.

42N3A

Status description

error: syntax error or access rule violation - incompatible conditional query. All `{ [context] }` need to either return rows or update the graph.

42N3B

Status description

error: syntax error or access rule violation - incompatible number of return columns. All `{ [context] }` must return the same number of columns.

42N3C

Status description

error: syntax error or access rule violation - invalid use of `CALL { … }`. Not possible to enclose `{ [clause] }` in `CALL { … }`. Use `CALL () { … }` instead.

42N40

Status description

error: syntax error or access rule violation - single relationship pattern required. The `{ [fun] }` function must contain one relationship pattern.

42N41

Status description

error: syntax error or access rule violation - missing `|-expression`. The reduce function requires a `| expression` after the accumulator.

42N42

Status description

error: syntax error or access rule violation - unsupported sub-path binding. Sub-path assignment is not supported.

42N44

Status description

error: syntax error or access rule violation - inaccessible variable. It is not possible to access the variable

`{ [variable] }` declared before the `{ [clause] }` clause when using `{ [groupingConstructs] }`.

42N45

Status description

error: syntax error or access rule violation - unexpected end of input. Unexpected end of input, expected `CYPHER`, `EXPLAIN`, `PROFILE` or a query.

42N47

Status description

error: syntax error or access rule violation - invalid use of `UNION` and `CALL IN TRANSACTIONS`. `CALL { … } IN TRANSACTIONS` is not supported in `… UNION …`.

42N48

Status description

error: syntax error or access rule violation - no such function. The function `{ [fun] }` was not found. Verify that the spelling is correct.

42N49

Status description

error: syntax error or access rule violation - unsupported normal form. Unknown Normal Form: `{ [input] }`.

42N50

Status description

error: syntax error or access rule violation - procedure return column not defined. The procedure return column `{ [arg] }` is not defined for this procedure. Available output columns are: `{ [argList] }`. Verify that the spelling is correct.

42N51

Status description

error: syntax error or access rule violation - invalid parameter. Invalid parameter `{ [param] }`.

42N52

Status description

error: syntax error or access rule violation - invalid value type. `{ [input] }` is not a recognized Cypher type.

42N53

Status description

error: syntax error or access rule violation - unsafe usage of repeatable elements. The quantified path pattern may yield an infinite number of rows under match mode `REPEATABLE ELEMENTS`. Add an upper bound to the quantified path pattern.

42N54

Status description

error: syntax error or access rule violation - unsupported match mode. The match mode `{ [matchMode] }` is not supported.

42N56

Status description

error: syntax error or access rule violation - unsupported use of properties. Properties are not supported in the `{ [fun] }` function.

42N57

Status description

error: syntax error or access rule violation - invalid use of data-modifications in expressions. `{ [expr] }` cannot contain any updating clauses.

42N58

Status description

error: syntax error or access rule violation - unsupported use of nesting. Nested `CALL { … } IN TRANSACTIONS` is not supported.

42N59

Status description

error: syntax error or access rule violation - variable already defined. Variable `{ [variable] }` already declared.

42N60

Status description

error: syntax error or access rule violation - unsupported combination of match mode and path mode. `REPEATABLE ELEMENTS` with `{ [pathMode] }` path mode is not supported.

42N61

Status description

error: syntax error or access rule violation - unsupported mixing of different path modes. Mixing `{ [pathModeList] }` in the same graph pattern is not supported. Split the pattern into separate `MATCH`

clauses instead.

42N62

Status description

error: syntax error or access rule violation - variable not defined. Variable `{ [variable] }` not defined.

42N63

Status description

error: syntax error or access rule violation - inner type with different nullability. All inner types in a Closed Dynamic Union must be nullable, or be appended with `NOT NULL`.

42N64

Status description

error: syntax error or access rule violation - at least one node or relationship required. A quantified or parenthesized path pattern must have at least one node or relationship pattern.

42N65

Status description

error: syntax error or access rule violation - node variable not bound. The `{ [fun] }` function requires bound node variables when it is not part of a `MATCH …`.

42N66

Status description

error: syntax error or access rule violation - relationship variable already bound. Bound relationships are not allowed in calls to the `{ [fun] }` function.

42N68

Status description

error: syntax error or access rule violation - duplicate variable definition. Variables cannot be defined more than once in a `{ [clause] }` clause.

42N69

Status description

error: syntax error or access rule violation - function not allowed inside expression. The `{ [fun] }` function is only allowed as a top-level element and not inside an `{ [expr] }`.

42N70

Status description

error: syntax error or access rule violation - function without required WHERE clause. The function `{ [fun] }` requires a WHERE clause.

42N71

Status description

error: syntax error or access rule violation - incomplete query. A query must conclude with a `RETURN` clause, a `FINISH` clause, an update clause, a unit subquery call, or a procedure call without a `YIELD` clause.

42N72

Status description

error: syntax error or access rule violation - graph function only supported on composite databases. Calling graph functions is only supported on composite databases. Use the name directly or connect to a composite database with the desired constituents.

42N73

Status description

error: syntax error or access rule violation - invalid placement of `USE` clause. The `USE` clause must be the first clause of a query or an operand to `… UNION …` . In a `CALL` sub-query, it can also be the second clause if the first clause is an importing `WITH`.

42N74

Status description

error: syntax error or access rule violation - invalid nested USE clause. Failed to access `{ [db] 1 }` and `{ $db2 }`. Child USE clauses must target the same graph as their parent query. Run in separate (sub)queries instead.

42N75

Status description

error: syntax error or access rule violation - invalid use of graph function. A call to the graph function `{ [fun] }` is only allowed as the top-level argument of a `USE` clause.

42N76

Status description

error: syntax error or access rule violation - unfulfillable hints. The hint(s) `{ [hintList] }` cannot be fulfilled.

42N77

Status description

error: syntax error or access rule violation - missing hint predicate. The hint `{ [hint] }` cannot be fulfilled. The query does not contain a compatible predicate for `{ [entityType] }` on `{ [variable] }`.

42N78

Status description

error: syntax error or access rule violation - variable already bound. Node `{ [variable] }` has already been bound and cannot be modified by the `{ [clause] }` clause.

42N79

Status description

error: syntax error or access rule violation - invalid `USE` clause. The `USE` clause is not required for administration commands. Retry the query without the `USE` clause, and it will be routed automatically.

42N7A

Status description

error: syntax error or access rule violation - disjoint by requires concurrent transactions. `'DISJOINT BY'` can only be used in `'CALL { … } IN CONCURRENT TRANSACTIONS'`.

42N7B

Status description

error: syntax error or access rule violation - non-deterministic disjoint by expression. `'DISJOINT BY'` expressions must be deterministic.

42N7C

Status description

error: syntax error or access rule violation - subquery in disjoint by expression. `'DISJOINT BY'` expressions must not contain subquery expressions.

42N81

Status description

error: syntax error or access rule violation - missing request parameter. Expected `{ [paramList] 1 }`, but got `{ [paramList] 2 }`.

42N82

Status description

error: syntax error or access rule violation - cannot drop database with aliases. The database identified by `{ [db] }` has one or more aliases. Drop the aliases `{ [aliasList] }` before dropping the database.

42N83

Status description

error: syntax error or access rule violation - impersonation disallowed while password change required. Cannot impersonate a user while password change required.

42N84

Status description

error: syntax error or access rule violation - `TERMINATE TRANSACTION` misses `YIELD` clause. `WHERE` clause without `YIELD` clause. Use `TERMINATE TRANSACTION … YIELD … WHERE …`.

42N85

Status description

error: syntax error or access rule violation - cannot specify both allowed and denied databases. Allowed and denied database options are mutually exclusive.

42N86

Status description

error: syntax error or access rule violation - wildcard in parameter. `{ [syntax] }` failed. Parameterized database and graph names do not support wildcards.

42N87

Status description

error: syntax error or access rule violation - database or alias with similar name exists. The database or alias name `{ [db] 1 }` conflicts with the name `{ [db] 2 }` of an existing database or alias.

42N88

Status description

error: syntax error or access rule violation - cannot grant privilege. Permission cannot be granted for `REMOVE IMMUTABLE PRIVILEGE`.

42N89

Status description

error: syntax error or access rule violation - invalid driver settings map. Failed evaluating the given driver settings. `{ [cause] }`

42N90

Status description

error: syntax error or access rule violation - cannot alter immutable composite database. Composite

databases cannot be altered (database: `{ [db] }`).

42N92

Status description

error: syntax error or access rule violation - cannot combine old and new auth provider syntax. Cannot combine old and new auth syntax for the same auth provider.

42N94

Status description

error: syntax error or access rule violation - incomplete `ALTER USER` command. `'ALTER USER'` requires at least one clause.

42N95

Status description

error: syntax error or access rule violation - provider-id combination already exists. The combination of provider and id is already in use.

42N96

Status description

error: syntax error or access rule violation - invalid user configuration. User has no auth provider. Add at least one auth provider for the user or consider suspending them.

42N97

Status description

error: syntax error or access rule violation - missing mandatory auth clause. Clause `{ [clause] }` is mandatory for auth provider `{ [auth] }`.

42N98

Status description

error: syntax error or access rule violation - cannot modify own user. Cannot modify the user record of the current user.

42N99

Status description

error: syntax error or access rule violation - cannot delete own user. Cannot delete the user record of the current user.

42NA0

Status description

error: syntax error or access rule violation - operations must be executed on constituent. Query contains operations that must be executed on the constituent.

42NA1

Status description

error: syntax error or access rule violation - graph access operations on composite database. Graph access operations are not supported on composite databases.

42NA2

Status description

error: syntax error or access rule violation - database operations on composite database. Database operations are not supported on composite databases.

42NA3

Status description

error: syntax error or access rule violation - schema operations on composite database. Schema operations are not supported on composite databases.

42NA4

Status description

error: syntax error or access rule violation - transaction operations on composite database. Transaction operations are not supported on composite databases.

42NA5

Status description

error: syntax error or access rule violation - accessing multiple graphs only supported on composite databases. Accessing multiple graphs in the same query is only supported on composite databases. Connect to a composite database with the desired constituents.

42NA6

Status description

error: syntax error or access rule violation - invalid alias target. Aliases are not allowed to target composite databases.

42NA7

Status description

error: syntax error or access rule violation - referenced database not found. No database is corresponding to `{ [db] }`. Verify that the elementId is correct.

42NA8

Status description

error: syntax error or access rule violation - invalid reference in command. Invalid reference in command `{ [cmd] }`.

42NA9

Status description

error: syntax error or access rule violation - system database rules. The system database supports a restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is allowed), a subset of show and terminate commands, and combinations of the two. `YIELD` and `RETURN` are also permitted when combined with procedure calls, show, or terminate commands.

42NAA

Status description

error: syntax error or access rule violation - incorrectly formatted graph reference. Incorrectly formatted graph reference `{ [input] }`. Expected a single quoted or unquoted identifier. Separate name parts should not be quoted individually.

42NAB

Status description

error: syntax error or access rule violation - not supported standalone call. `WHERE` is not supported in a standalone call. Use `CALL … YIELD … WHERE … RETURN …` instead.

42NAC

Status description

error: syntax error or access rule violation - invalid `CREATE DATABASE` statement. The backup metadata script contains an invalid or missing `CREATE DATABASE` statement. The metadata script must contain exactly one `CREATE DATABASE` statement and it must use the parameter `$database`.

42NAD

Status description

error: syntax error or access rule violation - no such auth rule. An auth rule with the name `{ [authRule] }` was not found. Verify that the spelling is correct.

42NAE

Status description

error: syntax error or access rule violation - auth rule already exists. An auth rule with the name `{ [authRule] }` already exists.

42NAM

Status description

error: syntax error or access rule violation - unsupported temporal function form in auth rule condition. `{ [input] }` cannot be used in auth rule conditions as it retrieves the current time. Only transaction start time is available at the time of auth rule evaluation. Use `{ [replacement] }` instead.

42NAP

Status description

error: syntax error or access rule violation - unsupported trim specification. Unknown trim specification: `{ [input] }`.

42NFC

Status description

error: syntax error or access rule violation - auth info validation error. Authentication and/or authorization could not be validated. See security logs for details.

42NFD

Status description

error: syntax error or access rule violation - credentials expired. Permission denied. The credentials you provided were valid, but must be changed before you can use this instance.

42NFE

Status description

error: syntax error or access rule violation - auth info expired. Authentication and/or authorization info expired.

42NFF

Status description

error: syntax error or access rule violation - permission/access denied. Access denied, see the security logs for details.

General processing exceptions

General processing exceptions occur when there is a general processing error, such as an internal error, deadlock, execution failure, invalid server state transition, constraint creation or drop failure, etc.

50N00

Status description

error: general processing exception - internal error. Internal exception raised `{ [msgTitle] }`: `{ [msg] }`

50N05

Status description

error: general processing exception - deadlock detected. Deadlock detected while trying to acquire locks. See log for more details.

50N06

Status description

error: general processing exception - remote execution client error. Remote execution failed. See cause for more details.

50N07

Status description

error: general processing exception - execution failed. Execution failed. See cause and debug log for details.

50N09

Status description

error: general processing exception - invalid server state transition. The server transitioned into a server state that is not valid in the current context: `{ [boltServerState] }`.

50N10

Status description

error: general processing exception - index drop failed. Unable to drop `{ [idxDescrOrName] }`.

50N11

Status description

error: general processing exception - constraint creation failed. Unable to create `{ [constrDescrOrName] }`.

50N12

Status description

error: general processing exception - constraint drop failed. Unable to drop `{ [constrDescrOrName] }`.

50N13

Status description

error: general processing exception - constraint validation error. Unable to validate constraint `{ [constrDescrOrName] }`.

50N14

Status description

error: general processing exception - constraint violation. A constraint imposed by the database was violated.

50N15

Status description

error: general processing exception - unsupported index operation. The system attemped to execute an unsupported operation on index `{ [idx] }`. See debug.log for more information.

50N16

Status description

error: general processing exception - remote execution transient error. Remote execution failed. See cause for more details.

50N17

Status description

error: general processing exception - remote execution database error. Remote execution failed. See cause for more details.

50N18

Status description

error: general processing exception - shard execution transient error. Communication with shard `{ [graph] }` failed. See cause for more details.

50N19

Status description

error: general processing exception - shard execution database error. Communication with shard `{ [graph] }` failed. See cause for more details.

50N20

Status description

error: general processing exception - shard execution client error. Communication with shard `{ [graph] }` failed. See cause for more details.

50N21

Status description

error: general processing exception - no such schema descriptor. The `{ [schemaDescrType] }` was not

found for `{ [schemaDescr] }`. Verify that the spelling is correct.

50N23

Status description

error: general processing exception - transaction retry aborted. Transaction retry aborted after `{ [count] }` attempts. Retry timed out with a maximum retry duration of `{ [timeAmount] }` `{ [timeUnit] }`.

50N24

Status description

error: general processing exception - sharded properties transaction handling database error. Unexpected exception while getting transaction state.

50N25

Status description

error: general processing exception - sharded properties transaction handling client error. Unexpected exception while getting transaction state.

50N26

Status description

error: general processing exception - invalid backup metadata script. The backup metadata script contains invalid syntax.

50N42

Status description

error: general processing exception - unexpected error. Unexpected error has occurred. See debug log for details.

System configuration or operation exceptions

System configuration or operation exception errors occur when there is an error in the system configuration or operation, such as procedure registration failure, a missing class field annotation, an unsupported injectable component type, duplicate field names, invalid map key type, etc.

51N00

Status description

error: system configuration or operation exception - procedure registration error. Failed to register procedure/function.

51N01

Status description

error: system configuration or operation exception - class field annotation should be public, non-final, and non-static. The field `{ [procField] }` in the class `{ [procClass] }` is annotated as a `@Context` field, but it is declared as static. `@Context` fields must be public, non-final and non-static.

51N02

Status description

error: system configuration or operation exception - unsupported injectable component type. Unable to set up injection for procedure `{ [procClass] }`. The field `{ [procField] }` has type `{ [procFieldType] }` which is not a supported injectable component.

51N03

Status description

error: system configuration or operation exception - unable to access field. Unable to set up injection for `{ [procClass] }`, failed to access field `{ [procField] }`.

51N04

Status description

error: system configuration or operation exception - missing class field annotation. The field `{ [procField] }` on `{ [procClass] }` must be annotated as a `@Context` field in order to store its state.

51N05

Status description

error: system configuration or operation exception - class field should be public and non-final. The field `{ [procField] }` on `{ [procClass] }` must be declared non-final and public.

51N06

Status description

error: system configuration or operation exception - missing argument name. The argument at position `{ [pos] }` in `{ [procMethod] }` requires a `@Name` annotation and a non-empty name.

51N07

Status description

error: system configuration or operation exception - invalid ordering of default arguments. The `{ [procFun] }` contains a non-default argument after a default argument. Non-default arguments are not allowed to be positioned after default arguments.

51N08

Status description

error: system configuration or operation exception - exactly one @UserAggregationResult method and one @UserAggregationUpdate method required. The class `{ [procClass] }` must contain exactly one `@UserAggregationResult` method and exactly one `@UserAggregationUpdate` method.

51N09

Status description

error: system configuration or operation exception - @UserAggregationUpdate method must be public and void. The `@UserAggregationUpdate` method `{ [procMethod] }` of `{ [procClass] }` must be public and have the return type `void`.

51N10

Status description

error: system configuration or operation exception - aggregation method not public. The method `{ [procMethod] }` of `{ [procClass] }` must be public.

51N11

Status description

error: system configuration or operation exception - class not public. The class `{ [procClass] }` must be public.

51N12

Status description

error: system configuration or operation exception - class not void. The procedure `{ [proc] }` has zero return columns and must be defined as void.

51N13

Status description

error: system configuration or operation exception - procedure or function name already in use. Unable to register the procedure or function `{ [procFun] }` because the name is already in use.

51N14

Status description

error: system configuration or operation exception - duplicate field name. The procedure `{ [proc] }` has a duplicate `{ [procFieldType] }` field, `{ [procField] }`.

51N15

Status description

error: system configuration or operation exception - invalid map key type. Type mismatch for map key.

Required `STRING`, but found `{ [valueType] }`.

51N16

Status description

error: system configuration or operation exception - invalid default value type. Type mismatch for the default value. Required `{ [valueType] }`, but found `{ [input] }`.

51N17

Status description

error: system configuration or operation exception - invalid procedure or function name. Procedures and functions cannot be defined in the root namespace, or use a reserved namespace. Use the package name instead (e.g., org.example.com.`{ [procFun] }`)`.

51N18

Status description

error: system configuration or operation exception - invalid method return type. The method `{ [procMethod] }` has an invalid return type. Procedures must return a stream of records, where each record is of a defined concrete class.

51N20

Status description

error: system configuration or operation exception - cannot inject field. The field `{ [procField] }` is not injectable. Ensure the field is marked as public and non-final.

51N21

Status description

error: system configuration or operation exception - procedure registry is busy. The procedure registration failed because the procedure registry was busy. Try again.

51N22

Status description

error: system configuration or operation exception - exhaustive shortest path search disabled. Finding the shortest path for the given pattern requires an exhaustive search. To enable exhaustive searches, set `cypher.forbid_exhaustive_shortestpath` to false.

51N23

Status description

error: system configuration or operation exception - cyclic shortest path search disabled. Cannot find the shortest path when the start and end nodes are the same. To enable this behavior, set

`dbms.cypher.forbid_shortestpath_common_nodes` to false.

51N24

Status description

error: system configuration or operation exception - insufficient resources for plan search. Could not find a query plan within given time and space limits.

51N25

Status description

error: system configuration or operation exception - database is busy. Cannot compile query due to excessive updates to indexes and constraints.

51N26

Status description

error: system configuration or operation exception - not supported in this version. `{ [item] }` is not available. This implementation of Cypher does not support `{ [feat] }`.

51N27

Status description

error: system configuration or operation exception - not supported in this edition. `{ [feat] }` is not supported in `{ [edition] }`.

51N28

Status description

error: system configuration or operation exception - not supported by this database. This Cypher command must be executed against the database `{ [db] }`.

51N29

Status description

error: system configuration or operation exception - not supported by this server. The command `{ [cmd] }` must be executed on the current `LEADER` server.

51N2A

Status description

error: system configuration or operation exception - not supported with auth disabled. The command `{ [cmd] }` is not available with auth disabled.

51N30

Status description

error: system configuration or operation exception - not supported with this configuration. `{ [item] }` is not supported in `{ [context] }`.

51N31

Status description

error: system configuration or operation exception - not supported. `{ [feat] }` is not supported in `{ [context] }`.

51N32

Status description

error: system configuration or operation exception - server panic. Server is in panic.

51N33

Status description

error: system configuration or operation exception - replication error. This member failed to replicate transaction, try again.

51N34

Status description

error: system configuration or operation exception - write transaction failed due to leader change. Failed to write to the database due to a cluster leader change. Retrying your request at a later time may succeed.

51N35

Status description

error: system configuration or operation exception - database location changed. The location of `{ [db] }` has changed while the transaction was running.

51N36

Status description

error: system configuration or operation exception - out of memory. There is not enough memory to perform the current task.

51N37

Status description

error: system configuration or operation exception - stack overflow. There is not enough stack size to perform the current task.

51N38

Status description

error: system configuration or operation exception - failed to acquire execution thread. There are insufficient threads available for executing the current task.

51N39

Status description

error: system configuration or operation exception - raft log corrupted. Expected set of files not found on disk. Please restore from backup.

51N40

Status description

error: system configuration or operation exception - unable to start database. Database `{ [db] }` failed to start. Try restarting it.

51N41

Status description

error: system configuration or operation exception - admin operation failed. Server or database admin operation not possible. Reason: `{ [msg] }`

51N43

Status description

error: system configuration or operation exception - cannot deallocate servers. Cannot deallocate server(s) `{ [serverList] }`.

51N44

Status description

error: system configuration or operation exception - cannot drop server. Cannot drop server `{ [server] }`.

51N45

Status description

error: system configuration or operation exception - cannot cordon server. Cannot cordon server `{ [server] }`.

51N46

Status description

error: system configuration or operation exception - cannot alter server. Cannot alter server `{ [server]`

`}`.

51N47

Status description

error: system configuration or operation exception - cannot rename server. Cannot rename server `{ [server] }`.

51N48

Status description

error: system configuration or operation exception - cannot enable server. Cannot enable server `{ [server] }`.

51N49

Status description

error: system configuration or operation exception - cannot alter database. Cannot alter database `{ [db] }`.

51N50

Status description

error: system configuration or operation exception - cannot recreate database. Cannot recreate database `{ [db] }`.

51N51

Status description

error: system configuration or operation exception - cannot create database. Cannot create database `{ [db] }`.

51N52

Status description

error: system configuration or operation exception - number of primaries out of range. Cannot `{ [action] }` database topology. Number of primaries `{ [count] }` needs to be at least 1 and may not exceed `{ [upper] }`.

51N53

Status description

error: system configuration or operation exception - number of secondaries out of range. Cannot `{ [action] }` database topology. Number of secondaries `{ [count] }` needs to be at least 0 and may not exceed `{ [upper] }`.

51N54

Status description

error: system configuration or operation exception - cannot reallocate. Failed to calculate reallocation for databases. `{ [msg] }`

51N55

Status description

error: system configuration or operation exception - cannot create additional database. Failed to create the database `{ [db] }`. The limit of databases is reached. Either increase the limit using the config setting `{ [cfgSetting] }` or drop a database.

51N56

Status description

error: system configuration or operation exception - topology out of range. The number of `{ [serverType] }` seeding servers `{ [count] 1 }` is larger than the desired number of `{ [allocType] }` allocations `{ $count2 }`.

51N57

Status description

error: system configuration or operation exception - generic topology modification error. Unexpected error while picking allocations. `{ [msg] }`

51N58

Status description

error: system configuration or operation exception - invalid shard topology. Invalid database shard topology. The number of `{ [context] }` `{ [count] }` needs to be at least 1 and may not exceed `{ [upper] }`.

51N59

Status description

error: system configuration or operation exception - internal resource exhaustion. The DBMS is unable to handle the request, please retry later or contact the system operator. More information is present in the logs.

51N60

Status description

error: system configuration or operation exception - unable to check enterprise license acceptance. The DBMS is unable to determine the enterprise license acceptance status.

51N61

Status description

error: system configuration or operation exception - index population failed. Index `{ [idx] }` population failed.

51N62

Status description

error: system configuration or operation exception - index is in a failed state. Unable to use index `{ [idx] }` because it is in a failed state. See logs for more information.

51N63

Status description

error: system configuration or operation exception - index is still populating. Index `{ [idx] }` is not ready yet. Wait until it finishes populating and retry the transaction.

51N64

Status description

error: system configuration or operation exception - index dropped while sampling. The index dropped while sampling.

51N65

Status description

error: system configuration or operation exception - vector index dimensionality mismatch. Vector index `{ [idx] }` has a configured dimensionality of `{ [dim] 1 }`, but the provided vector has a dimension `{ [dim] 2 }`.

51N66

Status description

error: system configuration or operation exception - resource exhaustion. Insufficient resources to complete the request.

51N67

Status description

error: system configuration or operation exception - invalid CDC selector type. Unexpected CDC selector `{ [selectorType] 1 }` at `{ [input] }`, expected selector to be a `{ [selectorType] 2 }` selector.

51N68

Status description

error: system configuration or operation exception - CDC is disabled for this database. Change Data Capture is not currently enabled for this database.

51N69

Status description

error: system configuration or operation exception - system database is immutable. It is not possible to perform `{ [operation] }` on the system database.

51N70

Status description

error: system configuration or operation exception - bolt is not enabled. Cannot get routing table for `{ [db] }` because Bolt is not enabled. Please update your configuration such that `server.bolt.enabled` is set to true.

51N71

Status description

error: system configuration or operation exception - unsupported operation of a sharded database. Feature: `{ [feat] }` is not available in a sharded database.

51N72

Status description

error: system configuration or operation exception - memory pool out of memory. Failed to allocate memory in a memory pool. See `{ [cfgSetting] }` in the neo4j.conf file.

51N73

Status description

error: system configuration or operation exception - transaction memory limit reached. The transaction uses more memory than is allowed. The maximum allowed size for a transaction can be configured with `{ [cfgSetting] }` in the neo4j.conf file.

51N74

Status description

error: system configuration or operation exception - maximum number of transactions reached. Failed to start a new transaction. The limit of concurrent transactions is reached. Increase the number of concurrent transactions using `{ [cfgSetting] }` in the neo4j.conf file.

51N76

Status description

error: system configuration or operation exception - upgrade failed. The upgrade to a new Neo4j version failed.

51N77

Status description

error: system configuration or operation exception - not supported in this store format. `{ [feat] }` is not supported in `{ [storeFormat] }` store format.

51N7A

Status description

error: system configuration or operation exception - no admin user candidate. No admin user candidate is found. Use `neo4j-admin dbms set-default-admin` to select a valid admin.

Procedure exceptions

Procedure exceptions occur when there is an error in executing a procedure, such as when the procedure execution fails due to a client error, when the procedure cannot be invoked on a primary, when the number of arguments to checkConnectivity is invalid, etc.

52N01

Status description

error: procedure exception - procedure execution timeout. Execution of the procedure `{ [proc] }` timed out after `{ [timeAmount] }` `{ [timeUnit] }`.

52N02

Status description

error: procedure exception - procedure execution client error. Execution of the procedure `{ [proc] }` failed due to a client error.

52N03

Status description

error: procedure exception - invalid procedure execution mode. Execution of the procedure `{ [proc] }` failed due to an invalid specified execution mode `{ [procExeMode] }`.

52N05

Status description

error: procedure exception - must invoke procedure on secondary. Cannot invoke procedure on this member because it is not a secondary for the database `{ [db] }`.

52N06

Status description

error: procedure exception - invalid number of arguments to checkConnectivity. Unexpected number of arguments (expected 0-2 but received `{ [count] }`).

52N07

Status description

error: procedure exception - invalid port argument to checkConnectivity. Unrecognised port name `{ [port] }` (valid values are: `{ [portList] }`.

52N08

Status description

error: procedure exception - invalid server id argument to checkConnectivity. Unable to parse server id `{ [server] }`.

52N09

Status description

error: procedure exception - procedure execution database error. Execution of the procedure `{ [proc] }` failed due to a database error.

52N10

Status description

error: procedure exception - invalid address key. An address key is included in the query string provided to the GetRoutingTableProcedure, but its value could not be parsed.

52N11

Status description

error: procedure exception - generic topology procedure error. An unexpected error has occurred: `{ [msg] }`.

52N12

Status description

error: procedure exception - cannot change default database. The previous default database `{ [db] }` is still running.

52N13

Status description

error: procedure exception - new default database does not exist. New default database `{ [db] }` does

not exist.

52N14

Status description

error: procedure exception - system cannot be default database. System database cannot be set as default.

52N16

Status description

error: procedure exception - invalid procedure argument list. Invalid arguments to procedure.

52N17

Status description

error: procedure exception - quarantine change failed. Setting/removing the quarantine marker failed: `{ [msg] }`.

52N18

Status description

error: procedure exception - too many seeders. The number of seeding servers `{ [countSeeders] }` is larger than the defined number of allocations `{ [countAllocs] }`.

52N19

Status description

error: procedure exception - no such seeder. The specified seeding server with id `{ [server] }` was not found. Verify that the spelling is correct.

52N21

Status description

error: procedure exception - failed to clean the system graph. Failed to clean the system graph.

52N22

Status description

error: procedure exception - invalid procedure argument. Invalid argument `{ [field] }` for `{ [procParam] }` on procedure `{ [proc] }`. The expected format of `{ [procParam] }` is `{ [procParamFmt] }`.

52N23

Status description

error: procedure exception - non-reloadable namespace. The following namespaces are not reloadable: `{ [namespaceList] }`

52N24

Status description

error: procedure exception - failed to reload procedures. Failed to reload procedures. See logs for more information.

52N25

Status description

error: procedure exception - JMX error. JMX error while accessing `{ [param] }`. See logs for more information.

52N26

Status description

error: procedure exception - invalid change identifier. `{ [param] }` is not a valid change identifier.

52N27

Status description

error: procedure exception - invalid commit timestamp. The commit timestamp for the provided transaction ID does not match the one in the transaction log.

52N28

Status description

error: procedure exception - invalid change identifier and transaction id. `{ [transactionId] }` is not a valid change identifier. Transaction ID does not exist.

52N29

Status description

error: procedure exception - outdated change identifier. Given ChangeIdentifier describes a transaction that occurred before any enrichment records exist.

52N30

Status description

error: procedure exception - future change identifier. Given ChangeIdentifier describes a transaction that hasn't yet occurred.

52N31

Status description

error: procedure exception - wrong database. Change identifier `{ [param] }` does not belong to this database.

52N32

Status description

error: procedure exception - invalid sequence number. Change identifier `{ [param] 1 }` has an invalid sequence number `{ $param2 }`.

52N33

Status description

error: procedure exception - procedure invocation failed. Failed to invoke procedure/function `{ [sig] }` caused by: `{ [msg] }`.

52N34

Status description

error: procedure exception - procedure restricted. `{ [fun] }` is restricted and accesses database internals. Procedure restriction is controlled by the dbms.security.procedures.unrestricted setting. Only un-restrict procedures you can trust with access to database internals.

52N35

Status description

error: procedure exception - procedure compilation failed. Failed to compile procedure defined in `{ [procClass] }`: `{ [msg] }`

52N36

Status description

error: procedure exception - invalid procedure argument with API documentation hint. Invalid argument `{ [field] }` for `{ [procParam] }` on procedure `{ [proc] }`. The expected format of `{ [procParam] }` is outlined in the `{ [procParamFmt] }` API documentation.

52N37

Status description

error: procedure exception - procedure execution error. Execution of the procedure `{ [proc] }` failed.

52N38

Status description

error: procedure exception - cdc start position not found. Cannot find a start position in the logs.

52N39

Status description

error: procedure exception - cdc scanner inactive. The log scanner is no longer active.

52N40

Status description

error: procedure exception - Reconciliation failed during writing the topology graph, transaction may not be committed.

52N41

Status description

error: procedure exception - The key value for `{ [field] }` in the query string cannot be parsed when getting a routing table.

52U00

Status description

error: procedure exception - custom procedure execution error cause. Execution of the procedure `{ [proc] }` failed due to `{ [msgTitle] }`: `{ [msg] }`

Function exceptions

53N33

Status description

error: function exception - function invocation failed. Failed to invoke function `{ [sig] }` caused by: `{ [msg] }`

53N34

Status description

error: function exception - function restricted. `{ [fun] }` is restricted and accesses database internals. User-defined function restriction is controlled by the `dbms.security.procedures.unrestricted` setting. Only un-restrict user-defined functions you can trust with access to database internals.

53N35

Status description

error: function exception - function compilation failed. Failed to compile function defined in `{ [funClass] }`: `{ [msg] }`

53N37

Status description

error: function exception - function execution error. Execution of the function `{ [fun] }` failed.

53U00

Status description

error: function exception - custom function execution error cause. Execution of the function `{ [fun] }` failed due to `{ [msgTitle] }`: `{ [msg] }`.

Dependent object errors

G1001

Status description

error: dependent object error - edges still exist

[[08000]] = 08000

Status description

error: connection exception

[[08007]] = 08007

Status description

error: connection exception - transaction resolution unknown

[[08N00]] = 08N00

Status description

error: connection exception - unable to connect to database. Unable to connect to database `{ [db] }`. Unable to get bolt address of the leader. Check the status of the database. Retrying your request at a later time may succeed.

[[08N01]] = 08N01

Status description

error: connection exception - unable to write to database. Unable to write to database `{ [db] }` on this server. Server-side routing is disabled. Either connect to the database leader directly or enable server-side routing by setting `{ [cfgSetting] }=true`.

Possible solution

Either connect to the database directly using the driver or interactively with the `:use mydb` command, or enable server-side routing by setting `dbms.routing.enabled=true` in the configuration.

[[08N02]] = 08N02

Status description

error: connection exception - unable to connect to database. Unable to connect to database `{ [db] }`. Server-side routing is disabled. Either connect to `{ $db }` directly, or enable server-side routing by setting `{ [cfgSetting] }=true`.

GQLSTATUS error

```
08N02: connection exception - unable to connect to database. Unable to connect to database `mydb`. Server-side routing is disabled. Either connect to `mydb` directly, or enable server-side routing by setting `dbms.routing.enabled=true`.
```

Possible solution

Either connect to the database directly using the driver or interactively with the `:use mydb` command, or enable server-side routing by setting `dbms.routing.enabled=true` in the configuration.

[[08N03]] = 08N03

Status description

error: connection exception - failed to write to graph. Failed to write to graph `{ [graph] }`. Check the defined access mode in both driver and database.

[[08N04]] = 08N04

Status description

error: unable to route use clause. Routing with `{ [clause] }` is not supported in embedded sessions. Connect to the database directly or try running the query using a Neo4j driver or the HTTP API.

[[08N05]] = 08N05

Status description

error: connection exception - unable to route administration command. Routing administration commands is not supported in embedded sessions. Connect to the system database directly or try running the query using a Neo4j driver or the HTTP API.

[[08N06]] = 08N06

Status description

error: connection exception - protocol error. General network protocol error.

[[08N07]] = 08N07

Status description

error: connection exception - not the leader. This member is not the leader.

Possible solution

No write operations are allowed directly on this database. Connect to the leader directly or enable server-side routing by setting `dbms.routing.enabled=true` in the configuration.

[[08N08]] = 08N08

Status description

error: connection exception - database is read only. This database is read only on this server.

[[08N09]] = 08N09

Status description

error: connection exception - database unavailable. The database `{ [db] }` is currently unavailable. Check the database status. Retry your request at a later time.

[[08N10]] = 08N10

Status description

error: connection exception - invalid server state. Message `{ [msg] }` cannot be handled by session in the `{ [boltServerState] }` state.

[[08N11]] = 08N11

Status description

error: connection exception - request error. The request is invalid and could not be processed by the server. See cause for further details.

[[08N12]] = 08N12

Status description

error: connection exception - failed to parse bookmark. Failed to parse the supplied bookmark. Verify it is correct or check the debug log for more information.

[[08N13]] = 08N13

Status description

error: connection exception - database not up to requested bookmark. The database `{ [db] }` is not up to the requested bookmark `{ [transactionId] 1 }`. The latest transaction ID is `{ $transactionId2 }`.

[[08N14]] = 08N14

Status description

error: connection exception - alias chains are not permitted. Unable to provide a routing table for the database identifed by the alias `{ [alias] 1 }` because the request comes from another alias `{ $alias2 }` and alias chains are not permitted.

[[08N15]] = 08N15

Status description

error: connection exception - no such routing policy. Policy definition of the routing policy `{ [routingPolicy] }` could not be found. Verify that the spelling is correct.

[[08N16]] = 08N16

Status description

error: connection exception - general driver client error. Remote execution failed with message `{ [msg] }`.

[[08N17]] = 08N17

Status description

error: connection exception - general driver transient error. Remote execution failed with message `{ [msg] }`.

[[08N18]] = 08N18

Status description

error: connection exception - general driver database error. Remote execution failed with message `{ [msg] }`.

[[08N19]] = 08N19

Status description

error: connection exception - shard execution transient error. Communication with shard `{ [graph] }` failed with message `{ [msg] }`.

[[08N20]] = 08N20

Status description

error: connection exception - shard execution database error. Communication with shard `{ [graph] }` failed with message `{ [msg] }`.

[[08N21]] = 08N21

Status description

error: connection exception - shard execution client error. Communication with shard `{ [graph] }` failed with message `{ [msg] }`.

[[22000]] = 22000

Status description

error: data exception

[[22003]] = 22003

Status description

error: data exception - numeric value out of range. The numeric value `{ [value] }` is outside the required range.

[[22004]] = 22004

Status description

error: data exception - null value not allowed

[[22007]] = 22007

Status description

error: data exception - invalid date, time, or datetime format

[[22012]] = 22012

Status description

error: data exception - division by zero

[[22015]] = 22015

Status description

error: data exception - interval field overflow

[[22G03]] = 22G03

Status description

error: data exception - invalid value type

[[22G05]] = 22G05

Status description

error: data exception - invalid date, time, or datetime function field name

[[22G06]] = 22G06

Status description

error: data exception - invalid date, time, or datetime function field value

Scenario

A temporal field, such as `year`, `day`, `hour`, `minute`, in the map provided argument of one of the temporal functions `date()`, `time()`, `localtime()`, `datetime()`, `localdatetime()` has an invalid value.

Possible solution

Adjust the value.

[[22G08]] = 22G08

Status description

error: data exception - invalid duration function field value

Scenario

A temporal field, such as `years`, `days`, `hours`, `minutes`, in the map provided argument of one of the temporal functions `duration()` has an invalid value.

Possible solution

Adjust the value.

[[22G0I]] = 22G0I

Status description

error: data exception - invalid duration field. `{ [field] }` is not a valid duration field.

Example scenario

Try running the following invalid query:

```
RETURN duration({dogYears: datetime()});
```

The query returns an error with GQLSTATUS 22G0I and status description:

GQLSTATUS error

```
22G0I: data exception - invalid duration field. `dogYears` is not a valid duration field.
```

[[22N00]] = 22N00

Status description

error: data exception - unsupported value. The provided value is unsupported and cannot be processed.

[[22N01]] = 22N01

Status description

error: data exception - invalid type. Expected the value `{ [value] }` to be of type `{ [valueTypeList] }`, but was of type `{ [valueType] }`.

[[22N02]] = 22N02

Status description

error: data exception - specified negative numeric value. Expected `{ [option] }` to be a positive number but found `{ [value] }` instead.

[[22N03]] = 22N03

Status description

error: data exception - specified numeric value out of range. Expected `{ [component] }` to be of type `{ [valueType] }` and in the range `{ [lower] }` to `{ [upper] }` but found `{ [value] }`.

Example scenario

Try providing a value of `1005` for the `millisecond` field:

```
RETURN time({hour: 1, minute: 1, second: 1, millisecond: 1005});
```

The query returns a chain of errors where GQLSTATUS 22N03 is the cause of GQLSTATUS 22007:

GQLSTATUS error chain

```
22N03: data exception - specified numeric value out of range. Expected 'Millisecond' to be of type INTEGER
and in the range 0 to 999 but found 1005.
  22007: data exception - invalid date, time, or datetime format
```

Possible solution

Provide an integer value in the valid range for the `millisecond` field:

```
RETURN time({hour: 1, minute: 1, second: 2, millisecond: 5});
```

[[22N04]] = 22N04

Status description

error: data exception - invalid input value. Invalid input `{ [input] }` for `{ [context] }`. Expected `{ [inputList] }`.

[[22N05]] = 22N05

Status description

error: data exception - input failed validation. Invalid input `{ [input] }` for `{ [context] }`.

[[22N06]] = 22N06

Status description

error: data exception - required input missing. Invalid input. `{ [inputList] }` needs to be specified.

Changed in 2025.03

[[22N07]] = 22N07

Status description

error: data exception - invalid pre-parser option key. Invalid pre-parser option(s): `{ [optionList] }`.

[[22N08]] = 22N08

Status description

error: data exception - invalid pre-parser combination. Invalid pre-parser option, cannot combine `{ [option] 1 }` with `{ $option2 }`.

[[22N09]] = 22N09

Status description

error: data exception - conflicting pre-parser combination. Invalid pre-parser option, cannot specify multiple conflicting values for `{ [option] }`.

[[22N10]] = 22N10

Status description

error: data exception - invalid pre-parser option value. Invalid pre-parser option, specified `{ [input] }` is not valid for option `{ [option] }`. Valid options are: `{ [optionList] }`.

[[22N11]] = 22N11

Status description

error: data exception - invalid argument. Invalid argument: cannot process `{ [input] }`.

[[22N12]] = 22N12

Status description

error: data exception - invalid date, time, or datetime format. Invalid argument: cannot process `{ [input] }`.

[[22N13]] = 22N13

Status description

error: data exception - invalid time zone. Specified time zones must include a date component.

[[22N14]] = 22N14

Status description

error: data exception - invalid temporal value combination. Cannot select both `{ [temporal] 1 }` and `{ $temporal2 }`.

[[22N15]] = 22N15

Status description

error: data exception - invalid temporal component. Cannot read the specified `{ [component] }` component from `{ [temporal] }`.

[[22N16]] = 22N16

Status description

error: data exception - invalid import value. Importing entity values to a graph with a USE clause is not supported. Attempted to import `{ [expr] }` to `{ [graph] }`.

[[22N18]] = 22N18

Status description

error: data exception - incomplete spatial value. A `{ [crs] }` POINT must contain `{ [mapKeyList] }`.

[[22N19]] = 22N19

Status description

error: data exception - invalid spatial value. A `POINT` must contain either `x` and `y`, or `latitude` and `longitude`.

[[22N20]] = 22N20

Status description

error: data exception - invalid spatial value dimensions. Cannot create POINT with `{ [dim] 1 }D` coordinate reference system (CRS) and `{ [value] }` coordinates. Use the equivalent `{ $dim2 }D` coordinate reference system instead.

[[22N21]] = 22N21

Status description

error: data exception - unsupported coordinate reference system. Unsupported coordinate reference system (CRS): `{ [crs] }`.

[[22N22]] = 22N22

Status description

error: data exception - invalid spatial value combination. Cannot specify both coordinate reference system (CRS) and spatial reference identifier (SRID).

[[22N23]] = 22N23

Status description

error: data exception - invalid latitude value. Cannot create WGS84 POINT with invalid coordinate: `{ [coordinates] }`. The valid range for the latitude coordinate is [-90, 90].

[[22N24]] = 22N24

Status description

error: data exception - invalid coordinate arguments. Cannot construct a `{ [valueType] }` from `{ [coordinates] }`.

[[22N25]] = 22N25

Status description

error: data exception - invalid temporal arguments. Cannot construct a `{ [valueType] }` from `{ [temporal] }`.

[[22N26]] = 22N26

Status description

error: data exception - unsupported rounding mode. Unknown rounding mode. Valid values are: `CEILING`, `FLOOR`, `UP`, `DOWN`, `HALF_EVEN`, `HALF_UP`, `HALF_DOWN`, `UNNECESSARY`.

[[22N27]] = 22N27

Status description

error: data exception - invalid entity type. Invalid input `{ [input] }` for `{ [context] }`. Expected to be `{ [valueTypeList] }` `{ [hint] }`.



Note:

The `{ [hint] }` segment is optional. When present, it contains additional guidance (for example: `Hint: use properties(…) on the right-hand side.`).

Example scenario

Try running the following query, which expects a map but receives a node or relationship:

```
CYPHER 25
MATCH (a)-->(b)
SET a = b;
```

The query returns a chain of errors where GQLSTATUS 22N27 is the cause of GQLSTATUS 22G03:

GQLSTATUS error chain

```
22N27: data exception - invalid entity type. Invalid input 'NODE' for b. Expected to be MAP. Hint: use
properties(...) on the right-hand side. (line 3, column 9 (offset: 34))
"SET a = b"
         ^
  22G03: data exception - invalid value type
```

Possible solution

To resolve this error, ensure that the input for `b` is a map. You can modify the query as follows:

```
CYPHER 25
MATCH (a)-->(b)
SET a = properties(b);
```

[[22N28]] = 22N28

Status description

error: data exception - overflow error. The result of the operation `{ [operation] }` has caused an overflow.

[[22N29]] = 22N29

Status description

error: data exception - unknown coordinate reference system. Unknown coordinate reference system (CRS).

[[22N30]] = 22N30

Status description

error: data exception - missing temporal unit. At least one temporal unit must be specified.

[[22N31]] = 22N31

Status description

error: data exception - invalid properties in merge pattern. The `{ [entityType] }` property `{ [propKey] }` in `{ [pat] }` is invalid. `MERGE` cannot be used with a graph element property value that is `{ [value] }`.

Explanation

This error occurs when a `MERGE` pattern includes a node or relationship property value that is not allowed, such as `null` or `NaN`. Because `MERGE` must be able to match or create a concrete graph element, these invalid values are rejected.

Example scenarios

Let's take the following example scenarios:

Null property value in a `MERGE` pattern

Try merging a node with a `null` property value as follows:

```
MERGE (p:Person {age: null});
```

The query returns a chain of errors where GQLSTATUS 22N31 is the cause of GQLSTATUS 22G03:

GQLSTATUS error chain

```
22N31: data exception - invalid properties in merge pattern. The node property `age` in '(:Person {age:
null})' is invalid. 'MERGE' cannot be used with a graph element property value that is null.
  22G03: data exception - invalid value type
```

In this case, provide a concrete value for the property, or remove it from the `MERGE` pattern.

NaN property value in a `MERGE` pattern

Try merging a relationship with a `NaN` property value as follows:

```
WITH 0.0/0.0 AS nan
MERGE (a:Person)-[:KNOWS {score: nan}]->(b:Person)
```

The query returns an error with GQLSTATUS 22N31 and the status description:

```
error: data exception - invalid properties in merge pattern. The relationship property 'score' in
'(:Person)-[:KNOWS {score: NaN}]->(:Person)' is invalid. 'MERGE' cannot be used with graph element
property value that is NaN.
```

In this case, provide a numeric value for the property, or remove it from the `MERGE` pattern.

[[22N32]] = 22N32

Status description

error: data exception - non-deterministic sort expression. `ORDER BY` expressions must be deterministic.

[[22N33]] = 22N33

Status description

error: data exception - invalid shortest path expression. Shortest path expressions must contain start and end nodes. Cannot find: `{ [variable] }`.

[[22N34]] = 22N34

Status description

error: data exception - invalid use of aggregate function. Cannot use `{ [funType] }` function inside an aggregate function.

[[22N35]] = 22N35

Status description

error: data exception - invalid date format. Cannot parse `{ [input] }` as a DATE. Calendar dates need to be specified using the format `YYYY-MM`, while ordinal dates need to be specified using the format `YYYY-DDD`.

[[22N36]] = 22N36

Status description

error: data exception - invalid temporal format. Cannot parse `{ [input] }` as a `{ [valueType] }`.

[[22N37]] = 22N37

Status description

error: data exception - invalid coercion. Cannot coerce `{ [value] }` to `{ [valueType] }`.

[[22N38]] = 22N38

Status description

error: data exception - invalid function argument. Invalid argument to the function `{ [fun] }`.

[[22N39]] = 22N39

Status description

error: data exception - unsupported property value type. Value `{ [value] }` cannot be stored in properties.

[[22N40]] = 22N40

Status description

error: data exception - non-assignable temporal component. Cannot assign `{ [component] }` of a `{ [valueType] }`.

Example scenario

Try providing a string value for the `minute` field:

```
RETURN time({hour: 10, minute: '20', second: 23});
```

The query returns a chain of errors where GQLSTATUS 22N40 is the cause of GQLSTATUS 22G06:

GQLSTATUS error chain

```
22N40: data exception - non-assignable temporal component. Cannot assign 'minute' of a STRING.
  22G06: data exception - invalid date, time, or datetime function field value
```

Possible solution

Provide an integer value for the `minute` field

```
RETURN time({hour: 10, minute: 20, second: 23});
```

[[22N41]] = 22N41

Status description

error: data exception - merge node uniqueness constraint violation. The `MERGE` clause did not find a matching node `{ [variable] }` and cannot create a new node due to conflicts with existing uniqueness constraints.

[[22N42]] = 22N42

Status description

error: data exception - merge relationship uniqueness constraint violation. The `MERGE` clause did not find a matching relationship `{ [variable] }` and cannot create a new relationship due to conflicts with existing uniqueness constraints.

[[22N43]] = 22N43

Status description

error: data exception - unable to load external resource. Could not load external resource from `{ [url] }`.

[[22N44]] = 22N44

Status description

error: data exception - parallel runtime disabled. Parallel runtime has been disabled, enable it or upgrade to a bigger Aura instance.

[[22N46]] = 22N46

Status description

error: data exception - unsupported use of parallel runtime. Parallel runtime does not support updating queries or a change in the state of transactions. Use another runtime.

[[22N47]] = 22N47

Status description

error: data exception - invalid parallel runtime configuration. No workers are configured for the parallel runtime. Set `server.cypher.parallel.worker_limit` to a larger value.

[[22N48]] = 22N48

Status description

error: data exception - unable to use specified runtime. Cannot use the specified runtime `{ [runtime] }` due to `{ [cause] }` not being supported. Use another runtime.

[[22N49]] = 22N49

Status description

error: data exception - CSV buffer size overflow. Cannot read a CSV field larger than the set buffer size. Ensure the field does not have an unterminated quote, or increase the buffer size via `dbms.import.csv.buffer_size`.

[[22N51]] = 22N51

Status description

error: data exception - graph reference not found. A graph reference with the name `{ [db] }` was not found. Verify that the spelling is correct.

Changed in 2025.03

[[22N52]] = 22N52

Status description

error: data exception - invalid combination of `PROFILE` and `EXPLAIN`. `PROFILE` and `EXPLAIN` cannot be combined.

[[22N53]] = 22N53

Status description

error: data exception - invalid use of PROFILE. Cannot `PROFILE` query before results are materialized.

[[22N54]] = 22N54

Status description

error: data exception - invalid map. Multiple conflicting entries specified for `{ [mapKey] }`.

[[22N55]] = 22N55

Status description

error: data exception - required key missing from map. Map requires key `{ [mapKey] }` but was missing from field `{ [field] }`.

[[22N56]] = 22N56

Status description

error: data exception - protocol message length limit overflow. Protocol message length limit exceeded (limit: `{ [boltMsgLenLimit] }`).

[[22N57]] = 22N57

Status description

error: data exception - invalid protocol type. Protocol type is invalid. Invalid number of struct components (received `{ [count] 1 }` but expected `{ [count] 2 }`).

[[22N58]] = 22N58

Status description

error: data exception - invalid spatial component. Cannot read the specified `{ [component] }` component from `{ [value] }`.

[[22N59]] = 22N59

Status description

error: data exception - token does not exist. The `{ [tokenType] }` token with id `{ [tokenId] }` does not exist.

[[22N60]] = 22N60

Status description

error: data exception - illegal element. Encountered illegal `{ [item] }` element. Reason: `{ [msg] }`

GQLSTATUS error

```
22N60: data exception - illegal element. Encountered illegal root element. Reason: Expected struct.
```

[[22N62]] = 22N62

Status description

error: data exception - relationship type does not exist. The relationship type `{ [relType] }` does not exist.

[[22N63]] = 22N63

Status description

error: data exception - property key does not exist. The property key `{ [propKey] }` does not exist.

[[22N64]] = 22N64

Status description

error: data exception - constraint does not exist. The constraint `{ [constrDescrOrName] }` does not exist.

[[22N65]] = 22N65

Status description

error: data exception - equivalent constraint already exists. An equivalent constraint already exists: `{ [constrDescrOrName] }`.

[[22N66]] = 22N66

Status description

error: data exception - conflicting constraint already exists. A conflicting constraint already exists: `{ [constrDescrOrName] }`.

[[22N67]] = 22N67

Status description

error: data exception - duplicated constraint name. A constraint with the same name already exists: `{ [constr] }`.

[[22N68]] = 22N68

Status description

error: data exception - dependent constraint managed individually. Dependent constraints cannot be managed individually and must be managed together with its graph type.

[[22N69]] = 22N69

Status description

error: data exception - index does not exist. The index `{ [idxDescrOrName] }` does not exist.

Changed in 2025.07

Example scenario

Try dropping a nonexistent index:

```
DROP INDEX `nonExisting`;
```

The query returns a chain of errors where GQLSTATUS 22N69 is the cause of GQLSTATUS 50N10:

GQLSTATUS error chain

```
22N69: data exception - index does not exist. The index 'nonExisting' does not exist.
  50N10: general processing exception - index drop failed. Unable to drop 'nonExisting'.
```

[[22N70]] = 22N70

Status description

error: data exception - equivalent index already exists. An equivalent index already exists: `{ [idxDescrOrName] }`.

[[22N71]] = 22N71

Status description

error: data exception - duplicated index name. An index with the same name already exists: `{ [idx] }`.

Changed in 2025.07

Example scenario

Try creating an index when there is another existing index with that name:

Given:

```
CREATE INDEX `myIndex` FOR (n:Person) ON (n.name);
```

Try creating another index with the same name:

```
CREATE INDEX `myIndex` FOR (n:Book) ON (n.title);
```

The query returns an error with GQLSTATUS 22N71 and status description:

GQLSTATUS error

```
22N71: data exception - duplicated index name. An index with the same name already exists: `myIndex`
```

[[22N73]] = 22N73

Status description

error: data exception - constraint conflicts with existing index. Constraint conflicts with already existing index `{ [idxDescrOrName] }`.

[[22N74]] = 22N74

Status description

error: data exception - index conflicts with existing constraint. An index that belongs to the constraint `{ [constr] }` contains a conflicting index.

[[22N75]] = 22N75

Status description

error: data exception - constraint contains duplicated tokens. The constraint specified by `{ [constrDescrOrName] }` includes a label, relationship type, or property key with name `{ [token] }` more than once.

[[22N76]] = 22N76

Status description

error: data exception - index contains duplicated tokens. The index specified by `{ [idxDescrOrName] }` includes a label, relationship type, or property key with name `{ [token] }` more than once.

[[22N77]] = 22N77

Status description

error: data exception - property presence verification failed. `{ [entityType] }` (`{ [entityId] }`) with `{ [tokenType] }` `{ [token] }` must have the following properties: `{ [propKeyList] }`.

[[22N78]] = 22N78

Status description

error: data exception - property type verification failed. `{ [entityType] }` (`{ [entityId] }`) with `{ [tokenType] }` `{ [token] }` must have the property `{ [propKey] }` with value type `{ [valueType] }`.

[[22N79]] = 22N79

Status description

error: data exception - property uniqueness constraint violated. Property uniqueness constraint violated: `{ [reasonList] }`.

[[22N80]] = 22N80

Status description

error: data exception - index entry conflict. Index entry conflict: `{ [value] }`.

[[22N81]] = 22N81

Status description

error: data exception - expression type unsupported here. Invalid input: `{ [exprType] }` is not supported in `{ [context] }`.

[[22N82]] = 22N82

Status description

error: data exception - input contains invalid characters. Input `{ [input] }` contains invalid characters for `{ [context] }`. Special characters may require that the input is quoted using backticks.

[[22N83]] = 22N83

Status description

error: data exception - input consists of too many components. Expected name to contain at most `{ [upper] }` components separated by `'.'`.

[[22N84]] = 22N84

Status description

error: data exception - string too long. Expected the string to be no more than `{ [upper] }` characters long.

[[22N85]] = 22N85

Status description

error: data exception - string too short. Expected the string to be at least `{ [lower] }` characters long.

[[22N86]] = 22N86

Status description

error: data exception - numeric range 0 disallowed. Expected a nonzero number.

[[22N88]] = 22N88

Status description

error: data exception - not a valid CIDR IP. `{ [input] }` is not a valid CIDR IP.

[[22N89]] = 22N89

Status description

error: data exception - new password cannot be the same as the old password. Expected the new

password to be different from the old password.

[[22N90]] = 22N90

Status description

error: data exception - property type unsupported in constraint. `{ [valueType] }` is not supported in property type constraints.

Changed in 2025.05

[[22N91]] = 22N91

Status description

error: data exception - cannot convert alias local to remote or remote to local. Failed to alter the specified database alias `{ [alias] }`. Altering remote alias to a local alias or vice versa is not supported. Drop and recreate the alias instead.

[[22N92]] = 22N92

Status description

error: data exception - missing RETURN. This query requires a RETURN clause.

[[22N93]] = 22N93

Status description

error: data exception - missing `YIELD`. A required `YIELD` clause is missing.

[[22N94]] = 22N94

Status description

error: data exception - invalid `YIELD *`. `YIELD *` is not supported in this context. Explicitly specify which columns to yield.

[[22N95]] = 22N95

Status description

error: data exception - parsing JSON exception. Invalid JSON input. Please check the format.

[[22N96]] = 22N96

Status description

error: data exception - mapping JSON exception. Unable to map the JSON input. Please verify the structure.

[[22N97]] = 22N97

Status description

error: data exception - unexpected struct tag. Unexpected struct tag: `{ [value] }`.

[[22N98]] = 22N98

Status description

error: data exception - wrong first field during deserialization. Unable to deserialize request. Expected first field to be `{ [field] }`, but was `{ [value] }`.

[[22N99]] = 22N99

Status description

error: data exception - wrong token during deserialization. Unable to deserialize request. Expected `{ [token] }`, found `{ [value] }`.

[[22NA0]] = 22NA0

Status description

error: data exception - invalid property-based access control rule. Failed to administer property rule.

[[22NA1]] = 22NA1

Status description

error: data exception - invalid property-based access control rule involving non-commutative expressions. The property `{ [propKey] }` must appear on the left hand side of the `{ [operation] }` operator.

[[22NA2]] = 22NA2

Status description

error: data exception - invalid property-based access control rule involving multiple properties. The expression: `{ [expr] }` is not supported. Property rules can only contain one property.

[[22NA3]] = 22NA3

Status description

error: data exception - invalid property-based access control rule involving NaN. 'NaN' is not supported for property-based access control.

[[22NA4]] = 22NA4

Status description

error: data exception - invalid property-based access control rule involving comparison with `NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`.

[[22NA5]] = 22NA5

Status description

error: data exception - invalid property-based access control rule involving `IS NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`. Use `IS NULL` instead.

[[22NA6]] = 22NA6

Status description

error: data exception - invalid property-based access control rule involving `IS NOT NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`. Use `IS NOT NULL` instead.

[[22NA7]] = 22NA7

Status description

error: data exception - invalid property-based access control rule involving nontrivial predicates. The expression: `{ [expr] }` is not supported. Only single, literal-based predicate expressions are allowed for property-based access control.

[[22NA8]] = 22NA8

Status description

error: data exception - parsing JSON failure. Underlying error: `{ [cause] }`.

[[22NA9]] = 22NA9

Status description

error: data exception - unexpected map entry. Invalid input. Unexpected key `{ [mapKey] }`, expected keys are `{ [mapKeyList] }`.

[[22NAA]] = 22NAA

Status description

error: data exception - invalid list for property-based access control rule. The expression `{ [expr] }` is not supported. Lists containing `{ [exprType] }` values are not supported for property-based access control.

Example scenario

Try adding a property-based access control rule including a list with a `NULL` type:

```
GRANT TRAVERSE ON GRAPH * FOR (n) WHERE n.prop1 IN ["string value", NULL] TO role;
```

The query returns a chain of errors where GQLSTATUS 22NAA is the cause of GQLSTATUS 22NA0:

GQLSTATUS error chain

```
22NAA: data exception - invalid list for property-based access control rule. The expression 'n.prop1 IN
["string value", NULL]' is not supported. Lists containing NULL values are not supported for property-
based access control.
  22NA0: data exception - invalid property-based access control rule. Failed to administer property rule.
```

[[22NAB]] = 22NAB

Status description

error: data exception - mixed type list for property-based access control rule. The expression `{ [expr] }` is not supported. All elements in a list must be literals of the same type for property-based access control.

Example scenario

Try adding a property-based access control rule that contains a list of mixed types:

```
GRANT TRAVERSE ON GRAPH * FOR (n) WHERE n.prop1 IN ["string value", 5] TO role;
```

The query returns a chain of errors where GQLSTATUS 22NAB is the cause of GQLSTATUS 22NA0:

GQLSTATUS error chain

```
22NAB: data exception - mixed type list for property-based access control rule. The expression 'n.prop1 IN
["string value", 5]' is not supported. All elements in a list must be literals of the same type for
property-based access control.
  22NA0: data exception - invalid property-based access control rule. Failed to administer property rule.
```

[[22NAC]] = 22NAC

Status description

error: data exception - characters after quote in CSV field. Characters after an ending quote in a CSV field are not supported. See `{ [input] }` at position `{ [pos] }`. This is read as `{ [variable] }`.

Explanation

When a CSV file contains a field that starts with a quote and has content after the ending quote, the file is formatted incorrectly, and trying to read such a file with `LOAD CSV` will result in an error.

Example scenario

For example, let's say you have a CSV file, called `somefile.csv`, with the following content:

```
x,y
"x",1
"y"y,2
```

Try running the following query:

```
LOAD CSV 'somefile.csv' AS line
RETURN count(line);
```

The query returns an error with GQLSTATUS 22NAC and status description:

GQLSTATUS error

```
22NAC: data exception - characters after quote in CSV field. Characters after an ending quote in a CSV
field are not supported. See 'somefile.csv' at position 14. This is read as `y"y` .
```

[[22NAD]] = 22NAD

Status description

error: data exception - missing end quote in CSV field. Missing end quote at position `{ [pos] }` in `{ [input] }`.

Explanation

When a CSV file contains a field that starts with a quote but misses an ending quote, the file is formatted incorrectly and trying to read such a file with `LOAD CSV` will result in an error.

Example scenario

For example, let's say you have a CSV file, called `somefile.csv`, with the following content:

```
x,y
"x",1
"y,2
```

Try running the following query:

```
LOAD CSV 'somefile.csv' AS line
RETURN count(line);
```

The query returns an error with GQLSTATUS 22NAD and status description:

GQLSTATUS error

```
22NAD: data exception - missing end quote in CSV field. Missing end quote at position 15 in
'somefile.csv'.
```

[[22NAE]] = 22NAE

Status description

error: data exception - multi-line field in illegal CSV context. Multi-line fields are illegal in this context. Verify that there is not a missing end quote in `{ [input] }` at position `{ [pos] }`.

Explanation

This error is thrown when a CSV file contains a field that spans multiple lines, but the context in which it is being read does not allow multi-line fields. This often occurs when there is a missing end quote in a quoted field.

Example scenario

For example, let's say you have a CSV file, called `somefile.csv`, with the following content:

```
x,y
"x",1
"y
",2
```

And that the `neo4j` database logging is configured with specific allowed multiline files, but where `somefile.csv` is not among them.

Try running the following query:

```
LOAD CSV 'somefile.csv' AS line
RETURN count(line);
```

The query returns an error with GQLSTATUS 22NAE and status description:

GQLSTATUS error

```
22NAE: data exception - multi-line field in illegal CSV context. Multi-line fields are illegal in this
context. Verify that there is not a missing end quote in 'somefile.csv' at position 15.
```

[[22NB0]] = 22NB0

Status description

error: data exception - invalid property-based access control rule involving `WHERE` and `IS NULL`. The property value access rule pattern `{ [pred] }` always evaluates to `NULL`. Use `WHERE` syntax in combination with `IS NULL` instead.

[[22NB1]] = 22NB1

Status description

error: data exception - type mismatch. Type mismatch: expected to be `{ [valueTypeList] }` but was `{ [input] }`.

Explanation

An operand to an operation or the argument to a parameter has a type that is not assignable to one of the types expected for the operand or argument.

Example scenarios

Let's take the following example scenarios:

Type mismatch on an operand

Try computing a number modulo a string:

```
RETURN 123 % "abc";
```

The query returns a chain of errors where GQLSTATUS 22NB1 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NB1: data exception - type mismatch. Type mismatch: expected to be FLOAT or INTEGER but was 'STRING'.
(line 1, column 14 (offset: 13))
"RETURN 123 % "abc""
              ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, the second operand of the modulo operations (`%`) has to be a number (`FLOAT` or `INTEGER`) but was a string.

Type mismatch on a function argument

Try invoking the `head()` function on a string:

```
RETURN head("abc");
```

The query returns a chain of errors where GQLSTATUS 22NB1 is the cause of GQLSTATUS 22G03:

GQLSTATUS error chain

```
22NB1: data exception - type mismatch. Type mismatch: expected to be LIST but was 'STRING'. (line 1,
column 13 (offset: 12))
"RETURN head("abc")"
             ^
  22G03: Data exception - invalid value type
```

In this case, the argument of `head()` function has to be a list but is a string. Change the argument to a list or use a different function.

[[22NB2]] = 22NB2

Status description

error: data exception - incompatible graph type dependence. Graph type `{ [graphTypeDependence] 1 }` constraint `{ [constrDescrOrName] 1 }` is incompatible with graph type `{ $graphTypeDependence2 }` constraint `{ $constrDescrOrName2 }` because they have different graph type dependence.

[[22NB3]] = 22NB3

Status description

error: data exception - token presence verification failed. `{ [entityType] }` (`{ [entityId] }`) with `{ [tokenType] 1 }` `{ [token] 1 }` must have the `{ $tokenType2 }` `{ $token2 }`.

[[22NB4]] = 22NB4

Status description

error: data exception - endpoint label presence verification failed. Relationship ({ [entityId] 1 }) with type { [relType] } requires its { [endpointType] } node ({ $entityId2 }) to have the label { [label] }.

[[22NB5]] = 22NB5

Status description

error: data exception - unsupported time zone identifier. Unknown time zone identifier { [input] }.

[[22NB6]] = 22NB6

Status description

error: data exception - input empty. Invalid input. { [item] } is not allowed to be an empty string.

[[22NB7]] = 22NB7

Status description

error: data exception - element id unsupported on composite database. It is not supported to create element ids on composite databases. Create the element id for { [entityType] } { <<$entityId>> } on the constituent instead.

[[22NB8]] = 22NB8

Status description

error: data exception - invalid Neo4j type. { [input] } is not a recognized Neo4j type.

[[22NB9]] = 22NB9

Status description

error: data exception - invalid inner list type. Lists cannot have { [typeDescription] } as an inner type in this context.

Example scenario

Try creating a property type constraint with a list of a union as property type:

```
CREATE CONSTRAINT myConstraint
FOR (n:Label)
REQUIRE n.prop IS :: LIST<INTEGER NOT NULL | FLOAT NOT NULL>;
```

The query returns a chain of errors where GQLSTATUS 22NB9 is the cause of GQLSTATUS 22N90, which is the cause of GQLSTATUS 50N11:

GQLSTATUS error chain

```
22NB9: data exception - invalid inner list type. Lists cannot have a union of types as an inner type in
this context.
  22N90: data exception - property type unsupported in constraint. LIST<INTEGER NOT NULL | FLOAT NOT NULL>
is not supported in property type constraints.
  50N11: general processing exception - constraint creation failed. Unable to create 'node property type
constraint'.
```

[[22NBA]] = 22NBA

Status description

error: data exception - omitting mandatory field for property type constraints for vectors. Property type constraints for vectors need to define both coordinate type and dimension.

Example scenario

Try creating a property type constraint for a vector, omitting the dimension:

```
CREATE CONSTRAINT myConstraint
FOR (n:Label)
REQUIRE n.prop IS :: VECTOR<INTEGER>;
```

The query returns a chain of errors where GQLSTATUS 22NBA is the cause of GQLSTATUS 22N90, which is the cause of GQLSTATUS 50N11:

GQLSTATUS error chain

```
22NBA: data exception - omitting mandatory field for property type constraints for vectors. Property type
constraints for vectors need to define both coordinate type and dimension.
  22N90: data exception - property type unsupported in constraint. VECTOR<INTEGER NOT NULL> is not
supported in property type constraints.
  50N11: general processing exception - constraint creation failed. Unable to create 'node property type
constraint'.
```

[[22NBC]] = 22NBC

Status description

error: data exception - index belongs to constraint. Index belongs to constraint `{ [constrDescrOrName] }`.

Example scenario

This error code is used when the user tries drop an index belonging to a constraint.

Possible solution

Drop the constraint instead.

[[22NBD]] = 22NBD

Status description

error: data exception - unsupported struct tag. Unsupported struct tag: `{ [value] }`.

Example scenario

This error occurs when the server receives a struct tag that it understands but which is not included in the negotiated Bolt protocol version. This typically means the client driver has added an incorrect struct tag.

Possible solution

Ensure only valid struct tags are sent for the negotiated protocol version.

[[22NBE]] = 22NBE

Status description

error: data exception - invalid vector dimensions. Invalid vector dimensions. The number of vector dimensions must be between `{ [count] 1 }` and `{ $count2 }`, but is `{ $count3 }`.

Example scenario

This error occurs when the server receives a vector value from a driver and that vector value has an unsupported number of dimensions.

[[22NBF]] = 22NBF

Status description

error: data exception - property value too big. Property value of type `{ [typeDescription] }` is too big (more than `{ [bytes] }` bytes): `{ [value] }`

Example scenario

This error occurs when a property value cannot be stored because its size exceeds the limit.

[[22NBG]] = 22NBG

Status description

error: data exception - invalid vector coordinates. Invalid vector coordinates. The vector coordinates must be finite, but are `{ [value] }`.

Example scenario

Try creating a vector with non-finite coordinates:

```
RETURN VECTOR([4.4028235e+38f], 1, FLOAT32)
```

The query returns an error with GQLSTATUS 22NBG and status description:

GQLSTATUS error

```
22NBG: data exception - invalid vector coordinates. Invalid vector coordinates. The vector coordinates
must be finite, but are [Infinity].
```

[[22NBH]] = 22NBH

Status description

error: data exception - property value size: `{ [bytes] }` of element id: `{ [entityId] }` is too large for the index: `{ [idx] }` you are trying to add it to. See the Operations Manual for index limitations.

Example scenario

This error occurs when a property value cannot be added to an index because its size exceeds the limit for the index.

[[22NC1]] = 22NC1

Status description

error: data exception - graph type element contains duplicated tokens. The graph type element includes a property key with name `{ [propKey] }` more than once.

Explanation

When defining a graph type, node and relationship element types may only define a property once.

Example scenario

Try setting a graph type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    (n:Node => {prop::STRING, prop::INTEGER})
};
```

The query returns a chain of errors where GQLSTATUS 22NC1 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC1: data exception - graph type element contains duplicated tokens. The graph type element includes a
property key with name `prop` more than once. (line 2, column 31 (offset: 61))
"    (n:Node => {prop::STRING, prop::INTEGER})"
                               ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, remove the duplicate property key definition or uniquely name the property keys:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Node => {prop :: STRING, prop2 :: INTEGER})
};
```

[[22NC2]] = 22NC2

Status description

error: data exception - node element type has no effect. The node element type `{ [label] }` must contain one or more implied labels, or at least one property type.

Explanation

When defining a graph type, node element types must have some effect; otherwise, they are not allowed.

Example scenario

Try setting a graph type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    (n:Node => )
};
```

The query returns a chain of errors where GQLSTATUS 22NC2 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC2: data exception - node element type has no effect. The node element type `Node` must contain one or
more implied labels, or at least one property type. (line 2, column 5 (offset: 35))
"    (n:Node => )"
     ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, define at least one property type or add an implied label, for example:

```
ALTER CURRENT GRAPH TYPE SET {
    (n:Node => { prop :: ANY NOT NULL })
};
```

[[22NC3]] = 22NC3

Status description

error: data exception - relationship element type has no effect. The relationship element type `{ [relType] }` must define a source, destination, or at least one property type.

Explanation

When defining a graph type, relationship element types must have some effect. To be effective, a relationship element type must define at least one of the following:

- a source node element type or label
- a destination node element type or label
- at least one property type

Example scenario

Try setting a graph type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    ()-[:REL =>]->()
};
```

The query returns a chain of errors where GQLSTATUS 22NC3 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC3: data exception - relationship element type has no effect. The relationship element type `REL` must
define a source, destination, or at least one property type. (line 2, column 5 (offset: 35))
"    ()-[:REL =>]->()"
     ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, define at least one property type or add a node element type, for example:

```
ALTER CURRENT GRAPH TYPE SET {
    ()-[:REL => {prop: ANY NOT NULL }]->()
};
```

[[22NC4]] = 22NC4

Status description

error: data exception - a label cannot be both identifying and implied. The label(s) `{ [labelList] }` are defined as both identifying and implied.

Explanation

When defining a graph type, labels and relationships must be assigned clear roles. A label cannot be both "identifying" (to uniquely identify nodes) and "implied" (automatically assigned based on other schema rules) at the same time. Attempting to define a label as both will result in this error, as it creates ambiguity in the schema definition and node identification process.

Example scenario

Try setting a graph type that defines the node element types `Person` and `Student` with the label `Person` also declared as an implied label on the `Student` node element type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => {name :: STRING}),
    (s:Student => :Person )
};
```

The query returns a chain of errors where GQLSTATUS 22NC4 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC4: data exception - a label cannot be both identifying and implied. The label(s) `Person` are defined
as both identifying and implied. (line 3, column 20 (offset: 86))
"    (s:Student => :Person )"
                   ^
```

```
  42001: syntax error or access rule violation - invalid syntax
```

One way to fix this is to add another label to both nodes and capture the property type on `name` there:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => :Named),
    (s:Student => :Named),
    CONSTRAINT FOR (:Named) REQUIRE name IS :: STRING
};
```

[[22NC5]] = 22NC5

Status description

error: data exception - graph type element not found. The `{ [entityType] }` element type referenced by `{ [graphTypeReference] }` does not exist.

Explanation

This error occurs when a graph type definition references an element type (such as a node or relationship) using an alias that has not been defined, or by an identifying reference where there is no graph type element identified by that reference. If the referenced node or relationship type does not exist, the operation cannot proceed and will return an error to indicate the missing element.

Example scenarios

Node element reference does not exist

Try setting a graph type with an identifying node element reference that does not exist as follows:

```
ALTER CURRENT GRAPH TYPE SET {
  (:Node =>)-[e:REL => { prop :: STRING }]->()
};
```

The query returns a chain of errors where GQLSTATUS 22NC5 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC5: data exception - graph type element not found. The node element type referenced by '(:`Node` =>)'
does not exist. (line 2, column 3 (offset: 33))
"  (:Node =>)-[e:REL => { prop :: STRING }]->()"
   ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, you can make the reference non-identifying, or you can define the node type element before this operation:

```
ALTER CURRENT GRAPH TYPE SET {
  (:Node)-[e:REL => { prop :: STRING }]->()
};
```

Relationship element reference does not exist

Try setting a graph type with a relationship element reference that does not exist as follows:

```
ALTER CURRENT GRAPH TYPE SET {
  CONSTRAINT FOR ()-[b]->() REQUIRE (b.prop) IS KEY
};
```

The query returns a chain of errors where GQLSTATUS 22NC5 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC5: data exception - graph type element not found. The relationship element type referenced by 'b' does
not exist. (line 2, column 18 (offset: 48))
"  CONSTRAINT FOR ()-[b]->() REQUIRE (b.prop) IS KEY"
                  ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, the reference could be fixed by providing a relationship type:

```
ALTER CURRENT GRAPH TYPE SET {
  CONSTRAINT FOR ()-[b:REL]->() REQUIRE (b.prop) IS KEY
};
```

Element reference omitted from constraint definition

Try defining a constraint with an empty node element type reference, which is not allowed as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    CONSTRAINT FOR () REQUIRE n.prop IS UNIQUE
};
```

The query returns a chain of errors where GQLSTATUS 22NC5 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC5: data exception - graph type element not found. The node element type referenced by 'n' does not
exist. (line 2, column 31 (offset: 61))
"    CONSTRAINT FOR () REQUIRE n.prop IS UNIQUE"
                               ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, the reference can be fixed by providing a label for the constraint:

```
ALTER CURRENT GRAPH TYPE SET {
    CONSTRAINT FOR (n:Node) REQUIRE n.prop IS UNIQUE
};
```

[[22NC6]] = 22NC6

Status description

error: data exception - independent constraint and node element type have the same label. The independent constraint `{ [constrDescrOrName] }` is using the same label `{ [label] }` as a node element type.

Explanation

This error occurs when a label is used both as an identifying label (defining a node element type) and as part of an independent constraint. A label cannot simultaneously serve as an identifying label and be referenced in an independent constraint.

Example scenario

Try setting a property existence constraint on a property for the label `:Person`, where `:Person` is also defined as an identifying label in the graph type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => {name :: STRING}),
    CONSTRAINT FOR (p:Person =>) REQUIRE p.name IS NOT NULL
};
```

The query returns a chain of errors where GQLSTATUS 22NC6 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC6: data exception - independent constraint and node element type have the same label. The independent
constraint 'Constraint( type='NODE PROPERTY EXISTENCE', schema=(:Person {name}) )' is using the same label
`Person` as a node element type. (line 3, column 20 (offset: 86))
"    CONSTRAINT FOR (p:Person =>) REQUIRE p.name IS NOT NULL"
                    ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, define the property existence constraint as part of the node element type:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => {name :: STRING NOT NULL})
};
```

[[22NC7]] = 22NC7

Status description

error: data exception - independent constraint and relationship element type have the same relationship type. The independent constraint `{ [constrDescrOrName] }` is using the same relationship type `{ [relType] }` as a relationship element type.

Explanation

This error occurs when a relationship type is used both as an identifying relationship type (defining a relationship element type) and as part of an independent constraint. A relationship type cannot simultaneously serve as an identifying relationship type and be referenced in an independent constraint.

Example scenario

Try setting a property type constraint on a property for the relationship type `:REL`, where `:REL` is also defined as an identifying relationship type in the graph type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
```

```
    ()-[:REL => {name :: ANY NOT NULL}]->(),
    CONSTRAINT FOR ()-[r:REL =>]->() REQUIRE r.name IS :: STRING
};
```

The query returns a chain of errors where GQLSTATUS 22NC7 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC7: data exception - independent constraint and relationship element type have the same relationship
type. The independent constraint 'Constraint( type='RELATIONSHIP PROPERTY TYPE', schema=()-[:REL {name}]-
(), propertyType=STRING )' is using the same relationship type `REL` as a relationship element type. (line
3, column 25 (offset: 100))
"    CONSTRAINT FOR ()-[r:REL =>]->() REQUIRE r.name IS :: STRING"
                        ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, define the property type constraint as part of the relationship element type:

```
ALTER CURRENT GRAPH TYPE SET {
    ()-[:REL => {name :: STRING NOT NULL}]->()
};
```

[[22NC8]] = 22NC8

Status description

error: data exception - graph type contains duplicated tokens. The graph type includes a label, a relationship type, or an alias with the name `{ [token] }` more than once.

Explanation

This error occurs when a graph type definition attempts to use the same name for more than one label, relationship type, or a type alias within the same scope. Each label, relationship type, or type alias must have a unique name to avoid ambiguity in the graph schema. To resolve this error, ensure all labels, relationship types, and type aliases are uniquely named within the graph type definition.

Example scenarios

Let's take the following example scenarios:

Aliases

Try defining two node element types using the same type alias as follows:

```
ALTER CURRENT GRAPH TYPE SET {
  ( p: Person => { prop :: STRING } ),
  ( p: Property => { prop :: STRING } )
};
```

The query returns a chain of errors where GQLSTATUS 22NC8 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC8: data exception - graph type contains duplicated tokens. The graph type includes a label, a
relationship type, or an alias with the name 'p' more than once. (line 3, column 5 (offset: 74))
```

```
"  ( p: Property => { prop :: STRING } )"
     ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, rename one of the type aliases, for example:

```
ALTER CURRENT GRAPH TYPE SET {
  ( pers: Person => { prop :: STRING } ),
  ( prop: Property => { prop :: STRING } )
};
```

Identifiers

Try defining two relationship element types using the same relationship type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
  ()-[n:REL => { prop :: STRING }]->(),
  ()-[m:REL => { prop :: STRING }]->()
};
```

The query returns a chain of errors where GQLSTATUS 22NC8 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC8: data exception - graph type contains duplicated tokens. The graph type includes a label, a
relationship type, or an alias with the name 'REL' more than once. (line 3, column 9 (offset: 79))
"  ()-[m:REL => { prop :: STRING }]->()"
        ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, delete the duplicate relationship type definition, since they are both equivalent:

```
ALTER CURRENT GRAPH TYPE SET {
  ()-[n:REL => { prop :: STRING }]->()
};
```

[[22NC9]] = 22NC9

Status description

error: data exception - invalid element type constraints in graph type. A `{ [entityType] 1 }` element type property `{ [context] }` constraint cannot be specified inline of a `{ [entityType] 2 }` element type.

Explanation

Property type and existence constraints must be defined as part of the property description, not as inline constraints on graph type elements.

Example scenario

Try defining an existent constraint inline of a node element type as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => {name :: STRING, age :: INT}) REQUIRE p.studId IS NOT NULL
```

```
};
```

The query returns a chain of errors where GQLSTATUS 22NC9 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NC9: data exception - invalid element type constraints in graph type. A node element type property
existence constraint cannot be specified inline of a node element type. (line 2, column 48 (offset: 78))
"    (p:Person => {name :: STRING, age :: INT}) REQUIRE p.studId IS NOT NULL"
                                                ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE SET {
    (p :Person => {name :: STRING, age :: INT, studId :: ANY NOT NULL })
};
```

[[22NCA]] = 22NCA

Status description

error: data exception - node element type already exists. A node element type identified by the label `{ [label] }` already exists in the graph type.

Explanation

This error occurs when attempting to add a new node element type to the graph type, but a node element type with the same label already exists.

Example scenario

Given the existing graph type:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => {name :: STRING })
};
```

Try adding a new node element type with the same label `:Person` as follows:

```
ALTER CURRENT GRAPH TYPE ADD {
    (p:Person => {name :: STRING, age :: INT })
};
```

The query returns a chain of errors where GQLSTATUS 22NCA is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCA: data exception - node element type already exists. A node element type identified by the label
`Person` already exists in the graph type.
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE ALTER {
    (p:Person => {name :: STRING, age :: INT })
};
```

[[22NCB]] = 22NCB

Status description

error: data exception - relationship element type already exists. A relationship element type identified by the relationship type `{ [relType] }` already exists in the graph type.

Explanation

This error occurs when attempting to add a new relationship element type to the graph type, but a relationship element type with the same type already exists.

Example scenario

Given the existing graph type:

```
ALTER CURRENT GRAPH TYPE SET {
    ()-[r:REL => {name :: STRING }]->()
};
```

Try adding a new relationship element type with the same type `:REL` as follows:

```
ALTER CURRENT GRAPH TYPE ADD {
     ()-[r:REL => {name :: STRING, age :: INT }]->()
};
```

The query returns a chain of errors where GQLSTATUS 22NCB is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCB: data exception - relationship element type already exists. A relationship element type identified
by the relationship type `REL` already exists in the graph type.
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE ALTER {
     ()-[r:REL => {name :: STRING, age :: INT }]->()
};
```

[[22NCC]] = 22NCC

Status description

error: data exception - node element type specified incorrectly. The node element type `{ [graphTypeElement] 1 }` identified by the label `{ [label] }` is different to the one specified in the query `{ [graphTypeElement] 2 }`.

Explanation

This error occurs when attempting to drop a node element type from the graph type, but the specified element type does not match the existing one for the given label.

When dropping an element type, it is not necessary to specify the full element type; you can simply specify the label. If you do specify an element type, it must match exactly with the existing one in the graph type.

Example scenario

Given the following graph type:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => {name :: STRING })
};
```

Try running the following query:

```
ALTER CURRENT GRAPH TYPE DROP {
    (p:Person => :Student)
};
```

The query returns a chain of errors where GQLSTATUS 22NCC is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCC: data exception - node element type specified incorrectly. The node element type (:`Person` =>
{`name` :: STRING}) identified by the label `Person` is different to the one specified in the query
(:`Person` => :`Student`).
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE DROP {
    (p:Person => )
};
```

It is not necessary to specify the full element type to drop it.

[[22NCD]] = 22NCD

Status description

error: data exception - relationship element type specified incorrectly. The relationship element type `{ [graphTypeElement] 1 }` identified by the relationship type `{ [relType] }` is different to the one specified in the query `{ [graphTypeElement] 2 }`.

Explanation

This error occurs when attempting to drop a relationship element type from the graph type, but the specified element type does not match the existing one for the given relationship type.

When dropping an element type, it is not necessary to specify the full element type; you can simply specify the relationship type. If you do specify an element type, it must match exactly with the existing one in the graph type.

Example scenario

Given the following graph type:

```
ALTER CURRENT GRAPH TYPE SET {
    ()-[r:REL => {name :: STRING }]->()
};
```

Try dropping the relationship element type as follows:

```
ALTER CURRENT GRAPH TYPE DROP {
     ()-[r:REL => {name :: STRING, age :: INT }]->()
};
```

The query returns a chain of errors where GQLSTATUS 22NCD is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCD: data exception - relationship element type specified incorrectly. The relationship element type ()-
[:`REL` => {`name` :: STRING}]->() identified by the relationship type `REL` is different to the one
specified in the query ()-[:`REL` => {`age` :: INTEGER, `name` :: STRING}]->().
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE DROP {
     ()-[r:REL => ]->()
};
```

It is not necessary to specify the full element type to drop it.

[[22NCE]] = 22NCE

Status description

error: data exception - node element type in use. The node element type identified by the label `{ [label] }` is referenced in the graph type element `{ [graphTypeReference] }` and cannot be dropped.

Explanation

This error occurs when attempting to drop a node element type from the graph type, but the node element type is still being referenced by a relationship element type in the graph type. To successfully drop the node element type, you must first drop any relationship element types that reference it or remove it from the relationship element types.

Example scenario

Given the following graph type:

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => {name :: STRING }),
    (p)-[:DRIVES => ]->(:Car)
};
```

Try dropping the node element type as follows:

```
ALTER CURRENT GRAPH TYPE DROP {
    (p:Person => )
};
```

The query returns a chain of errors where GQLSTATUS 22NCE is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCE: data exception - node element type in use. The node element type identified by the label `Person`
is referenced in the graph type element '()-[:`DRIVES` =>]->()' and cannot be dropped.
  42001: syntax error or access rule violation - invalid syntax
```

To fix this and drop both types, change the query as follows:

```
ALTER CURRENT GRAPH TYPE DROP {
    (:Person => ),
    ()-[:DRIVES => ]->()
};
```

[[22NCF]] = 22NCF

Status description

error: data exception - graph type constraint not supported. Graph type constraint definitions are not supported in the `{ [graphTypeOperation] }` operation.

Explanation

This error occurs when you try to specify a graph type constraint using the `DROP` or `ALTER` operations, which does not support such operations. Graph type constraints may only be dropped by name when using the `DROP` operation, and are completely invalid when using `ALTER`. This also applies to graph type constraints defined inline of the graph type elements using `IS KEY` and `IS UNIQUE`.

Example scenario

`ALTER` an existing constraint

Given the existing graph type:

```
ALTER CURRENT GRAPH TYPE SET {
    CONSTRAINT c1 FOR (s:Student) REQUIRE s.studentId :: INTEGER
};
```

Try altering the constraint as follows:

```
ALTER CURRENT GRAPH TYPE ALTER {
    CONSTRAINT c1 FOR (s:Student) REQUIRE s.studentId :: STRING
};
```

The query returns a chain of errors where GQLSTATUS 22NCF is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCF: data exception - graph type constraint not supported. Graph type constraint definitions are not
supported in the `ALTER GRAPH TYPE ALTER` operation. (line 2, column 5 (offset: 37))
"    CONSTRAINT c1 FOR (s:Student) REQUIRE s.studentId :: STRING"
     ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE DROP {
    CONSTRAINT c1
};

ALTER CURRENT GRAPH TYPE ADD {
    CONSTRAINT c1 FOR (s:Student) REQUIRE s.studentId :: STRING
};
```

Add a `KEY` constraint using `ALTER`

Given the existing graph type:

```
ALTER CURRENT GRAPH TYPE SET {
    ( :Person => {name :: STRING })
};
```

Try altering the constraint as follows:

```
ALTER CURRENT GRAPH TYPE ALTER {
    ( :Person => {name :: STRING IS KEY } )
};
```

The query returns a chain of errors where GQLSTATUS 22NCF is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCF: data exception - graph type constraint not supported. Graph type constraint definitions are not
supported in the `ALTER GRAPH TYPE ALTER` operation. (line 2, column 34 (offset: 66))
"    ( :Person => {name :: STRING IS KEY } )"
                                  ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE ADD {
    CONSTRAINT nameOfKeyConstraint FOR (p:Person) REQUIRE p.name IS KEY
};
```

`DROP` an undesignated constraint

Try dropping an undesignated constraint with an `ALTER CURRENT GRAPH TYPE DROP` query.

```
ALTER CURRENT GRAPH TYPE DROP {
    ( :Person => {name :: STRING IS KEY})
};
```

The query returns a chain of errors where GQLSTATUS 22NCF is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22NCF: data exception - graph type constraint not supported. Graph type constraint definitions are not
supported in the `ALTER GRAPH TYPE DROP` operation. (line 2, column 34 (offset: 65))
"    ( :Person => {name :: STRING IS KEY})"
                                 ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix this, change the query as follows:

```
ALTER CURRENT GRAPH TYPE DROP {
    CONSTRAINT nameOfKeyConstraint
};
```

[[22NCG]] = 22NCG

Status description

error: data exception - wrong index type. Expected the index `{ [idx] }` to be a `{ [idxType] 1 }` index but was a `{ [idxType] 2 }` index.

Example scenario

For example, assuming that you have a range index called `rangeIdx`, when trying to use it in a `SEARCH` clause:

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX rangeIdx
    FOR [1, 2, 3]
    LIMIT 5
  )
RETURN movie.title AS title;
```

The query returns an error with GQLSTATUS 22NCG and status description:

GQLSTATUS error

```
22NCG: data exception - wrong index type. Expected the index `rangeIdx` to be a vector index but was a
range index.
```

[[22ND1]] = 22ND1

Status description

error: data exception - operation not allowed for roles that are granted to an `AUTH RULE`. Invalid input: `'{`

`[query] }'` is not allowed for roles that are granted to an `AUTH RULE`.

Explanation

If an auth rule fails to evaluate, for example, because it depends on a claim that the user's auth token does not contain, Neo4j will default to not assigning that role to the user.

When a role contains denied privileges, not assigning the role actually removes the denies, which means the user ends up with more privileges than they should. To avoid this, roles cannot have both `DENY` privileges and be granted to an auth rule.

This exception is thrown when attempting to `DENY` a privilege from a role that is already used by an auth rule.

Example scenario

The example assumes that you have a database called `secretDb`, two roles, called `role` and `otherrole`, and an auth rule, called `authrule`, where `role` is granted to `authrule` and `otherrole` is not.

Try denying privileges to the role:

```
DENY MATCH {*} ON GRAPH secretDb NODES * TO otherrole, role;
```

The query returns a chain of errors where GQLSTATUS 22ND1 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
22ND1: data exception - operation not allowed for roles that are granted to an AUTH RULE. Invalid input:
'DENY MATCH {*} ON GRAPH secretDb NODES * TO role' is not allowed for roles that are granted to an AUTH
RULE.
  42001: syntax error or access rule violation - invalid syntax
```

The error message contains a subset of the original query that caused the issue. Since `otherrole` is not granted to an auth rule, it is not included.

Possible solutions

Consider if it is possible to implement the security model without using denied privileges.

For example, you can replace the generic deny from the Example scenario with the following more fine-grained alternative by granting privileges to the role for specific graphs instead of denying access to all nodes in the `secretDb` graph:

```
GRANT MATCH {*} ON GRAPH publicDb, documentationDb NODES * TO role, otherrole;
```

If a `DENY` is required, it must be set to a role that is directly assigned to users without using auth rules, to guarantee that it will always be applied.

[[22ND2]] = 22ND2

Status description

error: data exception - operation not allowed for roles with `DENY` privileges. Invalid input: `'{ [query] }'` is not allowed for roles with `DENY` privileges.

Explanation

If an auth rule fails to evaluate, for example, because it depends on a claim that the user's auth token does not contain, Neo4j will default to not assigning that role to the user.

When a role contains denied privileges, not assigning the role actually removes the denies, which means the user ends up with more privileges than they should. To avoid this, roles cannot have both `DENY` privileges and be granted to an auth rule.

This exception is thrown when attempting to grant a role to an auth rule when the role has denied privileges.

Example scenario

The example assumes that you have a database, called `secretDb`, two roles, called `role` and `otherrole`, and an auth rule, called `authrule`.

1. Deny matching privileges to the role `role` for all nodes in the `secretDb` graph:

   ```
   DENY MATCH {*} ON GRAPH secretDb NODES * TO role;
   ```

2. Try assigning both roles to the `authrule`:

   ```
   CYPHER 25 GRANT ROLE otherrole, role TO AUTH RULE authrule;
   ```

   The query returns a chain of errors where GQLSTATUS 22ND2 is the cause of GQLSTATUS 42001:

   GQLSTATUS error chain

   ```
   22ND2: data exception - operation not allowed for roles with DENY privileges. Invalid input: 'GRANT
   ROLE role TO AUTH RULE authrule' is not allowed for roles with DENY privileges.
     42001: syntax error or access rule violation - invalid syntax
   ```

   The error message contains a subset of the original query that caused the issue. Since `otherrole` does not have denied privileges, it is not included.

Possible solutions

Consider whether it is possible to implement the security model without using denied privileges, for example, by revoking granted privileges or reducing the scope of the grant statements.

For example, you can replace the generic deny from step 1 in the Example scenario with the following more fine-grained alternative:

1. Revoke the denied privilege to reproduce the original scenario:

```
REVOKE DENY MATCH {*} ON GRAPH secretDb NODES * FROM role;
```

2. Grant match privileges to the role for specific graphs instead of denying access to all nodes in the `secretDb` graph:

   ```
   GRANT MATCH {*} ON GRAPH publicDb, documentationDb NODES * TO role, otherrole;
   ```

   Then it is possible to assign both roles to the `authrule`.

3. Try assigning both roles to the `authrule`:

   ```
   CYPHER 25 GRANT ROLE otherrole, role TO AUTH RULE authrule;
   ```

   If a `DENY` is required, it must be set to a role that is directly assigned to users without using auth rules, to guarantee that it will always be applied.

[[22ND3]] = 22ND3

Status description

error: data exception - wrong property for vector search with filters. The property `{ [propKey] }` is not an additional property for vector search with filters on the vector index `{ [idx] }` .

Example scenario

For example, assuming that you have a vector index created by the following command:

```
CYPHER 25
CREATE VECTOR INDEX moviePlots
FOR (m:Movie) ON m.embedding
WITH [m.rating];
```

Try using a property key other than `rating` in the `WHERE` clause property predicate in a `SEARCH` clause:

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    WHERE movie.votes > 1000
    LIMIT 5
  )
RETURN movie.title AS title;
```

The query returns an error with GQLSTATUS 22ND3 and status description:

GQLSTATUS error

```
22ND3: data exception - wrong property for vector search with filters. The property `votes` is not an
additional property for vector search with filters on the vector index `moviePlots`.
```

Possible solution

If you want to use the property `votes` for vector search with filters, it must be added as an additional property to the vector index. For example:

1. Drop the index you want to add the property to:

```
DROP INDEX moviePlots;
```

2. Recreate the index by adding the new property to the index:

```
CYPHER 25
CREATE VECTOR INDEX moviePlots
FOR (m:Movie) ON m.embedding
WITH [m.rating, m.votes];
```

[[25G02]] = 25G02

Status description

error: invalid transaction state - catalog and data statement mixing not supported

[[25N01]] = 25N01

Status description

error: invalid transaction state - invalid combination of statement types. Failed to execute the query `{ [query] }` due to conflicting statement types (read query, write query, schema modification, or administration command). To execute queries in the same transaction, they must be either of the same type, or be a combination of schema modifications and read commands.

[[25N02]] = 25N02

Status description

error: invalid transaction state - unable to complete transaction. Unable to complete transaction. See debug log for details.

[[25N03]] = 25N03

Status description

error: invalid transaction state - concurrent access violation. Transaction is being used concurrently by another request.

[[25N04]] = 25N04

Status description

error: invalid transaction state - specified transaction does not exist. Transaction `{ [transactionId] }` does not exist.

[[25N05]] = 25N05

Status description

error: invalid transaction state - transaction closed. Transaction has been closed.

Changed in 2025.03

[[25N06]] = 25N06

Status description

error: invalid transaction state - transaction start failed. Failed to start transaction. See debug log for details.

[[25N08]] = 25N08

Status description

error: invalid transaction state - invalid transaction lease. The lease for the transaction is no longer valid.

[[25N09]] = 25N09

Status description

error: invalid transaction state - internal transaction failure. The transaction failed due to an internal error.

[[25N11]] = 25N11

Status description

error: invalid transaction state - conflicting transaction state. There was a conflict detected between the transaction state and applied updates. Please retry the transaction.

[[25N12]] = 25N12

Status description

error: invalid transaction state - index was dropped. Index `{ [idx] }` was dropped in this transaction and cannot be used.

[[25N13]] = 25N13

Status description

error: invalid transaction state - cannot access entity after removal. A `{ [entityType] }` was accessed after being deleted in this transaction. Verify the transaction statements.

[[25N14]] = 25N14

Status description

error: invalid transaction state - transaction termination client error. The transaction has been terminated.

Retry your operation in a new transaction, and you should see a successful result. Reason: `{ [msg] }`

[[25N15]] = 25N15

Status description

error: invalid transaction state - transaction termination database error. The transaction has been terminated. Retry your operation in a new transaction, and you should see a successful result. Reason: `{ [msg] }`

[[25N16]] = 25N16

Status description

error: invalid transaction state - transaction termination transient error. The transaction has been terminated. Retry your operation in a new transaction, and you should see a successful result. Reason: `{ [msg] }`

[[25N17]] = 25N17

Status description

error: invalid transaction state - implicit transaction required. The attempted operation requires an implicit transaction.

Example scenario

This error occurs when running a query using the `CALL { … } IN TRANSACTIONS` construct in an explicit transaction.

Possible solution

Make sure you use the driver API to create an implicit transaction, or, if using Browser/Query, use the `:auto` command.

[[2DN01]] = 2DN01

Status description

error: invalid transaction termination - commit failed. Failed to commit transaction.

Changed in 2025.04

[[2DN02]] = 2DN02

Status description

error: invalid transaction termination - constituent commit failed. Failed to commit constituent transaction. See debug log for details.

[[2DN03]] = 2DN03

Status description

error: invalid transaction termination - transaction termination failed. Failed to terminate transaction. See debug log for details.

[[2DN04]] = 2DN04

Status description

error: invalid transaction termination - constituent transaction termination failed. Failed to terminate constituent transaction. See debug log for details.

[[2DN05]] = 2DN05

Status description

error: invalid transaction termination - failed to apply transaction. There was an error on applying the transaction. See logs for more information.

[[2DN06]] = 2DN06

Status description

error: invalid transaction termination - failed to append transaction. There was an error on appending the transaction. See logs for more information.

[[2DN07]] = 2DN07

Status description

error: invalid transaction termination - inner transactions still open. Unable to commit transaction because it still have non-closed inner transactions.

[[40000]] = 40000

Status description

error: transaction rollback

[[40003]] = 40003

Status description

error: transaction rollback - statement completion unknown

[[40N01]] = 40N01

Status description

error: transaction rollback - rollback failed. Failed to rollback transaction. See debug log for details.

[[40N02]] = 40N02

Status description

error: transaction rollback - constituent rollback failed. Failed to rollback constituent transaction. See debug log for details.

[[42000]] = 42000

Status description

error: syntax error or access rule violation

[[42001]] = 42001

Status description

error: syntax error or access rule violation - invalid syntax

[[42006]] = 42006

Status description

error: syntax error or access rule violation - number of edge labels below supported minimum

[[42007]] = 42007

Status description

error: syntax error or access rule violation - number of edge labels exceeds supported maximum

[[42008]] = 42008

Status description

error: syntax error or access rule violation - number of edge properties exceeds supported maximum

[[42009]] = 42009

Status description

error: syntax error or access rule violation - number of node labels below supported minimum

[[42010]] = 42010

Status description

error: syntax error or access rule violation - number of node labels exceeds supported maximum

[[42011]] = 42011

Status description

error: syntax error or access rule violation - number of node properties exceeds supported maximum

[[42012]] = 42012

Status description

error: syntax error or access rule violation - number of node type key labels below supported minimum

[[42013]] = 42013

Status description

error: syntax error or access rule violation - number of node type key labels exceeds supported maximum

[[42014]] = 42014

Status description

error: syntax error or access rule violation - number of edge type key labels below supported minimum

[[42015]] = 42015

Status description

error: syntax error or access rule violation - number of edge type key labels exceeds supported maximum

[[42I00]] = 42I00

Status description

error: syntax error or access rule violation - invalid case expression. `CASE` expressions must have the same number of `WHEN` and `THEN` operands.

[[42I01]] = 42I01

Status description

error: syntax error or access rule violation - invalid `FOREACH`. Invalid use of `{ [clause] }` inside `FOREACH`.

[[42I02]] = 42I02

Status description

error: syntax error or access rule violation - invalid comment. Failed to parse comment. A comment starting with `/*` must also have a closing `*/`.

[[42I04]] = 42I04

Status description

error: syntax error or access rule violation - invalid expression. `{ [expr] }` cannot be used in a `{ [clause] }` clause.

[[42I05]] = 42I05

Status description

error: syntax error or access rule violation - invalid FIELDTERMINATOR. The FIELDTERMINATOR specified for LOAD CSV can only be one character wide.

[[42I06]] = 42I06

Status description

error: syntax error or access rule violation - invalid input. Invalid input `{ [input] }`, expected: `{ [valueList] }`.

[[42I07]] = 42I07

Status description

error: syntax error or access rule violation - invalid integer literal. The given `{ [valueType] }` literal `{ [input] }` is invalid.

[[42I08]] = 42I08

Status description

error: syntax error or access rule violation - invalid lower bound. The lower bound of the variable length relationship used in the `{ [fun] }` function must be 0 or 1.

[[42I10]] = 42I10

Status description

error: syntax error or access rule violation - invalid label expression. Mixing label expression symbols (`|`, `&`, `!`, and `%`) with colon (`:`) between labels is not allowed. This expression could be expressed as `{ [syntax] }`.

[[42I11]] = 42I11

Status description

error: syntax error or access rule violation - invalid name. A `{ [tokenType] }` name cannot be empty or contain any null-bytes: `{ [input] }`.

[[42I12]] = 42I12

Status description

error: syntax error or access rule violation - invalid nesting of quantified path patterns. Quantified path

patterns cannot be nested.

[[42I13]] = 42I13

Status description

error: syntax error or access rule violation - invalid number of procedure or function arguments. The procedure or function call does not provide the required number of arguments; expected `{ [count] 1 }` but got `{ $count2 }`. The procedure or function `{ [procFun] }` has the signature: `{ [sig] }`.

Explanation

This error occurs when:

- A procedure is called with fewer or more arguments than the mandatory parameters for that procedure.
- A function is invoked with fewer or more arguments than the mandatory parameters for that function.

Example scenarios

Let's take the following example scenarios:

Procedure call with fewer arguments than mandatory parameters

Try calling a procedure with fewer arguments than the mandatory parameters for that procedure:

```
CALL db.resampleIndex();
```

The query returns a chain of errors where GQLSTATUS 42I13 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I13: syntax error or access rule violation - invalid number of procedure or function arguments. The
procedure or function call does not provide the required number of arguments; expected 1 but got 0. The
procedure or function db.resampleIndex() has the signature: db.resampleIndex(indexName :: STRING). (line
1, column 1 (offset: 0))
"CALL db.resampleIndex()"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, provide one argument to the procedure call for the parameter `indexName`.

Procedure call with more arguments than mandatory parameters

Try calling a procedure with more arguments than the mandatory parameters for that procedure:

```
CALL db.resampleIndex("myIndex", "myOtherIndex");
```

The query returns a chain of errors where GQLSTATUS 42I13 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I13: syntax error or access rule violation - invalid number of procedure or function arguments. The
procedure or function call does not provide the required number of arguments; expected 1 but got 2. The
procedure or function db.resampleIndex() has the signature: db.resampleIndex(indexName :: STRING). (line
1, column 1 (offset: 0))
"CALL db.resampleIndex("myIndex", "myOtherIndex")"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, provide only a single argument to the procedure call for the parameter `indexName`.

Function invocation with fewer arguments than mandatory parameters

Try invoking a function with fewer arguments than the mandatory parameters for that function:

```
RETURN range(1);
```

The query returns a chain of errors where GQLSTATUS 42I13 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I13: syntax error or access rule violation - invalid number of procedure or function arguments. The
procedure or function call does not provide the required number of arguments; expected 2 but got 1. The
procedure or function range() has the signature: range(start :: INTEGER, end :: INTEGER) :: LIST<INTEGER>
or range(start :: INTEGER, end :: INTEGER, step :: INTEGER) :: LIST<INTEGER>. (line 1, column 8 (offset:
7))
"RETURN range(1)"
        ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, provide at least two arguments to the invocation of `range`: one for the `start` parameter and one for the `end` parameter. Alternatively, `range` also accepts three arguments: one argument for each of the parameters `start`, `end`, and `step`.

Function invocation with more arguments than mandatory parameters

Try invoking a function with more arguments than the mandatory parameters for that function:

```
RETURN range(1, 2, 3, 4);
```

The query returns a chain of errors where GQLSTATUS 42I13 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I13: syntax error or access rule violation - invalid number of procedure or function arguments. The
procedure or function call does not provide the required number of arguments; expected 3 but got 4. The
procedure or function range() has the signature: range(start :: INTEGER, end :: INTEGER) :: LIST<INTEGER>
or range(start :: INTEGER, end :: INTEGER, step :: INTEGER) :: LIST<INTEGER>. (line 1, column 8 (offset:
7))
"RETURN range(1, 2, 3, 4)"
        ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, provide at most three arguments to the invocation of `range`: one argument for each of the parameters `start`, `end`, and `step`. Alternatively, `range` also accepts two arguments: one for the `start` parameter and one for the `end` parameter.

[[42I14]] = 42I14

Status description

error: syntax error or access rule violation - invalid number of relationship types. Exactly one relationship type must be specified for `{ [variable] }`.

[[42I15]] = 42I15

Status description

error: syntax error or access rule violation - invalid number of statements. Expected exactly one statement per query but got: `{ [count] }`.

[[42I16]] = 42I16

Status description

error: syntax error or access rule violation - invalid point. Map with keys `{ [mapKeyList] }` is not a valid POINT. Use either Cartesian or geographic coordinates.

[[42I17]] = 42I17

Status description

error: syntax error or access rule violation - invalid quantifier. A quantifier must not have a lower bound greater than the upper bound.

[[42I18]] = 42I18

Status description

error: syntax error or access rule violation - reference to non-grouping sub-expression. The expression contains a non-grouping sub-expression `{ [variableList] }`. In an aggregating context, only grouping sub-expressions and constants are allowed.

Explanation

An aggregating expression can only consist of grouping sub-expressions, constants, and aggregate functions. A variable that is not a sub-expression, a constant, or an argument to an aggregate function cannot be used in an aggregating expression.

To be usable in an aggregating expression, variables must be either aggregated or single-valued per group, regardless of the queried graph. Grouping sub-expressions and constants ensures that each group is single-valued. In contrast, variables that are neither a grouping sub-expression nor a constant may have multiple values per group.

Example scenario

For example, the following query projects `a.name` as a grouping key but then references the non-grouped

variable `a` inside a map together with a `collect(…)` aggregation:

```
MATCH (a {name: 'Andres'})<-[:HAS_FATHER]-(child)
RETURN a.name, {foo: a.partner = 'Andres', kids: collect(child.name)};
```

The query returns a chain of errors where GQLSTATUS 42I18 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I18: syntax error or access rule violation - reference to non-grouping sub-expression. The expression
contains a non-grouping sub-expression `a`. In an aggregating context only grouping sub-expressions and
constants are allowed. (line 2, column 22 (offset: 71))
"RETURN a.name, {foo: a.partner = 'Andres', kids: collect(child.name)}"
                     ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, you can add the non-grouping expression as a grouping key. Be aware that this may change the result.

```
MATCH (a {name: 'Andres'})<-[:HAS_FATHER]-(child)
RETURN a.name, a.partner, {foo: a.partner = 'Andres', kids: collect(child.name)};
```

Alternatively, you can add the missing variable `a` as a grouping key on its own.

```
MATCH (a {name: 'Andres'})<-[:FATHER]-(child)
RETURN a.name, a, {foo: a.partner = 'Andres', kids: collect(child.name)};
```

[[42I19]] = 42I19

Status description

error: syntax error or access rule violation - invalid string literal. Failed to parse string literal. The query must contain an even number of non-escaped quotes.

[[42I20]] = 42I20

Status description

error: syntax error or access rule violation - invalid symbol in expression. Label expressions and relationship type expressions cannot contain `{ [input] }`. To express a label disjunction use `{ [labelExpr] }` instead.

[[42I21]] = 42I21

Status description

error: syntax error or access rule violation - invalid reference to variable out of scope. Not allowed to reference `{ [variableList] }` from within a parenthesized/quantified path pattern like `{ [pat] }` in the same graph pattern.

[[42I23]] = 42I23

Status description

error: syntax error or access rule violation - invalid quantified path pattern in shortest path. The `{ [fun] }` function cannot contain a quantified path pattern.

[[42I24]] = 42I24

Status description

error: syntax error or access rule violation - invalid use of aggregate function. Aggregate expression `{ [expr] }` is not allowed in this context.

[[42I25]] = 42I25

Status description

error: syntax error or access rule violation - invalid use of `CALL IN TRANSACTIONS`. `CALL { … } IN TRANSACTIONS` is not supported after a write clause.

[[42I26]] = 42I26

Status description

error: syntax error or access rule violation - invalid `DELETE`. `DELETE …` does not support removing labels from a node. Use `REMOVE …` instead.

[[42I27]] = 42I27

Status description

error: syntax error or access rule violation - invalid use of `DISTINCT` with non-aggregate function. `DISTINCT` cannot be used with the `{ [fun] }` function.

[[42I28]] = 42I28

Status description

error: syntax error or access rule violation - invalid use of importing `WITH`. Importing `WITH` can consist only of direct references to outside variables. `{ [input] }` is not allowed.

[[42I29]] = 42I29

Status description

error: syntax error or access rule violation - invalid use of `IS`. The `IS` keyword cannot be used together with multiple labels in `{ [input] }`. Rewrite the expression as `{ [replacement] }`.

[[42I30]] = 42I30

Status description

error: syntax error or access rule violation - invalid use of label expressions. Label expressions cannot be used in `{ [expr] }`.

[[42I31]] = 42I31

Status description

error: syntax error or access rule violation - invalid use of `MATCH`. `MATCH …` cannot directly follow an `OPTIONAL MATCH …`. Use a `WITH` clause between them.

[[42I32]] = 42I32

Status description

error: syntax error or access rule violation - invalid use of node and relationship pattern predicate. Node and relationship pattern predicates cannot be used in `{ [context] }`. They can only be used in a `MATCH` clause or inside a pattern comprehension.

[[42I33]] = 42I33

Status description

error: syntax error or access rule violation - invalid use of `NOT NULL`. Closed Dynamic Union types cannot be appended with `NOT NULL`, specify `NOT NULL` on inner types instead.

[[42I34]] = 42I34

Status description

error: syntax error or access rule violation - invalid use of pattern expression. A pattern expression can only be used to test the existence of a pattern. Use a pattern comprehension instead.

[[42I32-2]] = 42I32

Status description

error: syntax error or access rule violation - invalid use of relationship type expression. Relationship type expressions can only be used in `MATCH …`.

[[42I36]] = 42I36

Status description

error: syntax error or access rule violation - invalid use of `REPORT STATUS`. `REPORT STATUS` can only be used when specifying `ON ERROR CONTINUE` or `ON ERROR BREAK`.

[[42I37]] = 42I37

Status description

error: syntax error or access rule violation - invalid use of `RETURN *`. `RETURN *` is not allowed when there are no variables in scope.

[[42I38]] = 42I38

Status description

error: syntax error or access rule violation - invalid use of `RETURN`. `RETURN …` can only be used at the end of a query or subquery.

[[42I39]] = 42I39

Status description

error: syntax error or access rule violation - invalid use of shortest path function. Mixing the `{ [fun] }` function with path selectors, explicit match modes, or explicit path modes is not allowed.

Example scenario

Try running the following query `REPEATABLE ELEMENTS` with the `shortestPath()` function:

```
MATCH REPEATABLE ELEMENTS p = shortestPath((:A)-[:R*0..10]->(:B))
WHERE any(x IN nodes(p) WHERE x:X)
RETURN p;
```

The query returns a chain of errors where GQLSTATUS 42I39 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I39: syntax error or access rule violation - invalid use of shortest path function. Mixing the
shortestPath() function with path selectors, explicit match modes or explicit path modes is not allowed.
(line 1, column 31 (offset: 30))
"MATCH REPEATABLE ELEMENTS p = shortestPath((:A)-[:R*0..10]->(:B))"
                               ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solution

Replace `shortestPath()` with the `SHORTEST 1` path selector, and `allShortestPaths()` with the `SHORTEST ALL` path selector.

For example, the query above can be rewritten as:

```
MATCH REPEATABLE ELEMENTS p = SHORTEST 1 (:A)-[:R*0..10]->(:B)
  WHERE any(x IN nodes(p) WHERE x:X)
RETURN p;
```

[[42I40]] = 42I40

Status description

error: syntax error or access rule violation - invalid use of `UNION` and `UNION ALL`. `UNION` and `UNION ALL` cannot be combined.

[[42I41]] = 42I41

Status description

error: syntax error or access rule violation - invalid use of variable length relationship. Variable length relationships cannot be used in `{ [value] }`.

[[42I42]] = 42I42

Status description

error: syntax error or access rule violation - invalid use of `YIELD`. Cannot use `YIELD` on a call to a void procedure.

Explanation

Void procedures do not return a result, so no result column can be yielded from them. The `YIELD` clause is not applicable on a call to a void procedure.

Example scenarios

Let's take the following example scenarios:

Yielding a result column from a void procedure

Try yielding result columns from the call of a procedure that does not return a result:

```
CALL db.createLabel("A") YIELD label
RETURN label;
```

The query returns a chain of errors where GQLSTATUS 42I42 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I42: syntax error or access rule violation - invalid use of YIELD. Cannot use YIELD on a call to a void
procedure. (line 1, column 1 (offset: 0))
"CALL db.createLabel("A") YIELD label"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, call the procedure without a `YIELD` clause.

[[42I43]] = 42I43

Status description

error: syntax error or access rule violation - invalid use of `YIELD *`. `YIELD *` can only be used with a standalone procedure call.

[[42I45]] = 42I45

Status description

error: syntax error or access rule violation - invalid use of multiple path patterns. Multiple path patterns cannot be used in the same clause in combination with a selective path selector. `{ [action] }`

[[42I46]] = 42I46

Status description

error: syntax error or access rule violation - invalid use of a node pattern pair. Node pattern pairs are only supported for quantified path patterns.

[[42I47]] = 42I47

Status description

error: syntax error or access rule violation - parser error. Parser Error: `{ [msg] }`.

[[42I48]] = 42I48

Status description

error: syntax error or access rule violation - invalid use of a subquery in `MERGE`. Subqueries are not allowed in a `MERGE` clause.

[[42I49]] = 42I49

Status description

error: syntax error or access rule violation - invalid inequality operator. Unknown inequality operator `!=`. The operator for inequality in Cypher is `<>`.

[[42I50]] = 42I50

Status description

error: syntax error or access rule violation - token name too long. Invalid input `{ [input] }...`. A `{ [tokenType] }` name cannot be longer than `{ [value] }`.

[[42I51]] = 42I51

Status description

info: invalid call signature. The procedure or function `{ [procFun] }` must have the signature: `{ [sig] }`.

[[42I52]] = 42I52

Status description

error: syntax error or access rule violation - no longer valid syntax. `{ [msg] }`

[[42I53]] = 42I53

Status description

error: syntax error or access rule violation - unsupported coordinate type. Unknown coordinate type: `{ [input] }`.

[[42I54]] = 42I54

Status description

error: syntax error or access rule violation - invalid use of `INSERT`. `{ [cause] }` is not allowed in `INSERT`. Use `CREATE` or `{ [replacement] }`.

[[42I55]] = 42I55

Status description

error: syntax error or access rule violation - invalid use of dynamic label or type. Dynamic `{ [tokenType] }` using `$any()` are not allowed in `{ [clause] }`.

[[42I56]] = 42I56

Status description

error: syntax error or access rule violation - invalid relationship direction. Only directed relationships are supported in `{ [clause] }`.

[[42I57]] = 42I57

Status description

error: syntax error or access rule violation - invalid query ending. `{ [exprType] }` cannot contain a query ending with `{ [clause] }`.

[[42I58]] = 42I58

Status description

error: syntax error or access rule violation - invalid entity reference. Entity, `{ [expr] }`, cannot be created and referenced in the same clause.

[[42I59]] = 42I59

Status description

error: syntax error or access rule violation - dynamic entity type not allowed. Dynamic label and types are only allowed in `{ [clauseList] }` clauses.

[[42I60]] = 42I60

Status description

error: syntax error or access rule violation - invalid glob escaping. Each part of the glob (a block of text up until a dot) must either be fully escaped or not escaped at all.

[[42I61]] = 42I61

Status description

error: syntax error or access rule violation - missing LOOKUP INDEX function name. Missing function name for the LOOKUP INDEX.

[[42I62]] = 42I62

Status description

error: syntax error or access rule violation - unsupported distance metric. Unknown distance metric: `{ [input] }`.

[[42I63]] = 42I63

Status description

error: syntax error or access rule violation - wrong subclause order. `ORDER BY`, `{ [clause] }` and `LIMIT` can only be used in this order in `RETURN`.

Example scenario

Try using `ORDER BY` after `SKIP` and `LIMIT`:

```
UNWIND [3,5,1,3,7,10] AS x
RETURN x
SKIP 2
ORDER BY x;
```

The query returns a chain of errors where GQLSTATUS 42I63 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I63: syntax error or access rule violation - wrong subclause order. `ORDER BY`, `SKIP` and `LIMIT` can
only be used in this order in `RETURN`. (line 2, column 1 (offset: 27))
"RETURN x"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

[[42I64]] = 42I64

This error occurs only in Neo4j 2025.07. In later versions, the behavior of `NEXT` has been changed to allow aggregations in a `UNION` query.

Status description

error: syntax error or access rule violation - unsupported operation. `{ [msg] }` currently not supported after `NEXT { [context] }`.

Example scenario

For example, when aggregating in a `UNION` query after a `NEXT`:

```
CYPHER 25
UNWIND [1, 2] AS x
RETURN x

NEXT

RETURN COUNT(x)
UNION ALL
RETURN COUNT(x);
```

The query returns a chain of errors where GQLSTATUS 42I64 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I64: syntax error or access rule violation - unsupported operation. Aggregations are currently not
supported in `NEXT` when used in a `UNION`.
  42001: syntax error or access rule violation - invalid syntax
```

[[42I65]] = 42I65

Status description

error: syntax error or access rule violation - invalid pattern character. An invalid character is used in the pattern. Verify that all characters are supported by `{ [valueType] }`.

Example scenario

Try formatting using a year character on a time-only type:

```
RETURN format(time(), "yyyy") AS formatted;
```

The query returns a chain of errors where GQLSTATUS 42I65 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I65: syntax error or access rule violation - invalid pattern character. An invalid character is used in
the pattern. Verify that all characters are supported by `ZONED TIME`.
  42001: syntax error or access rule violation - invalid syntax
```

[[42I66]] = 42I66

Status description

error: syntax error or access rule violation - pattern parsing failed. Pattern parsing failed. Make sure that an even number of escapes are used in the pattern.

Example scenario

Try formatting a duration:

```
RETURN format(duration("P1Y"), "y 'year");
```

The query returns a chain of errors where GQLSTATUS 42I66 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I66: syntax error or access rule violation - pattern parsing failed. Pattern parsing failed. Make sure
that an even number of escapes are used in the pattern.
  42001: syntax error or access rule violation - invalid syntax
```

[[42I67]] = 42I67

Status description

error: syntax error or access rule violation - unsupported language feature. The query is parsable in `CYPHER { [version] 2 }`, but it is run in `CYPHER { [version] 1 }`. Consider changing the database default Cypher version using `ALTER DATABASE SET DEFAULT LANGUAGE` or prefix the query with `CYPHER { [feat] 2 }`.

Example scenario

For example, when using `LET` in a Cypher 5 query:

```
CYPHER 5
LET x = 1
RETURN x;
```

The query returns a chain of errors where GQLSTATUS 42I67 is the cause of GQLSTATUS 42I06, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I67: syntax error or access rule violation - unsupported language feature. The query is parsable in
`CYPHER 25`, but it is run in `CYPHER 5`. Consider changing the database default Cypher version using
`ALTER DATABASE SET DEFAULT LANGUAGE` or prefix the query with `CYPHER 25`. (line 2, column 1 (offset: 9))
"LET x = 1"
 ^
  42I06: syntax error or access rule violation - invalid input. Invalid input 'LET', expected: 'ALTER',
'ORDER BY', 'CALL', 'USING PERIODIC COMMIT', 'CREATE', 'LOAD CSV', 'START DATABASE', 'STOP DATABASE',
'DEALLOCATE', 'DELETE', 'DENY', 'DETACH', 'DROP', 'DRYRUN', 'FINISH', 'FOREACH', 'GRANT', 'INSERT',
'LIMIT', 'MATCH', 'MERGE', 'NODETACH', 'OFFSET', 'OPTIONAL', 'REALLOCATE', 'REMOVE', 'RENAME', 'RETURN',
'REVOKE', 'ENABLE SERVER', 'SET', 'SHOW', 'SKIP', 'TERMINATE', 'UNWIND', 'USE' or 'WITH'.
  42001: syntax error or access rule violation - invalid syntax
```

[[42I68]] = 42I68

Status description

error: syntax error or access rule violation - mismatched pattern. Pattern, `{ [input] 1 }`, does not match input, `{ [input] 2 }`. Verify that the pattern is valid for constructing `{ [valueType] }`.

Example scenario

Try parsing a duration string value:

```
RETURN duration("5 hours 12 minutes 15 seconds", "h 'hours' m 'minutes'") AS d;
```

The query returns a chain of errors where GQLSTATUS 42I68 is the cause of GQLSTATUS 22007:

GQLSTATUS error chain

```
42I68: syntax error or access rule violation - mismatched pattern. Pattern, `'h 'hours' m 'minutes''`,
does not match input, `'5 hours 12 minutes 15 seconds'`. Verify that the pattern is valid for constructing
`DURATION`.
  22007: data exception - invalid date, time, or datetime format
```

[[42I69]] = 42I69

Status description

error: syntax error or access rule violation - invalid search variable reference. `{ [variable] }` must reference a variable from the same `MATCH` statement.

Example scenario

Try using a variable from a previous `MATCH` statement as variable reference in a `SEARCH` clause:

```
CYPHER 25
MATCH (movie:Movie)
MATCH (m:Movie {title:'Matrix, The'})
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
RETURN movie.title AS title;
```

The query returns a chain of errors where GQLSTATUS 42I69 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I69: syntax error or access rule violation - invalid search variable reference. `movie` must reference a
variable from the same MATCH statement. (line 4, column 10 (offset: 77))
"  SEARCH movie IN ("
          ^
  42001: syntax error or access rule violation - invalid syntax
```

[[42I70]] = 42I70

Status description

error: syntax error or access rule violation - search clause with multiple bound variables. In order to have a search clause, a `MATCH` statement can only have one bound variable.

Example scenario

Try using a `SEARCH` clause in a `MATCH` statement with more than one bound variable:

```
Cypher 25
MATCH (movie:Movie)-[r]->()
```

```
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
RETURN movie.title AS title, r.rating AS rating;
```

The query returns a chain of errors where GQLSTATUS 42I70 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I70: syntax error or access rule violation - search clause with multiple bound variables. In order to
have a search clause, a MATCH statement can only have one bound variable. (line 2, column 22 (offset: 31))
"MATCH (movie:Movie)-[r]->()"
                       ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solution

The first iteration of the `SEARCH` clause comes with many restrictions on the `MATCH` statement. It is likely that some of these restriction will be lifted in future versions of Neo4j. For the time being, the Cypher query under Example scenario can be rewritten like this:

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
MATCH (movie)-[r]->()
RETURN movie.title AS title, r.rating AS rating;
```

[[42I71]] = 42I71

Status description

error: syntax error or access rule violation - search clause with invalid predicates. In order to have a search clause, a `MATCH` statement can only have predicates on the bound node or relationship.

Example scenario

Try using a `SEARCH` clause in a `MATCH` statement with predicates on other parts of the pattern than the bound variable:

```
CYPHER 25
MATCH (movie:Movie)-[]->(:Person)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
RETURN movie.title AS title;
```

The query returns a chain of errors where GQLSTATUS 42I71 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I71: syntax error or access rule violation - search clause with invalid predicates. In order to have a
search clause, a MATCH statement can only have predicates on the bound node or relationship. (line 2,
column 27 (offset: 36))
"MATCH (movie:Movie)-[]->(:Person)"
                          ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solution

The first iteration of the `SEARCH` clause comes with many restrictions on the `MATCH` statement. It is likely that some of these restriction will be lifted in future versions of Neo4j. For the time being, the Cypher query under Example scenario can be rewritten like this:

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
MATCH (movie)-[]->(:Person)
RETURN movie.title AS title;
```

[[42I72]] = 42I72

Status description

error: syntax error or access rule violation - search clause with too complex pattern. In order to have a search clause, a `MATCH` statement can only have a single node or relationship pattern and no selectors.

Example scenarios

Try using a `SEARCH` clause in a `MATCH` statement with a pattern with several pattern parts, a variable length or a selector:

Use of a `SEARCH` clause in a `MATCH` statement with a pattern with several pattern parts

```
Cypher 25
MATCH (movie:Movie), ()
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
RETURN movie.title AS title;
```

The query returns a chain of errors where GQLSTATUS 42I72 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I72: syntax error or access rule violation - search clause with too complex pattern. In order to have a
search clause, a MATCH statement can only have a single node or relationship pattern and no selectors.
(line 2, column 22 (offset: 31))
"MATCH (movie:Movie), ()"
                      ^
  42001: syntax error or access rule violation - invalid syntax
```

Use of a `SEARCH` clause in a `MATCH` statement with a variable length

```
CYPHER 25
MATCH (movie:Movie)-[]->{1,3}()
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
RETURN movie.title AS title;
```

The query returns a chain of errors where GQLSTATUS 42I72 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I72: syntax error or access rule violation - search clause with too complex pattern. In order to have a
search clause, a MATCH statement can only have a single node or relationship pattern and no selectors.
(line 2, column 7 (offset: 16))
"MATCH (movie:Movie)-[]->{1,3}()"
       ^
  42001: syntax error or access rule violation - invalid syntax
```

Use of a `SEARCH` clause in a `MATCH` statement with a selector

```
CYPHER 25
MATCH ANY SHORTEST ()-->(movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
RETURN movie.title AS title;
```

The query returns a chain of errors where GQLSTATUS 42I72 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I72: syntax error or access rule violation - search clause with too complex pattern. In order to have a
search clause, a MATCH statement can only have a single node or relationship pattern and no selectors.
(line 2, column 7 (offset: 16))
"MATCH ANY SHORTEST ()-->(movie:Movie)"
       ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

The first iteration of the `SEARCH` clause comes with many restrictions on the `MATCH` statement. It is likely that some of these restriction will be lifted in future versions of Neo4j. For the time being, the Cypher queries under Example scenarios can be rewritten like this:

Use of a `SEARCH` clause in a `MATCH` statement with a pattern with several pattern parts

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
MATCH (movie), ()
RETURN movie.title AS title;
```

Use of a `SEARCH` clause in a `MATCH` statement with a variable length

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
MATCH (movie)-[]->{1,3}()
RETURN movie.title AS title;
```

Use of a `SEARCH` clause in a `MATCH` statement with a selector

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 5
  )
MATCH ANY SHORTEST ()-->(movie)
RETURN movie.title AS title;
```

[[42I76]] = 42I76

Status description

error: syntax error or access rule violation - index or constraint value too long. The provided index or constraint `{ [item] }` `{ [input] }` (`{ [bytes] 1 }` bytes) exceeded limit of `{ [bytes] 2 }` bytes.

Explanation

This error occurs when a schema rule (index/constraint) entry is too large.

Example scenario

You are running a Neo4j 2026.01 binary with an earlier kernel version, on a block-format database, and you try to create an index with a very large name `$largeIndexName`, which would be a little under 8KiB or more when UTF-8 encoded. For example, `$largeIndexName` is approximately 8KiB long, and starts with the following:

```
:param { largeIndexName:  "A very long index name that would be encoded to around 8KiB of UTF-8 encoded
data should ideally be gracefully handled, etc.." };
```

When you run the following Cypher statement:

```
CREATE INDEX $largeIndexName
FOR (n:Person) ON (n.name);
```

The query returns an error with GQLSTATUS 42I76 and the status description:

GQLSTATUS error

```
42I76: syntax error or access rule violation - index or constraint value too long. The provided index or
constraint name 'A very long index name that would be encoded to around 8KiB of UTF-8 encoded data should
```

```
ideally be ...' (8315 bytes) exceeded limit of 8136 bytes.
```

Possible solution

Reduce the size of the index or constraint name so that it is under the byte limit.

[[42I73]] = 42I73

Status description

error: syntax error or access rule violation - invalid predicate for vector search with filters. The vector search filter predicate `{ [expr] }` must consist of one or more property predicates joined by `AND`, and the combined property predicates for each property must specify either an exact value (e.g. `x.prop = 1`), a half-bounded range (e.g. `x.prop >= 1`), or a bounded range (e.g. `x.prop > 1 AND x.prop < 100`). Note that this is not an exhaustive list of valid predicates, see documentation for all rules.

Example scenarios

For example, assuming that you have a vector index created by the following command:

```
CYPHER 25
CREATE VECTOR INDEX moviePlots
FOR (m:Movie) ON m.embedding
WITH [m.rating];
```

Example 91. Using `NOT` inside the `WHERE` subclause predicate in a `SEARCH` clause

Try using `NOT` inside the `WHERE` subclause predicate in a `SEARCH` clause:

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    WHERE NOT movie.rating = 8
    LIMIT 5
  )
RETURN movie.title AS title, movie.rating AS rating;
```

The query returns a chain of errors where GQLSTATUS 42I73 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I73: syntax error or access rule violation - invalid predicate for vector search with filters. The
vector search filter predicate 'NOT movie.rating = 8' must consist of one or more property predicates
joined by AND, and the combined property predicates for each property must specify either an exact
value (e.g. x.prop = 1), a half-bounded range (e.g. x.prop >= 1), or a bounded range (e.g. x.prop > 1
AND x.prop < 100). Note that this is not an exhaustive list of valid predicates, see documentation
for all rules. (line 6, column 11 (offset: 106))
"    WHERE NOT movie.rating = 8"
           ^
  42001: syntax error or access rule violation - invalid syntax
```

Example 92. Using multiple half-bounded ranges on the same property inside the `WHERE` subclause predicate in a `SEARCH` clause

Try using multiple half-bounded ranges on the same property inside the `WHERE` subclause predicate in a `SEARCH` clause:

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    WHERE movie.rating > 6 AND movie.rating > 8
    LIMIT 5
  )
RETURN movie.title AS title, movie.rating AS rating;
```

The query returns a chain of errors where GQLSTATUS 42I73 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I73: syntax error or access rule violation - invalid predicate for vector search with filters. The
vector search filter predicate 'movie.rating > 6 AND movie.rating > 8' must consist of one or more
property predicates joined by AND, and the combined property predicates for each property must
specify either an exact value (e.g. x.prop = 1), a half-bounded range (e.g. x.prop >= 1), or a
bounded range (e.g. x.prop > 1 AND x.prop < 100). Note that this is not an exhaustive list of valid
predicates, see documentation for all rules. (line 6, column 28 (offset: 123))
"    WHERE movie.rating > 6 AND movie.rating > 8"
                           ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solution

You can rewrite the predicates to comply with the vector search filter rules. For example, the predicate `NOT movie.rating < 7` can be rewritten to `movie.rating >= 7`, while the predicate `movie.rating > 6 AND movie.rating > 8` can be rewritten to `movie.rating > 8`. However, it is not always possible to rewrite the predicates. For the complete list of limitations, see Cypher Manual → Limitations of the SEARCH clause.

[[42I74]] = 42I74

Status description

error: syntax error or access rule violation - wrong variable for vector search with filters. The variable `{ [variable] 1 }` in a vector search filter property predicate must be the same as the search clause binding variable `{ [variable] 2 }`.

Example scenario

For example, assuming that you have a vector index created by the following command:

```
CYPHER 25
CREATE VECTOR INDEX moviePlots
FOR (m:Movie) ON m.embedding
WITH [m.rating];
```

Try using a variable other than `movie` inside the `WHERE` subclause predicate in a `SEARCH` clause in the

following query:

```
CYPHER 25
MATCH (m:Movie {title:'Matrix, The'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    WHERE m.rating >= 8
    LIMIT 5
  )
RETURN movie.title AS title, movie.rating AS rating;
```

The query returns a chain of errors where GQLSTATUS 42I74 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I74: syntax error or access rule violation - wrong variable for vector search with filters. The variable
`m` in a vector search filter property predicate must be the same as the search clause binding variable
`movie`. (line 7, column 11 (offset: 146))
"    WHERE m.rating >= 8"
          ^
  42001: syntax error or access rule violation - invalid syntax
```

[[42I75]] = 42I75

Status description

error: syntax error or access rule violation - self-referencing in search. The expression `{ [expr] }` in the search clause may not depend on the search clause binding variable `{ [variable] }`.

Example scenarios

For example, assuming that you have a vector index created by the following command:

```
CYPHER 25
CREATE VECTOR INDEX moviePlots
FOR (m:Movie) ON m.embedding
WITH [m.rating];
```

Example 93. Using the search binding variable in the `FOR` subclause of a vector search

Try using the search binding variable `movie` in the `FOR` subclause in the following query:

```
CYPHER 25
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR movie.embedding
    LIMIT 5
  )
RETURN movie.title AS title, movie.rating AS rating;
```

The query returns a chain of errors where GQLSTATUS 42I75 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I75: syntax error or access rule violation - self-referencing in search. The expression
```

```
'movie.embedding' in the search clause may not depend on the search clause binding variable `movie`.
(line 5, column 14 (offset: 91))
"    FOR movie.embedding"
             ^
  42001: syntax error or access rule violation - invalid syntax
```

Example 94. Using the search binding variable in the `WHERE` subclause of a `SEARCH` clause

Try using the search binding variable `movie` in both the left-hand and right-hand side of the predicate in the `WHERE` subclause in the following query:

```
CYPHER 25
MATCH (m:Movie {title:'Matrix, The'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    WHERE movie.rating = movie.year
    LIMIT 5
  )
RETURN movie.title AS title, movie.rating AS rating;
```

The query returns a chain of errors where GQLSTATUS 42I75 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I75: syntax error or access rule violation - self-referencing in search. The expression
'movie.year' in the search clause may not depend on the search clause binding variable `movie`. (line
7, column 31 (offset: 166))
"    WHERE movie.rating = movie.year"
                         ^
  42001: syntax error or access rule violation - invalid syntax
```

[[42I78]] = 42I78

Status description

error: syntax error or access rule violation - unsupported procedure or function in language version. The procedure or function is available in `CYPHER { [version] }`. Consider changing the database default Cypher version using `ALTER DATABASE SET DEFAULT LANGUAGE` or prefix the query with `CYPHER { [version] }`.

Example scenario

Try calling a procedure or function that is only available in `CYPHER 25` from a `CYPHER 5` query:

```
CYPHER 5
RETURN coll.indexOf() AS result;
```

The query returns a chain of errors where GQLSTATUS 42I78 is the cause of GQLSTATUS 42N48, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I78: syntax error or access rule violation - unsupported procedure or function in language version. The
```

```
procedure or function is available in `CYPHER 25`. Consider changing the database default Cypher version
using `ALTER DATABASE SET DEFAULT LANGUAGE` or prefix the query with `CYPHER 25`. (line 2, column 8
(offset: 16))
"RETURN coll.indexOf() AS result"
        ^
  42N48: syntax error or access rule violation - no such function. The function coll.indexOf() was not
found. Verify that the spelling is correct.
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

- Prefix the query with the Cypher version in which the procedure or function is available, for example, `CYPHER 25`.
- Change the database default Cypher version using `ALTER DATABASE SET DEFAULT LANGUAGE`.

[[42I79]] = 42I79

Status description

error: syntax error or access rule violation - invalid reference in subclause expression. Aggregation in subclause expression is not allowed to reference variables declared in the same clause `{ [argList] }`.

Explanation

A subclause expression is an expression used in a subclause (e.g., `WHERE` or `ORDER BY`) that belongs to a projection clause. If that expression is aggregating, it is not allowed to reference aliases introduced in the same projection clause; only variables incoming to the clause are allowed.

Example scenario

Try sorting by an aggregation over a variable that the same `RETURN` clause introduces:

Aggregates over parity, which is declared in the same clause

```
UNWIND [1, 2, 3, 4] AS x
RETURN x % 2 AS parity, sum(x) AS total
  ORDER BY sum(parity);
```

`sum(parity)` is an aggregation, and `parity` is introduced by the same `RETURN` clause.

The query returns a chain of errors where GQLSTATUS 42I79 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I79: syntax error or access rule violation - invalid reference in subclause expression. Aggregation in
subclause expression is not allowed to reference variables declared in the same clause: `parity`. (line 3,
column 12 (offset: 76))
"  ORDER BY sum(parity)"
           ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, aggregate over the underlying incoming expression instead of the same-clause alias:

Aggregates over the underlying expression x % 2

```
UNWIND [1, 2, 3, 4] AS x
RETURN x % 2 AS parity, sum(x) AS total
  ORDER BY sum(x % 2);
```

Alternatively, compute the value in a preceding clause so it is an incoming variable rather than one declared in the same clause:

Introduces parity in an earlier clause

```
UNWIND [1, 2, 3, 4] AS x
LET parity = x % 2
RETURN parity, sum(x) AS total
  ORDER BY sum(parity);
```

If the aggregation is meant to reuse a value already projected by the clause, reference that return item by its alias without re-aggregating it:

Sorts by the existing total aggregation

```
UNWIND [1, 2, 3, 4] AS x
RETURN x % 2 AS parity, sum(x) AS total
  ORDER BY total;
```

[[42I80]] = 42I80

Status description

error: syntax error or access rule violation - invalid grouping element. The grouping element `{ [expr] }` is not a valid grouping key. `{ [cause] }`

Explanation

The `GROUP BY` clause is used to group the results of a clause by one or more expressions. Each expression in the `GROUP BY` clause is called a grouping element.

- A grouping element may reference an alias introduced by the same `WITH` or `RETURN` clause only if the grouping element is exactly that alias.
- A non-simple grouping element, such as a property access, a function call, or an arithmetic expression, cannot contain an alias.
- A grouping element cannot contain an aggregation result, since aggregations are computed per group and cannot be grouping keys.

Example scenarios

Let's take the following example scenarios:

Invalid grouping on projection item alias

Try projecting `b AS a`, so that `a` is a projection item alias that shadows the incoming `a`, and then grouping by `a.p` — a property access on that alias:

```
UNWIND [{a: {p: 1}, b: 20}, {a: {p: 2}, b: 10}] AS row
WITH row.a AS a, row.b AS b
RETURN b AS a, count(*) AS cnt
  GROUP BY a.p;
```

Because `a` is a projection item alias, the only valid way to group by it is the simple reference `a`; the property access `a.p` is not a valid grouping element. The query returns a chain of errors where GQLSTATUS 42I80 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I80: syntax error or access rule violation - invalid grouping element. The grouping element 'a.p' is not
a valid grouping key. A grouping element that references the projection item alias `a` must be a simple
variable reference. (line 4, column 13 (offset: 126))
"  GROUP BY a.p"
             ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix the query, the preferred way is to reference the alias as a simple variable:

Group by the alias as a simple variable

```
UNWIND [{a: {p: 1}, b: 20}, {a: {p: 2}, b: 10}] AS row
WITH row.a AS a, row.b AS b
RETURN b AS a, count(*) AS cnt
  GROUP BY a;
```

To group on the incoming value, rename the projection item so it no longer shadows the incoming variable, then group by that expression:

Avoid shadowing the incoming variable

```
UNWIND [{a: {p: 1}, b: 20}, {a: {p: 2}, b: 10}] AS row
WITH row.a AS a, row.b AS b
RETURN b AS bb, count(*) AS cnt
  GROUP BY a.p;
```

Grouping element references an aggregation result

Try grouping by the aggregation alias `cnt`:

```
UNWIND [{a: 1}, {a: 2}] AS row
WITH row.a AS a
RETURN a, count(*) AS cnt
  GROUP BY a, cnt;
```

The query returns a chain of errors where GQLSTATUS 42I80 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42I80: syntax error or access rule violation - invalid grouping element. The grouping element 'cnt' is not
a valid grouping key. A grouping element cannot reference the aggregation `cnt`. (line 4, column 15
(offset: 87))
"  GROUP BY a, cnt"
               ^
  42001: syntax error or access rule violation - invalid syntax
```

To resolve the error remove the aggregation result from the grouping elements:

Group only by non-aggregating expressions

```
UNWIND [{a: 1}, {a: 2}] AS row
WITH row.a AS a
RETURN a, count(*) AS cnt
  GROUP BY a;
```

[[42N00]] = 42N00

Status description

error: syntax error or access rule violation - graph reference not found. A graph reference with the name `{ [db] }` was not found. Verify that the spelling is correct.

Changed in 2025.03

[[42N01]] = 42N01

Status description

error: syntax error or access rule violation - no such constituent graph exists in composite database. The constituent graph `{ [graph] }` was not found in the in composite database `{ [db] }`. Verify that the spelling is correct.

[[42N02]] = 42N02

Status description

error: syntax error or access rule violation - writing in read access mode. Writing in read access mode not allowed.

[[42N03]] = 42N03

Status description

error: syntax error or access rule violation - writing to multiple graphs. Writing to multiple graphs in the same transaction is not allowed. Use `CALL IN TRANSACTION` or create separate transactions in your application.

[[42N04]] = 42N04

Status description

error: syntax error or access rule violation - unsupported access of composite database. Failed to access database identified by `{ [db] 1 }` while connected to session database `{ $db2 }`. Connect to `{ $db3 }` directly.

[[42N05]] = 42N05

Status description

error: syntax error or access rule violation - unsupported access of standard database. Failed to access database identified by `{ [db] 1 }` while connected to composite session database `{ $db2 }`. Connect to `{ $db3 }` directly or create an alias in the composite database.

[[42N06]] = 42N06

Status description

error: syntax error or access rule violation - unsupported action on composite database. `{ [action] }` is not supported on composite databases.

[[42N07]] = 42N07

Status description

error: syntax error or access rule violation - variable shadowing. The variable `{ [variable] }` is shadowing a variable with the same name from the outer scope and needs to be renamed.

[[42N08]] = 42N08

+[role=label—changed-2025.03]

Status description

error: syntax error or access rule violation - no such procedure. The procedure `{ [procFun] }` was not found. Verify that the spelling is correct.

Explanation

The procedure with the given name does not exist.

Example scenarios

Let's take the following example scenarios:

Calling a not existing procedure

Try calling a procedure `db.labelsX` that does not exist or is not available:

```
CALL db.labelsX() YIELD label
RETURN label;
```

The query returns a chain of errors where GQLSTATUS 42N08 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N08: syntax error or access rule violation - no such procedure. The procedure db.labelsX() was not
found. Verify that the spelling is correct.
  42001: syntax error or access rule violation - invalid syntax
```

In this case, verify the spelling of the procedure you want to call.

Available procedures are listed in `SHOW PROCEDURES`. Hence, you can verify if a procedure is available to you with:

```
SHOW PROCEDURES YIELD name, signature WHERE name = "db.labelsX";
```

If the procedure is not available, this query returns an empty result. If the procedure is available, this query returns the name and the signature of the procedure.

[[42N09]] = 42N09

Status description

error: syntax error or access rule violation - no such user. A user with the name `{ [user] }` was not found. Verify that the spelling is correct.

[[42N0A]] = 42N0A

Status description

error: syntax error or access rule violation - invalid shard target. `{ [action] }` is not allowed with a shard target. Connect to `{ [db] 1 }` in order to read or write data from `{ db2 }`.

[[42N0B]] = 42N0B

Status description

error: syntax error or access rule violation - cannot replace sharded database. The database identified by `{ [db] 1 }` is sharded. Drop the database `{ [db] 2 }` before recreating.

[[42N0D]] = 42N0D

Status description

error: syntax error or access rule violation - cannot call function from this context. The function `{ [fun] }` cannot be called from the current context. It can only be used `{ [context] }`.

Explanation

This error is thrown when a function is called from a context that is not allowed for that function.

Example scenario

The example assumes that you have at least one OIDC auth provider present in both `dbms.security.authorization_providers` and in `dbms.security.abac.authorization_providers`.

Try calling an internal security function like `abac.oidc.user_attribute('clearance')` from a regular Cypher query.

```
CYPHER 25
RETURN abac.oidc.user_attribute('clearance');
```

The query returns an error chain where GQLSTATUS 42N0D is the cause of GQLSTATUS 53N37.

GQLSTATUS error

```
42N0D: syntax error or access rule violation - cannot call function from this context. The function
abac.oidc.user_attribute() cannot be called from the current context. It can only be used in 'CREATE AUTH
RULE' and 'ALTER AUTH RULE' commands.
  53N37: function exception - function execution error. Execution of the function
abac.oidc.user_attribute() failed.
```

Possible solution

You can rewrite the query to avoid calling the function from an unsupported context. For example, you can use the function within an `AUTH RULE` creation or alteration command.

```
CREATE AUTH RULE authrule
  SET CONDITION abac.oidc.user_attribute('clearance') = 'high';
```

[[42N0E]] = 42N0E

Status description

error: syntax error or access rule violation - cannot call function without metadata. The function `{ [fun] }` cannot be called without metadata.

Explanation

This error occurs when the metadata required by the function is missing from the context supplied. This is an internal error which would only ever appear in the logs.

Example scenario

A function is called without the required metadata.

[[42N0F]] = 42N0F

Status description

error: syntax error or access rule violation - cannot call function without metadata for realm. The function `{ [fun] }` cannot be called without metadata for realm: `{ [auth] }`.

Explanation

This error occurs when the metadata for a specific security realm is missing from the context supplied. This is an internal error which would only ever appear in the logs.

Example scenario

A function is called with a metadata dictionary, but there is no entry for the required realm.

[[42N10]] = 42N10

Status description

error: syntax error or access rule violation - no such role. A role with the name `{ [role] }` was not found. Verify that the spelling is correct.

[[42N11]] = 42N11

Status description

error: syntax error or access rule violation - graph reference already exists. A graph reference with the name `{ [dbList] }` already exists.

Changed in 2025.05

[[42N12]] = 42N12

Status description

error: syntax error or access rule violation - user already exists. A user with the name `{ [user] }` already exists.

[[42N13]] = 42N13

Status description

error: syntax error or access rule violation - role already exists. A role with the name `{ [role] }` already exists.

[[42N14]] = 42N14

Status description

error: syntax error or access rule violation - invalid use of command. `{ [clause] }` cannot be used together with `{ [cmd] }`.

[[42N15]] = 42N15

Status description

error: syntax error or access rule violation - invalid use of reserved keyword. `{ [syntax] }` is a reserved keyword and cannot be used in this place.

[[42N16]] = 42N16

Status description

error: syntax error or access rule violation - unsupported index or constraint. Only single property `{ [idxType] }` are supported.

[[42N17]] = 42N17

Status description

error: syntax error or access rule violation - unsupported request. `{ [input] }` is not allowed on the `system` database.

Explanation

This error occurs when you try to run a query on the `system` database, which is not allowed.

Example scenarios

Let's take the following example scenarios:

Invalid clause

Try running a query containing a Cypher clause that is not allowed on the `system` database, for example, a `MATCH` clause:

```
MATCH (n) RETURN n;
```

The query returns a chain of errors where GQLSTATUS 42NA9 is the cause of GQLSTATUS 42N17, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NA9: syntax error or access rule violation - system database rules. The system database supports a
restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is
allowed), a subset of show and terminate commands, and combinations of the two. 'YIELD' and 'RETURN' are
also permitted when combined with procedure calls, show, or terminate commands.
  42N17: syntax error or access rule violation - unsupported request. 'MATCH' is not allowed on the system
database.
  42001: syntax error or access rule violation - invalid syntax
```

To solve this, run the query against a standard database.

Invalid procedure

Try running a procedure that is not allowed on the `system` database. For example, you run the following query that contains the `db.createLabel()` procedure, which is not allowed, and `db.index.fulltext.listAvailableAnalyzers()`, which is allowed:

```
CALL db.createLabel('Label')
CALL db.index.fulltext.listAvailableAnalyzers()
YIELD analyzer
RETURN analyzer;
```

The query returns a chain of errors where GQLSTATUS 42NA9 is the cause of GQLSTATUS 42N17, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NA9: syntax error or access rule violation - system database rules. The system database supports a
restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is
allowed), a subset of show and terminate commands, and combinations of the two. 'YIELD' and 'RETURN' are
also permitted when combined with procedure calls, show, or terminate commands.
  42N17: syntax error or access rule violation - unsupported request. 'CALL db.createLabel' is not allowed
on the system database.
  42001: syntax error or access rule violation - invalid syntax
```

To solve this, either remove the disallowed procedure or run the query against a standard database.

Invalid procedure call structure

Try running a query that contains an allowed procedure but an unallowed clause. For example, the `YIELD` clause in the following query is allowed for procedure calls, but the `WHERE` clause is not:

> **Note:**
>
> This example is valid only for versions prior to Neo4j 2026.06. From 2026.06 onwards, the `WHERE` clause, when part of procedure calls, is allowed on the `system` database, and the following example query works as expected.

```
CALL db.index.fulltext.listAvailableAnalyzers()
YIELD analyzer, description
WHERE analyzer = 'swedish'
RETURN description;
```

The query returns a chain of errors where GQLSTATUS 42NA9 is the cause of GQLSTATUS 42N17, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NA9: syntax error or access rule violation - system database rules. The system database supports a
restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is
allowed), a subset of show and terminate commands, and combinations of the two. 'YIELD' and 'RETURN' are
also permitted when combined with procedure calls, show, or terminate commands.
  42N17: syntax error or access rule violation - unsupported request. 'WHERE' is not allowed on the system
database.
  42001: syntax error or access rule violation - invalid syntax
```

To solve this, either remove the `WHERE` clause or run the query against a standard database.

Invalid show command

Try running a `SHOW` command that is not allowed on the `system` database, for example, `SHOW INDEXES`:

```
SHOW INDEXES
YIELD type, createStatement
WHERE type <> 'LOOKUP'
RETURN createStatement;
```

The query returns a chain of errors where GQLSTATUS 42NA9 is the cause of GQLSTATUS 42N17, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NA9: syntax error or access rule violation - system database rules. The system database supports a
restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is
allowed), a subset of show and terminate commands, and combinations of the two. 'YIELD' and 'RETURN' are
also permitted when combined with procedure calls, show, or terminate commands.
  42N17: syntax error or access rule violation - unsupported request. 'SHOW INDEXES' is not allowed on the
system database.
  42001: syntax error or access rule violation - invalid syntax
```

To solve this, run the query against a standard database.

[[42N18]] = 42N18

Status description

error: syntax error or access rule violation - read-only database. The database is in read-only mode.

[[42N19]] = 42N19

Status description

error: syntax error or access rule violation - duplicate clause. Duplicate `{ [syntax] }` clause.

[[42N20]] = 42N20

Status description

error: syntax error or access rule violation - empty list range operator. The list range operator `[ ]` cannot be empty.

[[42N21]] = 42N21

Status description

error: syntax error or access rule violation - unaliased return item. Expression in `{ [clause] }` must be aliased (use AS).

[[42N22]] = 42N22

Status description

error: syntax error or access rule violation - single return column required. A `COLLECT` subquery must end with a single return column.

[[42N23]] = 42N23

Status description

error: syntax error or access rule violation - missing reference to aggregation function. The aggregating function must be included in the `{ [clause] }` clause for use in `ORDER BY …`.

[[42N24]] = 42N24

Status description

error: syntax error or access rule violation - missing `WITH`. A `WITH` clause is required between `{ [input] 1 }` and `{ $input2 }`.

[[42N25]] = 42N25

Status description

error: syntax error or access rule violation - missing `YIELD`. Procedure call inside a query does not support naming results implicitly. Use `YIELD` instead. Available output columns are: `{ [variableList] }`.

Explanation

When a non-void procedure is called inside a query, the results must be explicitly named using a `YIELD` clause. Without `YIELD`, the query fails. The error message includes the available output columns to help construct the correct `YIELD` clause.

Example scenario

Try calling the built-in `db.labels()` procedure inside a query without `YIELD`:

```
MATCH (n)
CALL db.labels()
RETURN *;
```

The query returns a chain of errors where GQLSTATUS 42N25 is the cause of GQLSTATUS 42001:

GQLSTATUS error

```
42N25: syntax error or access rule violation - missing YIELD. Procedure call inside a query does not
support naming results implicitly. Use YIELD instead. Available output columns are: `label`. (line 2,
column 1 (offset: 10))
"CALL db.labels()"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, add a `YIELD` clause naming the output columns you need:

```
MATCH (n)
CALL db.labels() YIELD label
RETURN n, label;
```

To discover a procedure's output signature, use `SHOW PROCEDURES`:

```
neo4j@system> SHOW PROCEDURES YIELD name, signature WHERE name = 'db.labels';
+-----------------------------------------------+
| name        | signature                       |
+-----------------------------------------------+
| "db.labels" | "db.labels() :: (label :: STRING)" |
+-----------------------------------------------+

1 row
```

```
ready to start consuming query after 55 ms, results consumed after another 2 ms
```

[[42N26]] = 42N26

Status description

error: syntax error or access rule violation - multiple join hints on same variable. Multiple join hints for the same variable `{ [variable] }` are not supported.

[[42N28]] = 42N28

Status description

error: syntax error or access rule violation - patterns or variables not statically inferrable. Only statically inferrable patterns and variables are allowed in `{ [input] }`.

[[42N29]] = 42N29

Status description

error: syntax error or access rule violation - unbound variables in pattern expression. Pattern expressions are not allowed to introduce new variables: `{ [variable] }`.

[[42N31]] = 42N31

Status description

error: syntax error or access rule violation - specified number out of range. Expected `{ [component] }` to be `{ [valueType] }` in the range `{ [lower] }` to `{ [upper] }` but found `{ [value] }`.

[[42N32]] = 42N32

Status description

error: syntax error or access rule violation - invalid use of parameter map. Parameter maps cannot be used in `{ [keyword] }` patterns. Use a literal map instead.

[[42N34]] = 42N34

Status description

error: syntax error or access rule violation - path bound in quantified path pattern. Path cannot be bound in a quantified path pattern.

[[42N35]] = 42N35

Status description

error: syntax error or access rule violation - unsupported path selector or explicit path mode in path pattern. The path selector or explicit path mode `{ [selectorOrPathMode] }` is not supported within quantified or parenthesized path patterns.

Example scenario

Try using `SHORTEST` or an explicit path mode like `ACYCLIC` within a quantified path pattern:

Use the `SHORTEST` path selector within a quantified path pattern

```
MATCH ()(SHORTEST 5 PATHS (a)-[r]->(b))*()
RETURN r;
```

The query returns a chain of errors where GQLSTATUS 42N35 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N35: syntax error or access rule violation - unsupported path selector or explicit path mode in path
pattern. The path selector or explicit path mode SHORTEST 5 PATHS is not supported within quantified or
parenthesized path patterns. (line 1, column 27 (offset: 26))
"MATCH ()(SHORTEST 5 PATHS (a)-[r]->(b))*() RETURN r"
                           ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix the error, move the `SHORTEST` clause outside of the quantified path pattern:

```
MATCH SHORTEST 5 PATHS ()((a)-[r]->(b))*()
RETURN r;
```

Use the `ACYCLIC` path mode within a quantified path pattern

```
MATCH ()(ACYCLIC (a)-[r]->(b))*()
RETURN r;
```

The query returns a chain of errors where GQLSTATUS 42N35 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N35: syntax error or access rule violation - unsupported path selector or explicit path mode in path
pattern. The path selector or explicit path mode ACYCLIC is not supported within quantified or
parenthesized path patterns. (line 1, column 18 (offset: 17))
"MATCH ()(ACYCLIC (a)-[r]->(b))*() RETURN r"
                  ^
  42001: syntax error or access rule violation - invalid syntax
```

To fix the error, move the `ACYCLIC` clause outside of the quantified path pattern:

```
MATCH ACYCLIC ()((a)-[r]->(b))*()
RETURN r;
```

[[42N36]] = 42N36

Status description

error: syntax error or access rule violation - procedure call without parentheses. Procedure call is missing parentheses.

[[42N37]] = 42N37

Status description

error: syntax error or access rule violation - invalid use of relationship pattern predicates in variable length relationships. Relationship pattern predicates cannot be use in variable length relationships.

[[42N38]] = 42N38

Status description

error: syntax error or access rule violation - duplicate return item name. Return items must have unique names.

[[42N39]] = 42N39

Status description

error: syntax error or access rule violation - incompatible return column names. All `{ [context] }` must have the same return column names. Use `AS` to ensure columns have the same name.

Changed in 2025.03

Explanation

Where tabular results of branches of a query are combined into one tabular result, e.g. in `UNION` or `WHEN`, the branch results have to have the same column names.

Example scenarios

Let's take the following example scenarios:

Return columns without explicit names in `UNION`

Try running `UNION` queries with unnamed return columns:

```
RETURN 1
UNION
RETURN 2;
```

The query returns a chain of errors where GQLSTATUS 42N39 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N39: syntax error or access rule violation - incompatible return column names. All UNION subqueries must
have the same return column names. Use `AS` to ensure columns have the same name. (line 2, column 1
(offset: 9))
"UNION"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, the unnamed return columns are automatically named, which can cause the effective column names to differ (`` `1` `` vs. `` `2` ``). Explicitly name the return columns to the same name.

Named return columns in `UNION`

Try running `UNION` queries with differently named return columns:

```
RETURN 1 AS a
UNION
RETURN 2 AS b;
```

The query returns a chain of errors where GQLSTATUS 42N39 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N39: syntax error or access rule violation - incompatible return column names. All UNION subqueries must
have the same return column names. Use `AS` to ensure columns have the same name. (line 2, column 1
(offset: 14))
"UNION"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, the explicitly provided names of the return columns are different (`a` vs. `b`). Adjust the names to be the same.

Named return columns in `WHEN`

Try running conditional queries with differently named return columns:

```
WHEN true THEN RETURN 1 AS a
WHEN false THEN RETURN 2 AS b
ELSE RETURN 3 AS a;
```

The query returns a chain of errors where GQLSTATUS 42N39 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N39: syntax error or access rule violation - incompatible return column names. All conditional queries
must have the same return column names. Use `AS` to ensure columns have the same name. (line 2, column 1
(offset: 29))
"WHEN false THEN RETURN 2 AS b"
 ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, the explicitly provided names of the return columns are different (`a` vs. `b` vs. `a`). Adjust the names to be the same.

[[42N3A]] = 42N3A

Status description

error: syntax error or access rule violation - incompatible conditional query. All `{ [context] }` need to either return rows or update the graph.

[[42N3B]] = 42N3B

Status description

error: syntax error or access rule violation - incompatible number of return columns. All `{ [context] }` must return the same number of columns.

[[42N3C]] = 42N3C

Status description

error: syntax error or access rule violation - invalid use of `CALL { … }`. Not possible to enclose `{ [clause] }` in `CALL { … }`. Use `CALL () { … }` instead.

[[42N40]] = 42N40

Status description

error: syntax error or access rule violation - single relationship pattern required. The `{ [fun] }` function must contain one relationship pattern.

[[42N41]] = 42N41

Status description

error: syntax error or access rule violation - missing `|-expression`. The reduce function requires a `| expression` after the accumulator.

[[42N42]] = 42N42

Status description

error: syntax error or access rule violation - unsupported sub-path binding. Sub-path assignment is not supported.

[[42N44]] = 42N44

Status description

error: syntax error or access rule violation - inaccessible variable. It is not possible to access the variable `{ [variable] }` declared before the `{ [clause] }` clause when using `{ [groupingConstructs] }`.

Explanation

A `WITH` or `RETURN` clause that uses `DISTINCT`, an aggregation, or a `GROUP BY` clause defines a new set of available variables: grouping keys, projection item aliases, and aggregation results. Any of its subclauses (`ORDER BY`, `WHERE`) can therefore reference only those grouping keys, projection item aliases, and aggregation results, not the variables that are in scope before the clause. Referencing a variable that is declared before such a clause, and that is not a grouping key, projection item alias, or an aggregation result, is not possible and results in this error.

Example scenario

Try grouping by `a` and aggregating with `count(*)`, and then sorting on `b`, which is not a grouping key, a

projection item, or an aggregation result:

```
UNWIND [{a: 1, b: 10}, {a: 1, b: 20}, {a: 2, b: 30}] AS row
WITH row.a AS a, row.b AS b
RETURN a, count(*) AS cnt
  GROUP BY a
  ORDER BY b;
```

Because `b` is declared before the `RETURN` clause and is not part of the grouping, it cannot be accessed by the `ORDER BY` subclause. The query returns a chain of errors where GQLSTATUS 42N44 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N44: syntax error or access rule violation - inaccessible variable. It is not possible to access the
variable `b` declared before the RETURN clause when using `DISTINCT`, an aggregation, or a `GROUP BY`
clause. (line 5, column 12 (offset: 138))
"  ORDER BY b"
           ^
  42001: syntax error or access rule violation - invalid syntax
```

To resolve the error, either add `b` as a grouping key (`GROUP BY a, b`), or order by a grouping key or aggregation result (for example, `ORDER BY a` or `ORDER BY cnt`). For example:

```
UNWIND [{a: 1, b: 10}, {a: 1, b: 20}, {a: 2, b: 30}] AS row
WITH row.a AS a, row.b AS b
RETURN a, count(*) AS cnt
  GROUP BY a, b
  ORDER BY b;
```

[[42N45]] = 42N45

Status description

error: syntax error or access rule violation - unexpected end of input. Unexpected end of input, expected `CYPHER`, `EXPLAIN`, `PROFILE` or a query.

[[42N47]] = 42N47

Status description

error: syntax error or access rule violation - invalid use of `UNION` and `CALL IN TRANSACTIONS`. `CALL { … } IN TRANSACTIONS` is not supported in `… UNION …`.

[[42N48]] = 42N48

Status description

error: syntax error or access rule violation - no such function. The function `{ [fun] }` was not found. Verify that the spelling is correct.

Explanation

The function with the given name does not exist.

Example scenarios

Let's take the following example scenarios:

Calling a not existing function

Try calling a function `piX` that does not exist or is not available:

```
RETURN piX();
```

The query returns a chain of errors where GQLSTATUS 42N48 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N48: syntax error or access rule violation - no such function. The function piX() was not found. Verify
that the spelling is correct. (line 1, column 8 (offset: 7))
"RETURN piX()"
        ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, verify the spelling of the function you want to invoke.

Available functions are listed in `SHOW FUNCTIONS`. Hence, you can verify if a function is available to you with:

```
SHOW FUNCTIONS YIELD name, signature WHERE name = "piX";
```

If the function is not available, this query returns an empty result. If the function is available, this query returns the name and the signature of the function.

[[42N49]] = 42N49

Status description

error: syntax error or access rule violation - unsupported normal form. Unknown Normal Form: `{ [input] }`.

[[42N50]] = 42N50

Status description

error: syntax error or access rule violation - procedure return column not defined. The procedure return column `{ [arg] }` is not defined for this procedure. Available output columns are: `{ [argList] }`. Verify that the spelling is correct.

Explanation

When calling a procedure with a `YIELD` clause, all yielded column names must match the procedure's output signature. If a column name is misspelled or does not exist in the procedure's output, the query will fail. The error message includes the available output columns to help identify the correct name.

Example scenario

For example, when calling the built-in `dbms.components()` procedure and yielding a non-existent column:

Returns a non-existent column `nonExistent`

```
CALL dbms.components() YIELD nonExistent
RETURN nonExistent;
```

The query returns a chain of errors where GQLSTATUS 42N50 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N50: syntax error or access rule violation - procedure return column not defined. The procedure return
column 'nonExistent' is not defined for this procedure. Available output columns are: 'edition', 'name'
and 'versions'. Verify that the spelling is correct. (line 1, column 30 (offset: 29))
"CALL dbms.components() YIELD nonExistent"
                             ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, replace the invalid column name with at least one of the available output columns listed in the error message:

Returns all available columns

```
CALL dbms.components() YIELD name, versions, edition
RETURN name, versions, edition;
```

Returns only the `name` *column*

```
CALL dbms.components() YIELD name
RETURN name;
```

To discover a procedure's output signature, use `SHOW PROCEDURES`:

```
neo4j@system> SHOW PROCEDURES YIELD name, signature WHERE name = 'dbms.components';
+-------------------------------------------------------------------------------------------------------------
-+
| name              | signature
|
+-------------------------------------------------------------------------------------------------------------
-+
| "dbms.components" | "dbms.components() :: (name :: STRING, versions :: LIST<STRING>, edition :: STRING)"
|
+-------------------------------------------------------------------------------------------------------------
-+

1 row
ready to start consuming query after 321 ms, results consumed after another 11 ms
```

[[42N51]] = 42N51

Status description

error: syntax error or access rule violation - invalid parameter. Invalid parameter `{ [param] }`.

[[42N52]] = 42N52

Status description

error: syntax error or access rule violation - invalid value type. `{ [input] }` is not a recognized Cypher type.

[[42N53]] = 42N53

Status description

error: syntax error or access rule violation - unsafe usage of repeatable elements. The quantified path pattern may yield an infinite number of rows under match mode `REPEATABLE ELEMENTS`. Add an upper bound to the quantified path pattern.

[[42N54]] = 42N54

Status description

error: syntax error or access rule violation - unsupported match mode. The match mode `{ [matchMode] }` is not supported.

[[42N56]] = 42N56

Status description

error: syntax error or access rule violation - unsupported use of properties. Properties are not supported in the `{ [fun] }` function.

[[42N57]] = 42N57

Status description

error: syntax error or access rule violation - invalid use of data-modifications in expressions. `{ [expr] }` cannot contain any updating clauses.

Explanation

The given expression is not allowed to contain updating clauses, `CREATE`, `INSERT`, `MERGE`, `SET`, `REMOVE`, and `DELETE`.

Example scenarios

Let's take the following example scenarios:

Data-modifications in an `EXISTS` subquery

Try setting a label in an `EXISTS` subquery:

```
RETURN EXISTS { MATCH (a:A) SET a:Found RETURN a };
```

The query returns a chain of errors where GQLSTATUS 42N57 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N57: syntax error or access rule violation - invalid use of data-modifications in expressions. 'Exists'
cannot contain any updating clauses. (line 1, column 8 (offset: 7))
"RETURN EXISTS { MATCH (a:A) SET a:Found RETURN a }"
        ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, remove the `SET` clause from the `EXISTS` subquery.

Data-modifications in an `COUNT` subquery

Try setting a property in an `COUNT` subquery:

```
RETURN COUNT { MATCH (a:A) SET a.prop = 1 RETURN a };
```

The query returns a chain of errors where GQLSTATUS 42N57 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N57: syntax error or access rule violation - invalid use of data-modifications in expressions. 'Count'
cannot contain any updating clauses. (line 1, column 8 (offset: 7))
"RETURN COUNT { MATCH (a:A) SET a.prop = 1 RETURN a }"
        ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, remove the `SET` clause from the `COUNT` subquery.

Data-modifications in a `COLLECT` subquery

Try setting a property in a `COLLECT` subquery:

```
RETURN COLLECT { UNWIND range(1,10) AS x CREATE (a:A {prop: x}) RETURN a };
```

The query returns a chain of errors where GQLSTATUS 42N57 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N57: syntax error or access rule violation - invalid use of data-modifications in expressions. 'Collect'
cannot contain any updating clauses. (line 1, column 8 (offset: 7))
"RETURN COLLECT { UNWIND range(1,10) AS x CREATE (a:A {prop: x}) RETURN a }"
        ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, remove the `CREATE` clause from the `COLLECT` subquery.

[[42N58]] = 42N58

Status description

error: syntax error or access rule violation - unsupported use of nesting. Nested `CALL { … } IN TRANSACTIONS` is not supported.

[[42N59]] = 42N59

Status description

error: syntax error or access rule violation - variable already defined. Variable `{ [variable] }` already declared.

Explanation

A clause tries to declare a variable already in scope.

Example scenarios

Let's take the following example scenarios:

Redefine a variable in `CREATE`

Try binding a variable that is already in scope in a `CREATE` clause:

```
MATCH (n)
CREATE (n);
```

The query returns a chain of errors where GQLSTATUS 42N59 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N59: syntax error or access rule violation - variable already defined. Variable `n` already declared.
(line 2, column 9 (offset: 18))
"CREATE (n)"
         ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, name the variable differently or descope the original variable using `WITH`.

Redefine a variable in `LET`

Try binding a variable that is already in scope in a `LET` clause:

```
UNWIND [1, 2] AS x
MATCH (n)
LET x = 2
RETURN n, x;
```

The query returns a chain of errors where GQLSTATUS 42N59 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N59: syntax error or access rule violation - variable already defined. Variable `x` already declared.
(line 3, column 5 (offset: 33))
"LET x = 2"
     ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, name the variable differently or descope the original variable using `WITH`.

Redefine a variable in `UNWIND`

Try binding a variable that is already in scope in a `UNWIND` clause:

```
WITH 2 AS x
MATCH (n)
UNWIND [1, 2] AS x
RETURN n, x;
```

The query returns a chain of errors where GQLSTATUS 42N59 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N59: syntax error or access rule violation - variable already defined. Variable `x` already declared.
(line 3, column 18 (offset: 39))
"UNWIND [1, 2] AS x"
                 ^
  42001: syntax error or access rule violation - invalid syntax
```

In this case, name the variable differently or descope the original variable using `WITH`.

[[42N60]] = 42N60

Status description

error: syntax error or access rule violation - unsupported combination of match mode and path mode. `REPEATABLE ELEMENTS` with `{ [pathMode] }` path mode is not supported.

Example scenario

For example, in the following query `REPEATABLE ELEMENTS` is used with the `TRAIL` path mode:

```
MATCH REPEATABLE ELEMENTS p = TRAIL (:A)-[:R]->{,10}(:B)
RETURN p;
```

The query returns a chain of errors where GQLSTATUS 42N60 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N60: syntax error or access rule violation - unsupported combination of match mode and path mode.
REPEATABLE ELEMENTS with TRAIL path mode is not supported. (line 1, column 31 (offset: 30))
"MATCH REPEATABLE ELEMENTS p = TRAIL (:A)-[:R]->{,10}(:B)"
                               ^
  42001: syntax error or access rule violation - invalid syntax
```

[[42N61]] = 42N61

Status description

error: syntax error or access rule violation - unsupported mixing of different path modes. Mixing `{ [pathModeList] }` in the same graph pattern is not supported. Split the pattern into separate `MATCH` clauses instead.

Example scenario

For example, in the following query `TRAIL` and `ACYCLIC` are used:

```
MATCH p = ACYCLIC (a:A)-[:P]->+(b:B),
      q = TRAIL (b)-[:Q]->+(a)
RETURN p, q;
```

The query returns a chain of errors where GQLSTATUS 42N61 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N61: syntax error or access rule violation - unsupported mixing of different path modes. Mixing ACYCLIC
and TRAIL in the same graph pattern is not supported. Split the pattern into separate MATCH clauses
instead. (line 1, column 7 (offset: 6))
"MATCH p = ACYCLIC (a:A)-[:P]->+(b:B),"
       ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solution

Split the graph pattern into separate `MATCH` clauses and use predicates to enforce relationship uniqueness across the set of paths.

For example, the above query can be rewritten as:

```
MATCH p = ACYCLIC (a:A)-[r1:P]->+(b:B)
MATCH q = TRAIL (b)-[r2:Q WHERE NOT r2 IN r1]->+(a)
RETURN p, q;
```

Here the predicate `NOT r2 IN r1` ensures that the relationships in path `q` do not overlap with those in path `p`, replicating the effect of Cypher's default match mode `DIFFERENT RELATIONSHIPS`.

[[42N62]] = 42N62

Status description

error: syntax error or access rule violation - variable not defined. Variable `{ [variable] }` not defined.

[[42N63]] = 42N63

Status description

error: syntax error or access rule violation - inner type with different nullability. All inner types in a Closed Dynamic Union must be nullable, or be appended with `NOT NULL`.

[[42N64]] = 42N64

Status description

error: syntax error or access rule violation - at least one node or relationship required. A quantified or parenthesized path pattern must have at least one node or relationship pattern.

[[42N65]] = 42N65

Status description

error: syntax error or access rule violation - node variable not bound. The `{ [fun] }` function requires bound node variables when it is not part of a `MATCH …`.

[[42N66]] = 42N66

Status description

error: syntax error or access rule violation - relationship variable already bound. Bound relationships are not allowed in calls to the `{ [fun] }` function.

[[42N68]] = 42N68

Status description

error: syntax error or access rule violation - duplicate variable definition. Variables cannot be defined more than once in a `{ [clause] }` clause.

[[42N69]] = 42N69

Status description

error: syntax error or access rule violation - function not allowed inside expression. The `{ [fun] }` function is only allowed as a top-level element and not inside an `{ [expr] }`.

[[42N70]] = 42N70

Status description

error: syntax error or access rule violation - function without required WHERE clause. The function `{ [fun] }` requires a WHERE clause.

[[42N71]] = 42N71

Status description

error: syntax error or access rule violation - incomplete query. A query must conclude with a `RETURN` clause, a `FINISH` clause, an update clause, a unit subquery call, or a procedure call without a `YIELD` clause.

[[42N72]] = 42N72

Status description

error: syntax error or access rule violation - graph function only supported on composite databases. Calling graph functions is only supported on composite databases. Use the name directly or connect to a composite database with the desired constituents.

[[42N73]] = 42N73

Status description

error: syntax error or access rule violation - invalid placement of `USE` clause. The `USE` clause must be the

first clause of a query or an operand to `… UNION …`. In a `CALL` sub-query, it can also be the second clause if the first clause is an importing `WITH`.

[[42N74]] = 42N74

Status description

error: syntax error or access rule violation - invalid nested USE clause. Failed to access `{ [db] 1 }` and `{ $db2 }`. Child USE clauses must target the same graph as their parent query. Run in separate (sub)queries instead.

[[42N75]] = 42N75

Status description

error: syntax error or access rule violation - invalid use of graph function. A call to the graph function `{ [fun] }` is only allowed as the top-level argument of a `USE` clause.

[[42N76]] = 42N76

Status description

error: syntax error or access rule violation - unfulfillable hints. The hint(s) `{ [hintList] }` cannot be fulfilled.

[[42N77]] = 42N77

Status description

error: syntax error or access rule violation - missing hint predicate. The hint `{ [hint] }` cannot be fulfilled. The query does not contain a compatible predicate for `{ [entityType] }` on `{ [variable] }`.

[[42N78]] = 42N78

Status description

error: syntax error or access rule violation - variable already bound. Node `{ [variable] }` has already been bound and cannot be modified by the `{ [clause] }` clause.

[[42N79]] = 42N79

Status description

error: syntax error or access rule violation - invalid `USE` clause. The `USE` clause is not required for administration commands. Retry the query without the `USE` clause, and it will be routed automatically.

Explanation

This error occurs when an administration command is prepended with a `USE` clause.

Example scenario

Try showing all the users with a `USE` clause in the query:

```
USE system
SHOW USERS;
```

The query returns an error with GQLSTATUS 42N79 and status description:

```
42N79: syntax error or access rule violation - invalid USE clause. The USE clause is not required for
administration commands. Retry the query without the USE clause, and it will be routed automatically.
```

[[42N7A]] = 42N7A

Status description

error: syntax error or access rule violation - disjoint by requires concurrent transactions. `'DISJOINT BY'` can only be used in `'CALL { … } IN CONCURRENT TRANSACTIONS'`.

Example scenario

Try using `DISJOINT BY` in a `CALL` subquery that runs `IN TRANSACTIONS` without the `CONCURRENT` keyword:

```
UNWIND [1, 2, 3] AS i
CALL (i) {
  MATCH (n {id: i})
  SET n.processed = true
} IN TRANSACTIONS DISJOINT BY (i);
```

The query returns a chain of errors where GQLSTATUS 42N7A is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N7A: syntax error or access rule violation - disjoint by requires concurrent transactions. 'DISJOINT BY'
can only be used in 'CALL { ... } IN CONCURRENT TRANSACTIONS'. (line 5, column 19 (offset: 96))
"} IN TRANSACTIONS DISJOINT BY (i)"
                   ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, you can add the `CONCURRENT` keyword if concurrent execution is intended. Be aware that this changes how the query executes, as the transaction batches then run concurrently.

```
UNWIND [1, 2, 3] AS i
CALL (i) {
  MATCH (n {id: i})
  SET n.processed = true
} IN CONCURRENT TRANSACTIONS DISJOINT BY (i);
```

Alternatively, you can remove the `DISJOINT BY` clause if sequential execution is intended. Sequential transactions run one at a time and do not need disjoint partitioning.

```
UNWIND [1, 2, 3] AS i
CALL (i) {
  MATCH (n {id: i})
  SET n.processed = true
} IN TRANSACTIONS;
```

[[42N7B]] = 42N7B

Status description

error: syntax error or access rule violation - non-deterministic disjoint by expression. `'DISJOINT BY'` expressions must be deterministic.

Example scenario

For example, when the `DISJOINT BY` expression is non-deterministic, such as `rand()`:

```
UNWIND [1, 2, 3] AS i
CALL (i) {
  MATCH (n {id: i})
  SET n.processed = true
} IN CONCURRENT TRANSACTIONS DISJOINT BY (rand());
```

The query returns a chain of errors where GQLSTATUS 42N7B is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N7B: syntax error or access rule violation - non-deterministic disjoint by expression. 'DISJOINT BY'
expressions must be deterministic. (line 5, column 43 (offset: 120))
"} IN CONCURRENT TRANSACTIONS DISJOINT BY (rand())"
                                           ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, replace the non-deterministic expression with a deterministic expression over the row variables, for example the variable itself.

```
UNWIND [1, 2, 3] AS i
CALL (i) {
  MATCH (n {id: i})
  SET n.processed = true
} IN CONCURRENT TRANSACTIONS DISJOINT BY (i);
```

[[42N7C]] = 42N7C

Status description

error: syntax error or access rule violation - subquery in disjoint by expression. `'DISJOINT BY'` expressions must not contain subquery expressions.

Example scenario

For example, when the `DISJOINT BY` expression contains a subquery expression, such as `COUNT { … }`:

```
UNWIND [1, 2, 3] AS i
CALL (i) {
  MATCH (n {count: COUNT { MATCH (m) WHERE m.id = i RETURN m }})
  SET n.processed = true
} IN CONCURRENT TRANSACTIONS DISJOINT BY (COUNT { MATCH (m) WHERE m.id = i RETURN m });
```

The query returns a chain of errors where GQLSTATUS 42N7C is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N7C: syntax error or access rule violation - subquery in disjoint by expression. 'DISJOINT BY'
expressions must not contain subquery expressions. (line 5, column 43 (offset: 165))
"} IN CONCURRENT TRANSACTIONS DISJOINT BY (COUNT { MATCH (m) WHERE m.id = i RETURN m })"
                                           ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, project the subquery expression before the `CALL` clause and import the result as a variable.

```
UNWIND [1, 2, 3] AS i
WITH COUNT { MATCH (m) WHERE m.id = i RETURN m } AS c
CALL (c) {
  MATCH (n {count: c})
  SET n.processed = true
} IN CONCURRENT TRANSACTIONS DISJOINT BY (c);
```

[[42N81]] = 42N81

Status description

error: syntax error or access rule violation - missing request parameter. Expected `{ [paramList] 1 }`, but got `{ [paramList] 2 }`.

Example scenario

Try running the following Cypher query without providing the parameters:

```
RETURN $a + $b + $c;
```

The query returns a chain of errors where GQLSTATUS 42N81 is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N81: syntax error or access rule violation - missing request parameter. Expected $`a`, $`b` and $`c`,
but got nothing.
  42001: syntax error or access rule violation - invalid syntax
```

[[42N82]] = 42N82

Status description

error: syntax error or access rule violation - cannot drop database with aliases. The database identified by

`{ [db] }` has one or more aliases. Drop the aliases `{ [aliasList] }` before dropping the database.

[[42N83]] = 42N83

Status description

error: syntax error or access rule violation - impersonation disallowed while password change required. Cannot impersonate a user while password change required.

[[42N84]] = 42N84

Status description

error: syntax error or access rule violation - `TERMINATE TRANSACTION` misses `YIELD` clause. `WHERE` clause without `YIELD` clause. Use `TERMINATE TRANSACTION … YIELD … WHERE …`.

[[42N85]] = 42N85

Status description

error: syntax error or access rule violation - cannot specify both allowed and denied databases. Allowed and denied database options are mutually exclusive.

[[42N86]] = 42N86

Status description

error: syntax error or access rule violation - wildcard in parameter. `{ [syntax] }` failed. Parameterized database and graph names do not support wildcards.

[[42N87]] = 42N87

Status description

error: syntax error or access rule violation - database or alias with similar name exists. The database or alias name `{ [db] 1 }` conflicts with the name `{ [db] 2 }` of an existing database or alias.

[[42N88]] = 42N88

Status description

error: syntax error or access rule violation - cannot grant privilege. Permission cannot be granted for `REMOVE IMMUTABLE PRIVILEGE`.

[[42N89]] = 42N89

Status description

error: syntax error or access rule violation - invalid driver settings map. Failed evaluating the given driver settings. `{ [cause] }`

[[42N90]] = 42N90

Status description

error: syntax error or access rule violation - cannot alter immutable composite database. Composite databases cannot be altered (database: `{ [db] }`).

[[42N92]] = 42N92

Status description

error: syntax error or access rule violation - cannot combine old and new auth provider syntax. Cannot combine old and new auth syntax for the same auth provider.

[[42N94]] = 42N94

Status description

error: syntax error or access rule violation - incomplete `ALTER USER` command. `'ALTER USER'` requires at least one clause.

[[42N95]] = 42N95

Status description

error: syntax error or access rule violation - provider-id combination already exists. The combination of provider and id is already in use.

[[42N96]] = 42N96

Status description

error: syntax error or access rule violation - invalid user configuration. User has no auth provider. Add at least one auth provider for the user or consider suspending them.

[[42N97]] = 42N97

Status description

error: syntax error or access rule violation - missing mandatory auth clause. Clause `{ [clause] }` is mandatory for auth provider `{ [auth] }`.

[[42N98]] = 42N98

Status description

error: syntax error or access rule violation - cannot modify own user. Cannot modify the user record of the current user.

[[42N99]] = 42N99

Status description

error: syntax error or access rule violation - cannot delete own user. Cannot delete the user record of the current user.

[[42NA0]] = 42NA0

Status description

error: syntax error or access rule violation - operations must be executed on constituent. Query contains operations that must be executed on the constituent.

[[42NA1]] = 42NA1

Status description

error: syntax error or access rule violation - graph access operations on composite database. Graph access operations are not supported on composite databases.

[[42NA2]] = 42NA2

Status description

error: syntax error or access rule violation - database operations on composite database. Database operations are not supported on composite databases.

[[42NA3]] = 42NA3

Status description

error: syntax error or access rule violation - schema operations on composite database. Schema operations are not supported on composite databases.

[[42NA4]] = 42NA4

Status description

error: syntax error or access rule violation - transaction operations on composite database. Transaction operations are not supported on composite databases.

[[42NA5]] = 42NA5

Status description

error: syntax error or access rule violation - accessing multiple graphs only supported on composite databases. Accessing multiple graphs in the same query is only supported on composite databases. Connect to a composite database with the desired constituents.

[[42NA6]] = 42NA6

Status description

error: syntax error or access rule violation - invalid alias target. Aliases are not allowed to target composite databases.

[[42NA7]] = 42NA7

Status description

error: syntax error or access rule violation - referenced database not found. No database is corresponding to `{ [db] }`. Verify that the elementId is correct.

[[42NA8]] = 42NA8

Status description

error: syntax error or access rule violation - invalid reference in command. Invalid reference in command `{ [cmd] }`.

Explanation

This error occurs as the cause of other exceptions, for example, when you try to:

- Grant or deny privileges to, or revoke privileges from, a role that does not exist.
- Grant or deny privileges to, or revoke privileges from, a role to perform administration commands on a database that does not exist.
- Revoke a role from a user in Cypher 25, where either the role or the user does not exist. In Cypher 5, this last example gives a notification instead, see Impossible revoke command.

Example scenario

For example, given that `role1` exists but `role2` does not, try granting a privilege to both roles:

```
GRANT WRITE ON GRAPH * TO role1, role2;
```

The query returns a chain of errors where GQLSTATUS 42N10 is the cause of GQLSTATUS 42NA8, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42N10: syntax error or access rule violation - no such role. A role with the name `role2` was not found.
Verify that the spelling is correct.
  42NA8: syntax error or access rule violation - invalid reference in command. Invalid reference in
command 'GRANT WRITE ON GRAPH * TO role2'.
  42001: syntax error or access rule violation - invalid syntax
```



Note:

The command in the status description might not be the exact command you typed. Internally, the Neo4j administration commands are separated into more specific sub-commands, and the Cypher shown corresponds to the sub-command that was executed

when the error occurred.

[[42NA9]] = 42NA9

Status description

error: syntax error or access rule violation - system database rules. The system database supports a restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is allowed), a subset of show and terminate commands, and combinations of the two. `YIELD` and `RETURN` are also permitted when combined with procedure calls, show, or terminate commands.

Explanation

This error occurs as the cause of 42N17, when you try to run a query on the `system` database, which is not allowed.

Example scenarios

Let's take the following example scenarios:

Invalid clause

Try running a query containing a Cypher clause that is not allowed on the `system` database. For example, `MATCH` clauses are not allowed on the `system` database:

```
MATCH (n) RETURN n;
```

The query returns a chain of errors where GQLSTATUS 42NA9 is the cause of GQLSTATUS 42N17, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NA9: syntax error or access rule violation - system database rules. The system database supports a
restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is
allowed), a subset of show and terminate commands, and combinations of the two. 'YIELD' and 'RETURN' are
also permitted when combined with procedure calls, show, or terminate commands.
  42N17: syntax error or access rule violation - unsupported request. 'MATCH' is not allowed on the system
database.
  42001: syntax error or access rule violation - invalid syntax
```

To solve this, run the query against a standard database.

Invalid procedure

Try running a procedure that is not allowed on the `system` database. For example, you run the following query that contains the `db.createLabel()` procedure, which is not allowed, and `db.index.fulltext.listAvailableAnalyzers()`, which is allowed:

```
CALL db.createLabel('Label')
CALL db.index.fulltext.listAvailableAnalyzers()
YIELD analyzer
RETURN analyzer;
```

The query returns a chain of errors where GQLSTATUS 42NA9 is the cause of GQLSTATUS 42N17, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NA9: syntax error or access rule violation - system database rules. The system database supports a
restricted set of Cypher clauses. The supported clauses include procedure calls (if the procedure is
allowed), a subset of show and terminate commands, and combinations of the two. 'YIELD' and 'RETURN' are
also permitted when combined with procedure calls, show, or terminate commands.
  42N17: syntax error or access rule violation - unsupported request. 'CALL db.createLabel' is not allowed
on the system database.
  42001: syntax error or access rule violation - invalid syntax
```

To solve this, either remove the disallowed procedure or run the query against a standard database.

[[42NAA]] = 42NAA

Status description

error: syntax error or access rule violation - incorrectly formatted graph reference. Incorrectly formatted graph reference `{ [input] }`. Expected a single quoted or unquoted identifier. Separate name parts should not be quoted individually.

Explanation

As of Cypher 25, database names and aliases cannot include name parts that are quoted individually. Either the whole name must be quoted, or the whole name must be unquoted. A recommended approach, which is consistent between different commands and Cypher versions, is to send in the database name or alias as a parameter rather than a string literal.

Example scenario

Try creating a database with using the following Cypher 25 command:

```
CYPHER 25 CREATE ALIAS `foo`.`bar` FOR DATABASE baz;
```

The query returns a chain of errors where GQLSTATUS 42NAA is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NAA: syntax error or access rule violation - incorrectly formatted graph reference. Incorrectly
formatted graph reference '`foo`.`bar`'. Expected a single quoted or unquoted identifier. Separate name
parts should not be quoted individually. (line 1, column 24 (offset: 23))
"CYPHER 25 CREATE ALIAS `foo`.`bar` FOR DATABASE baz"
                       ^
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, the preferred way is to replace the alias name with a parameter:

Parameters

```
{
  "aliasName": "foo.bar"
```

```
}
```

Query

```
CREATE ALIAS $aliasName FOR DATABASE baz;
```

Alternatively, if you use a string literal, the correct quoting for Cypher 25 will either be

```
CYPHER 25 CREATE ALIAS `foo.bar` FOR DATABASE baz;
```

or

```
CYPHER 25 CREATE ALIAS foo.bar FOR DATABASE baz;
```

[[42NAB]] = 42NAB

Status description

error: syntax error or access rule violation - not supported standalone call. `WHERE` is not supported in a standalone call. Use `CALL … YIELD … WHERE … RETURN …` instead.

Explanation

A standalone call of a procedure does not support a `WHERE` clause, i.e., it does not support the filtering of the procedure result. Either remove the `WHERE` clause or add a `RETURN` clause after the call to turn it into an in-query call.

Example scenario

Try filtering the result of the procedure `db.labels()` in a standalone call:

```
CALL db.labels() YIELD label WHERE label <> 'A';
```

The query returns a chain of errors where GQLSTATUS 42NAB is the cause of GQLSTATUS 42N71, which is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NAB: syntax error or access rule violation - not supported standalone call. WHERE is not supported in a
standalone call. Use `CALL ... YIELD ... WHERE ... RETURN ...` instead. (line 1, column 30 (offset: 29))
"CALL db.labels() YIELD label WHERE label <> 'A'"
                              ^
  42N71: syntax error or access rule violation - incomplete query. A query must conclude with a RETURN
clause, a FINISH clause, an update clause, a unit subquery call, or a procedure call without a YIELD
clause.
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

You can abstain from filtering to get a valid query:

```
CALL db.labels() YIELD label;
```

However, if you still want to filter the result of the procedure `db.labels()`, you can add a `RETURN` clause:

Query

```
CALL db.labels() YIELD label WHERE label <> 'A'
RETURN label;
```

[[42NAC]] = 42NAC

Status description

error: syntax error or access rule violation - invalid `CREATE DATABASE` statement. The backup metadata script contains an invalid or missing `CREATE DATABASE` statement. The metadata script must contain exactly one `CREATE DATABASE` statement and it must use the parameter `$database`.

Explanation

When using the `existingMetadata: use` option to restore a database from a backup, the backup metadata script must contain exactly one `CREATE DATABASE` statement. This statement must use the parameter `$database` to specify the name of the database being restored. If the metadata script is missing this statement or contains an invalid statement, the `CREATE DATABASE` operation will fail with this error.

Example scenario

Try creating a database using the following command:

```
CREATE DATABASE `$databaseString` OPTIONS {existingMetadata:'use', seedUri:'$uri', existingData:'use'};
```

If the backup at `$uri` is missing the required `CREATE DATABASE` statement, the command will return an error with GQLSTATUS 42NAC.

Possible solutions

To resolve this issue, use another backup with a valid metadata script.

[[42NAD]] = 42NAD

Status description

error: syntax error or access rule violation - no such auth rule. An auth rule with the name `{ [authRule] }` was not found. Verify that the spelling is correct.

Explanation

When referencing an authorization rule in a command, the specified auth rule must already exist. If the auth rule does not exist, the operation will fail with this error.

Example scenarios

Let's take the following example scenarios:

Dropping an auth rule

Try dropping an auth rule using the following command:

```
DROP AUTH RULE authRuleName;
```

If the auth rule `authRuleName` does not exist, the query will return a chain of errors where GQLSTATUS 42NAD is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NAD: syntax error or access rule violation - no such auth rule. An auth rule with the name
`authRuleName` was not found. Verify that the spelling is correct.
  42001: syntax error or access rule violation - invalid syntax
```

To resolve this issue, make sure that the specified auth rule exists before calling the drop command.

Another solution is to use the `IF EXISTS` clause in the drop command to avoid the error if the auth rule does not exist.

Granting a role to an auth rule

Try granting a role to an auth rule using the following command:

```
GRANT ROLE roleName TO AUTH RULE authRuleName;
```

If the auth rule `authRuleName` does not exist, the query will return a chain of errors where GQLSTATUS 42NAD is the cause of GQLSTATUS 42001.

To resolve this issue, make sure that the specified auth rule exists before calling the grant role command.

[[42NAE]] = 42NAE

Status description

error: syntax error or access rule violation - auth rule already exists. An auth rule with the name `{ [authRule] }` already exists.

Explanation

When creating an authorization rule, the specified auth rule name must be unique. If an auth rule with the same name already exists, the operation will fail with this error.

Example scenario

Try creating an auth rule using the following command:

```
CREATE AUTH RULE authRule SET CONDITION abac.oidc.user_attribute('start_date') > date.transaction('UTC');
```

If an auth rule named `authRule` does already exist, the query will return a chain of errors where GQLSTATUS 42NAE is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NAE: syntax error or access rule violation - auth rule already exists. An auth rule with the name
`authRule` already exists.
  42001: syntax error or access rule violation - invalid syntax
```

Possible solutions

To resolve this issue, make sure that there is no existing auth rule with the same name before calling the create command.

Another solution is to use the `IF NOT EXISTS` clause in the create command to avoid the error if the auth rule already exists. Then the auth rule will only be created if the name of the auth rule is unique.

It is also possible to use the `OR REPLACE` clause in the create auth rule command to overwrite the existing auth rule with the new definition.

[[42NAM]] = 42NAM

Status description

error: syntax error or access rule violation - unsupported temporal function form in auth rule condition. `{ [input] }` cannot be used in auth rule conditions as it retrieves the current time. Only transaction start time is available at the time of auth rule evaluation. Use `{ [replacement] }` instead.

Explanation

Auth rule conditions are evaluated at a different point in time than regular queries. When a temporal function such as `date()`, `datetime()`, `localdatetime()`, `localtime()`, or `time()` is called with a map argument whose only key is `timezone` (e.g. `date({timezone: 'UTC'})`), it retrieves the current time. Because the current time is not available during auth rule evaluation — only the transaction start time is — these calls are not allowed in auth rule conditions.

Use the `.transaction()` form of the temporal function instead, which returns the transaction start time:

Disallowed form	Suggested replacement
`date({timezone: 'UTC'})`	`date.transaction('UTC')`
`datetime({timezone: 'Europe/Stockholm'})`	`datetime.transaction('Europe/Stockholm')`

This error is returned as the cause of 42001.

Example scenarios

CREATE AUTH RULE with disallowed temporal function

Try creating an auth rule whose condition uses `date({timezone: 'UTC'})`:

```
CREATE AUTH RULE authRule SET CONDITION abac.oidc.user_attribute('start_date') > date({timezone: 'UTC'});
```

The query returns a chain of errors where GQLSTATUS 42NAM is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NAM: syntax error or access rule violation - unsupported temporal function form in auth rule condition.
'date({timezone: 'UTC'})' cannot be used in auth rule conditions as it retrieves the current time. Only
transaction start time is available at the time of auth rule evaluation. Use 'date.transaction('UTC')'
instead. (line 1, column 82 (offset: 81))
"CREATE AUTH RULE authRule SET CONDITION abac.oidc.user_attribute('start_date') > date({timezone: 'UTC'})"
                                                                                 ^

  42001: syntax error or access rule violation - invalid syntax
```

To resolve this, replace the function call with the `.transaction()` form:

```
CREATE AUTH RULE authRule SET CONDITION abac.oidc.user_attribute('start_date') > date.transaction('UTC');
```

ALTER AUTH RULE with disallowed temporal function

Try changing an auth rule whose condition uses `datetime({timezone: 'Europe/Stockholm'})`:

```
ALTER AUTH RULE authRule SET CONDITION abac.oidc.user_attribute('start_date') > datetime({timezone:
'Europe/Stockholm'});
```

The query returns a chain of errors where GQLSTATUS 42NAM is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NAM: syntax error or access rule violation - unsupported temporal function form in auth rule condition.
'datetime({timezone: 'Europe/Stockholm'})' cannot be used in auth rule conditions as it retrieves the
current time. Only transaction start time is available at the time of auth rule evaluation. Use
'datetime.transaction('Europe/Stockholm')' instead. (line 1, column 81 (offset: 80))
"ALTER AUTH RULE authRule SET CONDITION abac.oidc.user_attribute('start_date') > datetime({timezone:
'Europe/Stockholm'})"
                                                                                ^

  42001: syntax error or access rule violation - invalid syntax
```

To resolve this, replace the function call with the `.transaction()` form:

```
ALTER AUTH RULE authRule SET CONDITION abac.oidc.user_attribute('start_date') > datetime.transaction
('Europe/Stockholm');
```

[[42NAP]] = 42NAP

Status description

error: syntax error or access rule violation - unsupported trim specification. Unknown trim specification: `{ [input] }`.

Example scenario

Try trimming a string using an unrecognized trim specification instead of `LEADING`, `TRAILING`, or `BOTH`:

```
RETURN trim('INVALID', 'x', 'xxxhelloxxx') AS result;
```

The query returns a chain of errors where GQLSTATUS 42NAP is the cause of GQLSTATUS 42001:

GQLSTATUS error chain

```
42NAP: syntax error or access rule violation - unsupported trim specification. Unknown trim specification:
'INVALID'.
  42001: syntax error or access rule violation - invalid syntax
```

To resolve the error, use one of the supported trim specifications: `LEADING`, `TRAILING`, or `BOTH`:

```
RETURN trim('BOTH', 'x', 'xxxhelloxxx') AS result;
```

[[42NFC]] = 42NFC

Status description

error: syntax error or access rule violation - auth info validation error. Authentication and/or authorization could not be validated. See security logs for details.

[[42NFD]] = 42NFD

Status description

error: syntax error or access rule violation - credentials expired. Permission denied. The credentials you provided were valid, but must be changed before you can use this instance.

[[42NFE]] = 42NFE

Status description

error: syntax error or access rule violation - auth info expired. Authentication and/or authorization info expired.

[[42NFF]] = 42NFF

Status description

error: syntax error or access rule violation - permission/access denied. Access denied, see the security logs for details.

[[50N00]] = 50N00

Status description

error: general processing exception - internal error. Internal exception raised `{ [msgTitle] }`: `{ [msg] }`

[[50N05]] = 50N05

Status description

error: general processing exception - deadlock detected. Deadlock detected while trying to acquire locks. See log for more details.

[[50N06]] = 50N06

Status description

error: general processing exception - remote execution client error. Remote execution failed. See cause for more details.

[[50N07]] = 50N07

Status description

error: general processing exception - execution failed. Execution failed. See cause and debug log for details.

[[50N09]] = 50N09

Status description

error: general processing exception - invalid server state transition. The server transitioned into a server state that is not valid in the current context: `{ [boltServerState] }`.

[[50N10]] = 50N10

Status description

error: general processing exception - index drop failed. Unable to drop `{ [idxDescrOrName] }`.

[[50N11]] = 50N11

Status description

error: general processing exception - constraint creation failed. Unable to create `{ [constrDescrOrName] }`.

[[50N12]] = 50N12

Status description

error: general processing exception - constraint drop failed. Unable to drop `{ [constrDescrOrName] }`.

[[50N13]] = 50N13

Status description

error: general processing exception - constraint validation error. Unable to validate constraint `{ [constrDescrOrName] }`.

[[50N14]] = 50N14

Status description

error: general processing exception - constraint violation. A constraint imposed by the database was violated.

[[50N15]] = 50N15

Status description

error: general processing exception - unsupported index operation. The system attemped to execute an unsupported operation on index `{ [idx] }`. See debug.log for more information.

[[50N16]] = 50N16

Status description

error: general processing exception - remote execution transient error. Remote execution failed. See cause for more details.

[[50N17]] = 50N17

Status description

error: general processing exception - remote execution database error. Remote execution failed. See cause for more details.

[[50N18]] = 50N18

Status description

error: general processing exception - shard execution transient error. Communication with shard `{ [graph] }` failed. See cause for more details.

[[50N19]] = 50N19

Status description

error: general processing exception - shard execution database error. Communication with shard `{ [graph] }` failed. See cause for more details.

[[50N20]] = 50N20

Status description

error: general processing exception - shard execution client error. Communication with shard `{ [graph] }` failed. See cause for more details.

[[50N21]] = 50N21

Status description

error: general processing exception - no such schema descriptor. The `{ [schemaDescrType] }` was not found for `{ [schemaDescr] }`. Verify that the spelling is correct.

Example scenario

GQLSTATUS error

```
error: general processing exception - no such schema descriptor. The label property existence constraint
was not found for '(:Label1 {prop1})'. Verify that the spelling is correct.")
```

[[50N23]] = 50N23

Status description

error: general processing exception - transaction retry aborted. Transaction retry aborted after `{ [count] }` attempts. Retry timed out with a maximum retry duration of `{ [timeAmount] }` `{ [timeUnit] }`.

[[50N24]] = 50N24

Status description

error: general processing exception - sharded properties transaction handling database error. Unexpected exception while getting transaction state.

Example scenario

This error occurs when there is an issue on the graph shard while getting transaction state from a property shard. For example, the graph shard receives an unexpected response from a property shard.

[[50N25]] = 50N25

Status description

error: general processing exception - sharded properties transaction handling client error. Unexpected exception while getting transaction state.

Example scenario

This error occurs when there is an issue on the graph shard while getting transaction state from a property shard. For example, the graph shard times out while waiting for transaction state from property shards.

[[50N26]] = 50N26

Status description

error: general processing exception - invalid backup metadata script. The backup metadata script contains invalid syntax.

Explanation

When using the `existingMetadata: true` option to restore a database from a backup, the backup metadata script must contain valid Cypher statements. If the metadata script contains invalid syntax, the restore operation will fail with this error.

Example scenario

Try creating a database using the following command:

```
CREATE DATABASE `$databaseString` OPTIONS {existingMetadata:'use', seedUri:'$uri', existingData:'use'};
```

If the backup at `$uri` contains invalid Cypher syntax in its metadata script, the command will return an error with GQLSTATUS 50N26.

Possible solutions

To resolve this issue, use another backup with a valid metadata script.

[[50N42]] = 50N42

Status description

error: general processing exception - unexpected error. Unexpected error has occurred. See debug log for details.

[[51N00]] = 51N00

Status description

error: system configuration or operation exception - procedure registration error. Failed to register procedure/function.

[[51N01]] = 51N01

Status description

error: system configuration or operation exception - class field annotation should be public, non-final, and non-static. The field `{ [procField] }` in the class `{ [procClass] }` is annotated as a `@Context` field, but it is declared as static. `@Context` fields must be public, non-final and non-static.

[[51N02]] = 51N02

Status description

error: system configuration or operation exception - unsupported injectable component type. Unable to set up injection for procedure `{ [procClass] }`. The field `{ [procField] }` has type `{ [procFieldType] }` which is not a supported injectable component.

[[51N03]] = 51N03

Status description

error: system configuration or operation exception - unable to access field. Unable to set up injection for `{ [procClass] }`, failed to access field `{ [procField] }`.

[[51N04]] = 51N04

Status description

error: system configuration or operation exception - missing class field annotation. The field `{ [procField] }` on `{ [procClass] }` must be annotated as a `@Context` field in order to store its state.

[[51N05]] = 51N05

Status description

error: system configuration or operation exception - class field should be public and non-final. The field `{ [procField] }` on `{ [procClass] }` must be declared non-final and public.

[[51N06]] = 51N06

Status description

error: system configuration or operation exception - missing argument name. The argument at position `{ [pos] }` in `{ [procMethod] }` requires a `@Name` annotation and a non-empty name.

[[51N07]] = 51N07

Status description

error: system configuration or operation exception - invalid ordering of default arguments. The `{ [procFun] }` contains a non-default argument after a default argument. Non-default arguments are not allowed to be positioned after default arguments.

[[51N08]] = 51N08

Status description

error: system configuration or operation exception - exactly one @UserAggregationResult method and one @UserAggregationUpdate method required. The class `{ [procClass] }` must contain exactly one `@UserAggregationResult` method and exactly one `@UserAggregationUpdate` method.

[[51N09]] = 51N09

Status description

error: system configuration or operation exception - @UserAggregationUpdate method must be public and void. The `@UserAggregationUpdate` method `{ [procMethod] }` of `{ [procClass] }` must be public and have the return type `void`.

[[51N10]] = 51N10

Status description

error: system configuration or operation exception - aggregation method not public. The method `{ [procMethod] }` of `{ [procClass] }` must be public.

[[51N11]] = 51N11

Status description

error: system configuration or operation exception - class not public. The class `{ [procClass] }` must be public.

[[51N12]] = 51N12

Status description

error: system configuration or operation exception - class not void. The procedure `{ [proc] }` has zero return columns and must be defined as void.

Changed in 2025.03

[[51N13]] = 51N13

Status description

error: system configuration or operation exception - procedure or function name already in use. Unable to register the procedure or function `{ [procFun] }` because the name is already in use.

[[51N14]] = 51N14

Status description

error: system configuration or operation exception - duplicate field name. The procedure `{ [proc] }` has a duplicate `{ [procFieldType] }` field, `{ [procField] }`.

[[51N15]] = 51N15

Status description

error: system configuration or operation exception - invalid map key type. Type mismatch for map key. Required `STRING`, but found `{ [valueType] }`.

[[51N16]] = 51N16

Status description

error: system configuration or operation exception - invalid default value type. Type mismatch for the default value. Required `{ [valueType] }`, but found `{ [input] }`.

[[51N17]] = 51N17

Status description

error: system configuration or operation exception - invalid procedure or function name. Procedures and functions cannot be defined in the root namespace, or use a reserved namespace. Use the package name instead (e.g., org.example.com.`{ [procFun] }`)`.

[[51N18]] = 51N18

Status description

error: system configuration or operation exception - invalid method return type. The method `{ [procMethod] }` has an invalid return type. Procedures must return a stream of records, where each record is of a defined concrete class.

[[51N20]] = 51N20

Status description

error: system configuration or operation exception - cannot inject field. The field `{ [procField] }` is not injectable. Ensure the field is marked as public and non-final.

[[51N21]] = 51N21

Status description

error: system configuration or operation exception - procedure registry is busy. The procedure registration failed because the procedure registry was busy. Try again.

[[51N22]] = 51N22

Status description

error: system configuration or operation exception - exhaustive shortest path search disabled. Finding the shortest path for the given pattern requires an exhaustive search. To enable exhaustive searches, set `cypher.forbid_exhaustive_shortestpath` to false.

[[51N23]] = 51N23

Status description

error: system configuration or operation exception - cyclic shortest path search disabled. Cannot find the shortest path when the start and end nodes are the same. To enable this behavior, set `dbms.cypher.forbid_shortestpath_common_nodes` to false.

[[51N24]] = 51N24

Status description

error: system configuration or operation exception - insufficient resources for plan search. Could not find a query plan within given time and space limits.

[[51N25]] = 51N25

Status description

error: system configuration or operation exception - database is busy. Cannot compile query due to excessive updates to indexes and constraints.

[[51N26]] = 51N26

Status description

error: system configuration or operation exception - not supported in this version. `{ [item] }` is not available. This implementation of Cypher does not support `{ [feat] }`.

[[51N27]] = 51N27

Status description

Changed in 2025.03

error: system configuration or operation exception - not supported in this edition. `{ [feat] }` is not supported in `{ [edition] }`.

Explanation

This error occurs when trying to perform an operation that is not supported in the current edition of the database. For example, when you try changing a user's status, home database, or (after linked users) external auths/remove auth.

Example scenario

Try setting status for user `foo` to `suspended` using `SET STATUS` operation in the Neo4j Community edition:

```
ALTER USER foo
SET PASSWORD CHANGE REQUIRED
SET STATUS SUSPENDED";
```

The query returns an error with GQLSTATUS 51N27 and status description:

GQLSTATUS error

```
51N27: system configuration or operation exception - not supported in this edition. `SET STATUS` is not
supported in community edition.
```

[[51N28]] = 51N28

Status description

error: system configuration or operation exception - not supported by this database. This Cypher command must be executed against the database `{ [db] }`.

[[51N29]] = 51N29

Status description

error: system configuration or operation exception - not supported by this server. The command `{ [cmd] }` must be executed on the current `LEADER` server.

[[51N2A]] = 51N2A

Status description

error: system configuration or operation exception - not supported with auth disabled. The command `{ [cmd] }` is not available with auth disabled.

Explanation

This error occurs when auth is disabled and a user attempts to run a command that is not available unless auth is enabled.

Example scenario

Try changing the current user's password when auth is disabled:

```
ALTER CURRENT USER SET PASSWORD FROM $oldPassword TO $newPassword;
```

The query returns an error with GQLSTATUS 51N2A and status description:

GQLSTATUS error

```
51N2A: system configuration or operation exception - not supported with auth disabled. The command 'ALTER
CURRENT USER SET PASSWORD' is not available with auth disabled.
```

To fix this, either enable auth or change the password using the `ALTER USER` command:

```
ALTER USER $currentUsername SET PASSWORD $newPassword CHANGE NOT REQUIRED;
```

[[51N30]] = 51N30

Status description

error: system configuration or operation exception - not supported with this configuration. `{ [item] }` is not supported in `{ [context] }`.

[[51N31]] = 51N31

Status description

Changed in 2025.06

error: system configuration or operation exception - not supported. `{ [feat] }` is not supported in `{ [context] }`.

[[51N32]] = 51N32

Status description

error: system configuration or operation exception - server panic. Server is in panic.

[[51N33]] = 51N33

Status description

error: system configuration or operation exception - replication error. This member failed to replicate transaction, try again.

[[51N34]] = 51N34

Status description

error: system configuration or operation exception - write transaction failed due to leader change. Failed to write to the database due to a cluster leader change. Retrying your request at a later time may succeed.

[[51N35]] = 51N35

Status description

error: system configuration or operation exception - database location changed. The location of `{ [db] }` has changed while the transaction was running.

[[51N36]] = 51N36

Status description

error: system configuration or operation exception - out of memory. There is not enough memory to perform the current task.

[[51N37]] = 51N37

Status description

error: system configuration or operation exception - stack overflow. There is not enough stack size to perform the current task.

[[51N38]] = 51N38

Status description

error: system configuration or operation exception - failed to acquire execution thread. There are insufficient threads available for executing the current task.

[[51N39]] = 51N39

Status description

error: system configuration or operation exception - raft log corrupted. Expected set of files not found on disk. Please restore from backup.

[[51N40]] = 51N40

Status description

error: system configuration or operation exception - unable to start database. Database `{ [db] }` failed to start. Try restarting it.

[[51N41]] = 51N41

Status description

error: system configuration or operation exception - admin operation failed. Server or database admin operation not possible. Reason: `{ [msg] }`

[[51N43]] = 51N43

Status description

error: system configuration or operation exception - cannot deallocate servers. Cannot deallocate server(s) `{ [serverList] }`.

[[51N44]] = 51N44

Status description

error: system configuration or operation exception - cannot drop server. Cannot drop server `{ [server] }`.

[[51N45]] = 51N45

Status description

error: system configuration or operation exception - cannot cordon server. Cannot cordon server `{ [server] }`.

[[51N46]] = 51N46

Status description

error: system configuration or operation exception - cannot alter server. Cannot alter server `{ [server] }`.

[[51N47]] = 51N47

Status description

error: system configuration or operation exception - cannot rename server. Cannot rename server { [server] }.

[[51N48]] = 51N48

Status description

error: system configuration or operation exception - cannot enable server. Cannot enable server { [server] }.

[[51N49]] = 51N49

Status description

error: system configuration or operation exception - cannot alter database. Cannot alter database { [db] }.

[[51N50]] = 51N50

Status description

error: system configuration or operation exception - cannot recreate database. Cannot recreate database { [db] }.

[[51N51]] = 51N51

Status description

error: system configuration or operation exception - cannot create database. Cannot create database { [db] }.

[[51N52]] = 51N52

Status description

error: system configuration or operation exception - number of primaries out of range. Cannot { [action] } database topology. Number of primaries { [count] } needs to be at least 1 and may not exceed { [upper] }.

Changed in 2025.07

Explanation

This error occurs when trying to create or alter a database while providing a topology with primary count outside the supported range.

[[51N53]] = 51N53

Status description

error: system configuration or operation exception - number of secondaries out of range. Cannot `{ [action] }` database topology. Number of secondaries `{ [count] }` needs to be at least 0 and may not exceed `{ [upper] }`.

Changed in 2025.07

Explanation

This error occurs when trying to create or alter a database while providing a topology with secondary count outside the supported range.

[[51N54]] = 51N54

Status description

error: system configuration or operation exception - cannot reallocate. Failed to calculate reallocation for databases. `{ [msg] }`

[[51N55]] = 51N55

Status description

error: system configuration or operation exception - cannot create additional database. Failed to create the database `{ [db] }`. The limit of databases is reached. Either increase the limit using the config setting `{ [cfgSetting] }` or drop a database.

[[51N56]] = 51N56

Status description

error: system configuration or operation exception - topology out of range. The number of `{ [serverType] }` seeding servers `{ [count] 1 }` is larger than the desired number of `{ [allocType] }` allocations `{ $count2 }`.

[[51N57]] = 51N57

Status description

error: system configuration or operation exception - generic topology modification error. Unexpected error while picking allocations. `{ [msg] }`

[[51N58]] = 51N58

Status description

error: system configuration or operation exception - invalid shard topology. Invalid database shard topology. The number of `{ [context] }` `{ [count] }` needs to be at least 1 and may not exceed `{ [upper] }`.

[[51N59]] = 51N59

Status description

error: system configuration or operation exception - internal resource exhaustion. The DBMS is unable to handle the request, please retry later or contact the system operator. More information is present in the logs.

[[51N60]] = 51N60

Status description

error: system configuration or operation exception - unable to check enterprise license acceptance. The DBMS is unable to determine the enterprise license acceptance status.

[[51N61]] = 51N61

Status description

error: system configuration or operation exception - index population failed. Index `{ [idx] }` population failed.

[[51N62]] = 51N62

Status description

error: system configuration or operation exception - index is in a failed state. Unable to use index `{ [idx] }` because it is in a failed state. See logs for more information.

[[51N63]] = 51N63

Status description

error: system configuration or operation exception - index is still populating. Index `{ [idx] }` is not ready yet. Wait until it finishes populating and retry the transaction.

Explanation

An index cannot be used while its `state` is `POPULATING`, which occurs immediately after it is created. To check the `state` of an index — whether it is `ONLINE` (usable) or `POPULATING` (still being built; the `populationPercent` column shows the progress of the index creation) — run the following command: `SHOW INDEXES`. To wait for an index to come online, see procedures db.awaitIndex() and db.awaitIndexes().

[[51N64]] = 51N64

Status description

error: system configuration or operation exception - index dropped while sampling. The index dropped while sampling.

[[51N65]] = 51N65

Status description

error: system configuration or operation exception - vector index dimensionality mismatch. Vector index `{ [idx] }` has a configured dimensionality of `{ [dim] 1 }`, but the provided vector has a dimension `{ [dim] 2 }`.

Example scenario

For example, given that you have created the following vector index:

```
CREATE VECTOR INDEX indexName
FOR (n: Label) ON n.prop
OPTIONS {
  indexConfig: {`vector.dimensions`: 1048}
};
```

Try querying it with a vector with a dimension other than 1048:

```
CYPHER 25
MATCH (n)
  SEARCH n IN (
    VECTOR INDEX indexName
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 5
  )
RETURN n;
```

or

```
CALL db.index.vector.queryNodes('indexName', 5, [1, 2, 3]);
```

The query returns an error with GQLSTATUS 51N65 and a status description:

GQLSTATUS error

```
51N65: system configuration or operation exception - vector index dimensionality mismatch. Vector index
`indexName` has a configured dimensionality of 1048, but the provided vector has a dimension 3.
```

Possible solution

If you want to store and query vectors of varying dimensions in the same vector index, it cannot have a configured dimension. For example:

1. Drop the index you want to change the configuration of:

   ```
   DROP INDEX indexName;
   ```

2. Recreate the index without the configured dimension:

   ```
   CREATE VECTOR INDEX indexName
   FOR (n: Label) ON n.prop;
   ```

> **Note:**
>
> Depending on what you want to do with the failing query, the solution might be to omit the dimension when creating the index on vectors with different dimensions. However, keep in mind that in all other cases, it is recommended to provide a dimension, as this adds additional checks that ensure only vectors with the configured dimensions are indexed.

[[51N66]] = 51N66

Status description

error: system configuration or operation exception - resource exhaustion. Insufficient resources to complete the request.

[[51N67]] = 51N67

Status description

error: system configuration or operation exception - invalid CDC selector type. Unexpected CDC selector `{ [selectorType] 1 }` at `{ [input] }`, expected selector to be a `{ [selectorType] 2 }` selector.

[[51N68]] = 51N68

Status description

error: system configuration or operation exception - CDC is disabled for this database. Change Data Capture is not currently enabled for this database.

[[51N69]] = 51N69

Status description

error: system configuration or operation exception - system database is immutable. It is not possible to perform `{ [operation] }` on the system database.

[[51N70]] = 51N70

Status description

error: system configuration or operation exception - bolt is not enabled. Cannot get routing table for `{ [db] }` because Bolt is not enabled. Please update your configuration such that `server.bolt.enabled` is set to true.

[[51N71]] = 51N71

Status description

error: system configuration or operation exception - unsupported operation of a sharded database. Feature: `{ [feat] }` is not available in a sharded database.

[[51N72]] = 51N72

Status description

error: system configuration or operation exception - memory pool out of memory. Failed to allocate memory in a memory pool. See `{ [cfgSetting] }` in the neo4j.conf file.

[[51N73]] = 51N73

Status description

error: system configuration or operation exception - transaction memory limit reached. The transaction uses more memory than is allowed. The maximum allowed size for a transaction can be configured with `{ [cfgSetting] }` in the neo4j.conf file.

[[51N74]] = 51N74

Status description

error: system configuration or operation exception - maximum number of transactions reached. Failed to start a new transaction. The limit of concurrent transactions is reached. Increase the number of concurrent transactions using `{ [cfgSetting] }` in the neo4j.conf file.

[[51N76]] = 51N76

Status description

error: system configuration or operation exception - upgrade failed. The upgrade to a new Neo4j version failed.

Explanation

This error code can occur during a Neo4j upgrade to a new version in the rare case when something in the user's setup or DBMS environment prevents the upgrade. The error will be accompanied by a cause with a more concrete GQLSTATUS, describing what the user needs to change before retrying the upgrade.

[[51N77]] = 51N77

Status description

error: system configuration or operation exception - not supported in this store format. `{ [feat] }` is not supported in `{ [storeFormat] }` store format.

Example scenario

This error code is used when the user tries to use a feature that is unsupported by the store format, e.g. writing a property with an unsupported data type.

Possible solution

Use a supported data type, or use a different store format.

[[51N7A]] = 51N7A

Status description

error: system configuration or operation exception - no admin user candidate. No admin user candidate is found. Use `neo4j-admin dbms set-default-admin` to select a valid admin.

Explanation

When initializing the Neo4j DBMS for the first time, the system attempts to determine a default admin user. This user is responsible for performing administrative tasks on the database. If no valid admin user candidate can be found, the system will throw this error.

Example scenario

For example, before the database starts up for the first time, execute the following command to set a default admin user that does not exist:

```
bin/neo4j-admin dbms set-default-admin user-that-does-not-exist
```

Possible solutions

To resolve this issue, run `neo4j-admin dbms set-default-admin` again with a user that does exist in the system.

[[52N01]] = 52N01

Status description

error: procedure exception - procedure execution timeout. Execution of the procedure `{ [proc] }` timed out after `{ [timeAmount] }` `{ [timeUnit] }`.

[[52N02]] = 52N02

Status description

error: procedure exception - procedure execution client error. Execution of the procedure `{ [proc] }` failed due to a client error.

[[52N03]] = 52N03

Status description

error: procedure exception - invalid procedure execution mode. Execution of the procedure `{ [proc] }` failed due to an invalid specified execution mode `{ [procExeMode] }`.

[[52N05]] = 52N05

Status description

error: procedure exception - must invoke procedure on secondary. Cannot invoke procedure on this member because it is not a secondary for the database `{ [db] }`.

[[52N06]] = 52N06

Status description

error: procedure exception - invalid number of arguments to checkConnectivity. Unexpected number of arguments (expected 0-2 but received `{ [count] }`).

[[52N07]] = 52N07

Status description

error: procedure exception - invalid port argument to checkConnectivity. Unrecognised port name `{ [port] }` (valid values are: `{ [portList] }`.

[[52N08]] = 52N08

Status description

error: procedure exception - invalid server id argument to checkConnectivity. Unable to parse server id `{ [server] }`.

[[52N09]] = 52N09

Status description

error: procedure exception - procedure execution database error. Execution of the procedure `{ [proc] }` failed due to a database error.

[[52N10]] = 52N10

Status description

error: procedure exception - invalid address key. An address key is included in the query string provided to the GetRoutingTableProcedure, but its value could not be parsed.

[[52N11]] = 52N11

Status description

error: procedure exception - generic topology procedure error. An unexpected error has occurred: `{ [msg] }`.

Example scenario

For example, trying to set default number of primaries to 12:

```
CALL dbms.setDefaultAllocationNumbers(12,0);
```

The query returns a chain of errors where GQLSTATUS 52N11 is the cause of GQLSTATUS 52N02:

GQLSTATUS error chain

```
52N11: procedure exception - generic topology procedure error. An unexpected error has occurred: Number of
primaries '12' may not exceed 11 and needs to be at least 1
  52N02: procedure exception - procedure execution client error. Execution of the procedure
dbms.setDefaultAllocationNumbers() failed due to a client error.
```

[[52N12]] = 52N12

Status description

error: procedure exception - cannot change default database. The previous default database `{ [db] }` is still running.

[[52N13]] = 52N13

Status description

error: procedure exception - new default database does not exist. New default database `{ [db] }` does not exist.

[[52N14]] = 52N14

Status description

error: procedure exception - system cannot be default database. System database cannot be set as default.

[[52N16]] = 52N16

Status description

error: procedure exception - invalid procedure argument list. Invalid arguments to procedure.

[[52N17]] = 52N17

Status description

error: procedure exception - quarantine change failed. Setting/removing the quarantine marker failed: `{ [msg] }`.

Example scenario

For example, trying to lift a quarantine which does not exist:

```
CALL dbms.unquarantineDatabase("0c000000", "neo4j");
```

The query returns a chain of errors where GQLSTATUS 52N17 is the cause of GQLSTATUS 52N02:

GQLSTATUS error chain

```
52N17: procedure exception - quarantine change failed. Setting/removing the quarantine marker failed:
Cannot find server: 0c000000
  52N02: procedure exception - procedure execution client error. Execution of the procedure
dbms.unquarantineDatabase() failed due to a client error.
----
```

[[52N18]] = 52N18

Status description

error: procedure exception - too many seeders. The number of seeding servers `{ [countSeeders] }` is larger than the defined number of allocations `{ [countAllocs] }`.

[[52N19]] = 52N19

Status description

error: procedure exception - no such seeder. The specified seeding server with id `{ [server] }` was not found. Verify that the spelling is correct.

[[52N21]] = 52N21

Status description

error: procedure exception - failed to clean the system graph. Failed to clean the system graph.

[[52N22]] = 52N22

Status description

error: procedure exception - invalid procedure argument. Invalid argument `{ [field] }` for `{ [procParam] }` on procedure `{ [proc] }`. The expected format of `{ [procParam] }` is `{ [procParamFmt] }`.

[[52N23]] = 52N23

Status description

error: procedure exception - non-reloadable namespace. The following namespaces are not reloadable: `{ [namespaceList] }`

[[52N24]] = 52N24

Status description

error: procedure exception - failed to reload procedures. Failed to reload procedures. See logs for more information.

[[52N25]] = 52N25

Status description

error: procedure exception - JMX error. JMX error while accessing `{ [param] }`. See logs for more information.

[[52N26]] = 52N26

Status description

error: procedure exception - invalid change identifier. `{ [param] }` is not a valid change identifier.

[[52N27]] = 52N27

Status description

error: procedure exception - invalid commit timestamp. The commit timestamp for the provided transaction ID does not match the one in the transaction log.

[[52N28]] = 52N28

Status description

error: procedure exception - invalid change identifier and transaction id. `{ [transactionId] }` is not a valid change identifier. Transaction ID does not exist.

[[52N29]] = 52N29

Status description

error: procedure exception - outdated change identifier. Given ChangeIdentifier describes a transaction that occurred before any enrichment records exist.

[[52N30]] = 52N30

Status description

error: procedure exception - future change identifier. Given ChangeIdentifier describes a transaction that hasn't yet occurred.

[[52N31]] = 52N31

Status description

error: procedure exception - wrong database. Change identifier `{ [param] }` does not belong to this database.

[[52N32]] = 52N32

Status description

error: procedure exception - invalid sequence number. Change identifier `{ [param] 1 }` has an invalid

sequence number `{ $param2 }`.

[[52N33]] = 52N33

Status description

error: procedure exception - procedure invocation failed. Failed to invoke procedure/function `{ [sig] }` caused by: `{ [msg] }`.

[[52N34]] = 52N34

Status description

error: procedure exception - procedure restricted. `{ [fun] }` is restricted and accesses database internals. Procedure restriction is controlled by the dbms.security.procedures.unrestricted setting. Only un-restrict procedures you can trust with access to database internals.

[[52N35]] = 52N35

Status description

error: procedure exception - procedure compilation failed. Failed to compile procedure defined in `{ [procClass] }`: `{ [msg] }`

[[52N36]] = 52N36

Status description

error: procedure exception - invalid procedure argument with API documentation hint. Invalid argument `{ [field] }` for `{ [procParam] }` on procedure `{ [proc] }`. The expected format of `{ [procParam] }` is outlined in the `{ [procParamFmt] }` API documentation.

[[52N37]] = 52N37

Status description

error: procedure exception - procedure execution error. Execution of the procedure `{ [proc] }` failed.

[[52N38]] = 52N38

Status description

error: procedure exception - cdc start position not found. Cannot find a start position in the logs.

[[52N39]] = 52N39

Status description

error: procedure exception - cdc scanner inactive. The log scanner is no longer active.

[[52N40]] = 52N40

Status description

error: procedure exception - Reconciliation failed during writing the topology graph, transaction may not be committed.

[[52N41]] = 52N41

Status description

error: procedure exception - The key value for `{ [field] }` in the query string cannot be parsed when getting a routing table.

Explanation

This error occurs when trying to use a routing policy, but providing some invalid details on a routed request. Example fields include `policy` and `alias`.

[[52U00]] = 52U00

Status description

error: procedure exception - custom procedure execution error cause. Execution of the procedure `{ [proc] }` failed due to `{ [msgTitle] }`: `{ [msg] }`

[[53N33]] = 53N33

Status description

error: function exception - function invocation failed. Failed to invoke function `{ [sig] }` caused by: `{ [msg] }`

Example scenario

Try parsing a duration string value:

```
RETURN duration({hours: 1}, "hh-mm-ss") as r;
```

The query returns an errors with GQLSTATUS 53N33 and status description:

GQLSTATUS error

```
53N33: function exception - function invocation failed. Failed to invoke function duration(input :: ANY,
pattern = DEFAULT_TEMPORAL_ARGUMENT :: STRING) :: DURATION caused by: A pattern can only be used in
conjunction with a `STRING` input.
----
```

[[53N34]] = 53N34

Status description

error: function exception - function restricted. `{ [fun] }` is restricted and accesses database internals. User-defined function restriction is controlled by the `dbms.security.procedures.unrestricted` setting.

Only un-restrict user-defined functions you can trust with access to database internals.

[[53N35]] = 53N35

Status description

error: function exception - function compilation failed. Failed to compile function defined in `{ [funClass] }`: `{ [msg] }`

[[53N37]] = 53N37

Status description

error: function exception - function execution error. Execution of the function `{ [fun] }` failed.

[[53U00]] = 53U00

Status description

error: function exception - custom function execution error cause. Execution of the function `{ [fun] }` failed due to `{ [msgTitle] }`: `{ [msg] }`.

G1001

Status description

error: dependent object error - edges still exist

List of Neo4j error codes

This page lists the current Neo4j error codes, which are returned along side GQLSTATUS error codes, grouped by type.

> **Note:**
>
> Starting from 2025.04, using the error messages programmatically is deprecated. Because error messages might change in a future release, please parse or use GQLSTATUS error codes in your application code. For details, see GQLSTATUS error codes.

Client errors

This is a complete list of all client errors Neo4j may return, and what they mean.

Neo4j code	Description
`Neo.ClientError.ChangeDataCapture.InvalidIdentifier`	Invalid change identifier.

Neo4j code	Description
`Neo.ClientError.Cluster.NotALeader`	The request cannot be processed by this server. Write requests can only be processed by the leader.
`Neo.ClientError.Cluster.Routing`	Unable to route the request to the appropriate server
`Neo.ClientError.Database.DatabaseNotFound`	The request referred to a database that does not exist.
`Neo.ClientError.Database.ExistingAliasFound`	The request referred to a database with an alias.
`Neo.ClientError.Database.ExistingDatabaseFound`	The request referred to a database that already exists.
`Neo.ClientError.Database.IllegalAliasChain`	An illegal chain of aliases has been detected. This request cannot be executed.
`Neo.ClientError.General.ForbiddenOnReadOnlyDatabase`	This is a read-only database, writing or modifying the database is not allowed.
`Neo.ClientError.General.InvalidArguments`	The request contained fields that were empty or are not allowed.
`Neo.ClientError.General.TransactionOutOfMemoryError`	The transaction used more memory than was allowed. The maximum allowed size for a transaction can be configured with `db.memory.transaction.max_size` in the Neo4j configuration file (normally in *conf/neo4j.conf* or, if you are using Neo4j Desktop, found through the user interface).
`Neo.ClientError.General.UpgradeRequired`	This transaction required the database to be of a higher kernel version than it is. Make sure that the Neoj4 DBMS has been correctly upgraded before retrying this operation.
`Neo.ClientError.General.WriteOnReadOnlyAccessDatabase`	This database is in read-only mode, writing or modifying the database is not allowed.
`Neo.ClientError.Procedure.ProcedureCallFailed`	Failed to invoke a procedure. See the detailed error description for the exact cause.
`Neo.ClientError.Procedure.ProcedureNotFound`	A request referred to a procedure that is not registered with this database instance. If you are deploying custom procedures in a cluster setup, ensure all instances in the cluster have the procedure jar file deployed.
`Neo.ClientError.Procedure.ProcedureRegistrationFailed`	The database failed to register a procedure, refer to the associated error message for details.
`Neo.ClientError.Procedure.ProcedureTimedOut`	The procedure did not complete within the specified timeout. You may want to retry with a longer timeout.
`Neo.ClientError.Procedure.TypeError`	A procedure is using or receiving a value of an invalid type.
`Neo.ClientError.Request.Invalid`	The client provided an invalid request.

Neo4j code	Description
`Neo.ClientError.Request.InvalidFormat`	The client provided a request that was missing required fields or had values that are not allowed.
`Neo.ClientError.Request.InvalidUsage`	The client made a request but did not consume outgoing buffers in a timely fashion.
`Neo.ClientError.Routing.DbmsInPanic`	Server is in panic.
`Neo.ClientError.Routing.RoutingFailed`	Failed to route.
`Neo.ClientError.Schema.ConstraintAlreadyExists`	Unable to perform this operation because it will clash with a pre-existing constraint.
`Neo.ClientError.Schema.ConstraintNotFound`	The request (directly or indirectly) referred to a constraint that does not exist.
`Neo.ClientError.Schema.ConstraintValidationFailed`	A constraint imposed by the database was violated.
`Neo.ClientError.Schema.ConstraintViolation`	Added or changed index entry violates a constraint.
`Neo.ClientError.Schema.ConstraintWithNameAlreadyExists`	Unable to perform this operation because a constraint with that name already exists.
`Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists`	Unable to perform this operation because an equivalent schema rule already exists.
`Neo.ClientError.Schema.ForbiddenOnConstraintIndex`	A requested operation cannot be performed on the specified index because the index is part of a constraint. If you want to drop the index, you must also drop the constraint.
`Neo.ClientError.Schema.IndexAlreadyExists`	Unable to perform this operation because it will clash with a pre-existing index.
`Neo.ClientError.Schema.IndexMultipleFound`	The request referenced an index by its schema and multiple matching indexes were found.
`Neo.ClientError.Schema.IndexNotApplicable`	The request did not contain the properties required by the index.
`Neo.ClientError.Schema.IndexNotFound`	The request (directly or indirectly) referred to an index that does not exist.
`Neo.ClientError.Schema.IndexWithNameAlreadyExists`	Unable to perform this operation because an index with that name already exists.
`Neo.ClientError.Schema.RepeatedLabelInSchema`	Unable to create an index or a constraint because the schema had a repeated label.
`Neo.ClientError.Schema.RepeatedPropertyInCompositeSchema`	Unable to create an index or a constraint because the schema had a repeated property.

Neo4j code	Description
`Neo.ClientError.Schema.RepeatedRelationshipTypeInSchema`	Unable to create an index or a constraint because the schema had a repeated relationship type.
`Neo.ClientError.Schema.SchemaRuleEntrySizeLimitError`	A schema rule entry exceeded the maximum allowed size.
`Neo.ClientError.Schema.TokenLengthError`	A token name, such as a label, relationship type or property key is too long
`Neo.ClientError.Schema.TokenNameError`	An invalid name is used for a token (label, relationship type, property key). Tokens cannot be empty strings and cannot be null.
`Neo.ClientError.Security.AuthenticationRateLimit`	The client has provided incorrect authentication details too many times in a row.
`Neo.ClientError.Security.AuthorizationExpired`	The stored authorization info has expired. Please reconnect.
`Neo.ClientError.Security.CredentialsExpired`	The credentials have expired and need to be updated.
`Neo.ClientError.Security.Forbidden`	An attempt was made to perform an unauthorized action.
`Neo.ClientError.Security.TokenExpired`	The auth provider token has expired.
`Neo.ClientError.Security.Unauthorized`	The client is unauthorized due to authentication failure.
`Neo.ClientError.Statement.AccessMode`	The request could not be completed due to an access mode violation.
`Neo.ClientError.Statement.ArgumentError`	The statement is attempting to perform operations using invalid arguments.
`Neo.ClientError.Statement.ArithmeticError`	Invalid use of arithmetic, such as dividing by zero.
`Neo.ClientError.Statement.ConstraintVerificationFailed`	A constraint imposed by the statement is violated by the data in the database.
`Neo.ClientError.Statement.EntityNotFound`	The statement refers to a non-existent entity.
`Neo.ClientError.Statement.ExternalResourceFailed`	Access to an external resource failed.
`Neo.ClientError.Statement.InvalidTargetDatabaseError`	The specified database is not a valid target for this command.
`Neo.ClientError.Statement.NotSystemDatabaseError`	This is an administration command and it should be executed against the `system` database.
`Neo.ClientError.Statement.ParameterMissing`	The statement refers to a parameter that was not provided in the request.
`Neo.ClientError.Statement.PropertyNotFound`	The statement refers to a non-existent property.
`Neo.ClientError.Statement.RemoteExecutionClientError`	The database was unable to execute a remote part of the statement due to a client error.

Neo4j code	Description
`Neo.ClientError.Statement.RuntimeUnsupportedError`	This query is not supported by the chosen runtime.
`Neo.ClientError.Statement.SemanticError`	The statement is syntactically valid but expresses something that the database cannot do.
`Neo.ClientError.Statement.SyntaxError`	The statement contains invalid or unsupported syntax.
`Neo.ClientError.Statement.TypeError`	The statement is attempting to perform operations on values with types that are not supported by the operation.
`Neo.ClientError.Statement.UnsupportedAdministrationCommand`	This administration command is not supported.
`Neo.ClientError.Statement.UnsupportedOperationError`	This query performed an operation that is not supported in this context.
`Neo.ClientError.Transaction.ForbiddenDueToTransactionType`	The transaction is of the wrong type to service the request. For instance, a transaction that has had schema modifications performed in it cannot be used to subsequently perform data operations, and vice versa.
`Neo.ClientError.Transaction.InvalidBookmark`	The supplied bookmark cannot be interpreted. You should only supply a bookmark that was previously generated by Neo4j. Maybe you have generated your own bookmark, or modified a bookmark since it was generated by Neo4j.
`Neo.ClientError.Transaction.InvalidBookmarkMixture`	Mixing bookmarks generated by different databases is forbidden. You should only chain bookmarks that are generated from the same database. You may however chain bookmarks generated from the `system` database with bookmarks from another database.
`Neo.ClientError.Transaction.LockClientStopped`	The transaction has been terminated, so no more locks can be acquired. This may occur because the transaction ran longer than the configured transaction timeout, a human operator manually terminated the transaction, or the database shut down.
`Neo.ClientError.Transaction.Terminated`	Explicitly terminated by the user.
`Neo.ClientError.Transaction.TransactionAccessedConcurrently`	There were concurrent requests accessing the same transaction, which is not allowed.
`Neo.ClientError.Transaction.TransactionHookFailed`	Transaction hook failure.
`Neo.ClientError.Transaction.TransactionMarkedAsFailed`	The transaction was marked as both successful and failed. Because failure takes precedence, the transaction was rolled back.
`Neo.ClientError.Transaction.TransactionNotFound`	The request referred to a transaction that does not exist.

Neo4j code	Description
`Neo.ClientError.Transaction.TransactionTimedOut`	The transaction has not completed within the specified timeout (`db.transaction.timeout`). You may want to retry with a longer timeout.
`Neo.ClientError.Transaction.TransactionTimedOutClientConfiguration`	The transaction has not completed within the timeout specified at its start by the client. You may want to retry with a longer timeout.
`Neo.ClientError.Transaction.TransactionValidationFailed`	Transaction changes did not pass validation checks.

Transient errors

This is a complete list of all transient errors Neo4j may return, and what they mean.

Neo4j code	Description
`Neo.TransientError.ChangeDataCapture.FutureIdentifier`	Change identifier points to a future transaction that has not yet happened on this database instance.
`Neo.TransientError.Cluster.ReplicationFailure`	Replication failure.
`Neo.TransientError.General.DatabaseUnavailable`	The database is not currently available to serve your request, refer to the database logs for more details. Retrying your request at a later time may succeed.
`Neo.TransientError.General.MemoryPoolOutOfMemoryError`	The memory pool limit was exceeded. The corresponding setting can be found in the error message.
`Neo.TransientError.General.OutOfMemoryError`	There is not enough memory to perform the current task. Please try increasing `server.memory.heap.max_size` in the Neo4j configuration file (normally in *conf/neo4j.conf* or, if you are using Neo4j Desktop, found through the user interface). If you are running an embedded installation, you can increase the heap by using the `-Xmx` command line flag, and then restart the database.

Neo4j code	Description
`Neo.TransientError.General.StackOverFlowError`	There is not enough stack size to perform the current task. This is generally considered to be a database error, so please contact Neo4j support. You could try increasing the stack size; for example to set the stack size to `2M`, add `server.jvm.additional=-Xss2M` to in the Neo4j configuration file (normally in *conf/neo4j.conf* or, if you are using Neo4j Desktop, found through the user interface). If you are running an embedded installation, you can add `-Xss2M` as command line flag.
`Neo.TransientError.General.TransactionMemoryLimit`	There is not enough memory to perform the current task. Please try increasing `server.memory.off_heap.transaction_max_size` in the Neo4j configuration file (normally in *conf/neo4j.conf* or, if you are using Neo4j Desktop, found through the user interface), and then restart the database.
`Neo.TransientError.Invalid.ResourceExhaustion`	The server has rejected this request as a resource is exhausted at the moment. You can retry at a later time. For further details see server logs.
`Neo.TransientError.Request.NoThreadsAvailable`	There are no available threads to serve this request at the moment. You can retry at a later time, or consider increasing max thread pool size for bolt connector(s).
`Neo.TransientError.Security.AuthProviderFailed`	An auth provider request failed.
`Neo.TransientError.Security.AuthProviderTimeout`	An auth provider request timed out.
`Neo.TransientError.Security.ModifiedConcurrently`	The user was modified concurrently to this request.
`Neo.TransientError.Statement.ExecutionTimeout`	The database was unable to execute the statement in a timely fashion.
`Neo.TransientError.Statement.RemoteExecutionTransientError`	The database was unable to execute a remote part of the statement due to a transient failure.
`Neo.TransientError.Transaction.BookmarkTimeout`	Bookmark wait timed out. Database has not reached the specified version.
`Neo.TransientError.Transaction.ConstraintsChanged`	Database constraints changed since the start of this transaction.

Neo4j code	Description
`Neo.TransientError.Transaction.DeadlockDetected`	This transaction, and at least one more transaction, has acquired locks in a way that it will wait indefinitely, and the database has aborted it. Retrying this transaction will most likely be successful.
`Neo.TransientError.Transaction.Interrupted`	Interrupted while waiting.
`Neo.TransientError.Transaction.LeaderSwitch`	The request could not be completed due to cluster leader switch.
`Neo.TransientError.Transaction.LeaseExpired`	The lease under which this transaction was started is no longer valid.
`Neo.TransientError.Transaction.LockAcquisitionTimeout`	Unable to acquire lock within configured timeout (`db.lock.acquisition.timeout`).
`Neo.TransientError.Transaction.MaximumTransactionLimitReached`	Unable to start new transaction since the maximum number of concurrently executing transactions is reached (`db.transaction.concurrent.maximum`). You can retry at a later time or consider increasing allowed maximum of concurrent transactions.
`Neo.TransientError.Transaction.Outdated`	Transaction has seen state which has been invalidated by applied updates while the transaction was active. Transaction may succeed if retried.
`Neo.TransientError.Transaction.QueryExecutionFailedOnTransaction`	The transaction was marked as failed because a query failed.

Database error

This is a complete list of all database errors Neo4j may return, and what they mean.

Neo4j code	Description
`Neo.DatabaseError.ChangeDataCapture.Disabled`	Change Data Capture is not currently enabled for this database.
`Neo.DatabaseError.ChangeDataCapture.ScanFailure`	Unable to read the Change Data Capture data for this database.
`Neo.DatabaseError.Database.DatabaseLimitReached`	The limit to number of databases has been reached.
`Neo.DatabaseError.Database.UnableToStartDatabase`	Unable to start database.
`Neo.DatabaseError.Database.Unknown`	Unknown database management error.

Neo4j code	Description
`Neo.DatabaseError.General.IndexCorruptionDetected`	The request (directly or indirectly) referred to an index that is in a failed state. The index needs to be dropped and recreated manually.
`Neo.DatabaseError.General.SchemaCorruptionDetected`	A malformed schema rule was encountered. Please contact your support representative.
`Neo.DatabaseError.General.StorageDamageDetected`	Expected set of files not found on disk. Please restore from backup.
`Neo.DatabaseError.General.UnknownError`	An unknown error occurred.
`Neo.DatabaseError.Schema.ConstraintCreationFailed`	Creating a requested constraint failed.
`Neo.DatabaseError.Schema.ConstraintDropFailed`	The database failed to drop a requested constraint.
`Neo.DatabaseError.Schema.IndexCreationFailed`	Failed to create an index.
`Neo.DatabaseError.Schema.IndexDropFailed`	The database failed to drop a requested index.
`Neo.DatabaseError.Schema.LabelAccessFailed`	The request accessed a label that did not exist.
`Neo.DatabaseError.Schema.PropertyKeyAccessFailed`	The request accessed a property that does not exist.
`Neo.DatabaseError.Schema.RelationshipTypeAccessFailed`	The request accessed a relationship type that does not exist.
`Neo.DatabaseError.Schema.SchemaRuleAccessFailed`	The request referred to a schema rule that does not exist.
`Neo.DatabaseError.Schema.SchemaRuleDuplicateFound`	The request referred to a schema rule that is defined multiple times.
`Neo.DatabaseError.Schema.TokenLimitReached`	The maximum number of tokens of this type has been reached, no more tokens of this type can be created.
`Neo.DatabaseError.Statement.ExecutionFailed`	The database was unable to execute the statement.
`Neo.DatabaseError.Statement.RemoteExecutionFailed`	The database was unable to execute a remote part of the statement.
`Neo.DatabaseError.Transaction.LinkedTransactionError`	The transaction was terminated because another transaction executing the same query encountered an error.
`Neo.DatabaseError.Transaction.TransactionCommitFailed`	The database was unable to commit the transaction.
`Neo.DatabaseError.Transaction.TransactionLogError`	The database was unable to write transaction to log.
`Neo.DatabaseError.Transaction.TransactionRollbackFailed`	The database was unable to roll back the transaction.
`Neo.DatabaseError.Transaction.TransactionStartFailed`	The database was unable to start the transaction.
`Neo.DatabaseError.Transaction.TransactionTerminationFailed`	The database was unable to terminate the transaction.

License

Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)

You are free to

Share

copy and redistribute the material in any medium or format

Adapt

remix, transform, and build upon the material

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms

Attribution

You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

NonCommercial

You may not use the material for commercial purposes.

ShareAlike

If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions

You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Notices

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation.

No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.

See https://creativecommons.org/licenses/by-nc-sa/4.0/ for further details. The full license text is available at https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.